Implementation and Verification of UCIe 2.0 Logical PHY Layer

---

Submitted to

The Department of Electronics and Communications Faculty of Engineering, Cairo University in Partial Fulfillment of the Requirements for The Degree of Bachelor of Science in Electronics and Electrical Communications Engineering

---

Submitted by

Ahmed Abdelraouf
Habibat-Alrahman Ashraf
Mohammed Ehab
Philopateer Awny
Shaza Adnan
Yomna Omar

Under Supervision of: Dr. Ibrahim Qamar

Technically Sponsored by

SIEMENS

June 2025

Acknowledgment

The successful completion of this graduation project and the delivery of its final report would not have been possible without the generous support and technical guidance of several individuals. We are truly grateful for the assistance we received throughout this academic year. The outcomes we were able to achieve are a direct reflection of the mentorship, encouragement, and shared expertise that supported us along the way.

We would like to express our heartfelt appreciation to our academic supervisor and mentor, **Prof. Dr. Ibrahim Qamar**, for his consistent support, timely guidance, and constructive feedback. His close follow-up and encouragement provided the foundation we needed to navigate both the challenges and milestones of this project.

We would like to extend our appreciation to **Eng. Ahmed Khedr**, Siemens team leader, for his early guidance and support in helping us set a strong foundation for the project.

We would also like to thank our project sponsors **Eng. Peter Eskander**, **Eng. Salma Ali**, **Eng. Mahmoud Ahmed** and **Eng. Mohamed Abdelkader** for their continuous involvement, invaluable feedback, and technical direction throughout the year. Their commitment to our learning extended far beyond scheduled meetings, as they provided detailed answers to our many questions—ranging from protocol-specific concerns in UCIe, to broader architectural topics in PCIe and other advanced interconnect standards. Their willingness to engage deeply with our work and guide us through complex scenarios had a significant impact on our development.

We are genuinely thankful for the collaborative effort and time invested by everyone who contributed to this experience.

Table of Contents

List of Figures 4
List of Tables 9
Terminologies 12
Abstract 15
Chapter 1: UCIe Introduction: 16
1.1. Previous Standards: 16
1.1.1. PCI Express (PCIe) 16
1.1.2. Compute Express Link (CXL) 16
1.1.3. Streaming Protocols (e.g., AXI-Stream) 17
1.1.4. Raw Die-to-Die Interface (RDI) 17
1.1.5. Advanced Interface Bus (AIB) 17
1.2. UCIe Motivation: 18
1.3. UCIe Layers Overview: 19
Chapter 2: Physical Layer: 24
2.1. Main Functionalities: 24
2.1.1. Link Initialization and Training: 24
2.1.2. Byte to Lane Mapping: 26
2.1.3. Valid Framing: 27
2.1.4. Clock Gating: 27
2.1.5. Transmitting and Receiving Sideband messages: 28
2.1.6. Scrambling & Training pattern generation: 34
2.1.7. Lane Reversal & Width Degradation & Speed Degradation: 35
2.2. Assumptions and Supported Functionalities: 36
2.3. Architecture: 39
2.4. Methodology: 43
Chapter 3: Physical Layer Design: 44
3.1. Link Training State Machine “LTSM”: 44
3.2. Controller: 57
3.2.1. RDI: 57
3.2.2. Phy_Module: 64
3.2.3. Phy_Module_Partner: 69
3.2.4. Controller Arbiter: 72
3.3. Sideband “SB”: 73
3.3.1. Sideband Transmitter “SB_TX”: 73
3.3.2. Sideband Receiver “SB_RX”: 96
3.4. Mainband “MB”: 111
3.4.1. Mainband Transmitter “MB_TX”: 111

3.4.2. Main band Receiver “MB_RX”:.....141
Chapter 4: UVM Basics:.....159
4.1. Motivation for UVM:.....159
4.2. UVM Testbench:.....160
4.2.1. Test.....160
4.2.2. Environment (env).....161
4.2.3. Agent.....161
4.2.4. UVM Configuration Object.....162
4.2.5. Coverage Collector.....163
4.2.6. UVM Class Library.....163
4.2.7. UVM phases.....164
4.2.8. Basic UVM Connections in Testbench Architecture.....165
4.3. Guidelines for Robust Verification of Complex Systems Using UVM.....166
4.3.1. Handling multiclock domain in UVM.....166
4.3.2. Interface Modeling and TB Arch. for Dual-Clock Domain Verification.....167
4.3.3. Managing Process Synchronization in UVM Testbench.....167
Chapter 5: Physical Layer Verification:.....169
5.1. Link Training State Machine “LTSM”:.....169
5.1.1. Interface Signals:.....169
5.1.2. UVM Testbench Architecture:.....169
5.1.3. Test Plan:.....171
5.1.4. Results:.....174
5.2. Controller:.....177
5.2.1. RDI:.....177
5.2.2. Phy_Module:.....189
5.2.3. Phy_Module_Partner:.....198
5.3. Sideband “SB”:.....204
5.3.1. Sideband Transmitter “SB_TX”:.....204
5.3.2. Sideband Receiver “SB_RX”:.....220
5.4. Mainband “MB”:.....232
5.4.1. Mainband Transmitter “MB_TX”:.....232
5.4.2. Mainband Receiver “MB_RX”:.....242
Chapter 6: Sub-Integration Phases:.....252
6.1. Sub-Integration Motive:.....252
6.2. Mainband Integration:.....252
6.2.1. Integration Steps:.....252
6.2.2. Bugs Found and Solved:.....254
6.2.3. Simulation results.....254

6.3. Sideband Integration: 256
6.3.1. Integration Steps: 256
6.3.2. Bugs Found and Solved 257
6.3.3. Simulation results 257
Chapter 7: End-To-End Verification Environment 259
7.1. Methodology: 259
7.1.1. Verification Reuse 259
7.1.2. Encapsulation 259
7.2. Hierarchical Verification Flow-steps 260
7.3.1. Active/Passive mode 260
7.3.2. Solving Namespace Collision Issue 260
7.3.3. Hierarchal interface versus a top interface with multiple modports 261
7.3.4. Scoreboard Architecture Design Strategy 261
7.3. E2E-UVM Architecture 263
7.4. Sequences & Simulation Scenarios 264
7.4.1. Reset_flow Sequence 264
7.4.2. Bringup_flow Sequence 264
7.4.3. Active_flow Sequence 266
7.4.4. Simulation Scenarios 266
7.5. Simulation Results 272
7.5.1. Bugs Found & Fixed 272
7.5.2. Coverage Achieved 274
Chapter 8: FPGA Flow 276
8.1. FPGA Overview 276
8.2. FPGA Flow 277
8.2.1. Linting 277
8.3. Challenges and Modifications: 286
Chapter 9: Future Work: 288
9.1. Migration to UCIe x64 Advanced Package 288
9.2. Lane Repair Mechanism 288
9.3. MMPL (Multi-Module PHY Logic) Integration 288
9.4. PHY Interoperability and Multi-Module Configuration Support 288
9.5. Integration of External Reference IPs as Predictors 289
References 290

List of Figures

Figure 1-1: UCIe layers and functionalities....20
Figure 1-2: Header Flit Format for Management Transport Protocol....20
Figure 1-3: Configurations of Protocol and Adapter Layers through FDI Interface....21
Figure 1-4: Data Transfer Timing Diagram through RDI Interface....22
Figure 1-5: Configurations of Adapter and Physical layers through RDI Inetrface....22
Figure 1-6: Physical Layer Components....23
Figure 2-1: State Definitions for Initialization in Specs....25
Figure 2-2: Link Training State Machine in Specs....26
Figure 2-3: Bit Arrangement Within Byte transfer....26
Figure 2-4: Byte Mapping for x32 interface....27
Figure 2-5: Valid framing example....27
Figure 2-6: Clock Gating Diagram....27
Figure 2-7: Example 64-bit Sideband Serial Packet Transfer....28
Figure 2-8: Sideband Packet Transmission: Back-to-Back....28
Figure 2-9: Mailbox Acessing Mechanism....29
Figure 2-10: Register access packet format.....29
Figure 2-11: RAR-packet fields definition.....30
Figure 2-12: Opcodes of messages.....30
Figure 2-13: RDI sideband: srcid and dstid encodings on RDI.....31
Figure 2-14: FDI sideband: srcid and dstid encodings on FDI.....31
Figure 2-15: UCIe Link sideband: srcid and dstid encodings for UCIe Link.....31
Figure 2-16: Completion packet field.....32
Figure 2-17: Completion message fields.....32
Figure 2-18: Message without data packet format.....33
Figure 2-19: Message with data packet format.....33
Figure 2-20: LFSR Seed Values.....34
Figure 2-21: LFSR Implementation.....34
Figure 2-22: Lane ID Standard Package Module....35
Figure 2-23: Per-Lane ID Format....35
Figure 2-24: Per_Lane ID Example....35
Figure 2-25: PHY Layer Architecture Level 1.....39
Figure 2-26: PHY Layer Architecture Level 2.....39
Figure 2-27: PHY Layer Architecture Level 3.....40
Figure 2-28: PHY Layer Functionalities Categories....41
Figure 3-1: LTSM States Diagrams....44
Figure 3-2: LTSM Interface....44
Figure 3-3: RDI Interface....57
Figure 3-4: Phy_Module Interface....65
Figure 3-5: Retrain Exit State Resolution....70
Figure 3-6: PHY Module Partner Interface....70
Figure 3-7: Two Level Arbitration....73
Figure 3-8: Side Band Transmitter Top Architecture....74
Figure 3-9: Side Band Transmitter Signals Interface....75
Figure 3-10: Side Band Transmitter LCLK Domain Architecture....78
Figure 3-11: Side Band Transmitter SB_CLK domain Architecture....78
Figure 3-12: SBTX Decoder Architecture....79
Figure 3-13: SBTX Decoder FSM States....81
Figure 3-14: SBTX D2D_Data_Integrity Architecture....82
Figure 3-15: SBTX Consecutive D2D messages on RDI Interface Spec.....83
Figure 3-16: Consecutive D2D messages on RDI Interface Handling....83

Figure 3-17: SBTX Message Credits in UCIe 84
Figure 3-18: SBTX Credit System Initialization 84
Figure 3-19: SBTX Credit Handler Architecture 85
Figure 3-20: SBTX Buffered Credits Return Mechanism 86
Figure 3-21: SBTX Sync FIFO Architecture 87
Figure 3-22: SBTX Priority Encoder Resend Mechanism 88
Figure 3-23: SBTX Priority Encoder “Arbiter” Architecture 88
Figure 3-24: SBTX the 4 Additional SBINIT Pattern Justification. 91
Figure 3-25: SBTX RESET State Transition Condition. 91
Figure 3-26: SBTX TX FSM Architecture 91
Figure 3-27: SBTX Signal controlling initialization Process Behavior 92
Figure 3-28: SBTX Non PMO Operation 92
Figure 3-29: SBTX SB CLK & Data Lane handler Architecture 93
Figure 3-30: Sideband Receiver Top Architecture 96
Figure 3-31: Sideband Receiver Interfcae 96
Figure 3-32: Sideband Receiver Detailed Architecture 99
Figure 3-33: Deserializer Diagram 99
Figure 3-34: Pattern Detector Diagram 101
Figure 3-35: Asynchronous FIFO Diagram 102
Figure 3-36: Decoder Diagram 104
Figure 3-37: Synchronous FIFO Diagram 106
Figure 3-38: Priority Register Diagram 107
Figure 3-39: Counter Diagram 110
Figure 3-40: Synchronizer Diagram 111
Figure 3-41: Lane Reversal feature mechanism 112
Figure 3-42: Normal and Reversed lanes connections. 113
Figure 3-43: Main-Band Transmitter Top Architecture 113
Figure 3-44: Main-Band Transmitter Semi-Detailed Architecture 115
Figure 3-45: Main-Band Transmitter Detailed Architecture 116
Figure 3-46: MB_TX Gray Synchronizer Block Architecture 117
Figure 3-47: MB_TX Pulse Synchronizer Block Architecture 119
Figure 348-: DFF Synchronizer Diagram 120
Figure 3-49: MB_TX DFF Synchronizer Block Architecture 121
Figure 350-: FIFO FULL & EMPTY flags generation 123
Figure 3-51: MB_TX Asynchronous FIFOs Block Architecture 124
Figure 352-: Same signal synchronized twice independently 126
Figure 353-: Waveforms for same signal synchronized twice– independently 127
Figure 3-54: Synchronized signal only once 127
Figure 3-55: MB_TX Controller Block Architecture 129
Figure 3-56: MB_TX VALTRAIN Pattern Generator Block Architecture 131
Figure 3-57: MB_TX Per-Lane ID Generator Block Architecture 134
Figure 3-58: MB_TX Byte-to-Lane Mapper Block Architecture 136
Figure 3-59: MB_TX Serializer Block Architecture 137
Figure 3-60: MB_TX Per-Lane LFSR Block Architecture 139
Figure 3-61: Main Band Receiver Top Architecture 141
Figure 3-62: Main Band Receiver Signals Interface 142
Figure 3-63: Main Band Receiver Detailed Architecture 145
Figure 3-64: Main Band Receiver Enable Generator Architecture 146
Figure 3-65: Bus Enable Synchronization 148
Figure 3-66: Positive Edge detector for the Valid Lane 148
Figure 3-67: Data Pending in Receiver Pipeline Spec. 149
Figure 3-68: Transferring Data in Multiples of 256B in D2D. 149

Figure 3-69: Main Band Receiver De-Mapper Architecture 150
Figure 3-70: Main Band Receiver LFSR Architecture 152
Figure 3-71: Main Band Receiver Data Comparator Architecture 154
Figure 3-72: Req-Ack Enable Synchronization 155
Figure 3-73: Main Band Receiver Valid Checker Architecture 156
Figure 3-74: Chunk Marker Functionality Clarification 158
Figure 4-1: Typical UVM Testbench Architecture 160
Figure 4-2: UVM Agent components 161
Figure 4-3: UVM Class Diagram 164
Figure 4-4: UVM phases diagram 164
Figure 4-5: Sequencer driver diagram 166
Figure 4-6: diagram illustrating the tb architecture 167
Figure 5-1: UVM architecture used for the LTSM environment. 170
Figure 5-2: Bug (1) MB Rx Verification 175
Figure 5-3: state_timer didnt return to zero after timeout 175
Figure 5-4: adapter exit not registered 175
Figure 5-5: LTSM Functional Coverage Report 176
Figure 5-6: LTSM Code Coverage 177
Figure 5-7: RDI responsibilities 178
Figure 5-8: UVM architecutre used for the RDI environment. 179
Figure 5-9: Timer 2 start signal latched 186
Figure 5-10: Racing between 2 different response generating always blocks 187
Figure 5-11: Certain outputs were Latched 187
Figure 5-12: RDI Code Coverage 188
Figure 5-13: RDI Functional Coverage 188
Figure 5-14: the blocks PHY_module interacts with 189
Figure 5-15: PHY-module UVM Testbench Architecture 190
Figure 5-16: handshake representation 193
Figure 5-17: diagram illustrating PHy-sequences 193
Figure 5-18: PHY-module bug0 194
Figure 5-19: PHY-module bug 1 195
Figure 5-20: PHY-module bug1 195
Figure 5-21: PHY-module bug-3 195
Figure 5-22: PHY-module coverage 196
Figure 5-23: phy-module functional coverage 0 197
Figure 5-24: PHY-module functional coverage 1 197
Figure 5-25: PHY-module-partner interaction with other block 198
Figure 5-26: Module partner UVM Testbench Architecture 199
Figure 5-27: Module partner bug-0 202
Figure 5-28: Module partner bug-1 202
Figure 5-29: Module partner bug-2 202
Figure 5-30: Module partner bug3 203
Figure 5-31: Module partner code coverage 203
Figure 5-32: Module partner functional coverage 204
Figure 5-33: UIRB Stimulus Generation Example 205
Figure 5-34: Encoder Message Stimulus Generation Example 205
Figure 5-35: RDI Transitions Stimulus Generation Example 205
Figure 5-36: Control Flags Stimulus Generation Example 206
Figure 5-37: Final SB Tx UVM Architecture 207
Figure 5-38: SB Tx IDLE Sequences 213
Figure 5-39: SB Tx Pattern Sequences 214
Figure 5-40: SB Tx Active Sequences 215

Figure 5-41: SB Tx Error Sequences 215
Figure 5-42: Bug (1) in SB Tx Verification 216
Figure 5-43: Bug (2) in SB Tx Verification 216
Figure 5-44:Bug (3) in SB Tx Verification 217
Figure 5-45: Bug (4) in SB Tx Verification 217
Figure 5-46: Bug (5) in SB Tx Verification 217
Figure 5-47: Bug (6) in SB Tx Verification 218
Figure 5-48: Bug (7) in SB Tx Verification 218
Figure 5-49: SB Tx Code Coverage 219
Figure 5-50: SB Tx Functional Coverage 219
Figure 5-51: Sideband Receiver interaction with other block 220
Figure 5-52: architectures with and without virtual sequence 221
Figure 5-53: sbrx-uvm environment 222
Figure 5-54: sideband-rx stimulus generation 226
Figure 5-55: clock gating handshakes 226
Figure 5-56: sb_clk message sequences 227
Figure 5-57: sb sequences with injited errors 227
Figure 5-58: lclk domain sequences 228
Figure 5-59: side band Rx bug-0 228
Figure 5-60: side band Rx bug-1 229
Figure 5-61: side band Rx bug-2 229
Figure 5-62: side band Rx bug-3 229
Figure 5-63: sideband Rx bug-4 229
Figure 5-64: sideband Rx code coverage 230
Figure 5-65: sideband RX functional coverage 230
Figure 5-66:Abstract MB Tx Interface 232
Figure 5-67: MB Tx UVM Testbench Architecture 233
Figure 5-68:MB Tx verification Sequences 237
Figure 5-69: Bug (1) in MB Tx Verification 239
Figure 5-70: Bug (2) in MB Tx Verification 239
Figure 5-71: Bug (3) in MB Tx Verification 240
Figure 5-72: Bug (4) in MB Tx Verification 240
Figure 5-73: Bug (5) in MB Tx Verification 240
Figure 5-74: Final MB Tx Code Coverage 241
Figure 5-75: Final MB Tx Functional Coverage 241
Figure 5-76: UVM Architecture followed for Mainband Receiver Verification Environment 243
Figure 5-77: All MB Rx sequences used in the verification environment 249
Figure 5-78: Bug (2) in MB Rx Verification 250
Figure 5-79: Bug (4) in MB Rx Verification 250
Figure 5-80: Bug (4) fixed in MB Rx Verification 250
Figure 5-81: MB Rx Functional Coverage 251
Figure 5-82: MB Rx Code Coverage 251
Figure 6-1: MB Top Environment 253
Figure 6-2: Wrong Lane Reversal Architecture 254
Figure 6-3: Separate MB Tx Code Coverage 254
Figure 6-4: Separate MB Rx Functional Coverage 255
Figure 6-5: Separate MB Rx Functional Coverage 255
Figure 6-6:SB Top Environment 257
Figure 6-7: Separate SB Tx Code Coverage 257
Figure 6-8: Separate SB Rx Code Coverage 257
Figure 6-9: Total SB Code Coverage 257
Figure 6-10: Separate SB Tx Functional Coverage 258

Figure 7-1:Un-Encapsulated Architecture 260
Figure 7-2: Hierarchical scoreboards for Independent Paths.....261
Figure 7-3: Chained Scoreboards Architecture.....262
Figure 7-4:Final Scoreboard Architecture 262
Figure 7-5 E2E UVM Architecture.....263
Figure 7-6: Final Transcript for 4 scenarios at once. 272
Figure 7-7 lane reversal bug 273
Figure 7-8 E2E code coverage.....274
Figure 7-9 E2E functional coverage 275
Figure 8-1: FPGA Flow 277
Figure 8-2 Linting Violations 1 278
Figure 8-3 Linting Violations 2 278
Figure 8-4 Linting Violations 3 278
Figure 8-5 BUFGCE.....280
Figure 8-6 Whole System CDC Paths 281
Figure 8-7 Frequency Limitation.....282
Figure 8-8 Synthesis Problem 1.....282
Figure 8-9 Synthesis Problem 2.....282
Figure 8-10 Synthesis Problem 3.....283
Figure 8-11 Synthesis Final Status 283
Figure 8-12: Dedicated Clock Pins (1). 284
Figure 8-13 Design Utilization due to IO Pins Limitation 284
Figure 8-14 final Implementation Status 284
Figure 8-15 Behavioral Simulation Result (MB Data sent).....285
Figure 8-16 Behavioral Simulation Result (SB Message Sent).....286
Figure 9-1: Example of Single Lane remapping implementation.....288
Figure 9-2:Implementation Example Showing Two Different Operating Modes 289
Figure 9-3:Suggested IP-Based E2E Verification Environment. 289

List of Tables

Table 3-1: LTSM Signals Description (sheet 1 of 2)....45
Table 3-2: LTSM Signals Description (sheet 2 of 2)....46
Table 3-3: LTSM States Description (sheet 1 of 4)....46
Table 3-4: LTSM States Description (sheet 2 of 4)....47
Table 3-5: LTSM States Description (sheet 3 of 4)....48
Table 3-6: LTSM States Description (sheet 4 of 4)....49
Table 3-7: MBINIT States Description (sheet 1 of 3)....49
Table 3-8: MBINIT States Description (sheet 2 of 3)....50
Table 3-9: MNINIT States Description (sheet 3 of 3)....51
Table 3-10: MBTRAIN States Description (sheet 1 of 6)....51
Table 3-11: MBTRAIN States Description (sheet 2 of 6)....52
Table 3-12: MBTRAIN States Description (sheet 3 of 6)....53
Table 3-13: MBTRAIN States Description (sheet 4 of 6)....54
Table 3-14: MBTRAIN States Description (sheet 5 of 6)....55
Table 3-15: MBTRAIN States Description (sheet 6 of 6)....56
Table 3-16: RDI Parameters....58
Table 3-17: RDI Signals Description (sheet 1 of 3)....58
Table 3-18: RDI Signals Description (sheet 2 of 3)....59
Table 3-19: RDI Signals Description (sheet 3 of 3)....60
Table 3-20: RDI States Description (sheet 1 of 3)....61
Table 3-21: RDI States Description (sheet 2 of 3)....62
Table 3-22: RDI States Description (sheet 3 of 3)....63
Table 3-23: Phy_Module Parameters....65
Table 3-24: Phy_Module Signals Description (sheet 1 of 4)....65
Table 3-25: Phy_Module Signals Description (sheet 2 of 4)....66
Table 3-26: Phy_Module Signals Description (sheet 3 of 4)....67
Table 3-27: Phy_Module Signals Description (sheet 4 of 4)....68
Table 3-28: Phy_Module_Partner Patameters....71
Table 3-29: Phy_Module_Partner Signals Description (sheet 1 of 2)....71
Table 3-30: Phy_Module_Partner Signals Description (sheet 2 of 2)....72
Table 3-31: SBTX Parameters....75
Table 3-32: SBTX Top Level Interface Signals (sheet 1 of 2)....75
Table 3-33: SBTX Top Level Interface Signals (sheet 2 of 2)....76
Table 3-34: SBTX Inter-Module signals (sheet 1 of 2)....76
Table 3-35: SBTX Inter-Module signals (sheet 2 of 2)....77
Table 3-36: SBTX Decoder Parameters....80
Table 3-37: SBTX Decoder Interface....80
Table 3-38: SBTX D2D_Data_Integrity Parameters....82
Table 3-39: SBTX D2D_Data_Integrity Interface....83
Table 3-40: SBTX Credit Handler Interface....85
Table 3-41: SBTX Sync. FIFO Parameters....87
Table 3-42: SBTX Sync. FIFO Interface....87
Table 3-43: SBTX Priority Encoder Parameters....89
Table 3-44: SBTX Priority Encoder Interface....89
Table 3-45: SBTX TX-FSM Parameters....91
Table 3-46: SBTX TX-FSM Interface (sheet 1 of 2)....91
Table 3-47: SBTX TX-FSM Interface (sheet 2 of 2)....92
Table 3-48: SBTX SB CLK Data lanes Handler Parameters....93
Table 3-49: SBTX SB CLK Data lanes Handler Interface....93
Table 3-50: Sideband Receiver Parameters....97

Table 3-51: Sideband Receiver Signals Description (sheet 1 of 2) 97
Table 3-52: Sideband Receiver Signals Description (sheet 2 of 2) 98
Table 3-53: Deserializer Parameters 100
Table 3-54: Deserializer Signals Description 100
Table 3-55: Pattern Detector Parameters 101
Table 3-56: Pattern Detector Signals Description 101
Table 3-57: Asynchronous FIFO Parameters 102
Table 3-58: Asynchronous FIFO Signals Description 102
Table 3-59: Decoder Parameters 104
Table 3-60: Decoder Signals Description (sheet 1 of 2) 104
Table 3-61: Decoder Signals Description (sheet 2 of 2) 105
Table 3-62: Synchronous FIFO Parameters 106
Table 3-63: Synchronous FIFO Signals Description (sheet 1 of 2) 106
Table 3-64: Synchronous FIFO Signals Description (sheet 2 of 2) 107
Table 3-65: Priority Register Parameters 108
Table 3-66: Priority Register Signals Description (sheet 1 of 2) 108
Table 3-67: Priority Register Signals Description (sheet 2 of 2) 109
Table 3-68: Counter Parameters 110
Table 3-69: Counter Signals Description 110
Table 3-70: Synchronizer Signals Description 111
Table 3-71: Main-Band Transmitter Signals Interface (Sheet 1 of 2) 114
Table 3-72: Main-Band Transmitter Signals Interface (Sheet 1 of 2) 115
Table 3-73: MB_TX Gray Synchronizer Signals 118
Table 3-74: MB_TX Gray Synchronizer Parameters 118
Table 3-75: MB_TX Pulse Synchronizer Signals 120
Table 3-76: MB_TX DFF Synchronizer Signals 122
Table 3-77: MB_TX Asynchronous FIFO Signals (Sheet 1 of 2) 124
Table 3-78: MB_TX Asynchronous FIFO Signals (Sheet 2 of 2) 125
Table 3-79: MB_TX Asynchronous FIFO Parameters 125
Table 3-80: MB_TX Controller Signals (Sheet 1 of 2) 129
Table 3-81: MB_TX Controller Signals (Sheet 2 of 2) 130
Table 3-82: MB_TX VALTRAIN Pattern Generator Signals 132
Table 3-83: MB_TX Per-Lane ID Generator Signals 134
Table 3-84: MB_TX Byte-to-Lane Mapper Signals 136
Table 3-85: MB_TX Serializer Signals 137
Table 3-86: MB_TX Per-Lane LFSR Signals (Sheet 1 of 2) 139
Table 3-87: MB_TX Per-Lane LFSR Signals (Sheet 2 of 2) 140
Table 3-88: MB_TX Per-Lane LFSR Parameters 140
Table 3-89: MBRX Top Level Interface Signals 143
Table 3-90: MBRX Top Level Interface Parameters 143
Table 3-91: MBRX Inter Module Signals (sheet 1 of 2) 143
Table 3-92: MBRX Inter Module Signals (sheet 2 of 2) 144
Table 3-93: MBRX Enable Generator Interface (sheet 1 of 2) 146
Table 3-94: MBRX Enable Generator Interface (sheet 2 of 2) 147
Table 3-95: MBRX De-Mapper Parameters 150
Table 3-96: MBRX De-Mapper Interface (sheet 1 of 2) 150
Table 3-97: MBRX De-Mapper Interface (sheet 2 of 2) 151
Table 3-98: MBRX LFSR Parameters 152
Table 3-99: MBRX LFSR Interface 153
Table 3-100: MBRX Data Comparator Parameters 154
Table 3-101: MBRX Data Comparator Parameters 155
Table 3-102: MBRX Valid Checker Interface 157

Table 5-1: LTSM Verification Plan (sheet 1 of 4).....171
Table 5-2: LTSM Verification Plan (sheet 2 of 4).....172
Table 5-3: LTSM Verification Plan (sheet 3 of 4).....173
Table 5-4: LTSM Verification Plan (sheet 4 of 4).....174
Table 5-5: RDI Verification Plan(sheet 1 of 7). 180
Table 5-6: RDI Verification Plan(sheet 2 of 7). 181
Table 5-7: RDI Verification Plan(sheet 3 of 7). 182
Table 5-8: RDI Verification Plan(sheet 4 of 7). 183
Table 5-9: RDI Verification Plan(sheet 5 of 7). 184
Table 5-10: RDI Verification Plan(sheet 6 of 7). 185
Table 5-11: RDI Verification Plan(sheet 7 of 7). 186
Table 5-12: Module Verification Plan (sheet 1 of 4).....190
Table 5-13: Module Verification Plan (sheet 2 of 4).....191
Table 5-14: Module Verification Plan (sheet 3 of 4).....192
Table 5-15: Module Verification Plan(sheet 4 of 4).....193
Table 5-16: Partner Verification Plan (sheet 1 of 3).....199
Table 5-17: Partner Verification Plan (sheet 2 of 3).....200
Table 5-18: Partner Verification Plan (sheet 3 of 3).....201
Table 5-19: SB Tx Verification Plan (sheet 1 of 5) 208
Table 5-20: SB Tx Verification Plan (sheet 2 of 5) 209
Table 5-21: SB Tx Verification Plan (sheet 3 of 5) 210
Table 5-22: SB Tx Verification Plan (sheet 4 of 5) 211
Table 5-23: SB Tx Verification Plan (sheet 5 of 5) 212
Table 5-24: SB Rx Verification Plan (sheet 1 of 3) 223
Table 5-25: SB Rx Verification Plan (sheet 2 of 3) 224
Table 5-26: SB Rx Verification Plan (sheet 3 of 3) 225
Table 5-27: MB Tx Verification Plan (Sheet 1 of 4). 234
Table 5-28: MB Tx Verification Plan (Sheet 2 of 4). 235
Table 5-29: MB Tx Verification Plan (Sheet 3 of 4). 236
Table 5-30: MB Tx Verification Plan (Sheet 4 of 4). 237
Table 5-31: MB Rx Verification Plan. 245
Table 7-1: Conigurable Sequence Parameters 265
Table 7-2: Scenario 1 Configurations for Both Dies. 267
Table 7-3: Scenario 2 Configurations for Both Dies. 268
Table 7-4: Scenario 3 Configurations for Both Dies. 269
Table 7-5: Scenario 4 Configurations for Both Dies. 270
Table 7-6: Scenario 5 Configurations for Both Dies. 271

Terminologies

Term	Definition
Ack	Acknowledge
Addr	Address
Advanced Package	This packaging technology is used for performance optimized applications and short reach interconnects.
B2B	Back-to-Back
BE	Byte Enable
chiplet	Integrated circuit die that contains a well-defined subset of functionality that is designed to be combined with other chiplets in a package.
CRC	Cyclic Redundancy Check
CXL	Compute eXpress Link
CXL 68B Flit Mode	This term is used to reference 68B Flit Mode related Protocol features defined in CXL Specification.
CXL 256B Flit Mode	This term is used to reference 256B Flit Mode related Protocol features defined in CXL Specification.
D2C	Data-to-Clock
D2D	Die-to-Die
Domain Reset	Used to refer to a hardware mechanism that sets or returns all UCIe registers and state machines associated with a given UCIe Link to their initialization values as specified in this document. It is required for both sides of the Link to have an overlapping time window such that they are both in domain reset concurrently.
DP	Downstream Port
DVSEC	Designated Vendor-Specific Extended Capability
E2E	End to end
FDI	Flit-Aware Die-to-Die Interface
FH	Flit Header
FIFO	First In, First Out
Flit	Link Layer unit of transfer
I/O	Input/Output
Lane	A pair of signals mapped to physical bumps, one for Transmission, and one for Reception. A xN UCIe Link is composed of N Lanes.
LCLK	Refers to the clock at which the Logical Physical Layer, Adapter and RDI/FDI are operating.
LFSR	Linear Feedback Shift Register
Link/UCIe Link	A Link or UCIe Link refers to the set of two UCIe components and their interconnecting Lanes which forms a dual-simplex communications path between the two components.
LSM	Adapter Link State Machine
LTSM	Link Training State Machine
LSB	Least Significant Bit
Mainband / MB	Connection that constitutes the main data path of UCIe. Consists of a forwarded clock, a data valid pin, and N Lanes of data per module.
MMPL	Multi-module PHY Logic
Module	UCIe main data path on the physical bumps is organized as a group of Lanes called a Module. For Standard Package, 16 Lanes constitute a single Module. For Advanced Package, 64 Lanes constitute a single Module.
MSB	Most Significant Bit

MTP	Management Transport Packet
Nak	Negatively acknowledge
NOP	No Operation
Packet	A block of data transmitted across a network.
PCIe (PCI Express)	Peripheral Component Interconnect Express.
PCIe Flit Mode	This term is used to reference Flit Mode related Protocol features
PCIe non-Flit Mode	This term is used to reference non-Flit Mode related Protocol features
PHY	Physical Layer
PMO	Sideband Performant Mode Operation.
RDI	Raw Die-to-Die Interface
RD_P[N]	Nth Physical Lane for Data Receiver
remote Link partner	This term is used throughout this specification to denote the logic associated with the far side of the UCIe Link; to denote actions or messages sent or received by the Link partner of a UCIe die.
Reserved	The contents, states, or information are not defined at this time. Using any Reserved area (for example, packet header bit-fields, configuration register bits) is not permitted. Reserved register fields must be read only and must return 0 (all 0s for multi-bit fields) when read. For packets transmitted and received over the UCIe Link (mainband or sideband), the Reserved bits must be cleared to 0b by the sender and ignored by the receiver. Reserved encodings for register and packet fields must not be used. Any implementation dependence on a Reserved field value or encoding will result in an implementation that is not UCIe-compliant. The functionality of such an implementation cannot be guaranteed in this or any future revision of this specification.
RVLD_P	Physical Lane for Valid Receiver
Rx	Receiver
RXCKSB	Physical Lane for sideband Clock Receiver
RXDATASB	Physical Lane for sideband Data Receiver
{<SBMSG>}	Sideband message requests or responses are referred to by their names enclosed in curly brackets.
serial packet	A 64-bit serial packet is defined on the sideband I/O interface to the remote chiplet
Sideband / SB	Connection used for parameter exchanges, register accesses for debug/compliance and coordination with remote partner for Link training and management. Consists of a forwarded clock pin and a data pin in each direction. The clock is fixed at 800 MHz regardless of the main data path speed.
Standard Package	This packaging technology is used for low cost and long reach interconnects using traces on organic package/substrate
TD_P[N]	Nth Physical Lane for Data Transmitter
TVLD_P	Physical Lane for Valid Transmitter
Tx	Transmitter
TXCKSB	Physical Lane for sideband Clock Transmitter
TXDATASB	Physical Lane for sideband Data Transmitter
UCIe	Universal Chiplet Interconnect express
UCIe-3D	Universal Chiplet Interconnect express for 3D packaging
UCIe-S	Used to denote x16 Standard Package module.

UCIe chiplet	A chiplet that complies with the UCIe specification.
UCIe die	This term is used throughout this specification to denote the logic associated with the UCIe Link on any given chiplet with a UCIe Link connection. It is used as a common noun to denote actions or messages sent or received by an implementation of UCIe.
UCIe Link	A UCIe connection between two chiplets. These chiplets are Link partners in the context of UCIe since they communicate with each other using a common UCIe Link.
UIE	Uncorrectable Internal Error
UiRB	UCIe Register Block
Unit Interval / UI	Given a data stream of a repeating pattern of alternating 1 and 0 values, the Unit Interval is the value measured by averaging the time interval between voltage transitions, over a time interval sufficiently long to make all intentional frequency modulation of the source clock negligible.
UP	Upstream Port
Vref	Reference voltage for receivers
Zero	Numerical value of 0 in a bit, field, or register, of appropriate width for that bit, field, or register.
RAR	Register Access Request

Abstract

The Universal Chiplet Interconnect Express (UCIe) is an open industry standard designed to enable high-bandwidth, low-latency die-to-die communication, allowing seamless integration of chiplets from multiple vendors within a single package. In this project, we designed, implemented, and verified a complete UCIe 2.0 Logical PHY Layer using Verilog and System Verilog, with all components developed entirely from scratch.

The PHY layer serves as a critical bridge, handling reliable data transfer between the UCIe Die-to-Die (D2D) layer and the remote PHY layer on the connected chiplet. The design incorporates all required logical PHY functionalities, including lane-level control, synchronization, and data multiplexing. A key architectural challenge at the top level was the presence of a large number of Clock Domain Crossings (CDCs) across internal module boundaries. Each CDC path was carefully managed through appropriate synchronization logic and verified using CDC-aware verification techniques to ensure data integrity and functional correctness.

Verification was performed using the Universal Verification Methodology (UVM) at both block and top levels. Individual environments were developed for each PHY submodule, and these were vertically reused and integrated into a unified top-level UVM testbench. The design was also synthesized for the Xilinx XCKU15P-FFVE1517-2-E FPGA to support further validation.

Chapter 1: UCIe Introduction:

1.1. Previous Standards:

Before the introduction of Universal Chiplet Interconnect Express (UCIe), several interconnect standards were used to facilitate communication between chiplets and dies within a package or across different packages. Some of the key standards include:

1.1.1. PCI Express (PCIe)

PCIe (Peripheral Component Interconnect Express) is a high-speed, serial point-to-point communication protocol developed by the PCI-SIG consortium. It is the de facto standard for connecting CPUs to peripherals such as GPUs, SSDs, and network cards. PCIe follows a layered architecture:

- **Transaction Layer**: Creates Transaction Layer Packets (TLPs) for memory and I/O transactions.
- **Data Link Layer**: Ensures integrity with ACK/NAK and flow control.
- **Physical Layer**: Handles SERDES-based transmission over PCB traces.

Limitations in Chiplet Systems:

- Designed for board-level communication—not optimized for ultra-short distances within a package.
- High power consumption due to SERDES overhead.
- Increased latency (~20–100 ns) because of packetization and layered stack.
- Limited bandwidth density compared to parallel, bump-based die-to-die links.

Integration with UCIe:
UCIe natively supports PCIe TLPs at its Logical Layer by encapsulating them into fixed-size **flits** (flow control units). This eliminates the need for traditional PCIe PHYs between dies, reducing power and latency.

1.1.2. Compute Express Link (CXL)

CXL is a high-speed cache-coherent interconnect protocol built on PCIe PHY, standardized by the CXL Consortium. It was developed to enable memory sharing and coherency between CPUs and accelerators.

Sub-protocols

- **CXL.io**: For initialization and configuration (PCIe-like).
- **CXL.cache**: Enables device caching of host memory.
- **CXL.mem**: Allows access to device-attached memory by the host CPU.

Limitations in Chiplet Systems:

- Built on PCIe PHY—inherits high latency and power drawbacks.
- Lacks native support for lane redundancy and error resilience.
- Not optimized for on-package, low-latency communication.

Integration with UCIe:
UCIe supports all three CXL protocols using unique Protocol IDs:

- PCIe/CXL.io
- CXL.cache
- CXL.mem

This enables CXL-based memory coherency between chiplets within a single package.

1.1.3. Streaming Protocols (e.g., AXI-Stream)

These protocols are widely used in signal-processing and AI accelerators for high-throughput, low-latency communication. Examples include:

- **AMBA AXI-Stream**: Valid/ready handshake model.
- **Custom protocols**: Proprietary formats for specific accelerators or memory controllers.

Limitations:

- Often lack reliability (e.g., error correction, retries).
- No address-based transactions.
- Usually point-to-point and not standardized.

Integration with UCIe:
UCIe introduces a **Streaming Protocol ID (e.g., 0x04)** and transport layer support for packetization, CRC, and flow control—bringing standardization and reliability to streaming interconnects.

1.1.4. Raw Die-to-Die Interface (RDI)

RDI is a simplified communication mode that uses the PHY layer directly without full protocol stacks. It is useful for:

- Reusing legacy PHYs like AIB, BoW, or OpenHBI.
- Simple point-to-point communication.

Features:

- Minimal latency and power.
- Source-synchronous clocking.
- No flow control or framing layers.

Support in UCIe:
RDI is supported with dedicated Protocol Types:

- Raw interface without flow control.
- Raw with minimal flow control.

1.1.5. Advanced Interface Bus (AIB)

AIB is a proprietary parallel die-to-die PHY developed by Intel for EMIB-based multi-chiplet integration. It uses a source-synchronous protocol and supports wide parallel buses.

Limitations:

- Proprietary—lacks wide industry adoption.
- Fixed bump pitch (55 µm) limits scalability.
- No integrated error handling (e.g., CRC, retry).
- No standard transport or protocol layers.

Role in UCIe:
UCIe can support legacy AIB-based PHYs using **Raw Die-to-Die Interface (RDI)** mode, enabling reuse of existing infrastructure while benefiting from UCIe's reliability features.

These standards were not specifically designed for the power, latency, and bandwidth requirements of modern chiplet-based architecture, leading to inefficiencies in performance, area, and energy consumption.

1.2. UCIe Motivation:

The semiconductor industry is transitioning from monolithic SoC designs to disaggregated, multi-die architectures (chiplets) due to the limitations of Moore's Law, rising fabrication costs, and the need for functional specialization.

This shift demands a standardized, high-performance die-to-die interconnect optimized for in-package communication. Existing protocols like PCIe and CXL were designed for board-level or rack-scale connectivity and are inefficient for chiplet integration due to their power, latency, and physical layer constraints.

Gaps in Existing Standards:

1. **PCIe and CXL Limitations**
   While PCIe and CXL dominate system-level interconnects, they rely on SERDES-based PHYs designed for long-reach channels (≥25 mm).

   For chiplet communication (≤2 mm in advanced packaging), these protocols incur unnecessary overhead:
   - High power consumption due to complex equalization and clock recovery.
   - Protocol stacks introduce latency unsuitable for tightly coupled workloads (e.g., cache-coherent CPU-GPU interactions).
   - Lack native support for lane repair, redundancy, or adaptive link tuning—critical for yield and reliability in multi-die systems.

2. **Proprietary or Domain-Specific Interfaces**
   Custom interconnects (e.g., AIB, AXI-Stream) are used in some chiplet designs but suffer from:
   - Vendor lock-in, limiting ecosystem interoperability.
   - No standardized protocol multiplexing or error handling.
   - Inability to seamlessly integrate with higher-level protocols like PCIe or CXL.

UCIe: A Unified Solution for Chiplet Interconnect

The **Universal Chiplet Interconnect Express (UCIe)** standard addresses these gaps by providing an open, layered, and scalable die-to-die interface optimized for in-package communication.

Key innovations from the UCIe specification include:

1. **Protocol Agnosticism and Flexibility**
   UCIe supports multiple protocol layers over a common physical interface:
   - **PCIe Flit Mode** and **CXL 256B/68B Flit Modes** for backward compatibility with existing ecosystems.
   - **Streaming Protocol** for low-latency, raw data transfers (e.g., AI/ML accelerators).
   - **Management Transport Protocol** for system manageability and debug.
   - **Raw Mode** for tunneling proprietary or third-party protocols.

This enables heterogeneous integration of chiplets (CPUs, GPUs, memory, I/O tiles) while preserving software compatibility with PCIe/CXL stacks.

2. **Optimized Physical Layer for In-Package Use**
   UCIe defines two PHY classes tailored for different packaging technologies:
   - **Standard Package (2D)**: Cost-effective, organic substrate interconnects (10–25 mm reach) for broader compatibility.

- **Advanced Package (2.5D)**: Short-reach (<2 mm), high-density interconnects (up to 1317 GB/s per die edge at 32 GT/s) with lane repair and redundancy.
- **UCIe-3D**: Vertical stacking with ultra-low latency (≤125 ps) and high bandwidth density (4000 GB/s/mm²).

Key PHY features:

- **Forwarded-clock architecture** eliminates complex CDR circuits, reducing power and latency.
- **Lane repair and redundancy** via spare lanes to improve yield.
- **Low-swing signaling** (0.4–0.85 V) for energy efficiency (as low as 0.05 pJ/bit in UCIe-3D).

3. **Reliability and Efficiency**
   - **Flit-based transport** with built-in CRC and retry mechanisms (BER ≤1e-27 for advanced packaging).
   - **Multi-protocol multiplexing** allows dynamic sharing of link bandwidth between protocols (e.g., CXL.io + CXL.mem).
   - **Standardized sideband channel** for link initialization, training, and management.

4. **Ecosystem and Scalability**
   - **Open standard** with defined bump maps, electrical specs, and interoperability rules.
   - **Support for retimers** to extend UCIe links off-package (e.g., for rack-scale disaggregation).
   - **UCIe Debug Architecture (UDA)** for pre-bond, post-bond, and system-level testing.

1.3. UCIe Layers Overview:

The **Universal Chiplet Interconnect Express (UCIe)** architecture is built on a **layered model** designed to enable reliable, high-performance, and interoperable communication between heterogeneous chiplets integrated into a single package. This layered architecture ensures modularity, scalability, and standardization across chiplet-based systems, making it easier to integrate IP from different vendors and support a wide range of workloads.

UCIe defines **three primary layers**, each with well-specified roles and responsibilities:

1. **Protocol Layer**
2. **Die-to-Die (D2D) Adapter Layer**
3. **Physical Layer**

Each of these layers works in coordination to transform high-level transactions into high-speed physical data transfers, and back again, while supporting features like flow control, reliability, low latency, and energy efficiency.

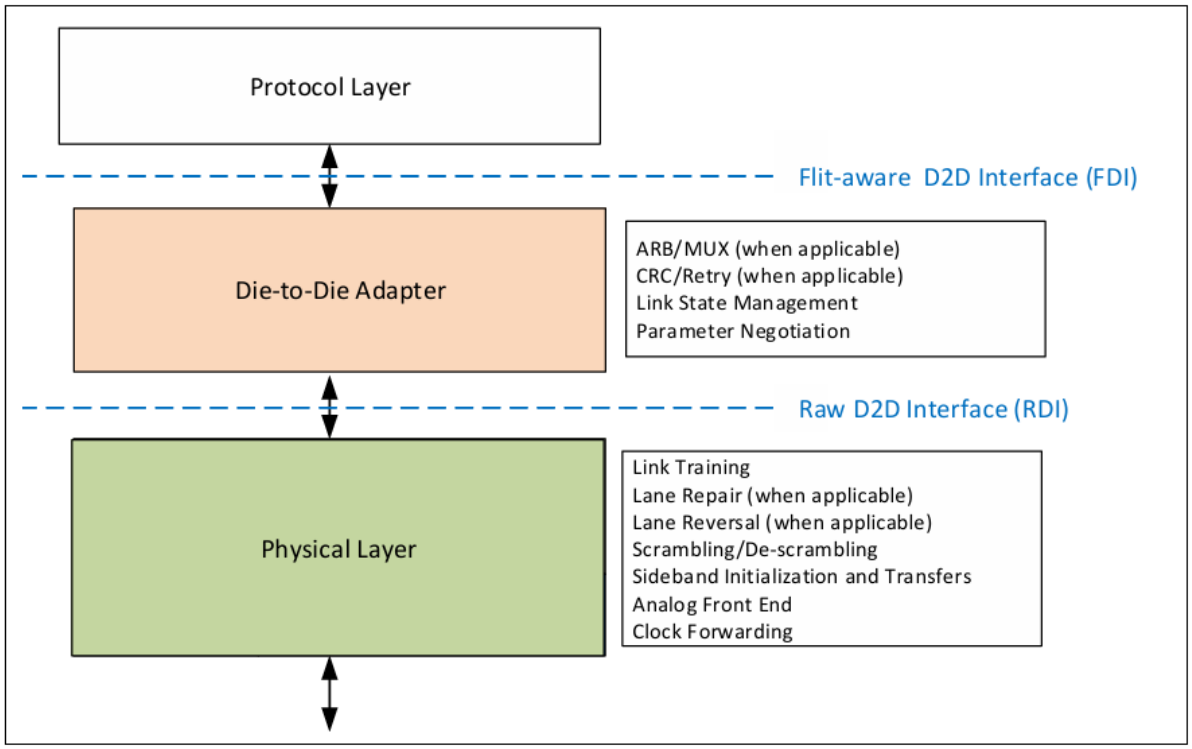


Figure 1-1: UCIe layers and functionalities

1. **Protocol Layer**
   The Protocol Layer is responsible for handling higher-level protocol-specific operations, such as transaction processing, flow control, and protocol-specific packet formatting. It supports multiple protocols, including:
   - **PCIe** (Peripheral Component Interconnect Express)
   - **CXL** (Compute Express Link)
   - **Streaming Protocol** (for custom/user-defined protocols)
   - **Management Transport Protocol** (for system manageability)

Figure 1-2 shows an example of Header Flit Format for management transport protocol.

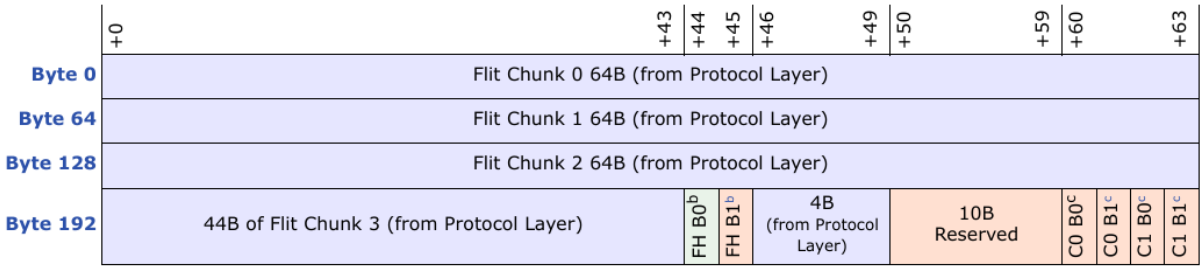


Figure 1-2: Header Flit Format for Management Transport Protocol

Key Responsibilities:
- Generates and processes protocol-specific packets (e.g., TLPs for PCIe, Flits for CXL).
- Manages protocol-level flow control and error handling (if not offloaded to the Adapter).
- Interfaces with the D2D Adapter via the **Flit-aware Die-to-Die Interface (FDI)**.

Figure 1-3 illustrates example configurations of the Protocol Layer and Adapter, including single and multi-protocol setups.

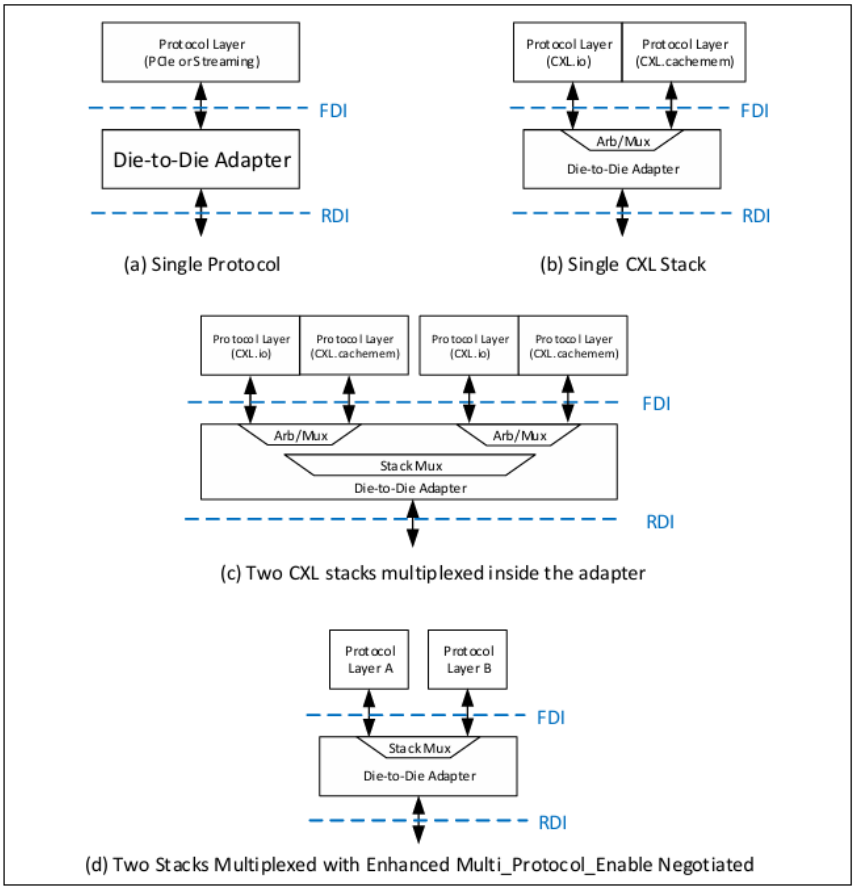


Figure 1-3: Configurations of Protocol and Adapter Layers through FDI Interface

2. **Die-to-Die (D2D) Adapter Layer**
   The D2D Adapter acts as a bridge between the Protocol Layer and the Physical Layer. It ensures reliable data transfer, handles multiplexing (if multiple protocols are used), and manages link state transitions.

   Key Responsibilities:
   - **Reliable Data Transfer**: Performs CRC computation and retry mechanisms for error recovery.
   - **Multiplexing**: Arbitrates between multiple Protocol Layers (if enabled) using stack identifiers.
   - **Link State Management**: Coordinates power states (e.g., L1, L2) and link initialization.
   - **Parameter Negotiation**: Exchanges capabilities with the remote link partner during initialization.

   Interfaces:
   - **Flit-aware Die-to-Die Interface (FDI)**: Connects to the Protocol Layer.
     - Used for transferring protocol Flits (e.g., 68B or 256B Flits) between the Protocol Layer and Adapter.
     - Supports dynamic clock gating and power management handshakes.

- **Raw Die-to-Die Interface (RDI)**: Connects to the Physical Layer.
  - Transfers raw data bytes after adding/adjusting Flit headers, CRC, or other metadata.
  - Manages lane repair, clock forwarding, and sideband communication.

Figure 1-4 shows an example for data transfer through RDI interface.

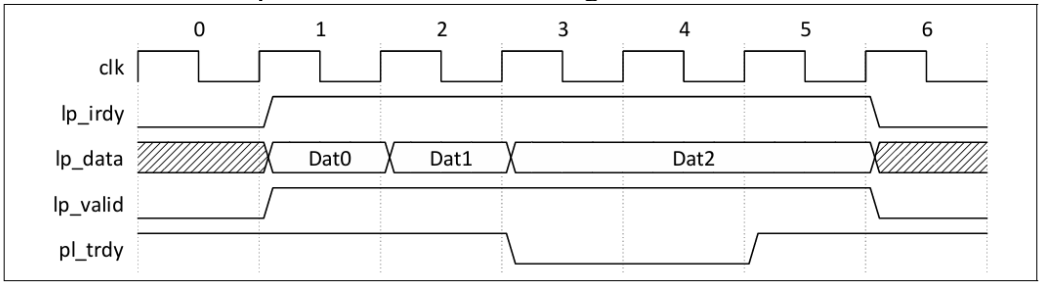


Figure 1-4: Data Transfer Timing Diagram through RDI Interface

Figure 1-5 shows the different configuration between Adapter layer and Physical layer through RDI interface.

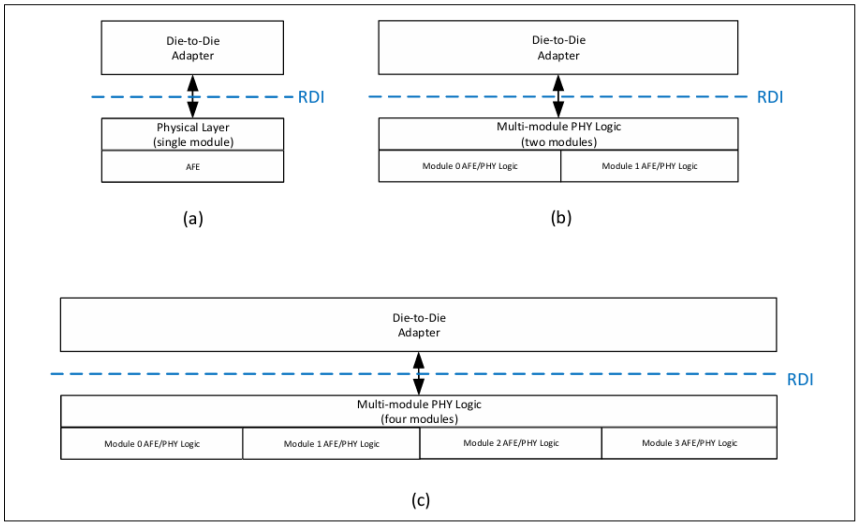


Figure 1-5: Configurations of Adapter and Physical layers through RDI Inetrface

3. Physical Layer

The Physical Layer handles electrical signaling, lane management, and low-level link training. It is divided into sub-components:

- **Logical Physical Layer:** Manages byte-to-lane mapping, clock gating, and sideband communication.
- **Electrical Layer:** Defines the analog front-end (AFE), transmitter/receiver specifications, and channel characteristics.
- **UCIe-3D Layer:** Supports 3D die stacking with vertical interconnect bumps.

Figure 1-6 shows the physical layer components.

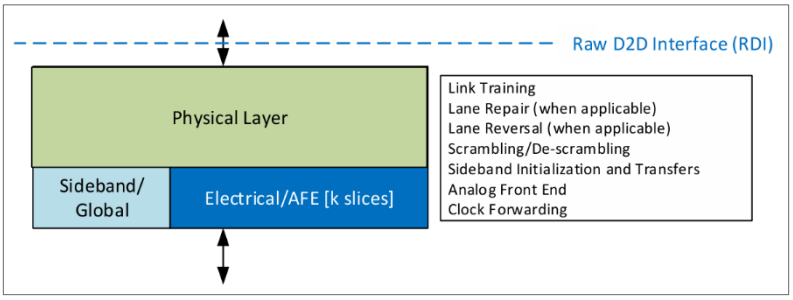


Figure 1-6: Physical Layer Components

Key Responsibilities:

- **Lane Management**: Performs lane reversal, repair, and redundancy remapping.
- **Link Training**: Initializes the link, calibrates eye width, and negotiates speed/width.
- **Sideband Communication**: Uses a dedicated low-speed link for control messages (e.g., parameter exchanges, register access).

Interfaces:

- **Raw Die-to-Die Interface (RDI)**: Connects to the D2D Adapter.
  - Transfers raw data bytes with framing signals (e.g., valid, track).
  - Supports clock gating and power management handshakes.
- **Sideband Interface**: Dedicated pins for out-of-band communication (e.g., Link training, management).

Chapter 2: Physical Layer:

2.1. Main Functionalities:

The Logical PHY comprehends the following functions:

- Link initialization, training and power management states
- Byte to Lane mapping for data transmission over Lanes
- Interconnect redundancy remapping (when required)
- Transmitting and receiving sideband messages
- Scrambling and training pattern generation
- Lane reversal
- Width degradation (when applicable)

2.1.1. Link Initialization and Training:

The Logical PHY layer in UCIe 2.0 orchestrates a structured sequence to bring up the physical link between chiplets, ensuring signal integrity, alignment, and robustness before data transmission begins. This process comprises multiple basic operations that are repeated across the link establishment which are:

2.1.1.1. Basic Link Training operations:

A. Data to Clock Point Test: This test evaluates the data integrity with clock through the link at a single PI phase. This is done in steps:
   1. The Transmitter configures its pattern generation parameters and sends a {Start Tx Init D to C point test req} message. The Receiver enables pattern comparison logic and replies with {Start Tx Init D to C point test resp} after setup.
   2. Transmitter resets its LFSR and sends {LFSR clear error req} to clear prior test states. Receiver resets its own LFSR and clears previous error logs, responding with {LFSR clear error resp}.
   3. Transmitter sends the configured test pattern over the Mainband lanes for the defined number of cycles.
      Pattern parameters are governed by Tx Pattern Generator Setup and Tx Pattern Mode Setup.
   4. Receiver compares incoming data per UI against the expected local pattern.
      Comparison outcomes are logged internally as part of the Rx compare setup logic.
   5. Transmitter sends {Tx Init D to C results req} to retrieve error logs.
      Receiver stops pattern comparison and returns results using {Tx Init D to C results resp}.
   6. Transmitter halts pattern generation and sends {End Tx Init D to C point test req}. Receiver acknowledges with {End Tx Init D to C point test resp}, marking test sequence completion.

B. Data to Clock Sweep Test: Transmitter-driven phase sweep test to evaluate receiver sampling margin across UI. This mode helps characterize system-level timing tolerance by shifting the sampling point (PI phase) and observing data validity across all lanes.
   The sequence of steps for this test are as follows:
   1. The Transmitter configures sweep parameters and sends a {Start Tx Init D to C eye sweep req} message. Receiver enables its pattern comparator and responds with {Start Tx Init D to C eye sweep resp} once configured.

2. Transmitter resets its LFSR and sends {LFSR clear error req} to clear state. Receiver resets its LFSR and clears prior error data, acknowledging with {LFSR clear error resp}.
3. Transmitter emits a test pattern defined by Tx Pattern Generator Setup over the Mainband for a configured number of cycles. Pattern ensures deterministic comparison for margin validation.
4. Receiver compares each UI of the incoming data stream to the expected local pattern. Match/mismatch results are logged internally as part of Rx compare setup.
5. Transmitter sends {Tx Init D to C results req} to request captured error data. Receiver stops pattern comparison and replies with {Tx Init D to C results resp} containing detailed results.
6. Transmitter sweeps through clock PI phases and repeats steps 2–5 at each phase point. This determines the valid sampling window (eye width) through iterative measurement.
7. Transmitter halts pattern output and sends {End Tx Init D to C eye sweep req}. Receiver responds with {End Tx Init D to C eye sweep resp}, completing the sweep procedure.

Those two basic operations are repeated across different LTSM states through link training where it can be Tx initiated or Rx initiated as well.

2.1.1.2. Link Training State Machine:

The **UCIe Link establishment process** is governed by a defined **state machine**.
Each phase of the bring-up process is encapsulated within a distinct state, enabling precise control, monitoring, and error recovery. The transition between states is driven by sideband detection, Mainband training, clock-data alignment, and protocol negotiation. The figures below provide a high-level overview of each **initialization state,** detailing its role within the overall training sequence, from initial reset to active link and runtime recovery. These states will be discussed in details in the design chapter later.

State	Description
RESET	This is the state following primary reset or exit from TRAINERROR.
SBINIT	Sideband initialization state where the sideband is detected, repaired (when applicable) and out of reset message is transmitted.
MBINIT	Following sideband initialization, Mainband (MB) is initialized at the lowest speed. Both dies perform on die calibration followed by interconnect repair (when applicable).
MBTRAIN	Mainband (Data, Clock and Valid signals) speed of operation is set to the highest negotiated data rate. Die-to-Die training of mainband is performed to center the clock with respect to Data.
LINKINIT	This state is used to exchange Adapter and Link management messages.
ACTIVE	This is the state in which transactions are sent and received.
L1/L2	Power Management state.
PHYRETRAIN	This state is used to begin the retrain flow for the Link during runtime.
TRAINERROR	State is entered when a fatal or non-fatal event occurs at any point during Link Training or operation.

Figure 2-1: State Definitions for Initialization in Specs

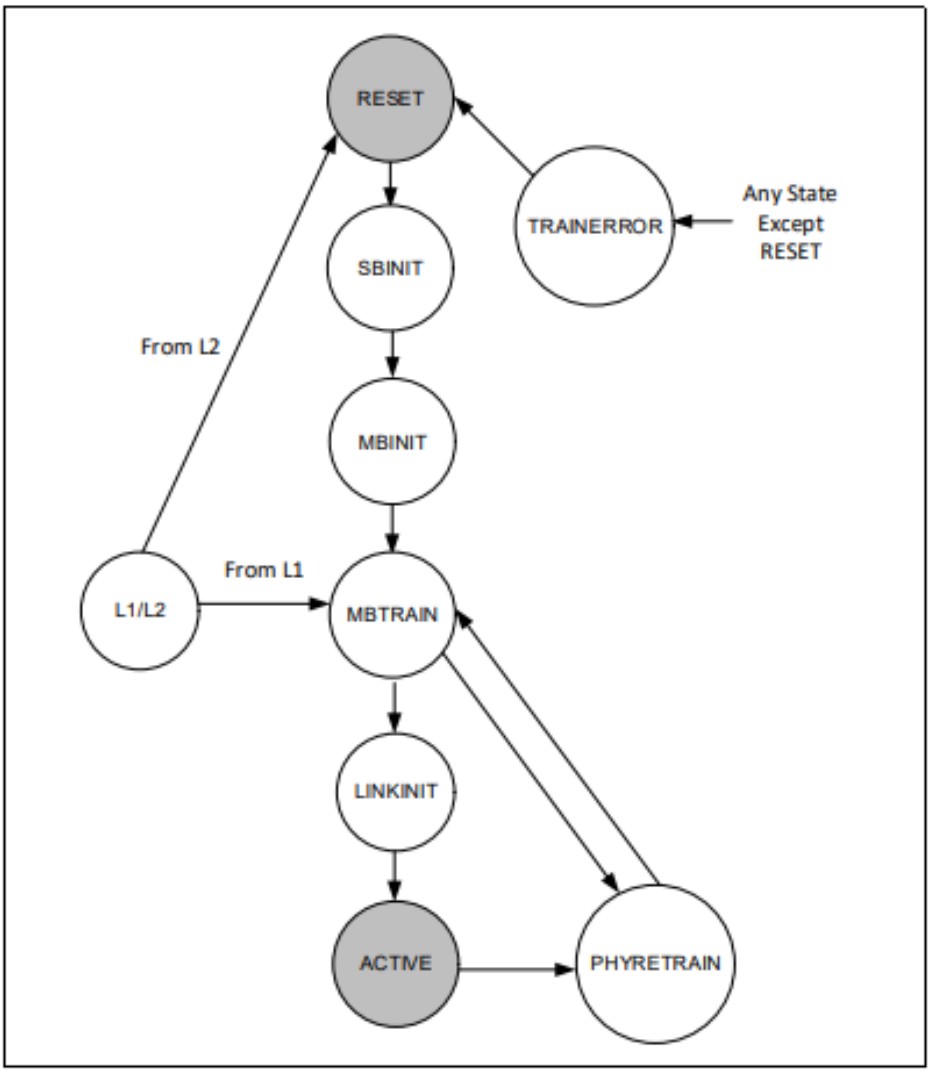


Figure 2-2: Link Training State Machine in Specs

2.1.2. Byte to Lane Mapping:

The D2D Layer transmits data in bytes, so the PHY maps each byte to a dedicated lane, for example Byte 0 (B0) to Lane 0, B1 to Lane 1, and so on preserving bit order (LSB-first). This fixed mapping ensures consistent byte-to-lane assignment across interface widths (x64/x32/x16), even with degraded modules. The PHY handles lane-to-package adaptations transparently, maintaining data integrity during transmission. Shown in figure (9) an example of bit mapping within a byte transfer showing also the correct valid frame sent within the byte transfer and in figure (10) the byte mapping for the x32 interface.

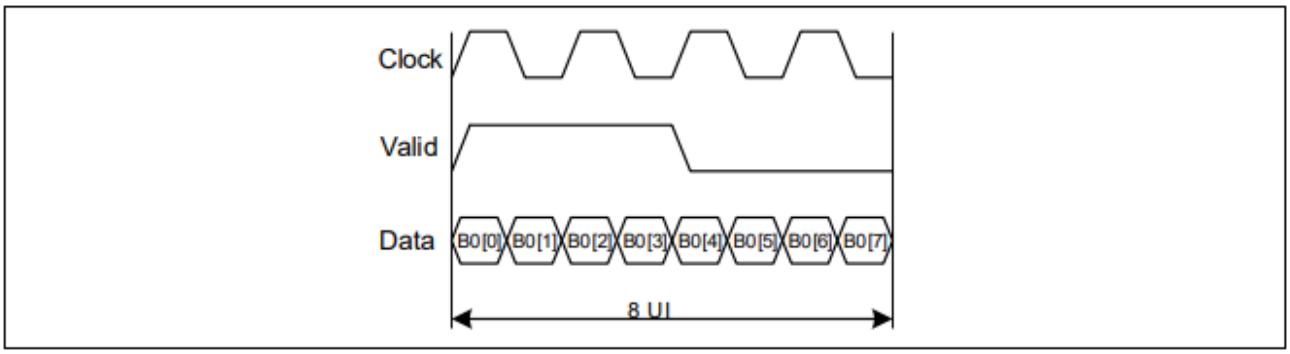


Figure 2-3: Bit Arrangement Within Byte transfer

UI \ Lane	0	1	2	3	4	5	6	7	...	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0 - 7	B0	B1	B2	B3	B4	B5	B6	B7	...	B16	B17	B18	B19	B20	B21	B22	B23	B24	B25	B26	B27	B28	B29	B30	B31
8 - 15	B32	B33	B34	B35	B36	B37	B38	B39	...	B48	B49	B50	B51	B52	B53	B54	B55	B56	B57	B58	B59	B60	B61	B62	B63
16 - 23	B64	B65	B66	B67	B68	B69	B70	B71	...	B80	B81	B82	B83	B84	B85	B86	B87	B88	B89	B90	B91	B92	B93	B94	B95
24 - 31	B96	B97	B98	B99	B100	B101	B102	B103	...	B112	B113	B114	B115	B116	B117	B118	B119	B120	B121	B122	B123	B124	B125	B126	B127
32 - 39	B128	B129	B130	B131	B132	B133	B134	B135	...	B144	B145	B146	B147	B148	B149	B150	B151	B152	B153	B154	B155	B156	B157	B158	B159
40 - 47	B160	B161	B162	B163	B164	B165	B166	B167	...	B176	B177	B178	B179	B180	B181	B182	B183	B184	B185	B186	B187	B188	B189	B190	B191
48 - 55	B192	B193	B194	B195	B196	B197	B198	B199	...	B208	B209	B210	B211	B212	B213	B214	B215	B216	B217	B218	B219	B220	B221	B222	B223
56 - 63	B224	B225	B226	B227	B228	B229	B230	B231	...	B240	B241	B242	B243	B244	B245	B246	B247	B248	B249	B250	B251	B252	B253	B254	B255

Figure 2-4: Byte Mapping for x32 interface

2.1.3. Valid Framing:

Valid signal is used to frame the transmitted data. For each 8-bit data packet, valid is asserted for the first 4 UI and de-asserted for 4 UI. This will allow data transfer in Raw Format or various Flit Formats using one or multiple valid frames. An example is shown in Figure 11 where Transfer 1 and Transfer 2 can be from the same Flit or different Flits.

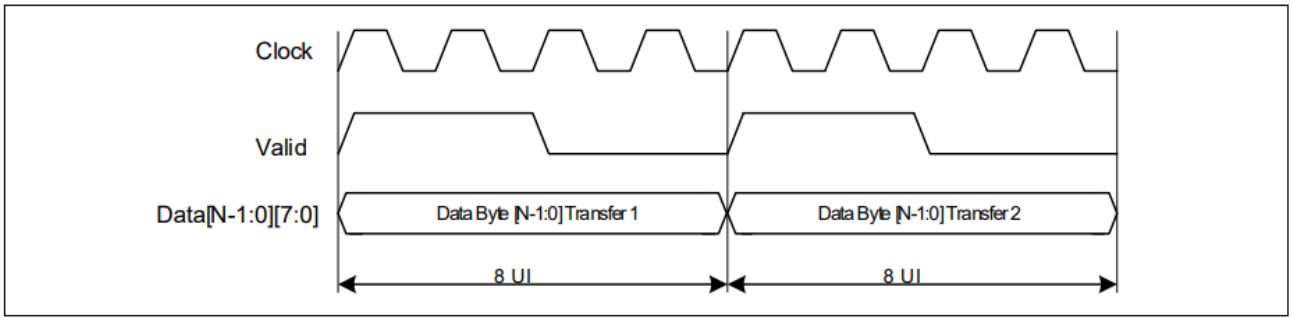


Figure 2-5: Valid framing example

2.1.4. Clock Gating:

Forwarded Mainband clocks on the UCIe Link must be gated when Valid signal is low after providing fixed 16 UI (8 cycles) of post-amble clock for half-rate clocking and 32 UI (8 cycles) of post-amble clock for quarter-rate clocking, unless free running clock mode is negotiated or Runtime Recalibration has been requested by the remote Link partner.

Free running clock mode is defined as the mode where the forwarded clock remains toggling even when the Transmitter for Valid is held low and there is no data transfer on the interface. This mode must be supported to allow disabling dynamic clock gating for normal operation or debug. This must be negotiated prior to Mainband Link training through parameter exchange.

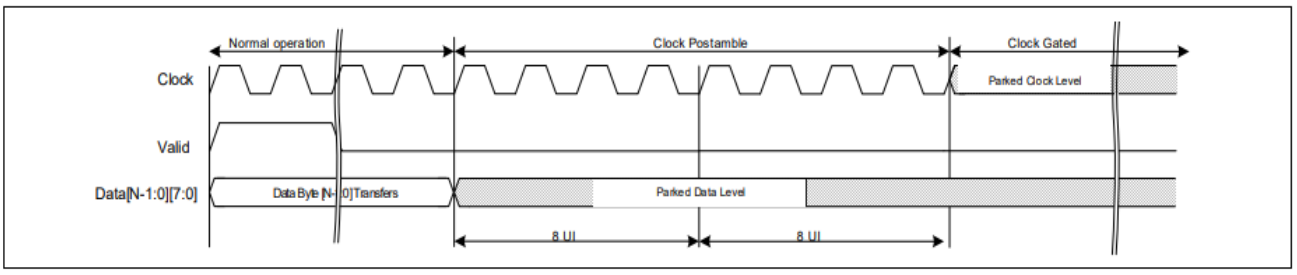


Figure 2-6: Clock Gating Diagram

2.1.5. Transmitting and Receiving Sideband messages:

The usage for the sideband Link is to provide an out of band channel for Link training and an interface for sideband access of registers of the Link partner. It is also used for Link Management Packets and parameter exchanges with remote Link partner.

Each module in the PHY Layer supports a sideband interface with a serial data and clock pin pair The sideband message formats are defined as a 64-bit header with no data, with 32 bits or 64 bits of data.

Sideband transmission is serial following a specific format where a 64-bit serial packet is defined on the I/O interface to the remote die as shown in Figure 13. 32-bit data is sent using the 64-bit serial packet with MSBs padded with 0b. Two sideband serial packets on the I/O interface are separated by a minimum of 32 bits low as shown in Figure 14. A sideband message with data would be transferred as a 64-bit header followed by 32 bits of low followed by 64-bit data followed by 32 bits of low and so on.

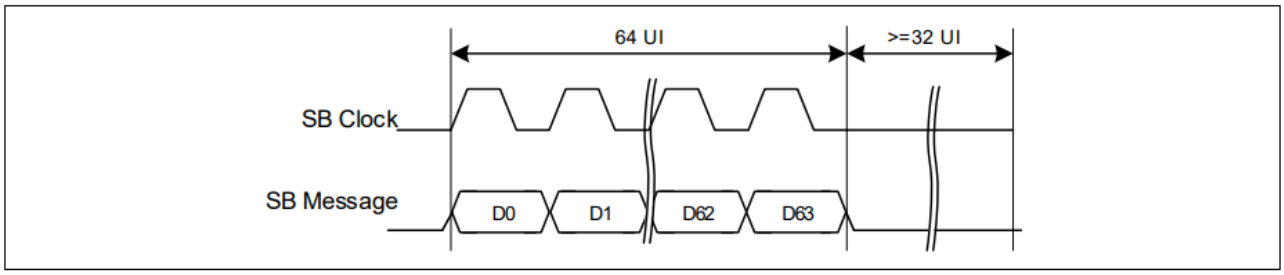


Figure 2-7: Example 64-bit Sideband Serial Packet Transfer

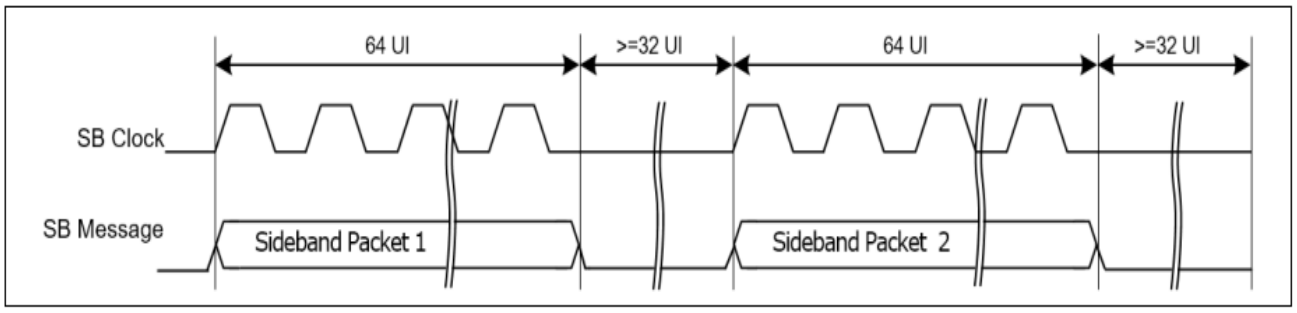


Figure 2-8: Sideband Packet Transmission: Back-to-Back

The Physical Layer is also responsible for framing and transporting sideband packets over the UCIe Link. Direct sideband access to remote die can originate from the Adapter or the Physical Layer. The Adapter forwards a remote die sideband access over RDI to the Physical Layer for framing and transport. These include register access requests, completions or messages.

2.1.5.1. Sideband Register Access Requests:

We have two types of register accessing request remote die register access request and local register access request

1. **Remote partner requests:** The Protocol Layer accesses remote die registers indirectly through a sideband mailbox mechanism. These mailbox registers are located within the Adapter, which is responsible for initiating remote register access requests upon receiving a corresponding access trigger via the (FDI). The figure attached below illustrates the mechanism by which the remote die mailbox is accessed, detailing both the role of the

Adapter and the responsibilities of the Physical Layer in this process. This mechanism enables indirect register access across dies.

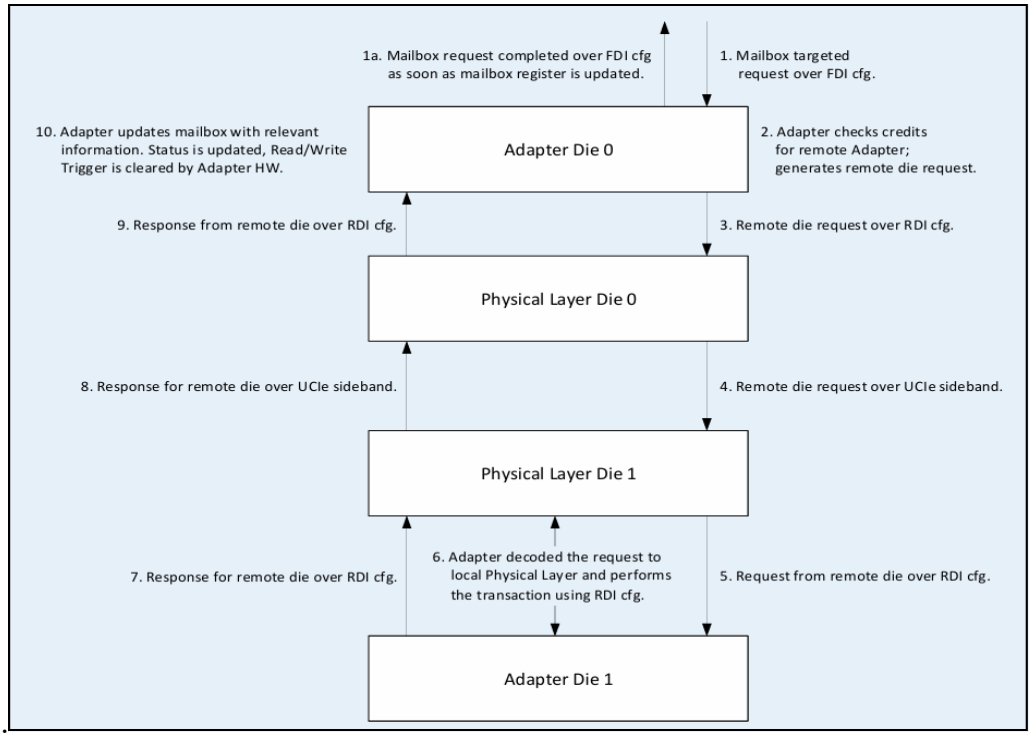


Figure 2-9: Mailbox Acessing Mechanism

2. **Local requests: The** protocol layer has the ability to request access to specificregisters via the lower layer, which acts as an intermediary to perform the actual read/write operations on the target hardware.

   To facilitate sideband, register communication between dies, a well-defined packet format is used for register access operations. This packet encapsulates all necessary control, address, and payload information required to initiate a remote register read or write. The format ensures reliable delivery and interpretation of register-level operations across the UCIe link.

   The structure of the register access packet is shown in the figure below. Each field of the packet plays a specific role in framing the transaction, ensuring data integrity, and managing protocol compliance.

Register Access Request																															
Bytes	3								2								1								0							
Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Header / Data	Header																															
Phase0	srcid			rsvd		tag[4:0]			be[7:0]								rsvd								ep	opcode[4:0]						
Phase1	dp	cp	cr	rsvd		dstid			addr[23:0]																							
Header / Data	Data (if applicable, can be 32 bits or 64 bits)																															
Phase2	data[31:0]																															
Phase3	data[63:32]																															

Figure 2-10: Register access packet format.

Each field within the register access packet plays a specific and essential role in defining the structure, control, and interpretation of register-level communication across the UCIe interface. the following figure presents a detailed explanation of each field, including its purpose, operational context, and any constraints or reserved values associated with different transaction types.

Field	Description
CP	Control Parity (CP) is the even parity of all the header bits excluding DP.
DP	Data Parity is the even parity of all bits in the data payload. If there is no data payload, this bit is set to 0b.
Cr	If 1b, indicates one credit return for credited sideband messages. This field is only used by the Adapter for remote Link partner's credit returns for E2E credits. It is not used for local FDI or RDI credit loops.
Addr[23:0]	Address of the request. Different opcodes use this field differently. See Table 7-6 for details. The following rules apply for the address field: For 64-bit request, Addr[2:0] is reserved. For 32-bit request, Addr[1:0] is reserved.
BE[7:0]	Byte Enables for the Request. It is NOT required to be contiguous. BE[7:4] are reserved if the opcode is for a 32-bit request.
EP	Data Poison. If poison forwarding is enabled, the completer can poison the data on internal errors. Setting the EP bit is optional, the conditions for setting it to 1 are implementation-specific. Typical usages involve giving additional FIT protection against data integrity errors on internal data buffers. A Receiver must not modify the contents of the target location for requests with data payload that have the EP bit set. It must return UR for the completion status of requests with an EP bit set.
Tag[4:0]	Tag is a 5-bit field generated by the requester, and it must be unique for all outstanding requests that require a completion. The original requester uses the Tag to associate returning completions with the original request.
Data	Payload. Can be 32 bits or 64 bits wide depending on the Opcode.

Figure 2-11: RAR-packet fields definition.

The figure below provides a comprehensive listing of all defined opcodes used across the protocol for messages, register access requests and completions. This mapping is critical for proper decoding and handling of sideband messages.

Opcode Encoding	Packet Type
00000b	32b Memory Read
00001b	32b Memory Write
00010b	32b DMS Register Read
00011b	32b DMS Register Write
00100b	32b Configuration Read
00101b	32b Configuration Write
01000b	64b Memory Read
01001b	64b Memory Write
01010b	64b DMS Register Read
01011b	64b DMS Register Write
01100b	64b Configuration Read
01101b	64b Configuration Write
10000b	Completion without Data
10001b	Completion with 32b Data
10010b	Message without Data
10111b	Management Port Messages without Data
11000b	Management Port Message with Data
11001b	Completion with 64b Data
11011b	Message with 64b Data
Other encodings	Reserved

Figure 2-12: Opcodes of messages.

The encoding of the source and destination fields is link-dependent, meaning it varies based on the particular link over which the message is conveyed.
The following figures outlines the encoding schemes used across different link configurations.

Field[a]	Description
srcid[2:0]	000b: Stack 0 Protocol Layer 001b: D2D Adapter 011b: Management Port Gateway (see Section 8.2) 100b: Stack 1 Protocol Layer other encodings are reserved.
dstid[2]	0b: Local die terminated request 1b: Remote die terminated request
dstid[1:0]	For Local die terminated requests: 10b: Physical Layer other encodings are reserved. For Remote die terminated Register Access Requests: dstid[1:0] is Reserved For Remote die terminated Register Access Completions: 01b: D2D Adapter other encodings are reserved. For Remote die terminated messages: 01b: D2D Adapter message 10b: Physical Layer message 11b: Management Port Gateway message (see Section 8.2)

Figure 2-13: RDI sideband: srcid and dstid encodings on RDI.

Field[a]	Description
srcid[2:0]	000b: Stack 0 Protocol Layer 100b: Stack 1 Protocol Layer other encodings are reserved.
dstid[2:0]	001b: D2D Adapter 010b: Physical Layer other encodings are reserved.

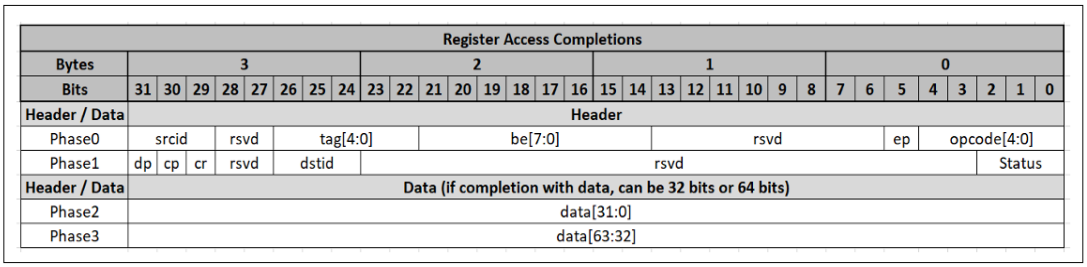
Figure 2-14: FDI sideband: srcid and dstid encodings on FDI.

Field	Description
srcid[2:0]	001b: D2D Adapter 010b: Physical Layer 011b: Management Port Gateway (see Section 8.2) other encodings are reserved
dstid[2]	1b: Remote die terminated request other encodings are reserved
dstid[1:0]	For Register Access requests: dstid[1:0] is Reserved. For Remote die terminated Register Access Completions: 01b: D2D Adapter other encodings are reserved. For Remote die terminated messages: 01b: D2D Adapter message 10b: Physical Layer message 11b: Management Port Gateway message (see Section 8.2)

Figure 2-15: UCIe Link sideband: srcid and dstid encodings for UCIe Link.

2.1.5.2. Sideband Completions:

After a register access request is successfully received and processed by the remote die, a register completion packet is generated and returned to the requester. This packet confirms the outcome of the operation whether a read, write, or error response and includes the relevant data (in the case of a read) along with status information. The completion packet ensures proper synchronization between both ends of the link and plays a critical role in maintaining protocol integrity. Attached below the completion packet format.



Register Access Completions																													
Bytes	3								2								1								0					
Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Header / Data	Header																															
Phase0	srcid			rsvd		tag[4:0]			be[7:0]								rsvd								ep	opcode[4:0]						
Phase1	dp	cp	cr	rsvd		dstid			rsvd																					Status		
Header / Data	Data (if completion with data, can be 32 bits or 64 bits)																															
Phase2	data[31:0]																															
Phase3	data[63:32]																															

Figure 2-16: Completion packet field.

Each field in the register completion packet is designed to convey essential information required for the correct interpretation and handling of the response. These fields include control bits to identify the packet type, status indicators to report success or failure, and, in the case of read operations, the actual data returned from the remote register. Additionally, fields such as the byte enable and tag ensure proper mapping to the original request and allow for partial updates or tracking of outstanding transactions. The following figure provides a detailed breakdown of each field, describing its purpose, width, and how it contributes to the overall protocol functionality.

Field	Description
`Tag[4:0]`	Completion Tag associated with the corresponding Request. The requester uses this to associate the completion with the original request.
`CP`	Control Parity. All fields other than "DP" and "CP" in the Header are protected by Control Parity, and the parity scheme is even (including reserved bits).
`DP`	Data Parity. All fields in data are protected by data parity, and the parity scheme is even.
`Cr`	If 1b, indicates one credit return for credited sideband messages. This field is only used by the Adapter for remote Link partner's credit returns for E2E credits. It is not used for local FDI or RDI credit loops.
`EP`	Data Poison. If poison forwarding is enabled, the completer can poison the data on internal errors. Setting the EP bit is optional, the conditions for setting it to 1 are implementation-specific. Typical usages involve giving additional FIT protection against data integrity errors on internal data buffers. A Receiver must not modify the contents of the target location for requests with data payload that have the EP bit set. It must return UR for the completion status of requests with an EP bit set.
`BE[7:0]`	Byte Enables for the Request. Completer returns the same value that the original request had (this avoids the requester from having to save off the BE value). BE[7:4] are reserved if the opcode is for a 32-bit request.
`Status[2:0]`	Completion Status 000b - Successful Completion (SC). This can be a completion with or without data, depending on the original request (it must set the appropriate Opcode). If the original request was a write, it is a completion without data. If the original request was a read, it is a completion with data. 001b - Unsupported Request (UR). On UCIe, this is a completion with 64b Data when a request is aborted by the completer, and the Data carries the original request header that resulted in UR. This enables easier header logging at the requester. Register Access requests that timeout must also return UR status, but for those the completion is without Data. 100b - Completer Abort (CA). On UCIe, this is a completion with 64b Data, and the Data carries original request header that resulted in UR. This enables easier header logging at the requester. 111b - Stall. Receiving a completion with Stall encoding must reset the timeout at the requester. Completer must send a Stall once every 4ms if it is not ready to respond to the original request. Other encodings are reserved. An error is logged in the Sideband Mailbox Status if a CA was received or if the number of timeouts exceed the programmed threshold. For timeouts below the programmed threshold, a UR is returned to the requester.
`Data`	Payload. 32 bits or 64 bits depending on the Opcode.

Figure 2-17: Completion message fields.

2.1.5.3. Sideband Messages:

Messages sent over the sideband interface can generally be grouped based on whether or not they carry extra data. Some messages—like **Link Management (LM)** and **Vendor Defined Packets (VDPs)—**consist only of a header and don't include any additional data. Others, such as **Parameter Exchange (PE)** messages or certain **Vendor Defined messages**, do carry a **64-bit data payload** alongside the header.

In addition, there are special **training handshake messages** that help bring up the link and keep both ends in sync. These are handled by the **Link Training State Machine (LTSM),** and depending on the specific training step, they can be sent either with or without data.

The figures below illustrate the detailed bit-level structure of each message type, including header fields such as source ID, destination ID, opcode, and message codes. This structured packetization enables robust and efficient parsing of control and data information at the receiver.

Messages without Data																													
Bytes	3								2								1								0					
Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Header / Data	Header																															
Phase0	srcid				rsvd				rsvd								msgcode[7:0]								rsvd						opcode[4:0]	
Phase1	dp	cp			rsvd				dstid								MsgInfo[15:0]								MsgSubcode[7:0]							

Figure 2-18: Message without data packet format.

Messages with data																																
Bytes	3								2								1								0							
Bits	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Header / Data	Header																															
Phase0	srcid				rsvd				rsvd								msgcode[7:0]								rsvd						opcode[4:0]	
Phase1	dp	cp			rsvd				dstid								MsgInfo[15:0]								MsgSubcode[7:0]							
Header / Data	Data																															
Phase2	data[31:0]																															
Phase3	data[63:32]																															

Figure 2-19: Message with data packet format.

2.1.5.4. Flow Control & Operation on RDI:

For each Transmitter associated with RDI, a design time parameter of the interface is used to determine the number of credits advertised by the Receiver, with a maximum of 32 credits. Each credit corresponds to 64 bits of header and 64 bits of potentially associated data. Thus, there is only one type of credit for all sideband packets, regardless of how much data they carry.

Every Transmitter/Receiver pair has an independent credit loop. For example, on RDI, credits are advertised from Physical Layer to Adapter for sideband packets transmitted from the Adapter to the Physical Layer; and credits are also advertised from Adapter to the Physical Layer for sideband packets transmitted from the Physical Layer to the Adapter. The Transmitter must check for available credits before sending Register Access requests and Messages.

The Transmitter must not check for credits before sending Register Access Completions, and the Receiver must guarantee unconditional sinking for any Register Access Completion packets. Messages carrying requests or responses consume a credit on RDI, but they must be guaranteed to make forward progress by the Receiver and not get blocked behind Register Access requests. RDI gives a dedicated signal for sideband credit returns across those interfaces.

All Receivers associated with RDI must check received messages for data or control parity errors, and these errors must be mapped to Uncorrectable Internal Errors (UIE) and transition RDI to LinkError state.

2.1.6. Scrambling & Training pattern generation:

Output data lanes are scrambled before sent on UCIe using Linear feedback shift register (LFSR). The LFSR uses the same polynomial as PCIe: G(X)=X23 + X21 + X16 + X8 + X5 + X2 + 1. The seed of the LFSR is Lane dependent, and based on the Logical Lane number and the seed value for Lane number is modulo 8 as shown in Figure 15.

Lane	Seed
0	23'h1DBFBC
1	23'h 0607BB
2	23'h1EC760
3	23'h18C0DB
4	23'h010F12
5	23'h19CFC9
6	23'h0277CE
7	23'h1BB807

Figure 2-20: LFSR Seed Values.

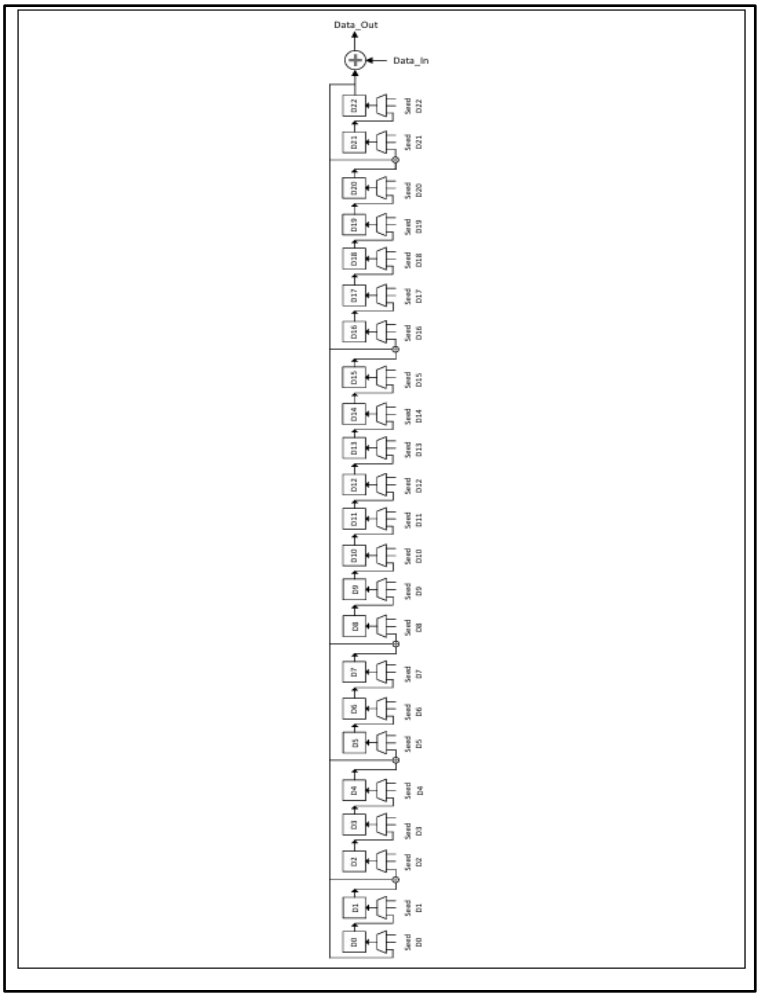


Figure 2-21: LFSR Implementation.

Through the Link Initialization and Training there are 4 patterns sent:

1. SBINIT pattern: this is a 64 UI of clock pattern clock pattern (1010…) and 32 UI low sent on Serial Sideband data Lane with clock at SBINIT state to detect clock and data and perform accurate synchronization.
2. Valid pattern: 128 iterations of (1111 0000) with the least significant bit first.
3. Per-Lane Id pattern: 128 iterations of Per-Lane Id which is a 16-bit ID unique for each lane used to detect lane reversal in Mainband data lanes. Its format is as shown below.

Logical Lane Number	Lane ID	Logical Lane Number	Lane ID
0	00000000b	8	00001000b
1	00000001b	9	00001001b
2	00000010b	10	00001010b
3	00000011b	11	00001011b
4	00000100b	12	00001100b
5	00000101b	13	00001101b
6	00000110b	14	00001110b
7	00000111b	15	00001111b

Figure 2-22: Lane ID Standard Package Module

Pattern	0	1	0	1	Lane ID[a]								0	1	0	1
Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Figure 2-23: Per-Lane ID Format

The Figure below shows lane ID examples and how it differs from one lane to another.

Lane 1	0	1	0	1	1	0	0	0	0	0	0	0	0	1	0	1
Lane 31	0	1	0	1	1	1	1	1	1	0	0	0	0	1	0	1
Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Figure 2-24: Per_Lane ID Example

4. LFSR pattern: 4K iterations of inserting zero input to the scrambler and sending the output.

2.1.7. Lane Reversal & Width Degradation & Speed Degradation:

PHY Layer must support Lane reversal of Mainband data lanes within a module. After detecting physical lane reversal in Link Initialization phase. For example, if physical Data Lane 0 on local die is connected to physical Data Lane (N-1) on the remote die (physical Data Lane 1 is connected to physical Data Lane N-2 and so on), the Die0 Tx should be able to reverse its lane mapping to account this physical lane reversal.

In the case of x16 Standard Package modules where Lane repair is not supported, resilience against faulty Lanes is provided by configuring the Link to a x8 width (Logical Lanes 0 to 7 or Logical Lanes 8 to 15, which exclude the faulty Lanes). For example, if one or more faulty Lanes are in logical Lane 0 to 7, the Link is configured to x8 width using logical Lanes 8 to 15. The configuration

is done during Link initialization or retraining. Transmitters of the disabled Lanes are tri-stated and Receivers are disabled. And if the width degradation is not an option (the faulty lanes are distributed not in a significant half) the speed degradation is applied where the both Dies scale down their speed trying to use the faulty data lanes.

2.2. Assumptions and Supported Functionalities:

Assumptions Considered During the Implementation of Our UCIe Protocol Variant

- **Memory-Mapped Control Access**
  It is assumed that a memory controller is present, facilitating access to the UIRB via memory-mapped registers. These registers, such as sw_exit, are accessible through standard read/write operations during both simulation and software-based interaction.

- **Message Completion – MBRX_Done**
  When the D2D layer initiates a PHY Retrain, the transmitter ensures any in-flight message is completely flushed by transmitting valid data over four consecutive cycles. This duration accounts for a worst-case message size of 256 bytes. Internally, the transmitter typically pipelines 64-byte chunks, requiring approximately three cycles to prepare each. If no new valid data is observed within three cycles after the final bit of the last chunk, the system concludes that transmission is complete, and the receiver is idle.

- **Inter-Message Timing**
  The time gap between consecutive message receptions is approximately 96 sbclk cycles and 120 lclk cycles, reflecting typical link utilization patterns and protocol processing delays**.**

- **Configuration Interface Clocking**
  All configuration space transactions are synchronized to the LCLK domain to maintain consistency and avoid metastability.

- **UIRB Response Time**
  The UIRB (Unified Interface Register Block) is designed to respond with the requested register data within a maximum of five Lclk cycles.

- **Read RAR Response Format**
  For read operations targeting remote accessible registers, the response includes the full 64-bit value of the requested register. Additionally, the byte enable (BE) field is attached within the completion packet and is subsequently interpreted by the D2D interface logic.

- **Fatal UIE Handling**
  Upon detection of a fatal Uncorrectable Internal Error (UIE), the D2D layer ceases data transmission across the interface, allowing the system to gracefully transition to a safe state.

- **Link Training Constraint**
  The link training process, triggered by start_UCIE_link, is assumed to only initiate when the system is in the RESET state, ensuring state consistency and avoiding undefined transitions.

- **Training Timeout Handling**
  If a timeout occurs during link training—specifically due to the repeated triggering of the sbinit_pattern— the pl_error signal is asserted to the adapter, indicating a protocol-level fault.

- **State Transition Protocol**
  The adapter is allowed to issue only one state request per transition of the pl_state_sts signal, ensuring serialized and traceable state changes.

- **Synchronized Die Behavior**
  Illegal protocol transitions are not expected from the remote die. Both dies adhere to a synchronized handshake mechanism, ensuring they move through protocol states together and maintain proper link alignment.

- **Internal Storage of Critical Fields**
  Some parameters required for message generation—such as the error threshold—are assumed to be stored internally within the module itself. This reduces memory access overhead and enhances performance**.**

Our UCIe protocol variant does not implement all the features outlined in the official specification. Below is a list of the selected features that have been implemented

- **Retimer Support**
  Our UCIe protocol variant does not support retimers and therefore does not extend connectivity beyond the package using off-package interconnect technologies such as electrical or optical cables.

- **Packaging Support**
  Our UCIe protocol variant supports the **Standard Package** option only. It does **not support the Advanced Package** option, as the design targets longer-reach, cost-effective interconnects rather than performance-optimized short-reach implementations and accordingly, Interconnect Redundancy Remapping is not supported.

- **Module Configuration Support**
  The **single-module configuration with x16 lanes** is supported, while **multi-module configurations** are not supported. The key difference lies in the **operational independence** of single-module configurations, where each module uses a dedicated Adapter and may function at varying data rates and widths. In contrast, multi-module configurations require tight synchronization, with all modules sharing a common Adapter and operating at the same data rate and width.

- **Flit Format Handling in Error Scenarios**
  Different protocols, such as **PCIe** and **CXL**, define varying flit formats—**68B** and **256B**, respectively. While these formats differ in size and structure, the **Main-Band** is designed to handle the **largest supported flit format** to ensure reliable operation during **error-triggered state transitions**. This capability ensures protocol flexibility and robust errors handling across diverse interface configurations.

- **Bit Reception Granularity**
  Single Data Rate **(SDR)** operation is supported, while Double Data Rate **(DDR)** is not supported. The difference lies in the data transfer rate per clock cycle—SDR transmits and receives one bit per cycle, ensuring deterministic timing and straightforward alignment, whereas DDR would require handling two bits per cycle, increasing complexity.

- **Main-Band Clock Gating Behavior**
  Free running clock mode is supported, while **mandatory clock gating** after the Valid signal de-asserts is not enforced in this mode. The key difference lies in the **clock behavior during inactive periods**—in standard operation, forwarded Main-Band clocks must be gated after providing a fixed postamble (16 UI for half-rate or 32 UI for quarter-rate), whereas in free running mode, the clocks continue toggling regardless of the Valid signal state.

- **Sideband PMO Capability**
  Sideband PMO is not supported, while standard sideband operation without PMO is supported. The difference lies in the **timing between data transfers**—when PMO is enabled, the 32-UI dead time between consecutive 64-UI sideband data transfers is eliminated, allowing back-to-back transmission. Without PMO, this **dead time remains required**, ensuring defined gaps between transfers.

- **Tx Pattern Mode Support**
  Continuous mode is supported, while **burst mode** is not supported. The difference lies in the **pattern transmission behavior during training test** continuous mode maintains a steady stream of test patterns, aligning with all currently defined training procedures.

- **LTSM State Support**
  All Link Training State Machine (LTSM) states are supported except for the Power Management state, which is not implemented as there is no power management mechanism Implemented.

- **Messages supported over SB Link**
  RARs, Completions, and D2D Requests are supported over the Sideband (SB) Link, while the Management Transport Protocol (MTP) and DMS messages are not supported. The difference lies in their purpose—RARs, Completions, and D2D Requests handle essential communication for runtime operations, whereas MTP and DMS messages are primarily used for diagnostics and support of the UCIe Debug & Test Architecture, which is not implemented in the current system.

- **Width Degradation**
  Only Width Degradation from 16 to 8 Lanes is supported as Interoperability is not supported in this variant where it allows for further degradation from 8 to 4.

- **No Speed Degradation Support:**
  The design does not support **dynamic link speed degradation**. As a result, it lacks resilience in adapting to deteriorating link quality and operates solely in single data rate (SDR) mode for mainband transmission.

- **Data Rate Support**
  All data rates from 4 to 32 Gbps are supported. This is achieved by adjusting the MBRX FIFO depth to accommodate the maximum clock ratio between the system clock and the selected data rate, ensuring reliable data handling across the full range of supported speeds.

- **RDI Interface Configuration**
  The RDI Interface for D2D messages is configured to use the maximum supported width of **32 bits** (other available options are 8 and 16 bits), while the RDI Interface for Main-Band data is set to the minimum supported width of **64 bytes** (other available options are 128 and 256 bytes).

2.3. Architecture:

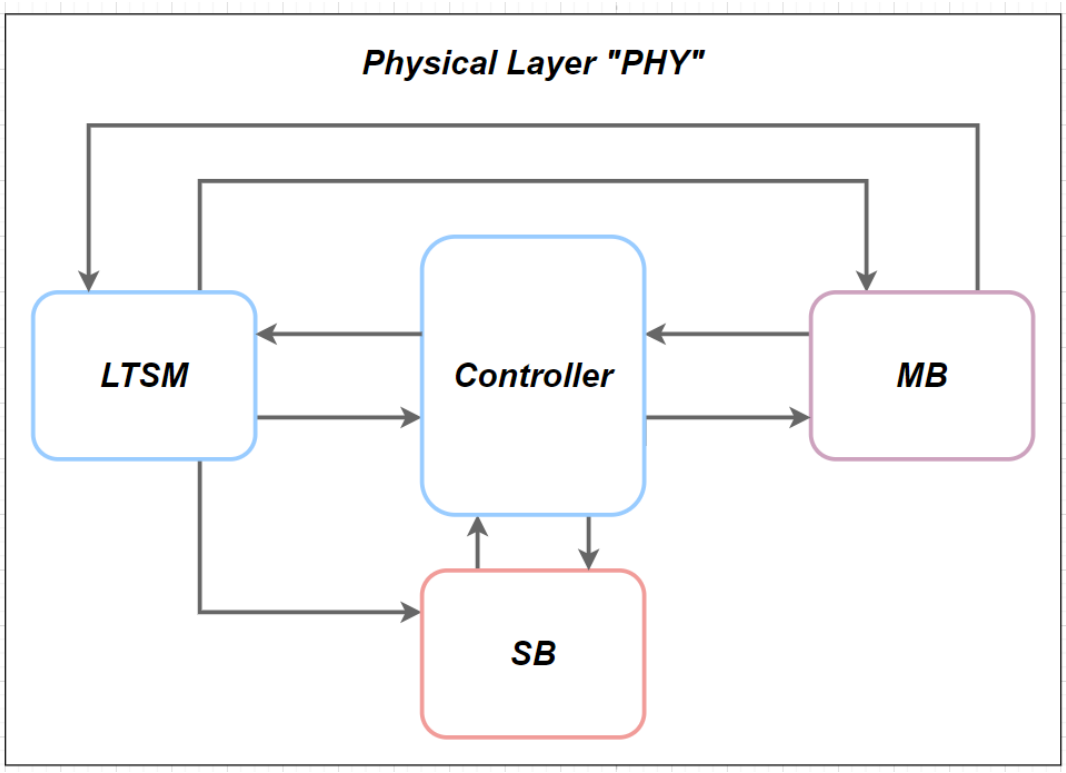


Figure 2-25: PHY Layer Architecture Level 1.

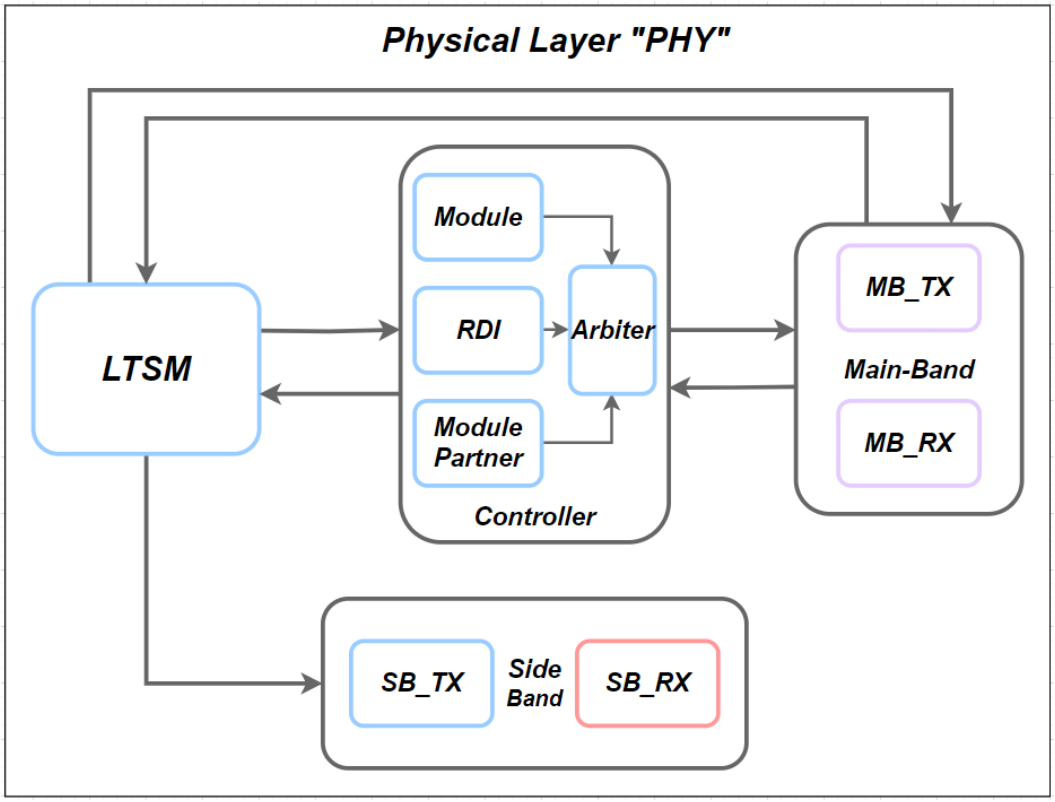


Figure 2-26: PHY Layer Architecture Level 2.

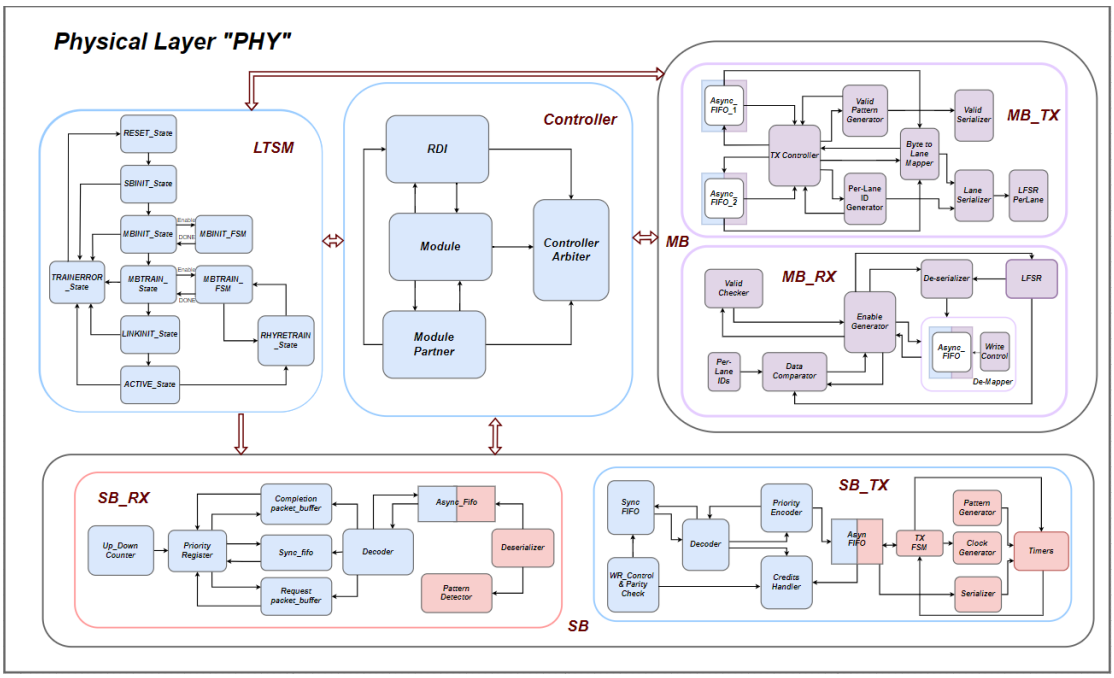


Figure 2-27: PHY Layer Architecture Level 3.

The PHY layer is responsible for a wide range of functionalities, which our architecture organizes into two primary categories: **Controlling** and **Data Forwarding**. Each of these categories is further subdivided into specific subcategories.

1. Controlling

This category includes the following subcomponents:

- **State Transition Management**: Handled by the **LTSM** module, which oversees the transition between operational states.
- **Training States Handshakes and Test Management**: Managed by the **Phy_Module** and **Module_Partner**, which are submodules within the **Controller Module**, in addition to the **MB Bring Up Phase Modules**.
- **RDI Interface Interaction**: This functionality is supported by the **SBTX LCLK Domain Blocks**, **SBRX LCLK Domain Blocks**, and the **RDI_Module**, another submodule within the **Controller Module**.

2. Data Forwarding

This category is divided into:

- **Main Data Handling**: Executed by the **MB Post Bring Up Phase Module**.
- **Control Message Handling**: Managed by the **SB Module**.

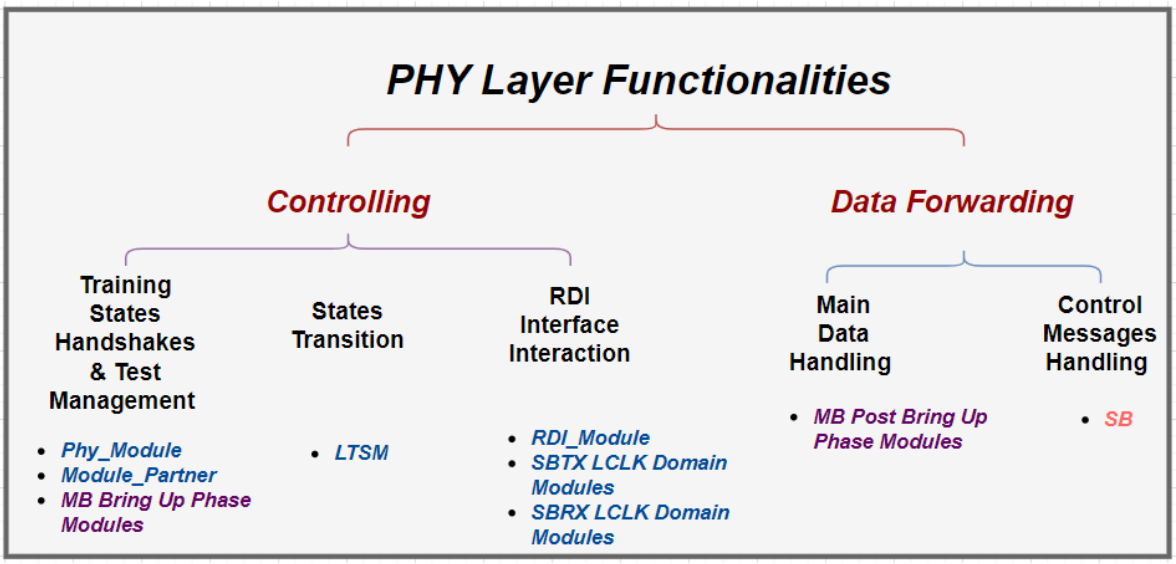


Figure 2-28: PHY Layer Functionalities Categories

SBTX & MBRX Architecture:
The architecture of both **MBRX** and **SBTX** are primarily based on a **modular design approach**, wherein the functionality assigned to each module is further decomposed into more specific categories depending on the **system's operational state.**

MBRX Architecture
In the case of MBRX, the architecture is divided into two main sets of sub-blocks:

- **Bring-Up Phase Blocks**: These are responsible for executing tests on the data lanes of the MB module. Since MB lanes are categorized into **Valid** and **Data** lanes, each lane has a **dedicated sub-block** tailored to handle its specific functionality.
- **Post Bring-Up Phase Blocks**: This phase is entered once all training states have been completed. These blocks take over the **main data handling** responsibilities, while the bring-up phase blocks transition to an idle state.

Additionally, the MBRX architecture adopts a **Control and Data Path separation model**, where all functional blocks are governed by centralized controllers responsible for their configuration and activation.

SBTX Architecture
The SBTX module interacts with various components such as the **SB lanes**, **D2D interface**, **UIRB**, and **SBRX**. For each of these external entities, a **dedicated sub-block** exists within the SBTX architecture to manage communication and coordination.
A key architectural feature of SBTX is its implementation of a **Distributed Triggering Mechanism**, wherein functional blocks support each other not by executing each other's tasks, but by generating **trigger signals**. These signals are then received and processed by the respective dedicated blocks, ensuring coordinated yet decoupled operation across the system.

MB_TX Architecture:
This part of the PHY layer is mainly responsible for handling the transmission of different data and valid tests and it also considered as the main transmitter of D2D data in ACTIVE state, so its architecture was surely needed to be divided into sub-blocks or sub-modules each dedicated for transmitting a specific test and one other block responsible for active data transmission, while keeping all these modules controlled by a control signals driven from a central MB_TX Controller to ensure correct tests and data handling without corruption or any data losses, where the MB_TX Controller makes its decisions based on the current LTSM state and status signals from the sub-modules blocks.

LTSM Architecture:
The **Link Training State Machine (LTSM)** is the core sequencing and orchestration unit within the PHY layer of the UCIe protocol. It governs the link's lifecycle, ensuring proper progression through its initialization, training, activation, and retraining phases. Architecturally, LTSM is composed of a **main control state machine**, two **nested sub-state machines** for initialization and training, and a set of **coordinating interfaces** with various subsystems such as the RDI, Main-Band (MB), and Side-Band (SB) layers.

At the heart of the LTSM lies a **primary finite state machine** that defines the operational phases of the PHY link. Each transition between these states is controlled by Side-Band handshakes, error indicators, timeout counters, and retraining requests.
The **MBINIT sub-state machine** operates only within the MBINIT main state and is responsible for executing the initial PHY configuration process.

The **MBTRAIN FSM** is a second-level state machine active only during the MBTRAIN phase. It governs the complete lane and timing calibration process including voltage reference validation, center alignment, and deskewing.

To ensure robustness, the LTSM incorporates a **timeout supervision mechanism** via a programmable counter. This counter resets on any sub-state change and is used to detect stall conditions within each state. If a state persists beyond a defined threshold (typically 8ms), the system forcibly transitions to **TRAINERROR** to recover the link.

The **LTSM module** is the central authority for PHY layer coordination in a UCIe link. With its hierarchical FSM design, robust timeout logic, and precise interface coordination, it ensures high-speed link initialization, adaptive training, retraining, and error recovery. Its modular sub-FSMs for MBINIT and MBTRAIN stages allow it to finely control each phase of the PHY setup, making it a critical part of a reliable chiplet-to-chiplet communication interface.

Controller Architecture:
The **Controller** block resides in the local die and plays a **pivotal role in PHY coordination**. It operates as a central controller interfacing with various PHY components such as the MB_RX, MB_TX, SB_RX, SBTX and the Link Training State Machine (LTSM). Its responsibilities span across tests handling, state transitions, errors handling, parameter resolution, and side-band message handling.
It ensures seamless collaboration between **MB_TX**, **MB_RX**, **SBTX**, **LTSM** and the **SB_RX** by distributing control through dedicated signals.
It is considered as the main brain of the PHY layer that decides the occurrence of all states transition.
This modular and signal-driven design ensures flexibility, scalability, and protocol compliance in high-speed interconnects such as UCIe.

2.4. Methodology:

To manage the complexity and evolving nature of the design and verification process, this project adopted an Agile development methodology as the underlying framework. Agile practices were applied consistently across both the FPGA prototyping flow and the UVM-based verification environment, enabling iterative development, early validation, and continuous improvement.

The FPGA implementation process benefited from Agile's **iterative structure**, which allowed the design to be refined through cycles of simulation, synthesis, and hardware bring-up.

This approach reduced integration risks and enabled **early detection of hardware-specific bugs**.

Within the UVM environment, Agile methodology allowed the team to **incrementally build reusable verification components** such as:

- Agents, monitors, and drivers for individual interfaces.
- Scoreboards and coverage models for tracking correctness and completeness.
- Sequences for protocol training, normal operation, and error injection.

Each sprint delivered new test scenarios or improved coverage, enabling a **coverage-driven verification strategy** that evolved with the design. Feedback loops between simulation results, waveform inspection, and assertion violations were tightly integrated into the Agile cycles.

Chapter 3: Physical Layer Design:

3.1. Link Training State Machine "LTSM":

The LTSM is a critical state machine in the UCIe protocol that manages the initialization, training, and operational status of the physical link between chiplets. It ensures reliable communication by handling:

- Link initialization
- Clock synchronization
- Signal integrity calibration
- Error recovery
- Power State Transitions

These functionalities have been achieved using the following LTSM module architecture as shown in figure 3-1:

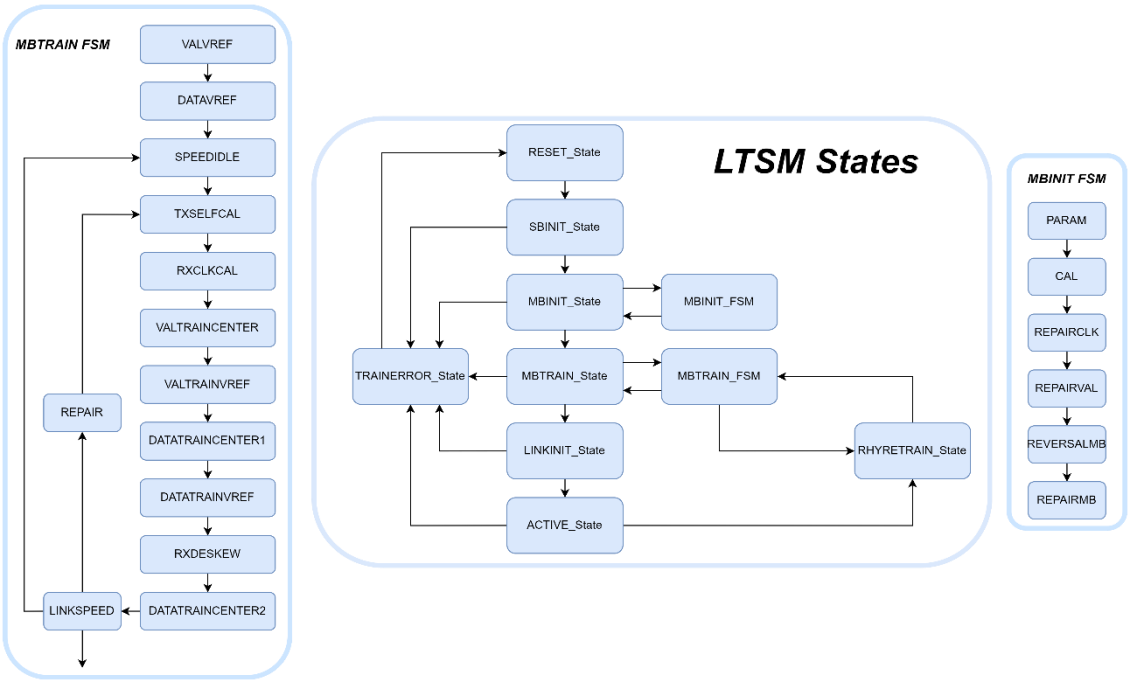


Figure 3-1: LTSM States Diagrams

LTSM has an interface with Controller, Configurable Space "UIRB", RDI Interface, Sideband Receiver, Mainband Transmitter, and Mainband Receiver as shown in figure 3-2:

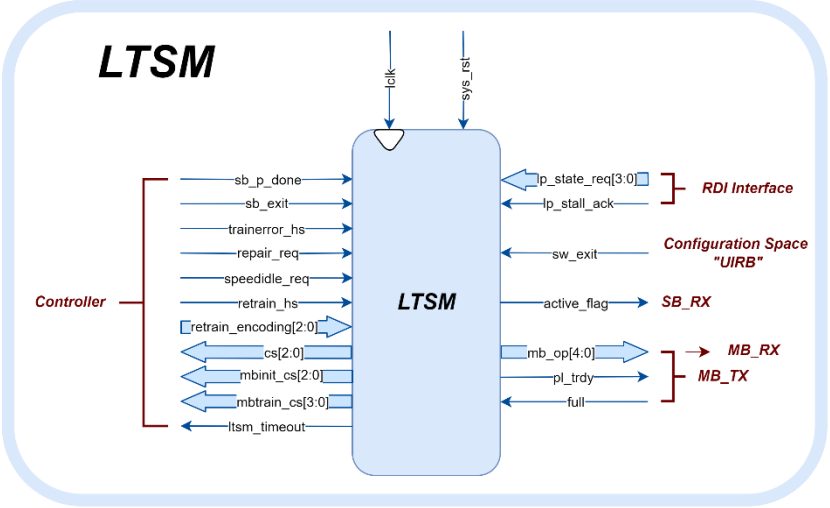


Figure 3-2: LTSM Interface

All signals of the LTSM top module are fully described in tables 3-1, and 3-2:

Table 3-1: LTSM Signals Description (sheet 1 of 2)

Port	Direction	Width	Description
lclk	Input	1	From Interface Link clock signal.
sys_rst		1	From Interface Active-low asynchronous link reset signal.
lp_state_req		4	From RDI Interface Adapter request to Physical Layer to request state change.
lp_stall_ack		1	From RDI Interface Signal is an indication from the Adapter Layer to the Physical Layer that all Flits are properly aligned and transmission is stalled.
sw_exit		1	From Configuration Space Signal indicates that software has written a 1 to the Start UCIe Link Training bit in the UCIe Link Control register (within the DVSEC), triggering the initiation or restart of the link training process.
full		1	From MB_TX Signal indicates if its active state FIFO is full or not for continuing or stopping accepting data from the RDI interface.
sb_p_done		1	From Controller Signal indicating that all in-progress sideband packets have finished transmission before entering the RESET state.
sb_exit		1	From Controller Signal indicating that all necessary sideband state transitions have completed, allowing the state machine to proceed to the next state.
trainerror_hs		1	From Controller Signal indicating that the handshake for entering the TRAINERROR state has completed.
repair_req		1	From Controller Signal that requests the LTSM to enter REPAIR state.
speedidle_req		1	From Controller Signal that requests the LTSM to enter SPEEDIDLE state.
retrain_hs		1	From Controller Signal indicating that the handshake for entering the RETRAIN state has completed.
retrain_encoding		3	From Controller Encoding that indicates the target state to transition to from the RETRAIN state.
cs	Output	3	To Controller A state variable representing the current state of the main LTSM.
mbinit_cs		3	To Controller A state variable representing the current sub-state of the MBINIT FSM.

Table 3-2: LTSM Signals Description (sheet 2 of 2)

Port	Direction	Width	Description
mbtrain_cs	Output	4	To Controller A state variable representing the current sub-state of the MBTRAIN FSM.
ltsm_timeout		1	To Controller Signal indicates that a timeout condition has occurred during the LTSM process when 8 milliseconds have elapsed without completing the required state transition.
active_flag		1	To SB_RX Signal that indicates the LTSM is currently in the **ACTIVE** state.
mb_op		5	To MB_TX and MB_RX Encoding that translates transitions from the LTSM encoding to the MB encoding.
pl_trdy		1	To MB_TX Signal indicates that PHY is ready to accept data from Adapter and data can be written in the MB_TX FIFO.

Description of LTSM States, MBINIT States, and MBTRAIN States is in tables 3-3, 3-4, 3-5, 3-6, 3-7, 3-8, 3-9, 3-10, 3-11, 3-12, 3-13, 3-14, and 3-15:

LTSM States Description:

Table 3-3: LTSM States Description (sheet 1 of 4)

State	Description	Transition
RESET	The **RESET** state is the initial state of the UCIe LTSM, ensuring a clean and controlled start before link initialization. In this state: • All new or pending sideband register access requests are treated as Unsupported Requests, with completions and credits returned to the Adapter. • Any pending completions are discarded. • Vendor Defined Messages are dropped or logged as errors. • No data transmission occurs, ensuring the link starts from a known, stable condition.	**Transition to SBINIT** when **either** of the following conditions is met after remaining in RESET for at least 4 ms: **Software-triggered:** • A write of 1 to the **Start UCIe Link Training** bit in the **UCIe Link Control register** (within the DVSEC). **Pattern detection:** • Detection of the **SBINIT pattern** on any sideband RX clock/data pair, defined as **two consecutive iterations** of 64-UI clock pattern followed by 32-UI low period.

Table 3-4: LTSM States Description (sheet 2 of 4)

State	Description	Transition
SBINIT	The **SBINIT** state is responsible for initializing the **Sideband (SB) interface**. In this state: • All new or pending sideband register access requests are treated as Unsupported Requests, with completions and credits returned to the Adapter. • Any pending completions are discarded. • Vendor Defined Messages are dropped or logged as errors. • No data transmission occurs, ensuring the link starts from a known, stable condition.	**Transition to MBINIT:** • SB pattern detection and {SBINIT Out of Reset} are successful. • {SBINIT done req} and {SBINIT done resp} messages are exchanged. **Transition to TRAINERROR:** • Sideband pattern or message detection fails after **8 ms timeout**. • {SBINIT done req} was sent, but no {SBINIT done resp} received. • Any pattern/handshake error during initialization process.
MBINIT	The **MBINIT** state is responsible for initializing the Mainband (MB) interface in Standard Package configurations. In this state: • The mainband link is initialized at the lowest supported data rate (4 GT/s). • Lane repair is not supported in Standard Package; instead, the state includes sub-states that: ▪ Verify the functional integrity of all active lanes. ▪ Perform width degradation if some lanes are found to be non-functional.	**Transition to MBTRAIN:** • All required MB interface checks are completed successfully. • Lane functionality has been verified. • Width degradation (if any) has been applied successfully. **Transition to TRAINERROR:** • Lane integrity checks fail, and width degradation is not viable. • timeout during initialization.
TRAINERROR	The **TRAINERROR** state acts as a transitional recovery state for handling fatal and non-fatal errors, as well as any conditions requiring a return to the RESET state. It may be entered during initialization, training, or due to manual intervention (e.g., the “Start UCIe Link training” bit set when not in RESET). It is also used when transitioning from a Link Up to a Link Down condition.	**Transition to RESET**: • All sideband transmissions have been completed. RDI not in LinkError.

Table 3-5: LTSM States Description (sheet 3 of 4)

State	Description	Transition
MBTRAIN	The **MBTRAIN** state is responsible for training the Mainband (MB) and it is used to setup operational speed and perform clock to data centering. This state uses many different sub-states to perform all the required calibration and training as the D2C point tests. UCIe Modules must enter each sub-state and the exit from each sub-state is coordinated between UCIe Module Partners through sideband handshakes. If a particular action within a sub-state is not needed, the UCIe Module is permitted to exit the sub-state through the relevant sideband handshake without performing the described operations in that sub-state. Devices enter this state once the MBINIT is completed. This state is common for both Advanced and Standard Packages.	**Transition to LINKINIT:** • All required MB interface checks and tests are completed successfully. • When a UCIe Module has sent and received the {MBTRAIN.LINKSPEED done resp} sideband message, which is the last sub-state in the MBTRAIN if no errors are encountered in it. **Transition to TRAINERROR:** • If errors encountered during tests and width degrade can't be applied and after completing the Trainerror sideband handshake messages. • If received an {TRAINERROR Entry req} sideband message. • Timeout occurs during training in any sub-state.
LINKINIT	The **LINKINIT** is used to exchange die to die Adapter and to complete initial Link management messages before entering Active state on RDI.	**Transition to ACTIVE:** • Physical layer initialization is complete, RDI is in Active state. **Transition to TRAINERROR:** • If the Trainerror sideband handshake messages are exchanged due to any errors. • Timeout occurs during link initializing.
ACTIVE	The **Active** state is the state in which transactions are sent and received. In this state, packets from upper layers can be exchanged between the two dies. All data in this state is scrambled using the scrambler LFSR described in MB_TX chapter. It is common for Advanced Package and Standard Package configurations.	**Transition to PHYRETRAIN:** • If any trigger of the PHYRETRAIN entry triggers occurs and after completing the required RDI handshakes (Stallreq/ack handshake mechanism) **Transition to TRAINERROR:** • If the Trainerror sideband handshake messages are exchanged due to any errors on the Link.

Table 3-6: LTSM States Description (sheet 4 of 4)

State	Description	Transition
PHYRETRAIN	The **PHYRETRAIN** State is used to begin the retrain flow for the Link during runtime. A die can enter PHY retrain for a number of reasons, which are: • Adapter directed PHY retrain: Adapter can direct the PHY to retrain for any reason it deems necessary • PHY initiated PHY retrain: Local PHY must initiate retrain on detecting a Valid framing error. • Remote die requested PHY retrain: Local PHY must enter PHY retrain on receiving a request from the remote die.	**Transition to MBTRAIN.TXSELFCAL:** • If the received resolved Retrain Encoding in the {PHYRETRAIN.retrain start resp} sideband message is 3'b001. **Transition to MBTRAIN.SPEEDIDLE:** • If the received resolved Retrain Encoding in the {PHYRETRAIN.retrain start resp} sideband message is 3'b010. **Transition to MBTRAIN.REPAIR:** • If the received resolved Retrain Encoding in the {PHYRETRAIN.retrain start resp} sideband message is 3'b100. **Transition to TRAINERROR:** • If the Trainerror sideband handshake messages are exchanged due to any errors or timeout occurs.

MBINIT States Description:

Table 3-7: MBINIT States Description (sheet 1 of 3)

State	Description	Transition
PARAM	The **MBINIT.PARAM** state handles the exchange of configuration parameters between the UCIe Module and its Link Partner to ensure PHY compatibility before mainband training. Parameters include voltage swing, data rate, clock mode, clock phase, module ID, width mode (UCIe-A x32 / UCIe-S x8), and sideband feature extensions.	**Transition to MBINIT.CAL:** • {MBINIT.PARAM configuration resp} is received correctly. • All parameters are resolved successfully and are interoperable. **Transition to TRAINERROR:** • No response or timeout (within implementation-defined window). • Parameters exchanged are **incompatible** or **unsupported**.

Table 3-8: MBINIT States Description (sheet 2 of 3)

State	Description	Transition
CAL	The **MBINIT.CAL** state is used to perform mainband PHY calibration, including: • Transmitter duty cycle correction • Receiver offset and Vref calibration • Other implementation-specific analog calibration steps	**Transition to MBINIT.REPAIRCLK:** • UCIe Module **receives {MBINIT.CAL Done resp}** in response to its request, indicating both ends are ready to proceed. **Transition to TRAINERROR:** • Timeout waiting for {MBINIT.CAL Done resp} • Sideband handshake failure • Internal error or invalid calibration state detected
REPAIRCLK	The **MBINIT.REPAIRCLK** state is used to check the functional operation of the clock and track lanes at the lowest supported data rate for Standard Package configurations. No physical lane repair is performed; only verification is done.	**Transition to MBINIT.REPAIRVAL:** • All three lanes (RCKP_L, RCKN_L, and RTRK_L) are **successfully validated** • {MBINIT.REPAIRCLK done req} and {done resp} handshake is completed **Transition to TRAINERROR:** • Failure to detect valid patterns on **any** of the three lanes • Timeout or sideband message error during request/response handshakes
REPAIRVAL	The **MBINIT.REPAIRVAL** state is used to functionally verify the Valid Lane (VLD) using a fixed pattern at the lowest supported data rate. No lane repair is performed in Standard Package; instead, the system checks that the Valid Lane operates correctly using a predefined pattern.	**Transition to MBINIT.REVERSALMB:** • Valid Lane (RVLD_L) pattern is detected successfully • {done req} and {done resp} handshake is completed **Transition to TRAINERROR:** • Failure in pattern detection on RVLD_L Any sideband communication failure or timeout during the sequence
REVERSALMB	The **MBINIT.REVERSALMB** state is used to detect whether lane reversal is needed based on per-lane data pattern matching. The UCIe Module verifies if the Data Lanes are routed correctly or reversed and applies reversal if required. This ensures that data is sampled on the correct physical lane regardless of routing.	**Transition to MBINITREPAIRMB:** • {done req} and {done resp} handshake completed successfully after initial pattern test passed, or reversal applied, and test passed **Transition to TRAINERROR:** • If both original and reversed tests fail • After completing the TRAINERROR handshake

Table 3-9: MNINIT States Description (sheet 3 of 3)

State	Description	Transition
REPAIRMB	The **MBINIT.REPAIRMB** state is responsible for verifying the mainband data lanes for functional integrity, performing width degradation if required, and preparing the link for transition to MBTRAIN. This state is only entered after successful Lane Reversal detection and application.	**Transition to MBTRAIN:** • Functional Lanes verified and width settled. **Transition to TRAINERROR:** • "Degrade not possible" sent or received.

MBTRAIN States Description:

Table 3-10: MBTRAIN States Description (sheet 1 of 6)

State	Description	Transition
VALVREF	Receiver reference voltage (Vref) to sample the incoming Valid is optimized in this state. The data rate on the mainband continues to be at the lowest supported data rate (4 GT/s). All data Lanes must be held low during Valid Lane reference voltage training. A pre-defined Receiver-initiated Data-to-Clock point test of 128 iterations of "VALTRAIN" Pattern is performed in this state which is 8'b11110000. Detection on a Receiver Lane is considered successful if "VALTRAIN" pattern detection errors are less than the set threshold. LFSR has no impact in this state as the data lanes are disabled.	**Transition to MBTRAIN.DATAVREF:** • If "VALTRAIN" pattern is detected successfully. • {end req} and {end resp} handshake is completed. **Transition to TRAINERROR:** • If "VALTRAIN" pattern test failed and errors exceeded the set error threshold. • After completing the TRAINERROR handshake. • If timeout occurs.

Table 3-11: MBTRAIN States Description (sheet 2 of 6)

State	Description	Transition
DATAVREF	Receiver reference voltage (Vref) to sample the incoming data is optimized in this state. The data rate on the UCIe Module mainband continues to be at the lowest supported data rate (4 GT/s). A pre-defined Receiver-initiated Data-to-Clock point test of 4K iterations of continuous mode LFSR Pattern is performed in this state. The LFSR pattern on Data must be accompanied by correct Valid framing on the Valid Lane.	**Transition to MBTRAIN.SPEEDIDLE:** • If LFSR pattern is detected successfully. • {end req} and {end resp} handshake is completed. **Transition to TRAINERROR:** • If LFSR pattern test failed and errors exceeded the set error threshold. • After completing the TRAINERROR handshake. • If timeout occurs.
SPEEDIDLE	This is an electrical idle state to allow frequency change.	**Transition to MBTRAIN.TXSELFCAL:** • {done req} and {done resp} handshake is completed. **Transition to TRAINERROR:** • If timeout occurs and after completing the TRAINERROR handshake.
TXSELFCAL	The UCIe Module calibrates its circuit parameters independent of the UCIe Module Partner.	**Transition to MBTRAIN.RXCLKCAL:** • {done req} and {done resp} handshake is completed. **Transition to TRAINERROR:** • If timeout occurs and after completing the TRAINERROR handshake.
RXCLKCAL	This state is used by the Module when ready to perform calibration on its Clock receive path.	**Transition to MBTRAIN.VALTRAINCENTER:** • When the required calibration (if any) is performed and {done req} and {done resp} handshake is completed. **Transition to TRAINERROR:** • If timeout occurs and after completing the TRAINERROR handshake.

Table 3-12: MBTRAIN States Description (sheet 3 of 6)

State	Description	Transition
VALTRAINCENTER	The **MBTRAIN.VALTRAINCENTER** state is used to ensure the valid signal is functional,so valid to clock training is performed before the data Lane training. The Receiver samples the pattern on Valid with the forwarded clock. A pre-defined Tansmitter-initiated Data-to-Clock point test of 128 iterations of "VALTRAIN" Pattern is performed in this state which is 8'b11110000. Detection on a Receiver Lane is considered successful if "VALTRAIN" pattern detection errors are less than the set threshold. LFSR has no impact in this state as the data lanes are disabled.	**Transition to MBTRAIN.VALTRAINVREF:** • If "VALTRAIN" pattern is detected successfully. • {done req} and {done resp} handshake is completed. **Transition to TRAINERROR:** • If "VALTRAIN" pattern test failed and errors exceeded the set error threshold. If timeout occurs and after completing the TRAINERROR handshake.

Table 3-13: MBTRAIN States Description (sheet 4 of 6)

State	Description	Transition
VALTRAINVREF	In this state, UCIe Module is permitted to optionally optimize the reference voltage (Vref) to sample the incoming Valid at the operating data rate. A pre-defined Receiver-initiated Data-to-Clock point test of 128 iterations of "VALTRAIN" Pattern is performed in this state which is 8'b11110000. Detection on a Receiver Lane is considered successful if "VALTRAIN" pattern detection errors are less than the set threshold. LFSR has no impact in this state as the data lanes are disabled.	**Transition to MBTRAIN.DATATRAINCENTER1:** • If "VALTRAIN" pattern is detected successfully. • {end req} and {end resp} handshake is completed. **Transition to TRAINERROR:** • If "VALTRAIN" pattern test failed and errors exceeded the set error threshold. • After completing the TRAINERROR handshake. • If timeout occurs.
DATATRAINCENTER1	In this state, the UCIe Module performs Data-to-Clock training (including valid). LFSR patterns must be used in this state. A pre-defined Transmitter-initiated Data-to-Clock point test of 4K iterations of continuous mode LFSR Pattern is performed in this state. The LFSR pattern on Data must be accompanied by correct Valid framing on the Valid Lane.	**Transition to MBTRAIN.DATATRAINVREF:** • If LFSR pattern test is succeeded. • {end req} and {end resp} handshake is completed. **Transition to TRAINERROR:** • If LFSR pattern test failed and errors exceeded the set error threshold. • After completing the TRAINERROR handshake. • If timeout occurs.

Table 3-14: MBTRAIN States Description (sheet 5 of 6)

State	Description	Transition
DATATRAINVREF	In this state, the UCIe Module is permitted to optionally optimize the reference voltage (Vref) on its data Receivers to optimize sampling of the incoming Data at the operating data rate. A pre-defined Receiver-initiated Data-to-Clock point test of 4K iterations of continuous mode LFSR Pattern is performed in this state. The LFSR pattern must be accompanied by Valid framing on the Valid Lane.	**Transition to MBTRAIN.RXDESKEW:** • If LFSR pattern test is succeeded. • {end req} and {end resp} handshake is completed. **Transition to TRAINERROR:** • If LFSR pattern test failed and errors exceeded the set error threshold. • After completing the TRAINERROR handshake. If timeout occurs.
RXDESKEW	The UCIe Module is permitted to optionally perform per Lane deskew on its Receivers to improve timing margin in this state. A pre-defined Receiver-initiated Data-to-Clock point test of 4K iterations of continuous mode LFSR Pattern is performed in this state. The LFSR pattern must be accompanied by Valid framing on the Valid Lane.	**Transition to MBTRAIN.DATATRAINCENTER2:** • If LFSR pattern test is succeeded. • {end req} and {end resp} handshake is completed. **Transition to TRAINERROR:** • If LFSR pattern test failed and errors exceeded the set error threshold. • After completing the TRAINERROR handshake. If timeout occurs.
DATATRAINCENTER2	This state is needed for the UCIe Module to recenter clock to aggregate data in case the UCIe Module Partner's Receiver performed a per Lane deskew. A pre-defined Transmitter-initiated Data-to-Clock point test of 4K iterations of continuous mode LFSR Pattern is performed in this state. The LFSR pattern must be accompanied by Valid framing on the Valid Lane.	**Transition to MBTRAIN.LINKSPEED:** • If LFSR pattern test is succeeded. • {end req} and {end resp} handshake is completed. **Transition to TRAINERROR:** • If LFSR pattern test failed and errors exceeded the set error threshold. • After completing the TRAINERROR handshake. If timeout occurs.

Table 3-15: MBTRAIN States Description (sheet 6 of 6)

State	Description	Transition
LINKSPEED	The **MBTRAIN.LINKSPEED** state is used by the UCIe Module to check the Link stability at the operating date rate. A pre-defined Transmitter-initiated Data-to-Clock point test of 4K iterations of continuous mode LFSR Pattern is performed in this state. The LFSR pattern on Data must be accompanied by correct Valid framing on the Valid Lane. The Receiver must be set up to perform per Lane comparison. Detection on a Receiver Lane is considered successful if total error count is less than the set threshold.	**Transition to LINKINIT:** • Triggered when the LFSR pattern test passes without errors. • Also occurs after sending and receiving the {MBTRAIN.LINKSPEED done resp} sideband message. **Transition to MBTRAIN.SPEEDIDLE:** • Occurs if errors are detected and lanes cannot be further width-degraded (already degraded), requiring speed reduction. • Also triggered after completing the SPEEDIDLE handshake sequence. **Transition to MBTRAIN.REPAIR:** • Initiated if errors occur and affected lanes can still be width degraded. • Also triggered upon receiving {MBTRAIN.LINKSPEED exit to repair resp} in response to a repair request. **Transition to TRAINERROR:** • Initiated after the TRAINERROR handshake or if a timeout occurs.
REPAIR	This state can be entered from **PHYRETRAIN** or from **MBTRAIN.LINKSPEED**. For Standard package, this state will be used for Link width degrade. For a x16 Standard package, if the Lanes encountering errors are all contained within the set of Lane 0 through Lane 7 or Lane 8 through Lane 15, the width must be degraded to a x8 Link using the set of Lanes without errors (Lane 0 ... Lane 7 OR Lane 8 ... Lane 15).	**Transition to MBTRAIN.TXSELCAL:** • If a degrade is possible or if all Lanes are functional. • If UCIe Module has sent and received the {MBTRAIN.REPAIR end resp} sideband message **Transition to TRAINERROR:** • If a “Degrade not possible” encoding is sent or received in the {MBTRAIN.REPAIR apply degrade req} SB message, the UCIe Module must exit to TRAINERROR after performing its handshake.. • If timeout occurs.

3.2. Controller:

3.2.1. RDI:

The RDI module handles communication between the physical layer and adapter layer in UCIE protocol. It manages state transitions, error handling, and packet exchanges according to the protocol specifications.
The RDI module is responsible for:

1. **State Management**:
   - Implement RDI state machine that responds to Adapter requests
   - Handles states: Nop, Active, LinkReset, Retrain, Disable, and LinkError
2. **Handshake Protocols**:
   - Stall request/acknowledge handshake
   - Wake request/acknowledge handshake
   - Clock request/acknowledge handshake
3. **Error Handling**:
   - Detects and processes link errors
   - Manages timeout conditions
   - Handles internal errors and packet errors
4. **Packet Processing**:
   - Processes incoming RDI packets
   - Generates appropriate response packets
   - Validates packet contents

RDI Module has an interface with RDI Interface, Controller, LTSM, Mainband, Sideband Transmitter, and Sideband Receiver as shown in figure 3-3:

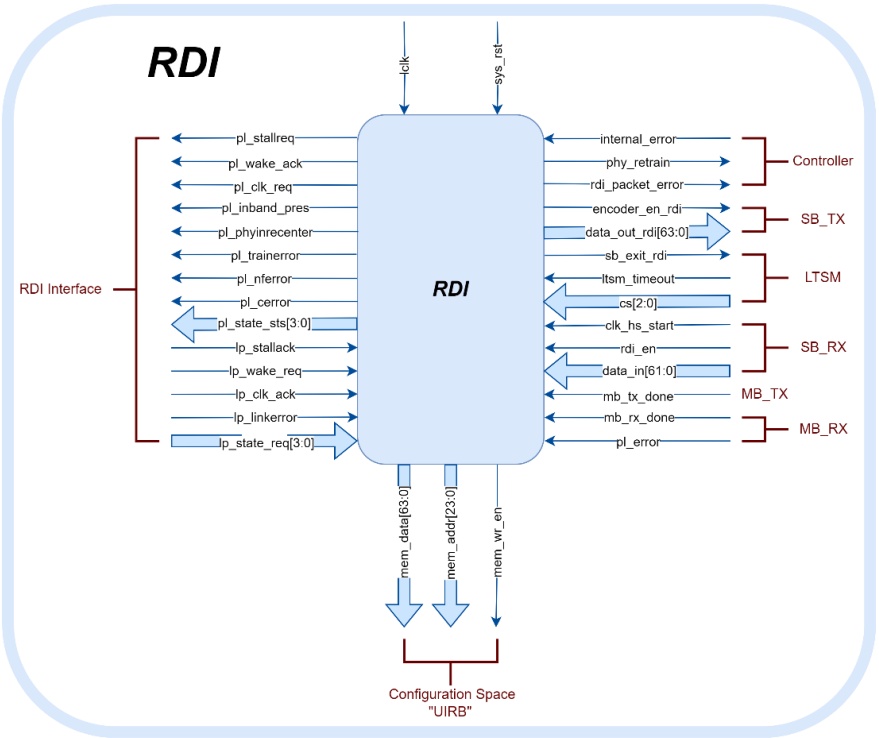


Figure 3-3: RDI Interface

All parameters and signals of the RDI top module are fully described in tables 3-16, 3-17, 3-18, and 3-19:

Table 3-16: RDI Parameters

Parameters	Width
DATA_WIDTH	64
ADDR_WIDTH	24

Table 3-17: RDI Signals Description (sheet 1 of 3)

Port	Direction	Width	Description
lclk	Input	1	From Interface Link clock signal.
sys_rst		1	From Interface Active-low asynchronous link reset signal.
internal_error		1	From Controller A signal that indicates an internal error condition detected within either the decoder_SBRX, phy_module, or phy_module_partner.
cs		3	From LTSM A state variable representing the current state of the main Link Training State Machine (LTSM). It is used to determine when the training sequence has completed.
ltsm_timeout		1	From LTSM Signal indicates that a timeout condition has occurred during the LTSM process without completing the required state transition.
clk_hs_start		1	From SB_RX Signal asserted to initiate the clock handshake with the adapter layer, requesting transition to normal clock mode.
rdi_en		1	From SB_RX Enable signal that activates rdi_module based on the decoded message type.
data_in		DATA_WIDTH-3	From SB_RX Decoded and validated data input.
mb_tx_done		1	From MB_TX A signal indicating that the Mainband Transmit (MB_TX) block has completed all pending operations in its pipeline and is ready to retrain.
mb_rx_done		1	From MB_RX A signal indicating that the Mainband Receiver (MB_RX) block has completed all pending operations in its pipeline and is ready to retrain.
pl_error		1	From MB_RX An indication from the Physical Layer to the Adapter that a framing-related error has been detected. This error is considered recoverable and triggers a transition to the RETRAIN state to re-establish link integrity.

Table 3-18: RDI Signals Description (sheet 2 of 3)

Port	Direction	Width	Description
lp_stallack	Input	1	From RDI Interface A signal from the Adapter to the Physical Layer indicating that the Flits have been aligned and transmission is stalled as requested.
lp_wake_req		1	From RDI Interface A request signal from the Adapter to the Physical Layer to disable clock gating on the Physical Layer's internal logic, effectively waking it up and enabling normal operation.
lp_clk_ack		1	From RDI Interface A response signal from the Adapter to the Physical Layer acknowledging that the clocks have been ungated and are now active.
lp_linkerror		1	From RDI Interface A signal from the Adapter to the Physical Layer indicating that a critical error has occurred, requiring the Link to transition to a Link Down state.
lp_state_req		4	From RDI Interface A 4-bit signal from the Adapter to the Physical Layer used to request a transition to a specific RDI state.
pl_stallreq	Output	1	To RDI Interface A signal from the Physical Layer to the Adapter requesting it to align the Transmitter at a Flit boundary and stall further Flit transmissions.
pl_wake_ack		1	To RDI Interface A response signal from the Physical Layer to the Adapter acknowledging that its internal clocks have been ungated and normal operation has resumed.
pl_clk_req		1	To RDI Interface A request signal from the Physical Layer to the Adapter to disable clock gating on the Adapter's internal logic, enabling clocks and allowing normal operation.
pl_inband_pres		1	To RDI Interface A signal from the Physical Layer to the Adapter indicating that the Die-to-Die has completed training successfully and is ready for the RDI transition to the Active state.
pl_phyinrecenter		1	To RDI Interface A signal from the Physical Layer to the Adapter indicating that the Physical Layer is currently performing training or retraining procedures to re-establish or optimize link integrity.

Table 3-19: RDI Signals Description (sheet 3 of 3)

Port	Direction	Width	Description
pl_trainerror	Output	1	To RDI Interface A signal asserted by the Physical Layer indicates the occurrence of a fatal error during link training or operation.
pl_nferror		1	To RDI Interface A signal from the Physical Layer to the Adapter indicating that a non-fatal error has been detected.
pl_cerror		1	To RDI Interface A signal from the Physical Layer to the Adapter indicating that a correctable error has been detected. This error does not impact on the data path and does not require a link retrain.
pl_state_sts		4	To RDI Interface A 4-bit status signal from the Physical Layer to the Adapter indicating the current state of the RDI interface.
mem_data		DATA_WIDTH	To Configuration Space A data bus carrying the information to be written into the configuration space for system setup or control purposes.
mem_addr		ADDR_WIDTH	To Configuration Space The address specifying the location in configuration space where the data is to be written.
mem_wr_en		1	To Configuration Space A control signal that enables writing mem_data to the specified mem_addr in the configuration space when asserted.
sb_exit_rdi		1	To LTSM Signal indicating that all necessary sideband state transitions have completed, allowing the state machine to proceed to the next state.
encoder_en_rdi		1	To SB_TX An enable signal that indicates data is valid and ready for transmission over the SB_TX.
data_out_rdi		DATA_WIDTH	To SB_TX The sideband message data to be transmitted from the current die to the remote die over the SB_TX.
rdi_packet_error		1	To Controller A signal indicating that an **error was detected** in a received **RDI sideband packet.**
phy_retrain		1	To Controller A signal indicating that the RDI is currently in the RETRAIN state. It acts as a flag to the LTSM to initiate a transition into the RETRAIN state.

Description of RDI States is in tables 3-20, 3-21, and 3-22:

RDI States Description:

Table 3-20: RDI States Description (sheet 1 of 3)

State	Description	Transition
Reset	The **Reset** state is a fundamental starting point for the Physical Layer in the UCIe link initialization process. It is entered upon de-assertion of the interface-level reset signal or as a result of transitions from the LinkReset, Disabled, LinkError, or L2 states. **In the Reset state:** • The Physical Layer is permitted to begin its internal initialization or training procedures. • The pl_state_sts signal remains asserted to indicate the Physical Layer is in the Reset state. • Exit from Reset is strictly controlled and can only occur after the Upper Layer (Adapter) explicitly requests a new state by changing the lp_state_req signal from NOP to a valid next-state encoding.	**Transition to Active:** • Occurs when lp_state_req holds value NOP for at least one clock cycle while pl_state_sts indicates Reset, and then changes to Active. • After successful completion of Active Entry handshakes, which vary by layer: ▪ Physical Layer: Exchanged Active Response sideband messages. ▪ Adapter Layer: Exchanged Active Status messages. **Transition to LinkReset/Disabled:** • When lp_state_req holds NOP for at least one cycle and then becomes LinkReset/Disabled. • if **remote Link partner** explicitly requests this transition via a sideband message. **Transition to LinkError:** • An internal error condition that maps to LinkError. • The assertion of the lp_linkerror signal by the Adapter. • A **remote LinkError request** received through a sideband message.
Active	The **Active** state represents the normal operational mode of the UCIe Link, where data transfers take place over the mainband and sideband interfaces. All transmitters and receivers are fully enabled, clock gating is disabled, and Flit exchange is fully functional.	**Transition to Retrain:** • lp_state_req == Retrain An internal retrain condition is detected while pl_state_sts == Active.
Retrain	The **Retrain** state is used to recover from link errors, or software-triggered resets. It allows the UCIe Link to re-establish synchronization and framing integrity without requiring a full LinkReset or disable. The **Retrain mechanism** ensures that errors can be corrected dynamically.	**Transition to Active:** • Active Entry handshakes begin **only after a NOP → Active** transition is detected on lp_state_req. • Transition completes once **Active handshakes** are exchanged **between Physical Layers.** **Transition to LinkReset / Disabled:** • If lp_state_req == LinkReset or lp_state_req == Disabled during Retrain

Table 3-21: RDI States Description (sheet 2 of 3)

State	Description	Transition
Active	The **Active** state represents the normal operational mode of the UCIe Link, where data transfers take place over the mainband and sideband interfaces. All transmitters and receivers are fully enabled, clock gating is disabled, and Flit exchange is fully functional.	**Transition to Retrain:** • lp_state_req == Retrain • An internal retrain condition is detected while pl_state_sts == Active.
Retrain	The **Retrain** state is used to recover from link errors, protocol-level issues, or software-triggered resets. It allows the UCIe Link to re-establish synchronization and framing integrity without requiring a full LinkReset or disable. The **Retrain mechanism** ensures that errors can be corrected dynamically, maintaining link robustness and reducing the need for full reinitialization.	**Transition to Active:** • Active Entry handshakes begin **only after a NOP → Active** transition is detected on lp_state_req. • Transition completes once **Active handshakes** are exchanged **between Physical Layers.** **Transition to LinkReset / Disabled:** • If lp_state_req == LinkReset or lp_state_req == Disabled during Retrain
LinkReset	The **LinkReset** state serves as a mechanism to reset the UCIe Link and its connected devices, similar to the **HotReset** behavior in PCIe. It ensures that both Protocol Layer and Adapter stacks are cleanly reset while allowing for the coordinated propagation of reset requests across the interface.	**Transition to Reset:** • A **reset pin or internal reset condition** is detected. • lp_state_req == Active while pl_state_sts == LinkReset and **all LinkReset operations are complete**. **Transition to Disabled:** • lp_state_req == Disabled. • An internal request to move to the Disabled state. **Transition to LinkError:** • An internal fatal error request. • Assertion of lp_linkerror while pl_state_sts == LinkReset.
Disabled	The **Disabled** state is used to gracefully shut down the UCIe Link without requiring a full hardware reset. It is typically used when the Link is intentionally turned off due to software, protocol-level, or implementation-specific requests.	**Transition to Reset:** • An internal **reset event** (e.g., a reset pin trigger). • Observation of lp_state_req == Active while pl_state_sts == Disabled, **after all cleanup actions** are complete. **Transition to LinkError:** • An **internal fatal error condition.** • Assertion of lp_linkerror while in the Disabled state.

Table 3-22: RDI States Description (sheet 3 of 3)

State	Description	Transition
Linkerror	The **LinkError** state indicates a critical fault condition on the UCIe Link, typically caused by uncorrectable errors or explicit requests from the Adapter or remote Link partner. It is designed to isolate and halt Link activity until software or reset mechanisms resolve the issue.	**Transition to Reset:** • **Internal reset** occurs (e.g., software clears error flags or reset pin is asserted). • lp_state_req == Active **and** lp_linkerror == 0 while in LinkError. • **Minimum 16 ms residency** in LinkError is met. • No internal condition prevents leaving LinkError.

Based on the functional architecture of the RDI-based Physical and Adapter Layer interaction, several critical handshake mechanisms are implemented to ensure safe, synchronized, and protocol-compliant transitions during various operational states.
These handshakes serve as synchronization points between the Adapter and the Physical Layer to manage clock gating, Flit transfer alignment, and state transitions.

The supported handshakes:
Stallreq/Ack Handshake
Used to gracefully halt Flit transfers at Flit boundaries during transitions from Active to Retrain, PM, LinkReset, or Disabled states.
It follows a 4-phase sequence:
pl_stallreq ↑ → lp_stallack ↑ → pl_stallreq ↓ → lp_stallack↓
Ensures Flit alignment and prevents protocol violations during link transitions.

Wake Req/Ack Handshake
Initiated by the Adapter via lp_wake_req to request clock ungating.
The PHY responds with pl_wake_ack once clk is ungated.
Used during transitions from clock-gated states (e.g., before lp_state_req, lp_cfg, or lp_linkerror).
Requires proper assertion timing and deassertion ordering to ensure safe operation.

Clock Req/Ack Handshake
Initiated by the PHY via pl_clk_req to ungate Adapter clocks.
The Adapter replies with lp_clk_ack when clocks are ready.
Mandatory before PHY-driven transitions (e.g., pl_inband_pres, or moving to LinkError/Active).
Requires proper edge spacing, ordered deassertion, and coordination with pl_state_sts.

3.2.2. Phy_Module:

The **Phy_Module** serves as the physical layer module controller responsible for initiating and managing communication with a remote die over the sideband link.
Its core responsibilities include:

1. **Sideband Message Transmission and Reception:**
    - Sends all required sideband request messages according to the current link training and management protocol.
    - Receives and validates response messages, ensuring the integrity and correctness of packet contents relevant to the various link states.
    - Monitors message exchanges across multiple finite state machines, including:
        - Link Training State Machine (LTSM)
        - Main Band Initialization FSM (MBINIT)
        - Main Band Training FSM (MBTRAIN)
2. **State Completion Signaling:**
    - Generates signals to indicate the completion of all necessary message exchanges for each state.
    - Coordinates state transitions based on successful message validation and timing requirements.
3. **Configuration Parameter Extraction and Memory Control:**
    - Extracts configuration parameters from incoming sideband messages.
    - Writes these parameters to the designated memory region in configuration space.
    - Controls memory access permissions and timing to ensure proper configuration updates.
4. **Mainband Test Initiation:**
    - Commands the Main Band (MB) subsystem to start relevant tests for each state of the link training and operation.
    - Manages test enabling signals to synchronize MB tests with sideband messaging sequences.
5. **Physical Layer Adaptation:**
    - Controls the Main Band transmitter behavior to:
        - Reverse lane polarity if required by link conditions.
        - Degrade link width adaptively to match the capabilities or status of the remote die.
    - Dynamically communicates the maximum supported link speed and manages speed degradation for compatibility or repair purposes.
6. **Mode Control for Decoder:**
    - Determines and signals the decoder of sideband transmitter decoder with mode based on the current operational state.
    - Enables the decoder to correctly interpret sideband messages during initialization and active state.
7. **Error Detection and Recovery:**
    - Detects packet errors and invalid message sequences during sideband exchanges.
    - Triggers link repair procedures or retraining sequences when error conditions are identified.
    - Maintains flags to manage speed degradation and repair requirements in a robust manner.

Phy_Module module has an interface with Phy_Module_Partner, RDI, RDI_Interface, Configuration Space, Mainband, Sideband, and LTSM as shown in figure 3-4:

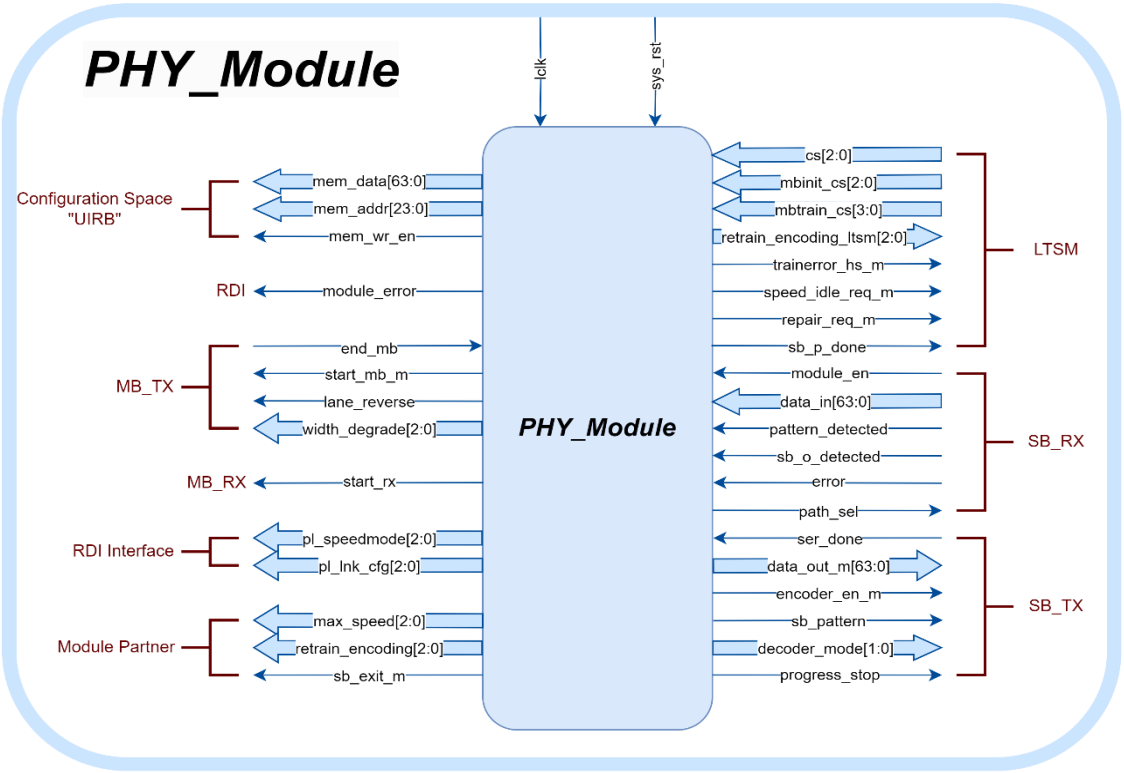


Figure 3-4: Phy_Module Interface

All parameters and signals of the Phy_Module top module are fully described in tables 3-23, 3-24, 3-25, 3-26, and 3-27:

Table 3-23: Phy_Module Parameters

Parameters	Width
DATA_WIDTH	64
ADDR_WIDTH	24

Table 3-24: Phy_Module Signals Description (sheet 1 of 4)

Port	Direction	Width	Description
lclk	Input	1	From Interface Link clock signal
sys_rst		1	From Interface Active-low link reset signal
cs		3	From LTSM A state variable representing the current state of the main LTSM.
mbinit_cs		3	From LTSM A state variable representing the current sub-state of the MBINIT FSM.
mbtrain_cs		4	From LTSM A state variable representing the current sub-state of the MBTRAIN FSM.
module_en		1	From SB_RX Enable signal that activates phy_module based on the decoded message type.

Table 3-25: Phy_Module Signals Description (sheet 2 of 4)

Port	Direction	Width	Description
data_in	Input	DATA_WIDTH	From SB_RX Decoded and validated data input.
pattern_detected		1	From SB_RX Flag asserted on detection of a fixed pattern twice synchronized to lclk.
sb_o_detected		1	From SB_RX Flag asserted on detection of SBINIT_out_of_reset packet synchronized to lclk.
error		1	From SB_RX A flag set when a parity error, message format error, or unrecognized message type is detected, indicating invalid data.
ser_done		1	From SB_TX Signal indicating that all sideband transmissions have been successfully completed.
end_mb		1	From MB_TX Signal indicating that the MB_TX block has completed the test related to the current state.
retrain_encoding_ltsm	Output	3	To LTSM Encoding that indicates the target state to transition to from the RETRAIN state.
trainerror_hs_m		1	To LTSM Signal indicating that the handshake for entering the TRAINERROR state has completed.
speed_idle_req_m		1	To LTSM Signal that requests the LTSM to enter SPEEDIDLE state.
repair_req_m		1	To LTSM Signal that requests the LTSM to enter REPAIR state.
sb_p_done		1	To LTSM Signal indicating that all in-progress sideband packets have finished transmission before entering the RESET state.
path_sel		1	To SB_RX Control signal used to demultiplex the output data from the Deserializer module. Based on its value, the data is routed either to the Pattern Detector block or the Asynchronous FIFO for further processing or storage.

Table 3-26: Phy_Module Signals Description (sheet 3 of 4)

Port	Direction	Width	Description
data_out_m	Output	DATA_WIDTH	To SB_TX The sideband message data to be transmitted from the current die to the remote die over the SB_TX.
encoder_en_m		1	To SB_TX An enable signal that indicates data is valid and ready for transmission over the SB_TX.
sb_pattern		1	To SB_TX Control signal that triggers the SB_TX block to begin transmitting the **SBINIT pattern** or the **SBINIT_out_of_reset message.**
decoder_mode		2	To SB_TX 2-bit signal Indicates the current operation performed by the decoder_SB_TX module, based on current state.
progress_stop		1	To SB_TX Control signal used to allow the completion of all in-progress sideband transmissions.
module_error		1	To RDI A signal that indicates an internal error detected within phy_module.
start_rx		1	To MB_RX Signal used to trigger MB_RX to begin executing the test associated with the current state.
start_mb_m		1	To MB_TX Signal used to trigger MB_TX to begin executing the test associated with the current state.
lane_reverse		1	To MB_TX Signal that indicates MB_TX should reverse its lane ordering.
width_degrade		3	To MB_TX A bus that indicates the currently active lane width for MB_TX.
pl_speedmode		3	To RDI Interface A bus that indicates the current operational link speed.
pl_lnk_cfg		3	To RDI Interface A bus that indicates the current link configuration negotiated by the Physical Layer.

Table 3-27: Phy_Module Signals Description (sheet 4 of 4)

Port	Direction	Width	Description
pl_lnk_cfg	Output	3	To RDI Interface A bus that indicates the current link configuration negotiated by the Physical Layer.
max_speed		3	To Phy_Module_Partner A bus that indicates the **maximum link speed** capability supported by the phy_module.
retrain_encoding		3	To Phy_Module_Partner A multi-bit bus that encodes the state that should follow the RETRAIN state, as determined by the outcome of the current test.
sb_exit_m		1	To Phy_Module_Partner Signal indicating that all necessary sideband state transitions for phy_module have completed, allowing the state machine to proceed to the next state.
mem_data		DATA_WIDTH	To Configuration Space A data bus carrying the information to be written into the configuration space for system setup or control purposes.
mem_addr		ADDR_WIDTH	To Configuration Space The address specifying the location in configuration space where the data is to be written.
mem_wr_en		1	To Configuration Space A control signal that enables writing mem_data to the specified mem_addr in the configuration space when asserted.

3.2.3. Phy_Module_Partner:

This block in the local die is mainly responsible for receiving the Side-Band Messages requests from the remote die and transmits back their responses.

It is considered the main controller in the LTSM states transition in nearly all its states except the LINKINIT and ACTIVE states where their transitions are controlled by the RDI.

It contributes with the Main-Band Transmitter (MB_TX) in any Receiver-initiated Data-to-Clock point tests at any state, where in this test, it receives a request from the remote die requesting to test its receiver lanes which are tested by receiving tests on it, so the Module Partner has to trigger the local die Main-Band Transmitter to enable its test pattern generators and start the test.

While it contributes with the Main-Band Receiver (MB_RX) in any Transmitter-initiated Data-to-Clock point tests at any state, where in this test, it receives a request from the remote die requesting to test its transmitter lanes which are tested by transmitting tests through them, so the Module Partner has to trigger the local die Main-Band Receiver to enable its test pattern comparators and be ready for the incoming test.

If an {TRAINERROR Entry req} Side-Band Message has been received at any time, this block will respond with the {TRAINERROR Entry resp} Side-Band Message at once and raises the "trainerror_hs_mp" to the LTSM triggering it to transition from the current state to TRAINERROR state.

Main block role in the PHY Layer of the UCIe Protocol:

- This block mainly interfaces with the **Main-Band Receiver (MB_RX)** as it is considered as a PHY Controller receiver which responds to the incoming requests most of the time.
- It is also responsible for the states transition in the **LTSM** by asserting the pulse signal **"sb_exit_mp"** which indicates that all the done or end handshakes of **Module** and **Module Partner** has finished successfully.
- This module is responsible for transmitting the different tests results along in the Side-Band message at different states. It receives these results from the **Main-Band Receiver (MB_RX)** through the signals **"valid_frame_error"**, **"detec_succ"** and **"faulty_lanes"**.
- It is also responsible for extracting the maximum error threshold supported from the input Side-Band message and pass it to the **Main-Band Receiver** through the signal **"error_threshold"** to use this error threshold in detecting the success of the input test where this happens if the total error count is less than the set error threshold.
- It is responsible for comparing the input local die data rate coming from the Controller Module Block in signal **"max_speed"** with that of the remote die coming through the Side-Band Message {MBINIT.PARAM configuration req}, then sends back the minimum between them through the Side-Band Message {MBINIT.PARAM configuration resp} in order for both dies to work on it.
- We can say that its only interface with the **Main-Band Transmitter (MB_TX)** is through the signals **"start_mb_mp"** which trigger the MB_TX to start sending the pattern test determined for the current state, and **"end_mb"** which is an input it waits on and indicates that the MB_TX has finished transmitting the test in order for further completing exchanging the rest of the Side-Band Messages.
- For the MB_RX, it extracts the active lanes encoding from the input Side-Band Messages **{MBINIT.REPAIRMB apply degrade req}** or **{MBTRAIN.REPAIR Apply degrade req}** and bypasses it to the MB_RX to applied on its receiver lanes.
- It also triggers the MB_RX to be ready to receive the test by asserting the signal **"test_resp_mp"** to allow the MB_RX to start and enable its comparison circuits.

- This module plays an important role in comparing the incoming Retrain Encoding through the Side-Band message **{PHYRETRAIN.retrain start req}** from the remote die (Die 1) with its local die (Die 0) Retrain Encoding coming from the Controller Module Block in signal **"retrain_encoding"**, then returns back the resolved Retrain Encoding through the Side-Band Message **{PHYRETRAIN.retrain start resp}** based on the following table which is taken from the UCIe specs document which is necessary for exiting from the PHYRETRAIN state to either one of the three states, TXSELFCAL, REPAIR or SPEEDIDLE:

Retrain reg Condition		Retrain Request Encoding		Resolved State Encoding		Exit
Die 0	Die 1	Die 0	Die 1	Die 0	Die 1	Both Dies
No Lane Errors	No Lane Errors	001b	001b	001b	001b	MBTRAIN.TXSELFCAL
No Lane Errors	Repair	001b	100b	100b	100b	MBTRAIN.REPAIR
No Lane Errors	Speed Degrade	001b	010b	010b	010b	MBTRAIN.SPEEDIDLE
Repair	No Lane Errors	100b	001b	100b	100b	MBTRAIN.REPAIR
Repair	Repair	100b	100b	100b	100b	MBTRAIN.REPAIR
Repair	Speed Degrade	100b	010b	010b	010b	MBTRAIN.SPEEDIDLE
Speed Degrade	No Lane Errors	010b	001b	010b	010b	MBTRAIN.SPEEDIDLE
Speed Degrade	Repair	010b	100b	010b	010b	MBTRAIN.SPEEDIDLE
Speed Degrade	Speed Degrade	010b	010b	010b	010b	MBTRAIN.SPEEDIDLE

Figure 3-5: Retrain Exit State Resolution

Figure 3-6 shows the overall top architecture of the Module Partner as a black box with only its interface inputs/outputs signals with the outside world:

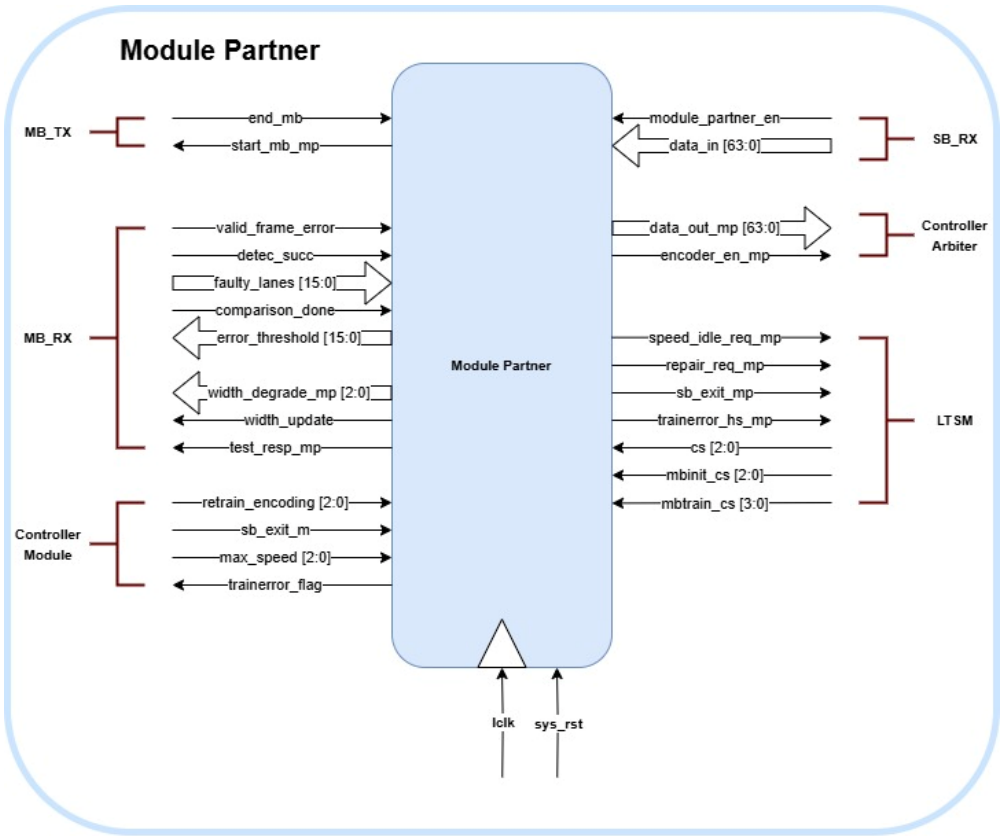


Figure 3-6: PHY Module Partner Interface

All parameters and signals of the Phy_Module_Partner top module are fully described in tables 3-28, and 3-29:

Table 3-28: Phy_Module_Partner Patameters

Parameters	Width	Description
DATA_WIDTH	64	Defines the data bus width of input and output Side-Band message from the Module Partner Block.

Table 3-29: Phy_Module_Partner Signals Description (sheet 1 of 2)

Port	Direction	Width	Description
lclk	Input	1	System clock domain signal.
sys_rst		1	Asynchronous active-low reset synchronized with the lclk domain.
module_partner_en		1	From SB_RX Decoder Block and indicates that there's an input data dedicated to the Module Partner.
data_in		Parametrized (DATA_WIDTH)	From SB_RX Decoder Block and indicates the input data SB msg.
cs		3	From LTSM and indicates the current state of LTSM main states.
mbinit_cs		3	From LTSM and indicates the current sub-state of MBINIT.
mbtrain_cs		4	From LTSM and indicates the current sub-state of MBTRAIN.
end_mb		1	From MB_TX and indicates that it had finished transmitting the test.
valid_frame_error		1	From MB_RX and indicates that there's an error detected during the VALTRAIN pattern test.
detec_succ		1	From MB_RX and indicates that the VALTRAIN pattern test has passed successfully with no errors.
faulty_lanes		16	From MB_RX and indicates if lanes in Per-Lane ID or LFSR tests passed or not.
comparison_done		1	From MB_RX and when asserted it indicates that the "faulty_lanes" signal has a valid value.
sb_exit_m		1	A signal from the Controller Module Block and determines if it has finished the SB msgs handshakes or not in order for the Module Partner to assert its "sb_exit_mp" if this signal is found asserted or to wait on it till it got asserted.
retrain_encoding_m		3	From Controller Module Block indicating the local die retrain encoding.
max_speed		3	From Controller Module Block indicating the local die data rate.
sb_exit_mp	Output	1	A signal going to the LTSM after both Module & Module Partner finishing their done SB msgs handshakes to allow states transition in LTSM.

Table 3-30: Phy_Module_Partner Signals Description (sheet 2 of 2)

Port	Direction	Width	Description
trainerror_hs_mp	Output	1	To LTSM to transition to TRAINERROR state after completing its entry SB msgs handshake.
trainerror_flag		1	To Module Block to make it start the trainerror entry handshake and it indicates if a wrong or unexpected or supposed msg has been sent to the Module Partner.
error_threshold		16	To MB_RX and determines the receiver error threshold.
start_mb_mp		1	To MB_TX to trigger it to start any test at different states.
width_degrade_mp		3	To MB_RX and indicates the local die receiver active logical lanes encoding.
width_update		1	A pulse signal to MB_RX to indicate that the “width_degrade_mp” signal has a valid encoding to register it.
data_out_mp		Parametrized (DATA_WIDTH)	To Controller Arbiter and holds the SB msg that will be sent through the SB_TX from the local die to the remote die.
encoder_en_mp		1	To Controller Arbiter to indicate that the Module Partner wants to send a valid msg in “data_out_mp” data bus signal.
speed_idle_req_mp		1	To LTSM to transition to SPEEDIDLE state after completing the handshake.
repair_req_mp		1	To LTSM to transition to REPAIR state after completing the handshake.
test_resp_mp		1	A pulse signal going to the MB_RX to make it know that a test has started and it will be receiving it soon.

3.2.4. Controller Arbiter:

Block Functionality

The **Controller Arbiter** serves as the Central Interface between the Controller's sub-modules—**Phy_Module**, **Module_Partner**, and **RDI_Module**. It handles write requests to the **SBTX Asynchronous FIFO**, coordinating with the **priority encoder** inside the SBTX block to manage message delivery into the FIFO. In addition to arbitration, the Controller Arbiter also functions as an **Information Insertion Entity**, responsible for appending common message fields such as **Source ID**, **Destination ID**, and computing **control and data parity bits**. This design promotes **hardware reuse**, as the parity bits are generated using shared **XOR logic** rather than being redundantly computed by each sub-module.

This architectural decision results in a **two-level arbitration structure**: one arbiter within the Controller and another within the SBTX block. The Controller Arbiter is implemented as a **finite state machine (FSM)**, with dedicated states for each sub-module. Crucially, it remains in the state corresponding to a requesting module until it completes transmission of both the **message header** and, if present, the **message data phase—ensuring proper serialization and transmission sequencing of multi-phase messages.**

Top Architecture

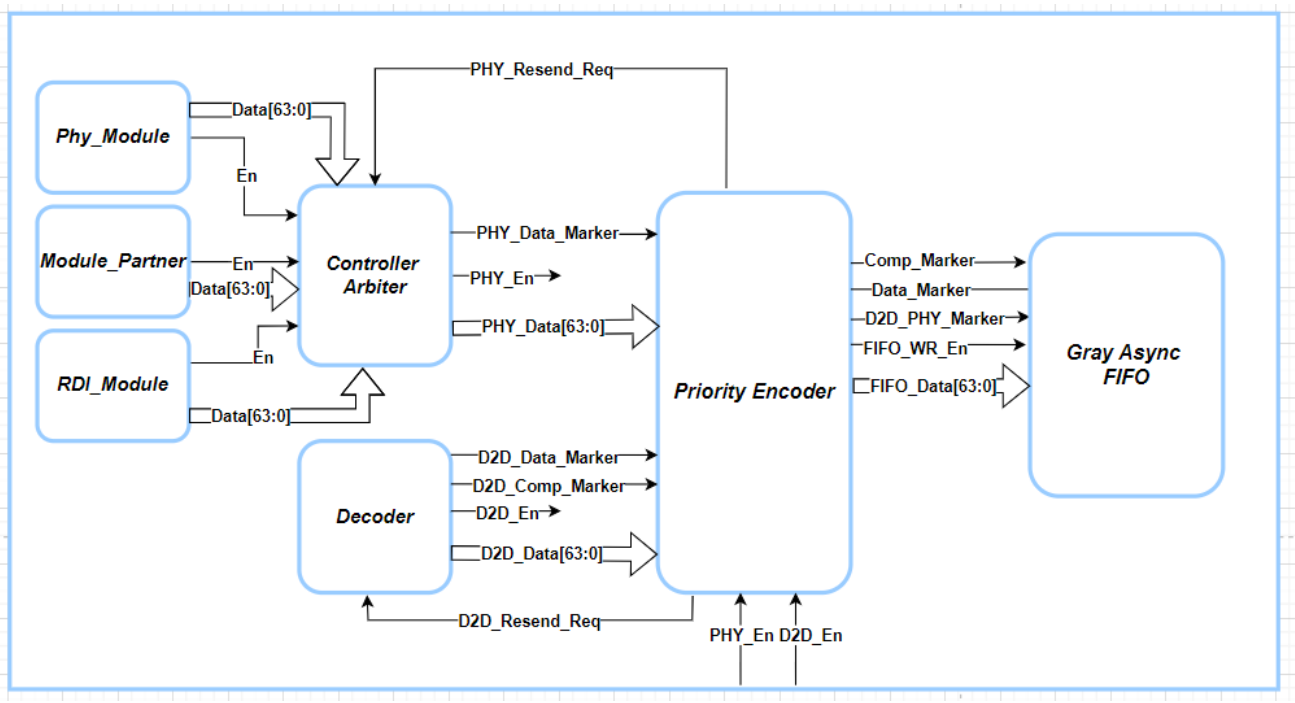


Figure 3-7: Two Level Arbitration

3.3. Sideband "SB":

3.3.1. Sideband Transmitter "SB_TX":

Block Functionality

The **Sideband Transmitter (SBTX)** serves as the **transmission endpoint of the system's control path**, dedicated to handling configuration and management traffic between dies. It plays a critical role in ensuring correct protocol behavior throughout both the **Bring-Up** and **operational** phases. Operating across **two asynchronous clock domains**, the SBTX is architected to reliably serialize and forward system messages while maintaining robust synchronization, protocol integrity, and system responsiveness.

Functional Summary:

- **Bring-Up Pattern Generation**
  Generates the **SBINIT pattern** and continuously transmits the "**SBINIT_out_oF_Reset**" sideband message until successful handshake with the remote die.

- **Safe Handling During Initialization**
  Detects **Remote Register Access Requests (RARs)** during reset or SBINIT phases and responds with **Unsupported Request (UR) completions**, **preserving credit flow integrity**.

- **Message Validation and Error Reporting**
  Receives messages from the **D2D layer**, performs **parity checks**, and either triggers the RDI's corrective path or proceeds with message decoding.

- **Message Classification and Dispatch**
  Differentiates between **remote messages** (forwarded to FIFO for SB transmission) and **local RARs** (serviced through direct interaction with the configuration space—**UIRB**).

- **Priority-Based Ingress Arbitration**
  Utilizes a **Priority Encoder** to manage message insertion into the FIFO, resolving contention between messages from the **Controller** and **D2D layer**.

- **Credit Management System**
  Tracks and returns **credits** for D2D messages only, maintaining credit discipline exclusively between the D2D layer and SBTX, while excluding Controller traffic from the credit flow.

Top Architecture

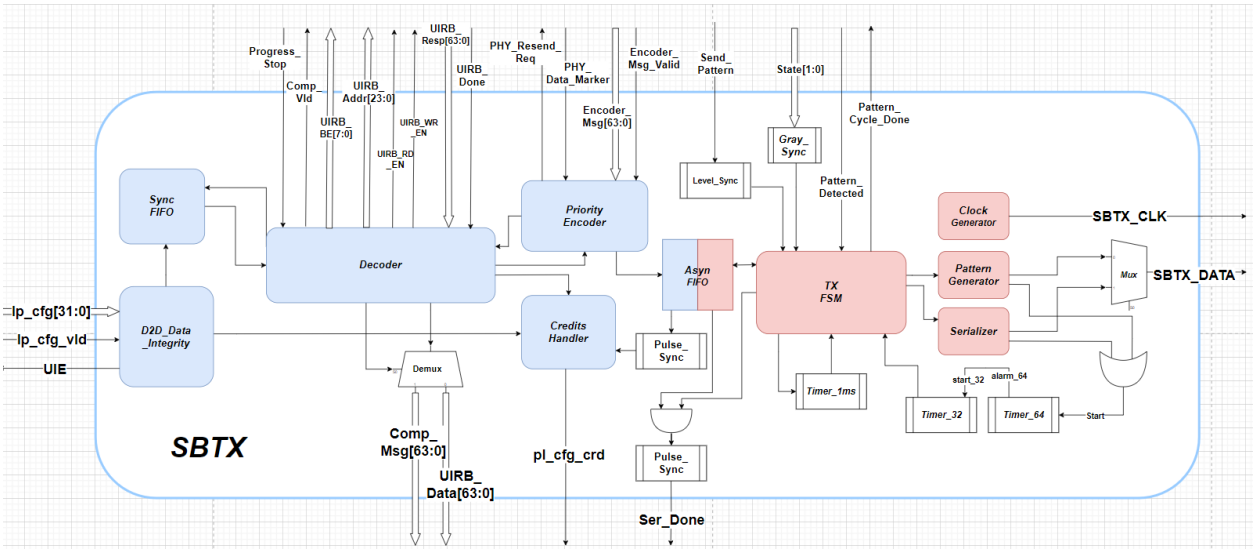


Figure 3-8: Side Band Transmitter Top Architecture

The SBTX module comprises multiple sub-blocks, with some operating in the LCLK domain and others in the SB_CLK domain.

LCLK Domain sub-blocks:

- D2D_Data_Integrity
- Synchronous FIFO
- Decoder
- Priority_Encoder
- Credits_Handler

SB_CLK Domain sub-blocks:

- TX_FSM
- SB CLK & Data Lanes Handler

Bridge: Gray Asynchronous FIFO.

Signals Interface

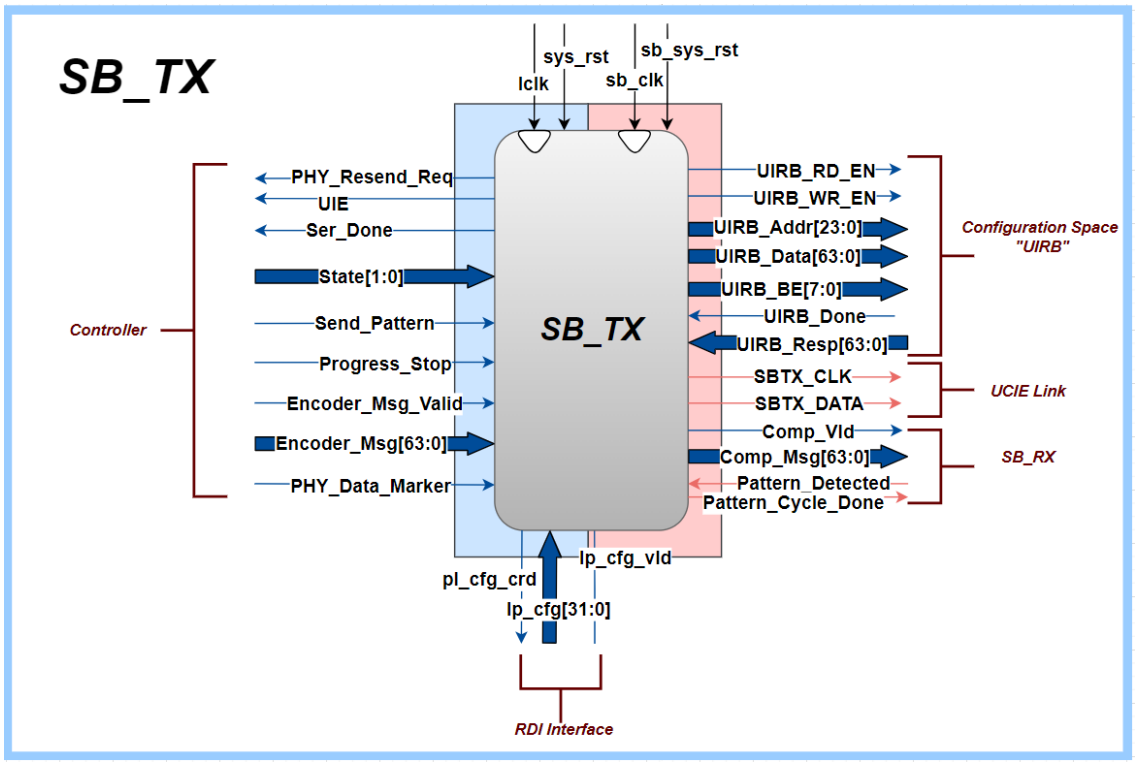


Figure 3-9: Side Band Transmitter Signals Interface

Table 3-31: SBTX Parameters

Parameter	Width
NC	32
DATA_WIDTH	64
ADDR_LENGTH	24
BE_LENGTH	8
STATE_WIDTH	2

Table 3-32: SBTX Top Level Interface Signals (sheet 1 of 2)

Top-Level Interface Signals				
Source/ Destination	**Signal**	**Direction**	**Width**	**Description**
UCIe Link	SBTX_CLK	Output	1	Data & Pattern are serialized on this lane to the Die Partner.
	SBTX_DATA	Output	1	Strobe that is accompanied with Data or Pattern serialized on SBTX_DATA lane.
RDI Interface	Pl_cfg	Output	NC	Represents one of the (2 or 4) phases of the D2D Message.
	Pl_cfg_vld	Output	1	When asserted, the data on the pl_cfg bus is valid.
	Pl_cfg_crd	Output	1	When asserted, one credit returned to the D2D layer.

Table 3-33: SBTX Top Level Interface Signals (sheet 2 of 2)

Source/ Destination	Signal	Direction	Width	Description
Configuration Space "UIRB"	UIRB_RD_EN	Output	1	The RAR is a Read request.
	UIRB_WR_EN	Output	1	The RAR is a Write request.
	UIRB_Addr	Output	ADDR_ LENGTH	The address of the register in the UIRB.
	UIRB_Data	Output	DATA_ WIDTH	Data to be written in the Register specified by the address bus in the UIRB when the RAR is a write request.
	UIRB_BE	Output	BE_ LENGTH	The Bytes to be Written in the Register from the forwarded 8 Bytes (Bit Making).
	UIRB_Done	Input	1	The Write/Read operation is done.
	UIRB_Resp	Input	DATA_ WIDTH	The Read Data from the Register specified by the address bus in the UIRB when the RAR is a Read request.

Table 3-34: SBTX Inter-Module signals (sheet 1 of 2)

Inter-Module Signals				
Source /Destination	**signal**	**Direction**	**Width**	**Description**
SBRX	Comp_vld	output	1	When asserted, the Message on the Comp_Mess bus is valid.
	Comp_Mess	output	DATA_WIDTH	Completion Message for the local RAR (Register Access Requests).
	Pattern_Cycle Done	output	1	It indicates that the 4 iterations sent after detecting the pattern on the Receiver Lane were successfully completed.
	Pattern_Detected	Input	1	Indicates that the pattern sent by the Die partner was detected twice on the Receiver Lane.
Controller	PHY_Resend_Req	output	1	Signal indicates that the Controller should resend its message again as the Priority_Encoder is currently handling the Decoder.
	Ser_Done	output	1	When asserted, it means that the FIFO is empty, and the last message is successfully serialized.

Table 3-35: SBTX Inter-Module signals (sheet 2 of 2)

Source /Destination	signal	Direction	Width	Description
Controller	UIE	output	1	When asserted, it means that a control or data parity error was detected in the D2D Message on RDI Interface.
	State	Input	STATE_WIDTH	LTSM State Encodings relevant to SBTX Operation.
	Send_Pattern	Input	1	When asserted, the pattern is transmitted continuously until it is detected by the remote side; it is then sent four additional times, after which the "SBINIT_out_of_Reset" message is transmitted repeatedly until this signal is de-asserted.
	Progress_Stop	Input	1	When asserted, this signal instructs the SBTX to halt reception and writing of messages from the Decoder, allowing the FIFO to drain in preparation for a safe transition from the ACTIVE state to a PHY training state.
	PHY_Data_Marker	Input	1	When asserted, it means that the Controller Message of type Data (has Data part).
	Encoder_Mess	Input	DATA_WIDTH	The Controller Message.
	Encoder_Mess_Valid	Input	1	When asserted, the Controller is requesting to write a message in the Async. FIFO.

Detailed Architecture

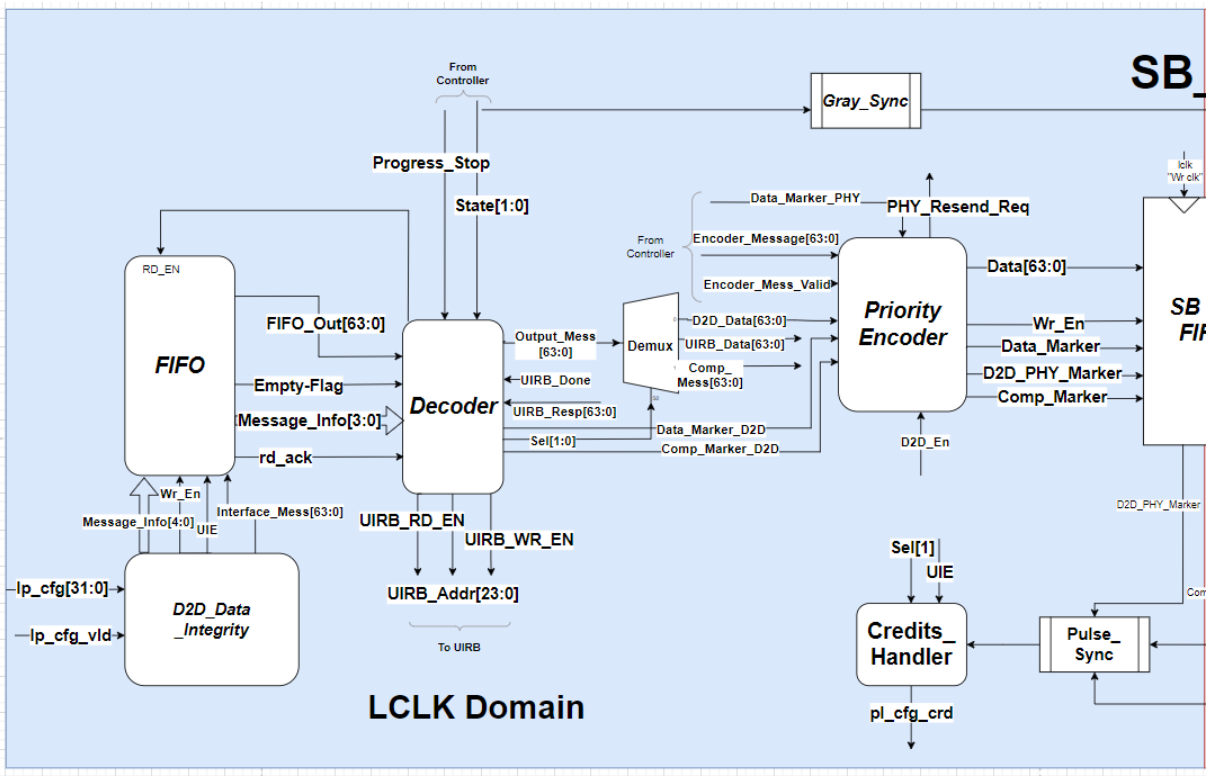


Figure 3-10: Side Band Transmitter LCLK Domain Architecture

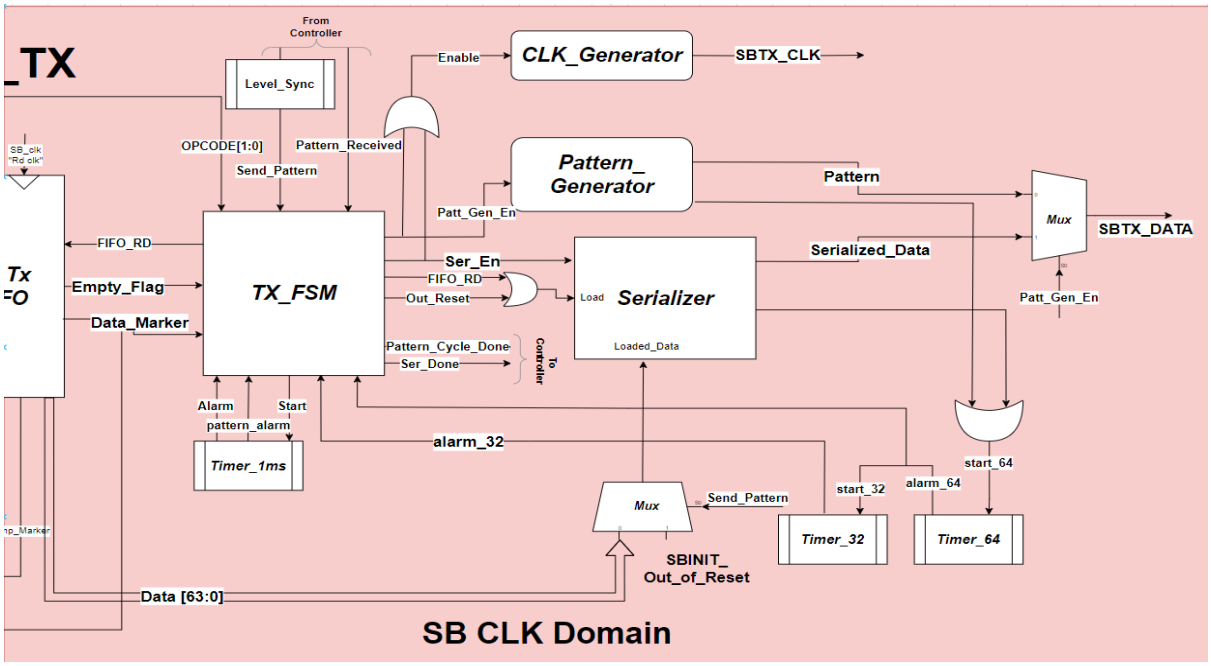


Figure 3-11: Side Band Transmitter SB_CLK domain Architecture.

Figure 3-10: Side Band Transmitter LCLK Domain Architecture and Figure 3-11: Side Band Transmitter SB_CLK domain show all the Modules constituting the Side Band Transmitter Submodules and all the Connections between them and the Upcoming Section would be Describing each Module Separately in Great Details.

Submodules Description

3.3.1.1. Decoder

Block Functionality:

Decoder is one of the most critical components in the **SBTX LCLK domain**, effectively acting as the control unit for data flow within this domain. Its functionality can be summarized as follows:

- It **processes incoming messages**, distinguishing between **local** and **remote** types.
- For **local register access requests (RARs)**, it interfaces with the **UIRB** to handle **read and write operations** targeting the configuration space. It waits for the corresponding response and, once received, formats a **completion message**, which is forwarded to the **SBRX** for transmission to the **D2D layer** via the **RDI interface**.
- If the message is **remote**, the Decoder interacts with the **priority encoder** to enqueue the message into a FIFO, preparing it for **serialization on the SB link** and subsequent transmission to the **partner die**.
- In the **RESET** or **SBINIT** states, it also processes messages from the **Sync. FIFO**. In this case, the Decoder:
  - Releases a **credit** to maintain credit system integrity.
  - Send a **completion message** for remote RARs, indicating an **Unsupported Request**, as specified by the protocol.

Dedicated Formatting Function:

Since completion messages may vary in format—such as **without data**, **with 32-bit data**, or **with 64-bit data**, the Decoder includes a dedicated F**ormatting Function** that generates the appropriate completion message based on its type and the current state.

Decoder's Role in Credit Management:

In addition, the Decoder contributes to the **credit system** by asserting specific signals to release credits in cases of local RAR completions and Unsupported Requests, ensuring smooth message handling and protocol compliance.

Block Architecture:

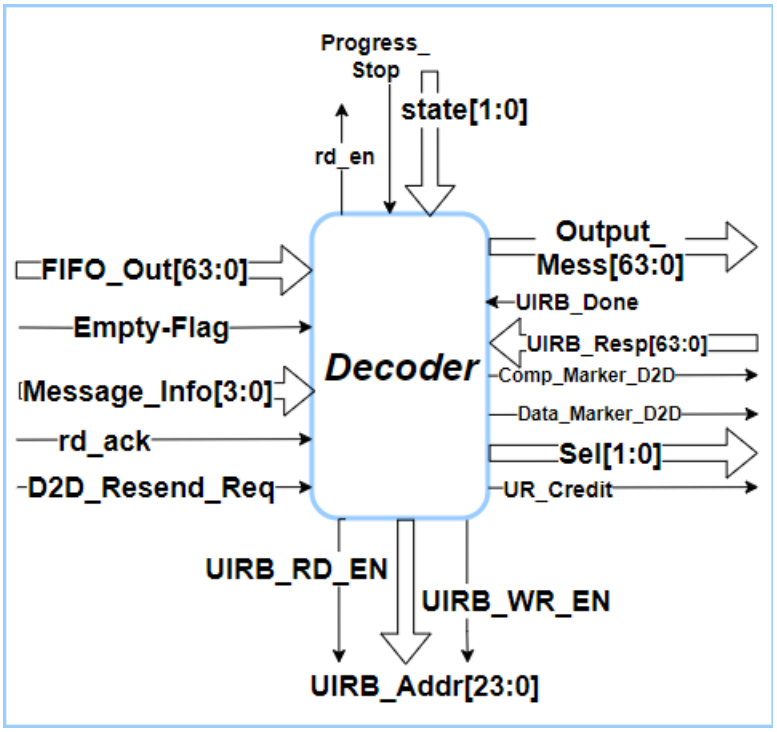


Figure 3-12: SBTX Decoder Architecture

Block Signals, Parameters & Configurability:

Table 3-36: SBTX Decoder Parameters

Parameter	Width
DATA_WIDTH	64
INFO_WIDTH	4
SEL_WIDTH	2

Table 3-37: SBTX Decoder Interface

Decoder Interface			
Signal	**Direction**	**Width**	**Description**
Rd_en	Output	1	The Decoder asserts it to read the D2D messages from the Sync. FIFO.
FIFO_Out	Input	DATA_WIDTH	D2D messages to be processed by the Decoder.
Empty_ Flag	Input	1	Indicates that There are no current messages from the D2D saved in Sync. FIFO.
Mess_Info	Input	INFO_WIDTH	Information bits about the nature of the D2D message.
Rd_ack	Input	1	When asserted, it confirms that the message was read from Sync. FIFO.
Ouput_Mess	Output	DATA_WIDTH	Message output by the Decoder whose destination is defined by the Sel bus.
UR_Credit	Output	1	Asserted to indicate that an UR request was handled and should be utilized by the Credits system.
Sel	Output	SEL_WIDTH	Selection signal for the de-mux to decide the destination of the output data of the decoder.
Comp_Marker_D2D	Output	1	The current D2D processed message is a completion message.
Data_Marker_D2D	Output	1	The current D2D processed message has data phases.
D2D_Resend_Req	Input	1	Asserted by the Priority Encoder to indicate that it cannot receive messages from the Decoder Currently due to being busy handling message from the Controller.
UIRB_WR_EN	**Already mentioned in** Error! Reference source not found. **&** Error! Reference source not found.		
UIRB_RD_EN			
UIRB_Addr			
UIRB_Done			
UIRB_Resp			
state			
Progress_stop			

Notes:

→ The Mess_Info bus simplifies D2D message handling by conveying key characteristics of the messages such as whether it is a RAR, a request/response, or a completion message—without requiring header processing. Most importantly, it also indicates whether the message includes data phases.

→ The Sel bus serves as the select signal for the de-multiplexer driven by the Decoder, which directs messages either to the UIRB, the D2D path, or for serialization over the SB link.

Clocking & CDC:

- **Clocking:** The block operates within the D2D (LCLK) clock domain.
- **CDC:** the block deals with signals from the Controller, UIRB and some other blocks of the Side Band Transmitter in the same domain so no CDC is introduced!

Special Features & Optimization:
State-Based Classification by Message Handling Behavior and Data Presence

Approach one: Combinational Decoder with Large Case Statement
This method uses a flat combinational structure (e.g., **a large case or if-else block**) to process each message type independently.
Why is it less suitable:

- It doesn't handle multi-cycle operations well (e.g., fetching data from a FIFO in a second cycle).
- It becomes large and hard to maintain as the number of message types grows.
- It lacks the ability to track temporal dependencies (e.g., wait for a response or a data fetch).

Approach two: In this design, the keyword is Type- Abstraction where the Decoder is implemented as a **finite state machine** (FSM), a deliberate choice driven by the observation that message handling depends primarily on the **State, message destination (local vs. remote) and whether a data payload is present**—not on the message type itself. This allows the design to consolidate similar handling patterns into shared states, reducing complexity.

Each group of functionally similar messages is assigned two states: one for messages without data, and a follow-up state for those with data. This ensures proper sequencing **and maintains alignment between headers and payloads, preventing protocol violations.**

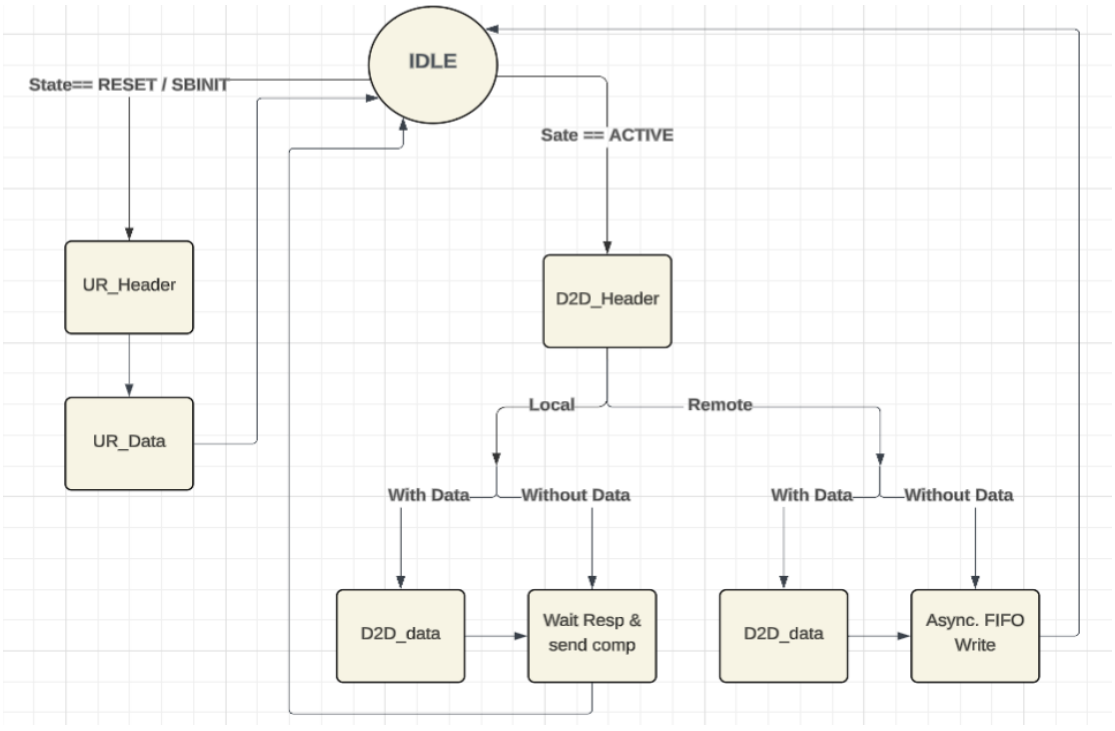


Figure 3-13: SBTX Decoder FSM States

3.3.1.2. D2D Data Integrity

Block Functionality:

The **D2D_Data_Integrity** module is responsible for receiving messages from the D2D interface via the RDI, verifying their integrity by checking both control and data parity errors, and—once validated—storing the complete message in the Sync FIFO with the two header phases grouped together and, if present, the two data phases grouped together, for subsequent processing by the Decoder.

This module also serves as a **central point for generating critical information bits** by analyzing the message header. These include:

- **Data_Marker** – Indicates that the message contains a data payload. It propagates through various blocks in the SBTX, such as:
  - The **Decoder**, which uses it to **fetch the corresponding data phases.**
  - The **Priority Encoder**, where it **triggers the priority lock mechanism.**
  - The **Transmit FSM**, which **withholds the ser_done signal** until both the header and data phases have been fully serialized.
- **Comp_Marker** – Identifies completion messages and is used by the credit system to prevent issuing credits for such messages, as specified by the protocol.
- **Req/Resp_Marker** – Marks request/response-type messages, enabling the Sync FIFO to handle them in a specialized manner.

Block Architecture:

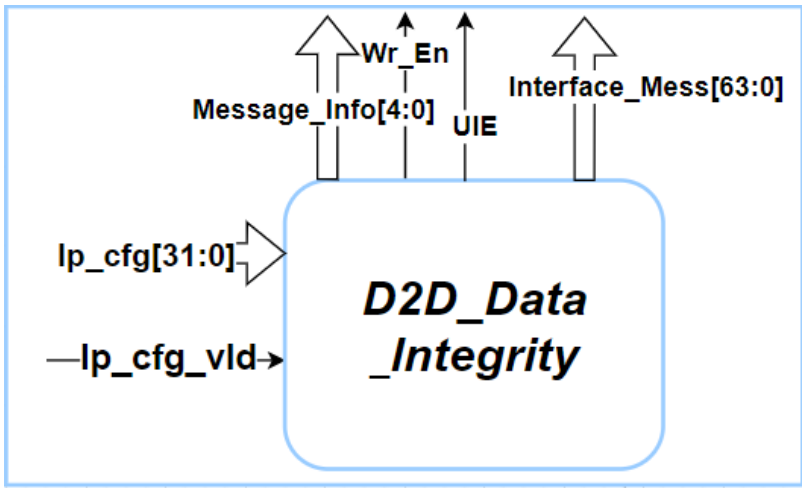


Figure 3-14: SBTX D2D_Data_Integrity Architecture

Block Signals, Parameters & Configurability:

Table 3-38: SBTX D2D_Data_Integrity Parameters

Parameter	Width
DATA_WIDTH	64
INFO_WIDTH	5

Table 3-39: SBTX D2D_Data_Integrity Interface

D2D_Data_Integrity Interface			
Signal	**Direction**	**Width**	**Description**
Interface_Mess	output	DATA_WIDTH	2 phases of the D2D messages that could be Header of Data.
WR_EN	output	1	**Used to trigger a writing operation in the Sync. FIFO**
UIE	output	1	When asserted, the module just received a parity-errored message from the D2D on the RDI Interface.
Mess_Info	output	INFO_WIDTH	Group of bits that convey the key characteristics of the message.
lp_cfg_vld	**Already mentioned in** Error! Reference source not found.		
lp_cfg			

Notes:

→ The Mess_Info bus includes one additional bit compared to the Decoder's Mess_Info bus in table (**Error! Reference source not found.**). This extra bit indicates whether the message is a Request/Response type, a property utilized exclusively by the Sync FIFO.

Clocking & CDC:

- **Clocking:** The block operates within the D2D (LCLK) clock domain.
- **CDC:** the block deals with signals from the RDI interface so no CDC is introduced!

Special Features & Optimization

Consecutive Messages on RDI Interface Handling

asserted independently for each phase. They must be asserted on consecutive clock cycles for transferring consecutive phases of the same packet. They may or may not assert on consecutive clock cycles when transferring phases of different packets. For packets with data, 64b of data is always transmitted over RDI or FDI; for 32b of valid payload, the most-significant 32b (Phase 4) of the packet are assigned to 0b before transfer.

Figure 3-15: SBTX Consecutive D2D messages on RDI Interface Spec.

To correctly handle writing the previous message into the FIFO after verifying parity errors, while also managing messages arriving in consecutive cycles, the module must operate in a **pipelined manner** that overlaps validation and storage stages without compromising data integrity.

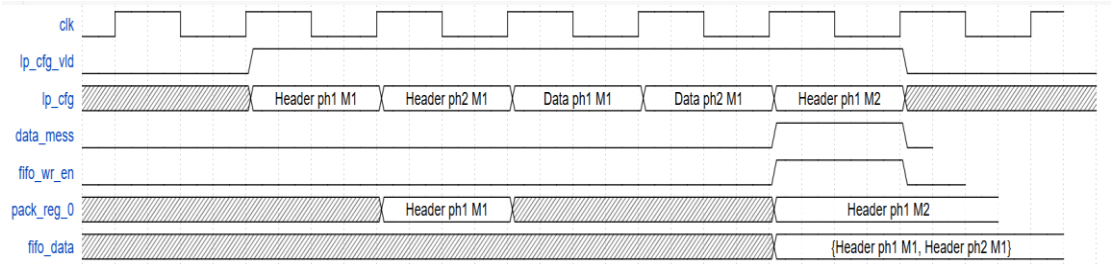


Figure 3-16: Consecutive D2D messages on RDI Interface Handling

3.3.1.3. Credits Handler

Block Functionality:

The Credits_Handler module represents the credit return logic of the SBTX and is responsible for maintaining the integrity of the credit system. It manages credit returns for various scenarios:

1. **Local RAR Completion:**
   When a local Register Access Request (RAR) is fully processed and the corresponding completion message is formed, the **comp_vld** signal is asserted. **This signal is used to trigger a credit return**, indicating that the resource consumed by the RAR is now free.

2. **Remote Message Handling via Async. FIFO:**
   Credits are also returned for remote messages once they are read from the Async. FIFO. Three key synchronized signals accompany each message:
   a. Data_Marker: Ensures that a credit is returned only once per message, regardless of whether it carries data or not, **since both consume the same credit**.
   b. Comp_Marker: Prevents credit return for completion messages, as per protocol, since completions do not consume credits.
   c. D2D/PHY_Marker: Distinguishes between D2D messages and controller-originated messages. **Only D2D messages are considered for credit return**, as the controller messages do not participate in the credit system.

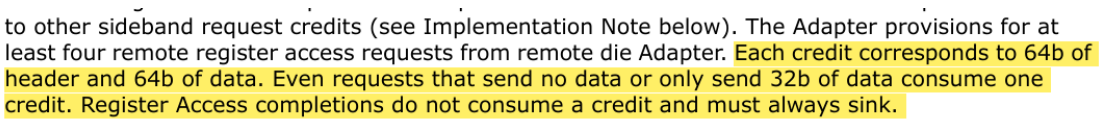

to other sideband request credits (see Implementation Note below). The Adapter provisions for at least four remote register access requests from remote die Adapter. Each credit corresponds to 64b of header and 64b of data. Even requests that send no data or only send 32b of data consume one credit. Register Access completions do not consume a credit and must always sink.

Figure 3-17: SBTX Message Credits in UCIe

Special Cases – UR and UIE Messages:

The module also handles credit return for Unsupported Requests (UR) and messages triggering Unexpected Interface Errors (UIE). These messages may occur during the Bring-Up phase or transitions such as TRAINERROR or PHYRETRAIN. Despite the system being in early or transitional states, credits must still be returned to maintain consistency, especially since D2D credit tracking is only updated upon exiting domain reset.

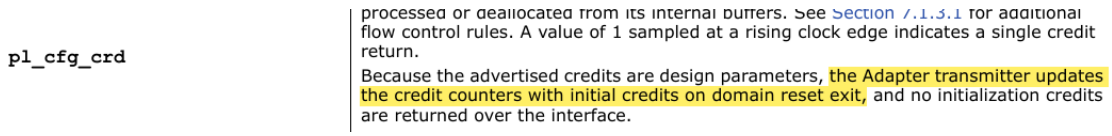


Figure 3-18: SBTX Credit System Initialization

Block Architecture:

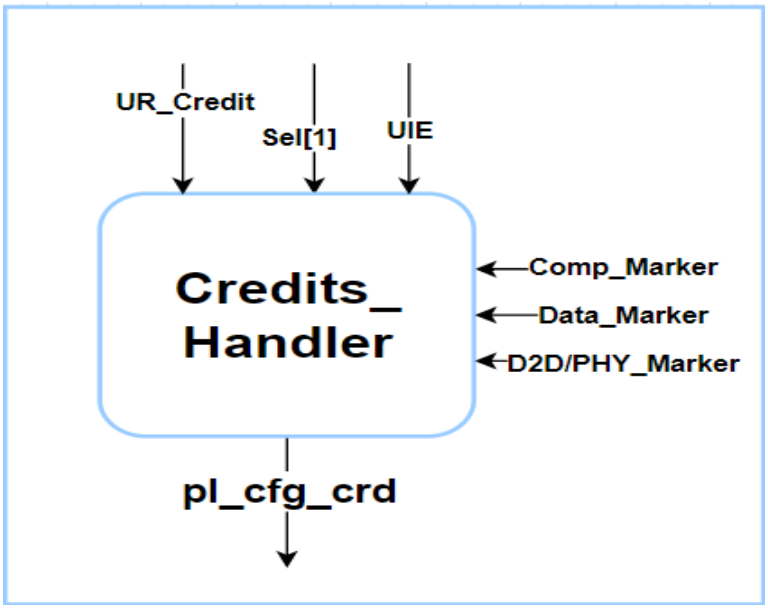


Figure 3-19: SBTX Credit Handler Architecture

Block Signals, Parameters & Configurability:

Table 3-40: SBTX Credit Handler Interface

Credits Handler Interface			
Signal	**Direction**	**Width**	**Description**
UR_Credit	Input	1	Asserted by the Decoder to notify the Credits system to release a credit.
Sel	Input	1	Asserted by the Decoder to notify the Credits system to release a credit.
UIE	Input	1	Asserted by the D2D_Data_Integrity module to notify the Credits system to release a credit.
Comp_Marker	Input	1	When asserted, this signal indicates that a completion message has exited the Asynchronous FIFO and is ready to be serialized onto the SB Link.
Data_Marker	Input	1	When asserted, this signal indicates that a Data message has exited the Asynchronous FIFO and is ready to be serialized onto the SB Link
D2D/PHY_Marker	Input	1	When asserted, this signal indicates that a D2D message has exited the Asynchronous FIFO and is ready to be serialized onto the SB Link
Lp_cfg_crd	**Already mentioned in** Error! Reference source not found.**.**		

Clocking & CDC:

- **Clocking:** The block operates within the D2D (LCLK) clock domain.
- **CDC:** the block deals with signals (Data_Marker, Comp_Marker, D2D/PHY_Marker).

From the output of the Async. FIFO (Read domain) so these signals need to be synchronized to the LCLK.
Since they are all Pulse signals so a **Toggle Synchronizer** would be suitable for this case.

Special Features & Optimization:
Buffered Credit Return Mechanism for Concurrent Event Handling in D2D Credits System

In this system, credit return signals serve as acknowledgments to the D2D layer, indicating that resources are available for new transactions. A credit is returned as a one-cycle, 1-bit pulse. These credit-triggering events originate from **multiple functional blocks**—such as the **Async. FIFO, Decoder, and D2D_Data_Integrity**—all of which can operate in parallel. Consequently, **multiple credit return conditions may occur in the same clock cycle**.

However, the credit return interface supports only a **single-bit pulse per cycle**, making it incapable of handling multiple simultaneous credit acknowledgments directly. To resolve this, a **credit counter** is implemented to buffer outstanding credit returns. This counter ensures that:

- All valid credit returns are eventually issued, preserving compliance with the D2D credit protocol.
- Surplus credit returns (exceeding the one-per-cycle limit) are queued and sequentially released in subsequent cycles.

This buffered approach guarantees deterministic credit behavior, prevents credit loss, and ensures smooth operation—even during periods of high concurrency—thereby maintaining system reliability and protocol integrity.

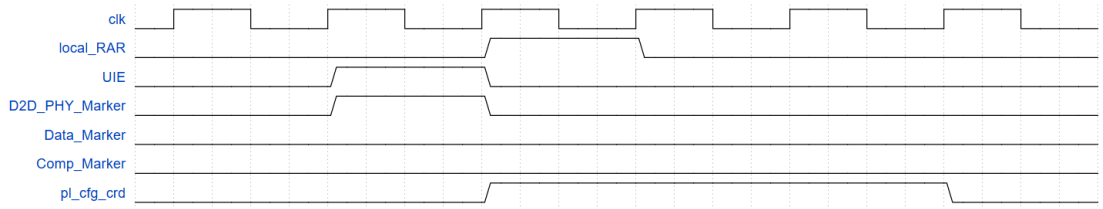


Figure 3-20: SBTX Buffered Credits Return Mechanism

3.3.1.4. Sync. FIFO

Block Functionality

the Sync. FIFO is mainly used to save messages from the RDI Interface (D2D Layer Messages).

1. **Depth Tailored for Worst-Case Throughput**
   The FIFO is designed with a depth of 64 to accommodate up to 32 D2D messages, considering that each message can span two entries (header and optional data). This depth accounts for the worst-case scenario where all incoming messages are header-only and supports potential accumulation due to the limited throughput of the decoder—**particularly during consecutive Local RAR requests, which may take multiple cycles to process individually**.

2. **Priority-Based Read Mechanism Using Pointer Manipulation**
   To ensure that higher-priority Req/Resp messages are not blocked by earlier but lower-priority Local RARs, the FIFO implements a priority mechanism. This allows it to output Req/Resp messages ahead of RARs, even if they were written later. **This is achieved through enhanced pointer control**, enabling **selective access** to entries based on message type while maintaining FIFO consistency.

Block Architecture

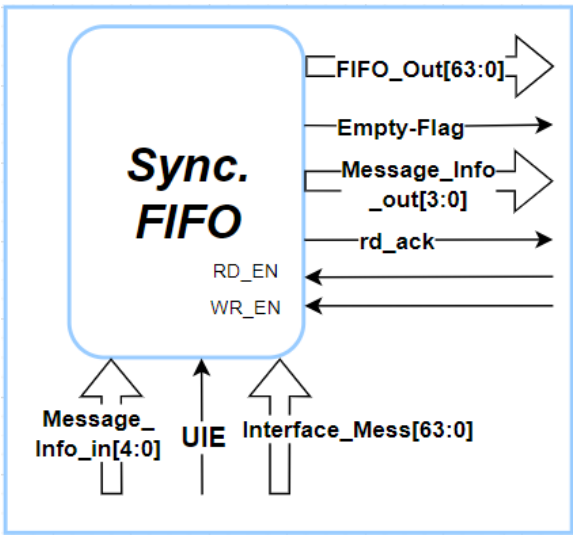


Figure 3-21: SBTX Sync FIFO Architecture

Block Signals, Parameters & Configurability:

Table 3-41: SBTX Sync. FIFO Parameters

Parameter	Width
DATA_WIDTH	64
INFO_WIDTH	5

Table 3-42: SBTX Sync. FIFO Interface

Sync. FIFO Interface			
Signal	**Direction**	**Width**	**Description**
RD_EN & WR_EN	Input	1	Write & Read enable of the FIFO generated by the D2D_Data_Integrity and Decoder modules respectively.
Empty_Flag	Output	1	Notify the Decoder when there are no more D2D messages to be processed
FIFO_Out	Output	DATA_WIDTH	D2D Message from the FIFO.
Mess_Info_Out	Output	INFO_WIDTH	Group of bits that convey the key characteristics of the message.
UIE	Input	1	When asserted, FIFO should not save the message on its data port.
Mess_Info_In	Input	INFO_WIDTH+1	Group of bits that convey the key characteristics of the message with additional bit representing whether the Message is Req/Resp or not.
Interface_Mess	Input	DATA_WIDTH	D2D message to be saved in the FIFO.

Clocking & CDC:

- **Clocking:** The block operates within the D2D (LCLK) clock domain.
- **CDC:** the block deals with signals from the Decoder & D2D_Data_Integrity Modules so no CDC is introduced*!*

3.3.1.5. Priority Encoder

Block Functionality:

The **Priority Encoder** serves as an arbiter between the Decoder and Controller, coordinating access to Async. FIFO. Its design is optimized for low-contention scenarios with rare simultaneous message requests, leveraging a lightweight mechanism that ensures message integrity without added buffering overhead.

1. **Priority Lock Mechanism for Synchronized Dual-Source Arbitration**
   The Priority Encoder acts as an **arbiter** between two sources—the Decoder and the Controller—both of which may request access to write messages into the Async. FIFO. To handle potential contention and ensure data integrity, it implements a **Priority Lock mechanism**. When either source requests access and asserts its Data_Marker, **the arbiter grants exclusive access** and locks priority to that source. It then ignores any further requests from the other source until the current source finishes writing both the header and associated data into the FIFO. This ensures atomicity of multi-phase message writes and prevents **interleaving** of partial messages from different sources.

2. **Resend-Based Flow Control Avoiding Data Buffering**
   Instead of **buffering or packing** incoming messages from both sources, the arbiter employs a **Resend protocol**. If a source attempts to write while another has locked the priority, it is simply not acknowledged; the source is expected to retry transmission after the Resend signal is de-asserted. This design choice is justified by the low message arrival rate from both sources: Controller messages are sparse, typically separated by ~120 LCLK cycles, and Decoder activity primarily begins after Controller modules (e.g., PHY and Partner) complete training. The only **rare overlap case** involves error messages from the RDI module during active operation. Given the **infrequency of such contention**, this Resend strategy avoids unnecessary complexity and resource usage (e.g., packing registers) in the arbiter.

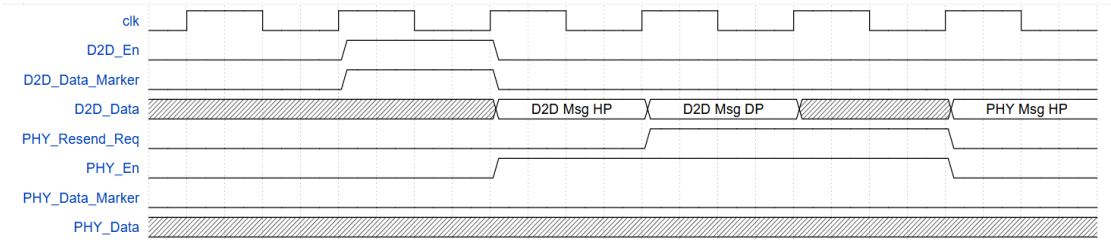


Figure 3-22: SBTX Priority Encoder Resend Mechanism

Block Architecture:

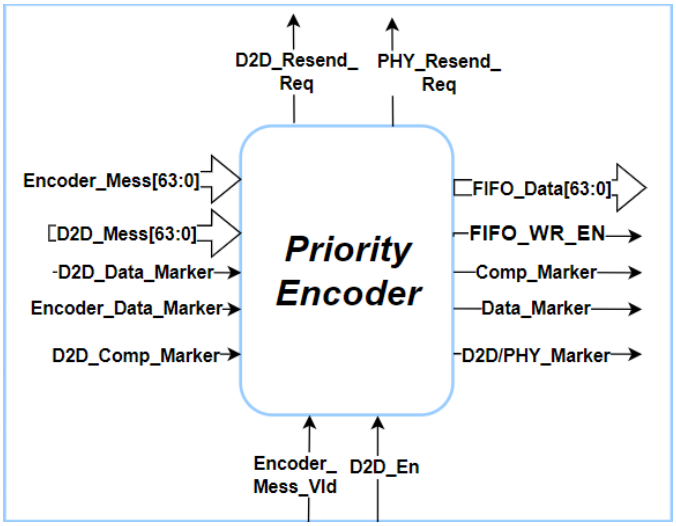


Figure 3-23: SBTX Priority Encoder "Arbiter" Architecture

Block Signals, Parameters & Configurability:

Table 3-43: SBTX Priority Encoder Parameters

Parameter	Width
DATA_WIDTH	64

Table 3-44: SBTX Priority Encoder Interface

Priority Encoder Interface			
Signal	**Direction**	**Width**	**Description**
D2D_En	Input	1	When asserted the Decoder is requesting to write a message in the Async. FIFO.
D2D_Data_Marker	Input	1	The message on D2D_Data has a Data phase.
D2D_Comp_Marker	Input	1	The message on D2D_Data is a completion message.
D2D_Data	Input	DATA_WIDTH	Remote D2D message.
D2D_Resend_Req	Input	1	Asserted to indicate that the module cannot handle the Decoder message due to being busy with handling a message from the Controller.
FIFO_WR_En	Output	1	FIFO Write Enable.
FIFO_Data	Output	DATA_WIDTH	D2D or Controller Message out of the FIFO.
Comp_Marker	Output	1	When asserted, this signal indicates that a completion message has exited the Asynchronous FIFO and is ready to be serialized onto the SB Link.
Data_Marker	Output	1	When asserted, this signal indicates that a Data message has exited the Asynchronous FIFO and is ready to be serialized onto the SB Link
D2D/PHY_Marker	Output	1	When asserted, this signal indicates that a D2D message has exited the Asynchronous FIFO and is ready to be serialized onto the SB Link
Encoder_Mess_Vld	**Already mentioned in** Error! Reference source not found.		
Encoder_Mess			
Encoder_Data_Marker			
PHY_Resend_Req			

Clocking & CDC:

- **Clocking:** The block operates within the D2D (LCLK) clock domain.
- **CDC:** the block deals with signals from the Decoder & Controller Modules so no CDC is introduced!

3.3.1.6. Tx FSM

Block Functionality:

The **TX FSM** is the first block operating in the **SB Clock (SB_CLK) domain**, responsible for driving data onto the **SBTX_DATA** lane. Its primary role is to control the transmission of either:

- Phases of valid messages.
- Training pattern (e.g., **SBINIT pattern**) during link initialization.

FSM Behavior and States:

- **During SBINIT**:
  - The FSM alternates between sending a predefined **pattern** for 1 ms and staying **idle** for 1 ms.
  - This sequence repeats **8 times**, aiming to ensure consistent pattern detection on the **SBRX** lane.
  - Once the pattern is detected, the FSM sends **4 additional patterns** to account for any timing skew between the two UCIe modules exiting reset *(2)*
  - It then transmits a **SBINIT_out_of_Reset** SB message until it's also detected.
- **In Active or Other States**:
  - The FSM monitors the **Async FIFO** for new messages.
  - If a message is present, it begins **serializing** its content onto the SBTX_DATA lane.

The FSM also generates an important signal required for UCIe functionality, as it monitors the Data_Marker of the message currently being serialized and asserts a ser_done signal only after the entire message—both header and data phases—has been fully serialized. This ser_done signal is then ANDed with the Empty flag of the Async FIFO to indicate that the FIFO is completely empty and the last message has been fully processed. This combined condition allows **the system to safely transition to the RESET state**, if required, ensuring that no message is left partially serialized and that protocol integrity is preserved during link shutdown or state changes *(3)*

FSM States:

The FSM uses a minimal state set to manage its functionality:

1. IDLE – Waiting for activity.
2. Pattern_Operational – Sending training pattern.
3. Pattern_Off – Holding line idle between pattern bursts.
4. Pattern_Detected – **Sending post-detection patterns**
5. Serialize – Serializing message phases from FIFO & “SBINIT_out_of_Reset “SB Message.

Why Mealy FSM?

This FSM is implemented as a **Mealy Machine** because:

- It must respond immediately to input signals like Timers Alarms, pattern_detected and fifo_empty
- It **enables and coordinates** modules based on real-time input conditions, ensuring correct timing and handshake behavior.
- Outputs (like enable signals) depend on both the **current state** and **external inputs**, aligning naturally with Mealy FSM semantics.

4. If the UCIe Module successfully detects the pattern, it stops sending data and clock on its sideband Transmitters after four more iterations of pattern in Step 1. This will allow for any time differences in both UCIe Modules coming out of RESET. The UCIe Module sideband Transmitter and Receiver must now be enabled to send and receive sideband messages, respectively.

Figure 3-24: SBTX the 4 Additional SBINIT Pattern Justification.

The exit from TRAINERROR to RESET is implementation specific. For cases when there is no error escalation (i.e., RDI is not in LinkError), it is recommended to exit TRAINERROR as soon as possible. For cases when there is error escalation (i.e., RDI is in LinkError), it is required for Physical Layer to be in TRAINERROR as long as RDI is in LinkError. To avoid problems with entering RESET while transmitting sideband packets, any in-progress sideband packets must finish transmission before entering RESET state.

Figure 3-25: SBTX RESET State Transition Condition.

Block Architecture:

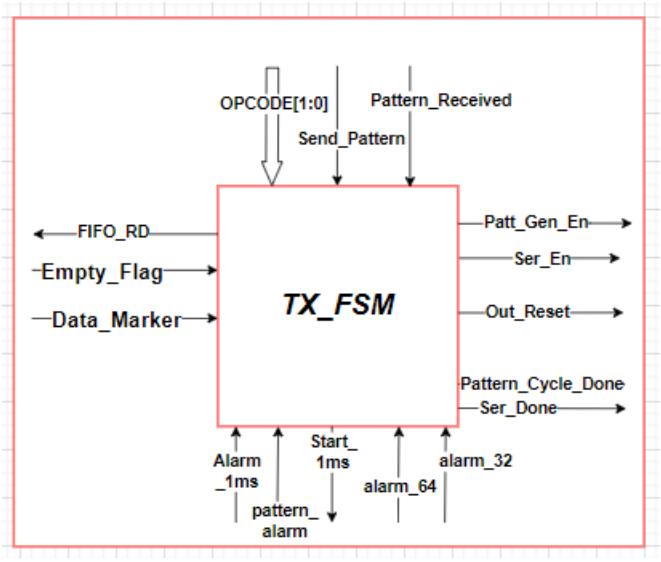


Figure 3-26: SBTX TX FSM Architecture

6.3 Block Signals, Parameters & Configurability:

Table 3-45: SBTX TX-FSM Parameters

Parameter	Width
OPCODE_LENGTH	2

Table 3-46: SBTX TX-FSM Interface (sheet 1 of 2)

TX FSM Interface			
Signal	**Direction**	**Width**	**Description**
Patt_Gen_En	Output	1	Enable signal to activate the pattern generator
Ser_En	Output	1	Enable signal to activate the Serializer
FIFO_RD	Output	1	Enable signal to read Messages from the Async. FIFO to be serialized.
Out_Reset	Output	1	Signal to load the SBINIT_out_of_Reset message into the Serializer.
Start_1ms	Output	1	Enable signal to initiate the 1 ms. timer that controls the alternation between transmitting and pausing the pattern
Alarm_32	Input	1	Fire flag of the 32 cycles alarm.
Alarm_64	Input	1	Fire flag of the 64 cycles alarm.

Table 3-47: SBTX TX-FSM Interface (sheet 2 of 2)

Signal	Direction	Width	Description
Patt_alarm	input	1	The Fire flag is used to halt pattern transmission even if the 1 ms. operation is still ongoing.
Alarm_1ms	input	1	Fire flag of the 64 1 ms .alarm.
Empty_Falg	Input	1	Async. FIFO Empty Flag
Data_Marker	Input	1	Used to indicate that the output message from the Async. FIFO has Data phase.
Patt_Received	**Already mentioned in** Error! Reference source not found..		
Send_Pattern			
OPCODE" State"			
Patt_Cycle_Done			
Ser_Done			

Clocking & CDC:

- **Clocking:** The block operates within the SB clock domain.
- **CDC:** the block deals with signals from the LCLK Domain OPCODE (bus) and Send_Pattern (signal)

For the Bus, since it reflects states of an FSM so a Gray Synchronizer would be suitable for this case. For the Signal, since the nature of the signal is a Level signal so a 2FF Synchronizer (known as Level Synchronizer) would be suitable.

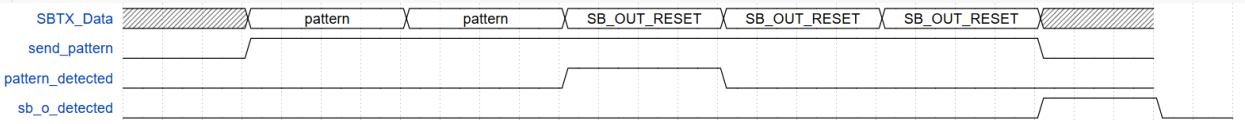


Figure 3-27: SBTX Signal controlling initialization Process Behavior

3.3.1.7. SB CLK & Data Lanes Handler

Block Functionality:

The SB CLK & Data Lanes Handler is a composite module responsible for driving the SBTX_DATA and SBTX_CLK lanes. It consists of several sub-modules—namely, the Serializer, Strobe Generator, and Pattern Generator—all of which are orchestrated by enable signals issued by the TX FSM. The Serializer operates as a simple shift register used for message serialization. The Strobe Generator functions as a clock gating unit that controls the emission of SBTX_CLK. The Pattern Generator, implemented as a toggle register, outputs the alternating pattern 1010... required for training sequences, typically repeated for 64 cycles. Additionally, two timers are employed to manage timing behavior during training phases, as Performant Mode Operation (PMO) is not supported in this implementation.

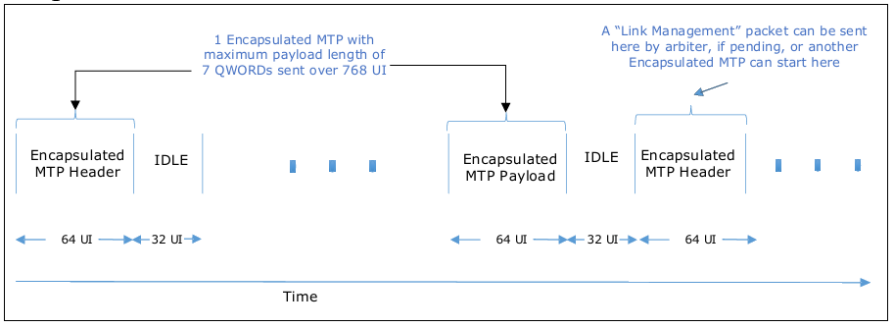


Figure 3-28: SBTX Non PMO Operation

Block Architecture:

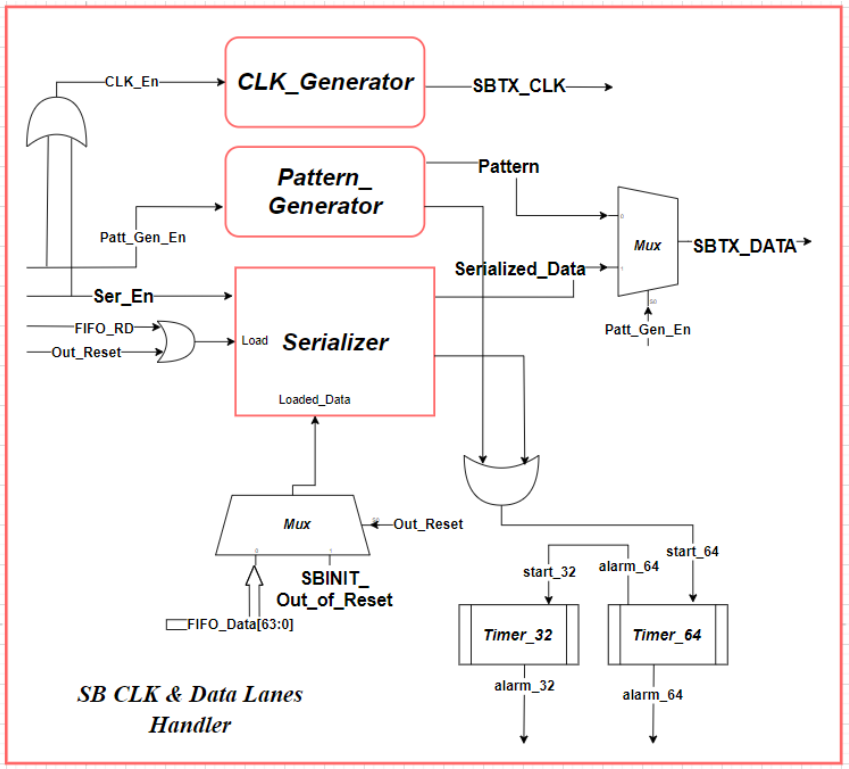


Figure 3-29: SBTX SB CLK & Data Lane handler Architecture

Block Signals, Parameters & Configurability:

Table 3-48: SBTX SB CLK Data lanes Handler Parameters

Parameter	Width
DATA_WIDTH	64

Table 3-49: SBTX SB CLK Data lanes Handler Interface

SB CLK & Data Lanes Handler Interface			
Signal	**Direction**	**Width**	**Description**
Patt_Gen_En	Input	1	Enable signal to activate the pattern generator.
Ser_En	Input	1	Enable signal to activate the Serializer.
CLK_En	Input	1	Enable signal to activate the Strobe Generator which is the ORing between the Patt_Gen_En & Ser_En.
FIFO_Data	Input	DATA_WIDTH	Message out of Async. FIFO.
Load	Input	1	The Serializer loading signal which is the ORing between the FIFO_RD & Out_Reset signals.
Out_Reset	Input	1	When asserted, the Serializer should keep loading the "SBINIT_out_of_Reset" message and serialize it.
Alarm_32	Output	1	Fire flag of the 32 cycles alarm.
Alarm_64	Output	1	Fire flag of the 64 cycles alarm.
SBTX_CLK	**Already mentioned in** Error! Reference source not found.**.**		
SBTX_DATA			

Notes:

→ The CLK_En signal is the ORing between the Patt_Gen_En & Ser_En signals because the Strobe is accompanied with both Data and Pattern sent on the SB Lane.

→ The Load signal of the Serializer is the ORing between the FIFO_RD & Out_Reset signals because the Serializer either serialize the Message read from the Async. FIFO. Or the "SBINIT_out_of_Reset" SB Message.

Clocking & CDC:

- **Clocking:** The block operates within the SB clock domain.
- **CDC:** the block deals with TX FSM signals so no CDC is introduced!

Special Features & Optimization:

Multiplexed Timer and Clock Resource Architecture for Unified Control of Training Pattern and Data Transmission in Non-PMO Mode Systems

The system operates in **two primary modes: training mode**, where a repeating pattern is transmitted, and **active mode**, where actual messages are sent. In both modes, transmission occurs in bursts of **64 cycles** of valid data, followed by **32 cycles** of idle (low) signal. This fixed timing pattern compensates for the lack of **Performant Mode Operation (PMO)** and helps clearly define valid data windows.

To manage this timing, the design employs **two timers**—one for the 64-cycle active phase and another for the 32-cycle idle phase. A shared **clock generator** produces a strobe signal used to gate data transmission in both modes. Since only one type of data (either pattern or message) is sent at a time, a **multiplexer** allows the **pattern generator** and **serializer** to share the same timers and strobe logic. This approach **reduces hardware overhead** while maintaining synchronized and deterministic timing behavior.

3.3.1.8. Asynchronous FIFO

✓ **Pointers Gray Encoding**

Gray encoding is used in asynchronous FIFOs to safely transfer pointer values between different clock domains. Unlike binary counters, **gray code changes only one bit at a time**, reducing the risk of metastability during synchronization. This ensures that pointers are accurately interpreted when crossing domains, preventing false full or empty conditions. By using gray code, FIFO design becomes more robust and avoids data corruption. Overall, it enables reliable operation in systems with unsynchronized clocks.

✓ **FIFO Size**

The **Asynchronous FIFO** in this system is designed to buffer messages coming from two main sources: the **Controller** and the **Decoder (RDI Module)**. Although both sources share the FIFO, they **operate in different system states**, which simplifies the depth analysis.

1. **Controller Message Scenario – During Training State**

   In the **training phase**, the following three controller modules may each generate a message:
   - **PHY Module** typically initiates training-related control messages.
   - **Module Partner** may respond to training requests.
   - **RDI Module**: handles error reporting or specific initialization steps.

These messages are **not concurrent** due to the natural spacing in training operations. However, a **worst-case** snapshot can include:

- A **request** initiated by one module.
- A **response** triggered by the partner.
- An **error or state message** generated by the RDI module.

➢ **Worst-case total during training: 3 messages**.

2. **Decoder (RDI) Scenario – During Active State**
   In the **active operating state**, only **remote messages** are written into the FIFO. These include:
   - **REQ/RESP Messages or Advertising Capabilities**: Regular messages exchanged as part of UCIe protocol communication. These are **handshake-based**—each request waits for a response before sending the next. This limits the number of simultaneous messages naturally.
   - **Register Access Requests (RARs)**: Sent from the D2D to access remote registers via a mailbox mechanism.

   Important limitations and conditions:
   - The **RAR mechanism allows for up to 4 outstanding transactions** because the system provisions **4 credits** for remote access.
   - However, there's **only one mailbox register**, meaning that **only one RAR can be active at any given time**.
   - The **additional credits** were originally designed to support **debug-related RARs**, which would arise due to register access timeouts. But since **debug RARs are not implemented**, those cases are not considered.

   ➢ **Worst-case total during active state: 5 remote messages**
   (1 RAR + 1 Completion + 1 REQ/RESP messages + 2 Advertise Capability)

FIFO Depth Calculation

- **Training phase max** = 3 controller messages
- **Active phase max** = 5 remote messages

To accommodate this and support **Gray-coded address synchronization** (commonly used in async FIFOs to ensure safe pointer transitions between clock domains), the FIFO depth is rounded up to the **nearest power of 2**.

✅ **Minimum FIFO depth required: 8 entries**

3.3.2. Sideband Receiver "SB_RX":

The SB RX (Sideband Receiver) is a critical component in the physical layer design that handles the reception and processing of sideband communication packets.

The SB RX module is responsible for:

- Receiving serial sideband data from the UCIE link.
- Converting it to parallel format.
- Validating and decoding the packets.
- Routing the processed data to appropriate system components and to the higher layer.
- Managing clock domain crossings between sideband and link clock domains.

These functionalities have been achieved using the following Sideband Receiver top module architecture as shown in figure 3-30:

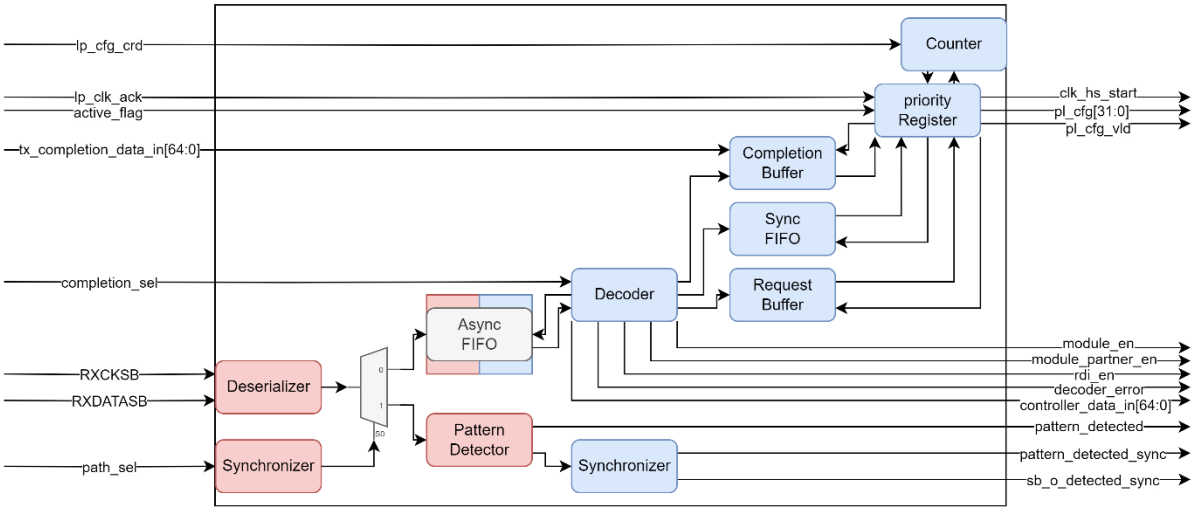


Figure 3-30: Sideband Receiver Top Architecture

Side band Receiver has an interface with Controller, UCIE Link, RDI Interface, Sideband Transmitter, and LTSM as shown in figure 3-31:

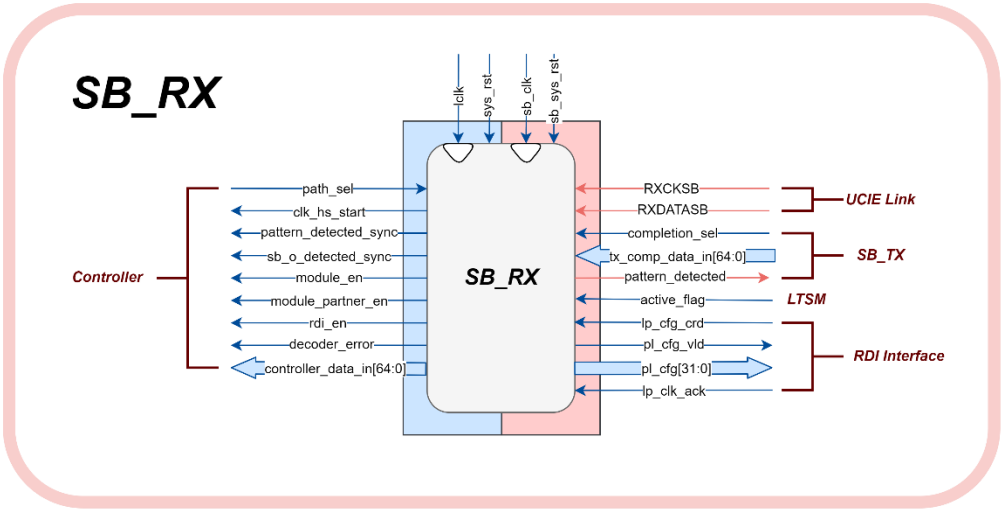


Figure 3-31: Sideband Receiver Interfcae

All parameters and signals of the SB RX (Sideband Receiver) top module are fully described in tables 3-51, 3-52, and 3-53:

Table 3-50: Sideband Receiver Parameters

Parameters	Width
DATA_WIDTH	64
CHUNK_WIDTH	32
FIFO_DEPTH	2,8,64
DESTINATION_SEL	2

Table 3-51: Sideband Receiver Signals Description (sheet 1 of 2)

Port	Direction	Width	Description
lclk	Input	1	From Interface Link clock signal.
sb_clk		1	From Interface Sideband clock signal.
sys_rst		1	From Interface Active-low asynchronous link reset signal.
sb_sys_rst		1	From Interface Active-low asynchronous side band reset signal.
RXCKSB		1	From UCIE Link UCIE Link Clock signal associated with serial data.
RXDATASB		1	From UCIE Link Serial data input received bit-by-bit.
completion_sel		1	From SB_TX This signal indicates that we receive a completion packet from SB_TX, so the decoder should register its data until SB_TX completes.
tx_completion_data_in		DATA_WIDTH+1	From SB_TX completion packet originating from the SB_TX module. This packet represents a response to a register access request initiated by the local die.
active_flag		1	From LTSM This signal indicates that the system is in **active state**, meaning the **clock is ungated** and logic operations can proceed.
lp_cfg_crd		1	From RDI Interface This signal indicates that the Adapter has space available in its buffer to receive packets. When this signal is asserted, it means increment credit counter.

Table 3-52: Sideband Receiver Signals Description (sheet 2 of 2)

Port	Direction	Width	Description
lp_clk_ack	Input	1	From RDI Interface Signal indicates the successful completion of the clock handshake process intended to remove clock gating.
path_sel		1	From Controller Control signal used to demultiplex the output data from the Deserializer module, the data is routed either to the Pattern Detector block or the Asynchronous FIFO for further processing or storage.
controller_data_in	Output	DATA_WIDTH+1	To Controller Decoded and validated data input from the Decoder module to the Controller.
clk_hs_start		1	To Controller This signal asserted to initiate the clock handshake with the adapter layer, requesting transition to normal clock mode.
module_en		1	To Controller Enable signal that activates phy_module based on the decoded message type.
module_partner_en		1	To Controller Enable signal that activates phy_module_partner based on the decoded message type.
rdi_en		1	To Controller Enable signal that activates rdi_module based on the decoded message type.
decoder_error		1	To Controller A flag set when a parity error, message format error, or unrecognized message type is detected, indicating invalid data.
sb_o_detected_sync		1	To Controller Synchronized flag to lclk asserted on detection of out_of_reset packet.
pattern_detected_sync		1	To Controller Synchronized flag to lclk asserted on detection of the fixed pattern twice.
pattern_detected		1	To SB_Tx Flag asserted on detection of the fixed pattern twice.
pl_cfg		CHUNK_WIDTH	To RDI Interface Half of the selected 64-bit packet.
pl_cfg_vld		1	To RDI Interface Signal is high when pl_cfg holds valid data.

Figure 3-32 shows the detailed architecture of Sideband Receiver.

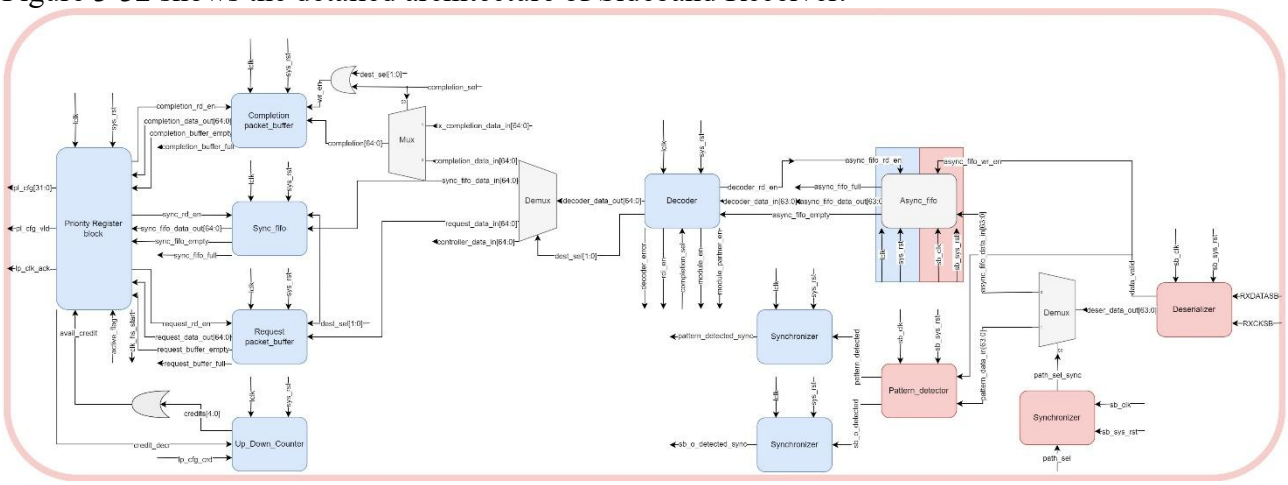


Figure 3-32: Sideband Receiver Detailed Architecture

Sideband Receiver Submodules Description:

3.3.2.1. Deserializer:

The **Deserializer module** functions as a serial-to-parallel converter that transforms incoming serial data into a parallel data word of configurable width (64 bits). It is specifically designed where the serial data stream is received using a dedicated clock signal (RXCKSB) that operates at the same nominal frequency as the system clock (sb_clk) but originates from an independent clock source. Although the frequencies are matched, the differing sources result in asynchronous behavior due to potential phase misalignment or jitter.

This module performs three primary functions:

- It **captures incoming serial bits** on the falling edge of the RXCKSB signal.
- It **accumulates the received bits** and assembles them into a complete parallel word of width defined by the parameter DATA_WIDTH.
- Once a full word is collected, it **safely transfers the output into the system clock domain** (sb_clk) by using proper clock domain crossing techniques to ensure data integrity and prevent metastability.

Figure 3-33 shows the block diagram of Deserializer:

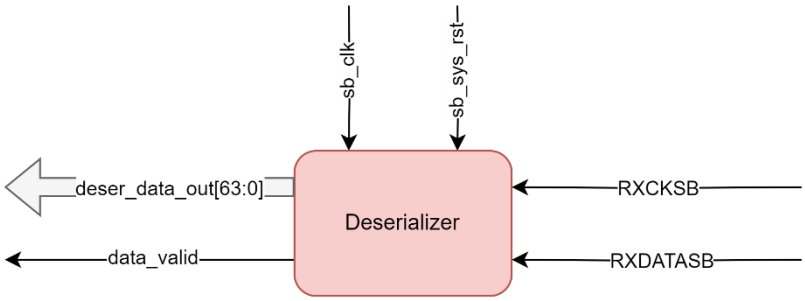


Figure 3-33: Deserializer Diagram

All parameters and signals of the Deserializer submodule are fully described in tables 3-54, and 3-55:

Table 3-53: Deserializer Parameters

Parameters	Width
DATA_WIDTH	64

Table 3-54: Deserializer Signals Description

Port	Direction	Width	Description
sb_clk	Input	1	From Interface System clock used for output and control; same frequency as RXCKSB, but different clock source.
sb_sys_rst		1	From Interface Active-low asynchronous reset signal.
RXCKSB		1	From UCIE Link Clock signal associated with serial data; sampling occurs on the **negative edge**.
RXDATASB		1	From UCIE Link Serial data input received bit-by-bit.
deser_data_out	Output	DATA_WIDTH	To 1x2 Demux Parallel output of width DATA_WIDTH; updated when a full word is assembled.
data_valid		1	To Pattern Detector and Asynchronous FIFO Asserted for one cycle of sb_clk when deser_data_out is valid.

3.3.2.2. Pattern Detector:

The **Pattern Detector module** monitors the pattern_data_in bus of width DATA_WIDTH (64 bits), and asserts two output signals pattern_detected and sb_o_detected when defined conditions are met, **based on LTSM state**:

- **pattern_detected:**
  Detects a fixed 64-bit pattern (0x5555555555555555) on two consecutive valid inputs. The output is asserted only on the **second occurrence**.
  - **This detection logic is enabled only during the RESET and SBINIT states**.

- **sb_o_detected:**
  Identifies the correct reception of a **SBINIT out-of-reset packet**.
  - **This detection is active only in the SBINIT state**

Figure 3-34 shows the block diagram of Pattern Detector:

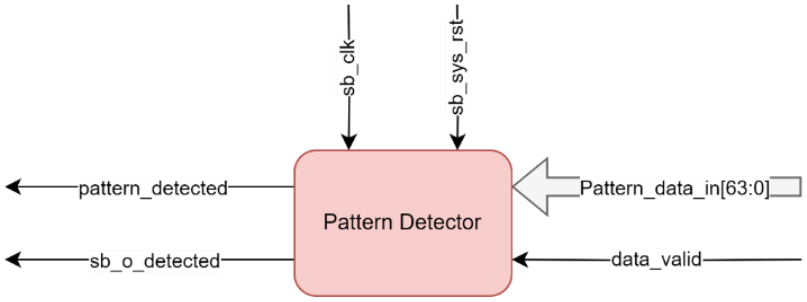


Figure 3-34: Pattern Detector Diagram

All parameters and signals of the Pattern Detector submodule are fully described in tables 3-56, and 3-57:

Table 3-55: Pattern Detector Parameters

Parameters	Width
DATA_WIDTH	64

Table 3-56: Pattern Detector Signals Description

Port	Direction	Width	Description
sb_clk	Input	1	From Interface System clock driving the detection logic.
sb_sys_rst		1	From Interface Active-low asynchronous reset signal.
pattern_data_in		DATA_WIDTH	From 1x2 Demux Parallel input data bus for pattern detection (width configurable).
data_valid		1	From Deserializer Enable signal indicating when pattern_data_in is valid.
pattern_detected	Output	1	To Synchronizer and SB_Tx Flag asserted on detection of the fixed pattern twice.
sb_o_detected		1	To Synchronizer Flag asserted on detection of SBINIT_out_of_reset packet.

3.3.2.3. Asynchronous FIFO:

The **Asynchronous FIFO module** is a dual-clock FIFO designed to **safely transfer 64-bit data across two asynchronous clock domains**: the **Sideband clock domain (sb_clk)** for writing and the **Link clock domain (lclk)** for reading.
The FIFO has a **depth of one entry**, where:

- **The entry stores valid sideband packet data,** which is written once every 96 cycles of the sideband clock. This interval is based on the fact that 64 bits are received serially and accumulated via a deserializer, with **each Unit Interval (UI) spanning 32 sideband clock cycles.**

The **write operation is enabled only after the correct detection of the SBINIT out-of-reset packet**. Once enabled, the FIFO remains active for the subsequent **Link Training and Status Machine (LTSM) states starting from SBINIT**, allowing reliable data transfer during sideband protocol handshaking and link bring-up stages.

Figure 3-35 shows the block diagram of Asynchronous FIFO:

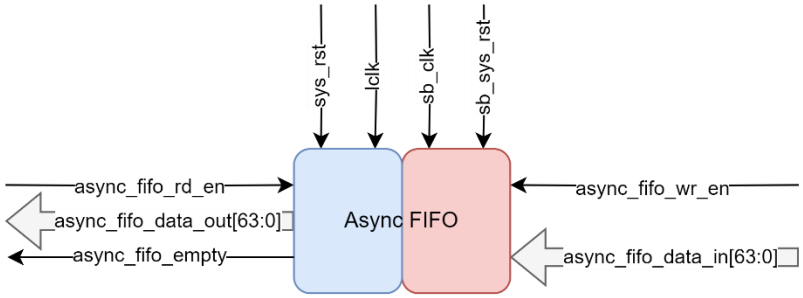


Figure 3-35: Asynchronous FIFO Diagram

All parameters and signals of the Asynchronous FIFO submodule are fully described in tables 3-58, and 3-59:

Table 3-57: Asynchronous FIFO Parameters

Parameters	Width
DATA_WIDTH	64

Table 3-58: Asynchronous FIFO Signals Description

Port	Direction	Width	Description
lclk	Input	1	From Interface Read domain clock.
sb_clk		1	From Interface Write domain clock.
sys_rst		1	From Interface Active-low asynchronous reset for the read domain.
sb_sys_rst		1	From Interface Active-low asynchronous reset for the write domain.
async_fifo_data_in		DATA_WIDTH	From 1x2 Demux 64-bit write data input.
async_fifo_wr_en		1	From Deserializer Write enable signal.
async_fifo_rd_en		1	From Decoder Read enable signal.
async_fifo_data_out	Output	DATA_WIDTH	To Decoder 64-bit data output.
async_fifo_empty		1	To Decoder High when FIFO is empty (prevents reading).

3.3.2.4. Decoder:

The **Decoder Module** serves as a critical interface that interprets incoming 64-bit data packets and directs them to appropriate system components based on message type and content. Its primary function is to analyze the raw input data, verify its integrity, decode the message type, and generate control signals that enable or disable specific modules accordingly.

The module tasks are:

1. **Message Parsing and Identification**

The module inspects the incoming 64-bit message, focusing particularly on bits [21:14], which encode the message type. This type field determines how the rest of the message should be interpreted, whether as a data packet, a control packet, a memory read/write command, or other specialized message formats.

2. **Header and Data Integrity Verification**

To ensure reliable operation, the decoder performs a parity check on each header by comparing a dedicated parity bit (bit 62) with the XOR of the remaining bits [61:0].

For messages with data, which typically consist of a header packet followed by one data packet, the decoder saves the parity flag from the header packet (dp_reg). This saved parity is then used to verify the parity of the subsequent data packet. This mechanism provides an additional layer of data integrity verification across message segments, ensuring that data packets are validated consistently with their headers.

3. **Control Signal Generation**

Depending on the decoded message type and the parity check results (including header-data parity verification), the decoder asserts or de-asserts several enable signals (module_en, module_partner_en, rdi_en) that activate different modules. This selective enabling ensures that only the relevant functional blocks process the current message.

4. **Message Sequencing and State Management**

Many messages span data word, beginning with a header followed by payload. The decoder maintains internal state flags (header_flag, waiting_for_data, data_flag, ignore_flag) to track its progress through these messages. After receiving a header with no errors, the decoder sets flags indicating it is awaiting subsequent data packets and uses the saved data parity flag to verify the integrity of these packets. This state management prevents data misinterpretation and helps synchronize decoding across clock cycles.

5. **Destination Selection and Routing**

The decoder uses the message content to set destination select signals (dest_sel), guiding the data to the appropriate units for processing. This routing ensures efficient data flow within the system and proper handling of diverse message types.

6. **Error Handling and Safety**

When an error is detected (such as parity failure or unrecognized message type), the decoder flags the error via decoder_error and disables all enable signals. This prevents corrupted or invalid data from propagating and potentially causing system malfunction.

7. **Header Data Packet Indicator**

The decoder's output includes an additional bit that signals whether the current header is followed by a data packet. This distinction is crucial for the credit-based flow control system, where each credit corresponds to a complete message consisting of both the header and its associated data packet. By indicating the presence of a data packet, the decoder helps the credit system decrement credits correctly (once per full message) preventing double decrementing when both header and data arrive separately. This ensures accurate tracking of available credits, avoids resource mismanagement, and maintains efficient data flow and buffer utilization within the system.

Figure 3-36 shows the block diagram of Decoder:

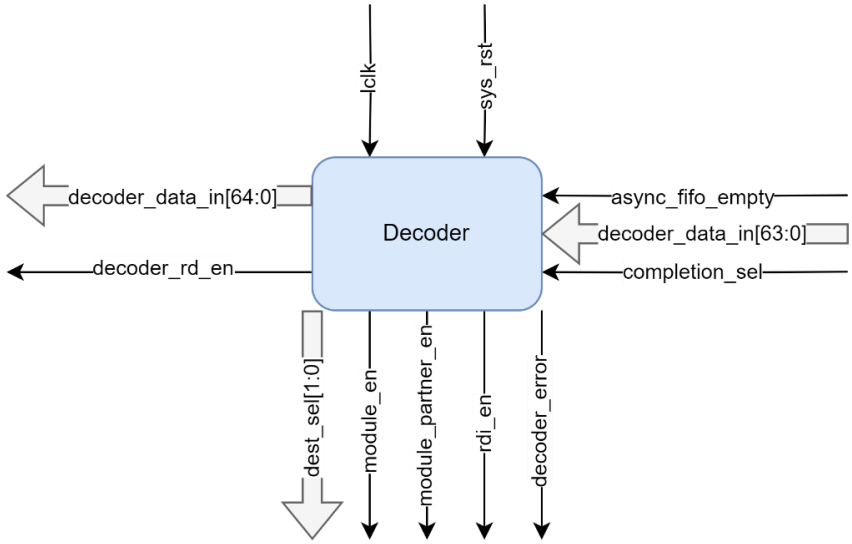


Figure 3-36: Decoder Diagram

All parameters and signals of the Decoder submodule are fully described in tables 3-60, 3-61, and 3-62:

Table 3-59: Decoder Parameters

Parameters	Width
DATA_WIDTH	64
DESTINATION_SEL	2

Table 3-60: Decoder Signals Description (sheet 1 of 2)

Port	Direction	Width	Description
lclk	Input	1	From Interface System clock signal.
sys_rst		1	From Interface Active-low asynchronous reset signal.
decoder_data_in		DATA_WIDTH	From Asynchronous FIFO The 64-bit input data bus carrying incoming message packets. Each packet may represent a header, or data payload.
async_fifo_empty		1	From Asynchronous FIFO This signal indicates that valid data is present in the FIFO and available for reading.
completion_sel		1	From SB_Tx This signal indicates that we receive a completion packet from sb_tx, so the decoder should register its data until sb_tx completes.

Table 3-61: Decoder Signals Description (sheet 2 of 2)

Port	Direction	Width	Description
decoder_data_out	Output	DATA_WIDTH+1	To 1x4 Demux Decoded and validated data output. Includes an additional status bit indicating whether the decoder is waiting for the remaining data of the current header packet.
decoder_rd_en	Output	1	To Asynchronous FIFO Control signal that enables reading packets from the asynchronous FIFO into the decoder module.
dest_sel	Output	DESTINATION_SEL	To 1x4 Demux A bus of signals selecting the destination block within the system for the current message data.
module_en	Output	1	To Controller Enable signal that activates phy_module based on the decoded message type.
module_partner_en	Output	1	To Controller Enable signal that activates phy_module_partner based on the decoded message type.
rdi_en	Output	1	To Controller Enable signal that activates rdi_module based on the decoded message type.
decoder_error	Output	1	To Controller A flag set when a parity error, message format error, or unrecognized message type is detected, indicating invalid data.

3.3.2.5. Synchronous FIFO:

The **Synchronous FIFO module** implements a parameterizable synchronous First-In-First-Out (FIFO) buffer, tailored for use in a sideband receiver datapath. It enables reliable, orderly, and synchronized storage of incoming packets based on the system clock, ensuring no data loss or corruption occurs during packet handoff between processing blocks.

This FIFO supports standard operations such as write and read through wr_en and rd_en signals, respectively, while asserting full and empty flags to prevent overflow or underflow conditions. The wr_data and rd_data ports are both (DATA_WIDTH+1) bits wide, where the extra bit is used to encode metadata—specifically, whether the current packet is a standalone header or a header followed by an associated data packet. This additional information is essential for ensuring protocol compliance and supporting a robust credit-based flow control mechanism.

This module is instantiated three times in the design to serve different traffic types with varying requirements in terms of priority and buffer depth:

1. **Completion Packet FIFO (Highest Priority)**
   Purpose: Stores completion packets received from the opposite die in response to local register access or from the local die.

Depth: 8 entries as the system supports up to 4 outstanding register access requests. Each completion can include both a header and a data packet (worst case), leading to a maximum of 8 packets.
Priority: This FIFO has the highest reading priority in the system without credit consumption.

2. **Request Packet FIFO (Lowest Priority)**
   Purpose: Buffers register access request packets that are to be processed locally received from the remote die.
   Depth: 2 entries as only 1 outstanding register access request is allowed from the opposite die. Since each request may contain a header and optional data, 2 entries are sufficient.
   Priority: This FIFO has the lowest read priority, as these requests are less time-critical than completion or general-purpose messages.

3. **General Message FIFO**
   Purpose: Handles all other packet types, including memory, control, and sideband messages.
   Depth: 64 entries as the system supports 32 credits for such messages, and since each message may consist of a header and a data packet, 64 entries are needed to buffer worst-case traffic.
   Priority: Medium priority, read only when higher-priority FIFOs (completion) are empty.

Figure 3-37 shows the block diagram of Synchronous FIFO:

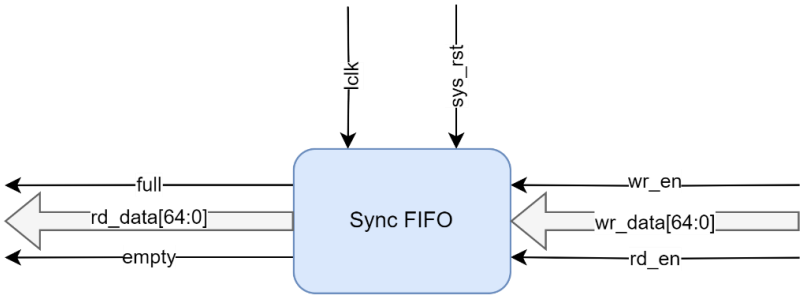


Figure 3-37: Synchronous FIFO Diagram

All parameters and signals of the Synchronous FIFO submodule are fully described in tables 3-63, 3-64, and 3-65:

Table 3-62: Synchronous FIFO Parameters

Parameters	Width
DATA_WIDTH	64
FIFO_DEPTH	2, 8, 64

Table 3-63: Synchronous FIFO Signals Description (sheet 1 of 2)

Port	Direction	Width	Description
lclk	Input	1	From Interface System clock signal.
sys_rst		1	From Interface Active-low asynchronous reset.
wr_data		DATA_WIDTH+1	From 1x4 Demux Input data to be written into FIFO. Extended by 1 bit for packet-type tagging.

Table 3-64: Synchronous FIFO Signals Description (sheet 2 of 2)

Port	Direction	Width	Description
wr_en	Input	1	From Decoder Write enable. When high and FIFO is not full, wr_data is stored into FIFO.
rd_en		1	From Priority Register Read enable. When high and FIFO is not empty, rd_data is updated.
rd_data	Output	DATA_WIDTH+1	To Priority Register Output data read from FIFO.
empty		1	To Priority Register FIFO empty flag. Asserted when no data is available for reading.

3.3.2.6. Priority Register:

The **Priority Register module** is responsible for managing prioritized data output from three FIFO buffers within the sideband receive path. It implements intelligent arbitration and sequencing logic to select one of three potential data sources, chunk the 64-bit packet into two 32-bit halves, and output valid register data.

This module:

1. Selects one valid data source (from the three FIFOs: **sync**, **completion**, or **request**) based on priority and **credit availability**.
2. Manages **read enable signals** to pull data safely from the selected FIFO.
3. **Chunks 64-bit data** into 32-bit halves, outputting one half at a time using reg_out, while storing the other half temporarily.
4. Asserts a **data-valid flag** and optionally a **credit-decrement signal** if the message is not a register access completion.
5. Ensure that **no data is sent while clock gating is active**, by enforcing a handshake protocol before transmission. Data output is only initiated once the **clock gating is removed**, either by completing the **clock handshake** with the adapter layer or entering the **active state**, guaranteeing that data is transferred only when the link is ready and power gating is properly handled.

Figure 3-38 shows the block diagram of Priority Register:

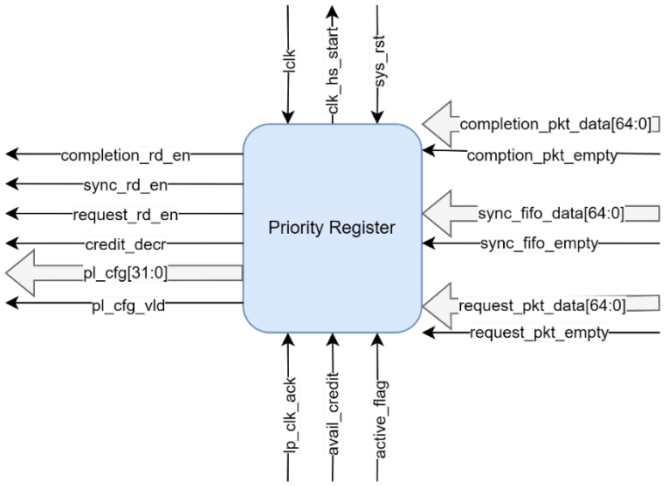


Figure 3-38: Priority Register Diagram

All parameters and signals of the Priority Register submodule are fully described in tables 3-66, 3-67, and 3-68:

Table 3-65: Priority Register Parameters

Parameters	Width
DATA_WIDTH	64
CHUNK_WIDTH	32

Table 3-66: Priority Register Signals Description (sheet 1 of 2)

Port	Direction	Width	Description
lclk	Input	1	From Interface System clock signal.
sys_rst		1	From Interface Active-low asynchronous reset signal.
completion_pkt_data		DATA_WIDTH+1	From Completion Buffer Packet Data from Completion_packet_buffer. Bit [64] is a flag for credit decrement; bits [63:0] carry the actual packet.
sync_fifo_data		DATA_WIDTH+1	From Synchronous FIFO Data from Sync_fifo. Bit [64] is a flag for credit decrement. Bits [63:0] carry the actual packet.
request_pkt_data		DATA_WIDTH+1	From Request Buffer Packet Data from Request_packet_buffer. Bit [64] is a flag for credit decrement. Bits [63:0] carry the actual packet.
completion_pkt_empty		1	From Completion Buffer Packet This signal indicates if Completion Packet Buffer is empty. Used to gate reading.
sync_fifo_empty		1	From Synchronous FIFO This signal indicates if the Sync FIFO is empty. Used to gate reading.
request_pkt_empty		1	From Request Buffer Packet This signal indicates if Request Packet Buffer is empty. Used to gate reading.
active_flag		1	From LTSM This signal indicates that the system is in the **active state**, meaning the **clock is ungated** and logic operations can proceed.
avail_credit		1	From Credit Counter This signal indicates if credits are available (for Sync and Request FIFOs). Required before initiating a read.

Table 3-67: Priority Register Signals Description (sheet 2 of 2)

Port	Direction	Width	Description
lp_clk_ack	Input	1	From RDI Interface This signal indicates the successful completion of the clock handshake process intended to **remove clock gating** and resume normal operation.
pl_cfg	Output	CHUNK_WIDTH	To RDI Interface Half of the selected 64-bit packet.
pl_cfg_vld		1	To RDI Interface This signal is high when pl_cfg holds valid data.
completion_rd_en		1	To Completion Buffer Packet Read enable for Completion_packet_buffer.
sync_rd_en		1	To Synchronous FIFO Read enable for Sync_fifo.
request_rd_en		1	To Request Buffer Packet Read enable for Request_packet_buffer.
credit_decr		1	To Credit Counter This signal indicates that credit should be decremented. Only asserted if the 65th bit ([64]) of the selected packet is 1.
clk_hs_start		1	To Controller This signal asserted to initiate the clock handshake with the adapter layer, requesting transition to normal clock mode.

3.3.2.7. Counter:

The **Counter module** implements a parameterizable up-down counter used to manage credit availability. The counter supports both increment (up) and decrement (down) operations but ensures that no change occurs when both signals are asserted simultaneously.

In this design, the counter:

- **Starts fully loaded** when reset is deasserted, representing **full credit availability**.
- **Increments** when up is high and down is low, unless they are already at maximum value.
- **Decrements** when down is high and up is low, unless they are already at zero.
- **Holds** when both up and down are either high or low.

Figure 3-39 shows the block diagram of Counter:

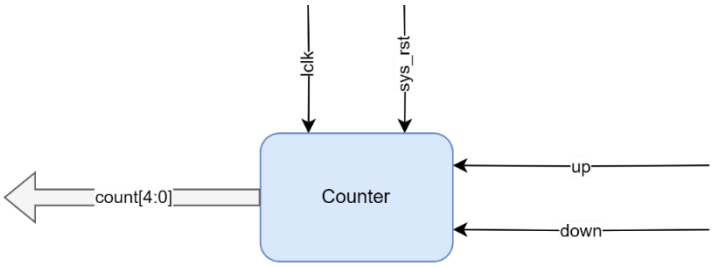


Figure 3-39: Counter Diagram

All parameters and signals of the Counter submodule are fully described in tables 3-69 and 3-70:

Table 3-68: Counter Parameters

Parameters	Width
WIDTH	5

Table 3-69: Counter Signals Description

Port	Direction	Width	Description
lclk	Input	1	From Interface System clock signal.
sys_rst	Input	1	From Interface Active-low asynchronous reset; sets counter to its maximum value when asserted (represents all credits available).
up	Input	1	From RDI Interface This signal requests to increment the counter by 1.
down	Input	1	From Priority Register This signal requests to decrement the counter by 1.
count	Output	WIDTH	To Priority Register The output bus represents the current number of available credits.

3.3.2.8. Synchronizer:

In asynchronous systems where different modules operate under distinct clock domains, metastability becomes a significant risk. To ensure reliable communication between such domains, dedicated synchronization logic must be employed. The Synchronizer module is a fundamental two-stage flip-flop synchronizer designed to safely transfer a single-bit control signal from one clock domain to another.

This module is instantiated **three times** in the sideband receive system to handle signals that must cross from one clock domain to another. Specifically:

- **Inputs to Controller ”pattern_detected and sb_o_detected”:**

Some input signals are originally generated in the sb_clk domain but are needed by the controller, which operates in the lclk domain. These signals are passed through instances of the Synchronizer to ensure reliable and stable handover between domains.

- **Outputs from the Controller “path_sel”:**

Conversely, the controller produces output signal that must be interpreted back in the sb_clk domain. This output is also synchronized using the same Synchronizer module when transitioning from lclk to sb_clk.

Figure 3-40 shows the block diagram of Synchronizer:

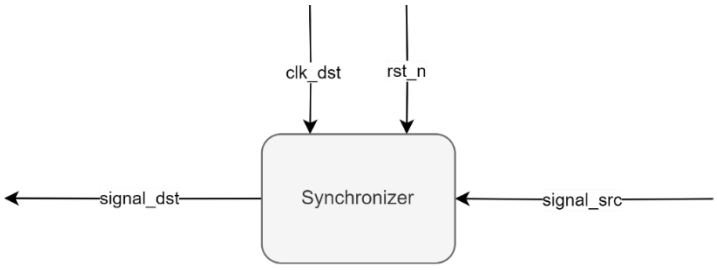


Figure 3-40: Synchronizer Diagram

All signals of the Synchronizer submodule are fully described in table 3-71:

Table 3-70: Synchronizer Signals Description

Port	Direction	Width	Description
clk_dst	Input	1	From Destination Domain Clock signal of the destination domain.
rst_n	Input	1	From Destination Domain Active-low asynchronous reset signal.
signal_src	Input	1	From Source Domain The asynchronous signal originating from the source domain.
signal_dst	Output	1	To Destination Domain The synchronized output of the signal, safe for use in the destination clock domain.

3.4. Mainband "MB":

3.4.1. Mainband Transmitter "MB_TX":

Block Functionality

- In this sub-chapter, we will discuss the main functionality of the Mainband Transmitter (MB_TX) part of the Physical Layer of UCIe.
- This part is mainly responsible for performing different link initialization and training tests, such as:
  - 128 iteration of VALTRAIN pattern for Valid Lanes (8'b11110000).
  - 128 iteration of Per-Lane ID pattern for Data Lanes (for ex.: Lane0 ID is 16'b0).
  - 4K iterations of LFSR pattern for Data Lanes.
- It is also responsible for data packets mapping into lanes, serializing and scrambling them, then finally sending them through the UCIe Link in the active state either lane reversed or in a direct connection.

Parameters and configurability:

- The Main-Band Transmitter part supports a configurable-lanes based on the “Lane_Reversal” signal which is decided by the PHY Controller if it will be applied or not and comes as an input to the MainBand Transmitter to be further applied on its lanes.

- The Lane Reversal feature is described by the following as stated in the UCIe specs document:
  - An example of Lane reversal is when physical Data Lane 0 on local die is connected to physical Data Lane (N-1) on the remote die (physical Data Lane 1 is connected to physical Data Lane N-2 and so on) where N = 8 for a x8 Standard Package, N = 16 for a x16 Standard Package. Valid, Clock, and sideband signals must not be reversed. Lane reversal is discovered and applied during initialization and training based on the training test results.

  - It is stated in the UCIe specs document that lane reversal is applied on the Transmitter (MB_TX) only and the Receiver (MB_RX) lanes **must not** be reversed. This is achieved by connecting the transmitter logical lanes (TD_L), which are the lanes after scrambling, in a reverse order with its transmitter physical lanes (TD_P), an example for a x16 Standard Package Module as in our case, TD_L[0] is connected to TD_P[15] and TD_L[15] is connected to TD_P[0] and so on for the rest of the lanes. After that, the physical lanes of the transmitter are connected in a reversed order too with the physical lanes of the receiver, an example for a x16 Standard Package Module as in our case, TD_P[0] is connected to RD_P[15] and TD_P[15] is connected to RD_P[0] and so on for the rest of the lanes in order to be de-scrambled by the same LFSR of each lane (same Seed value) to recover the original data and understand it, by which we will achieve lane reversing as stated in UCIe specs.

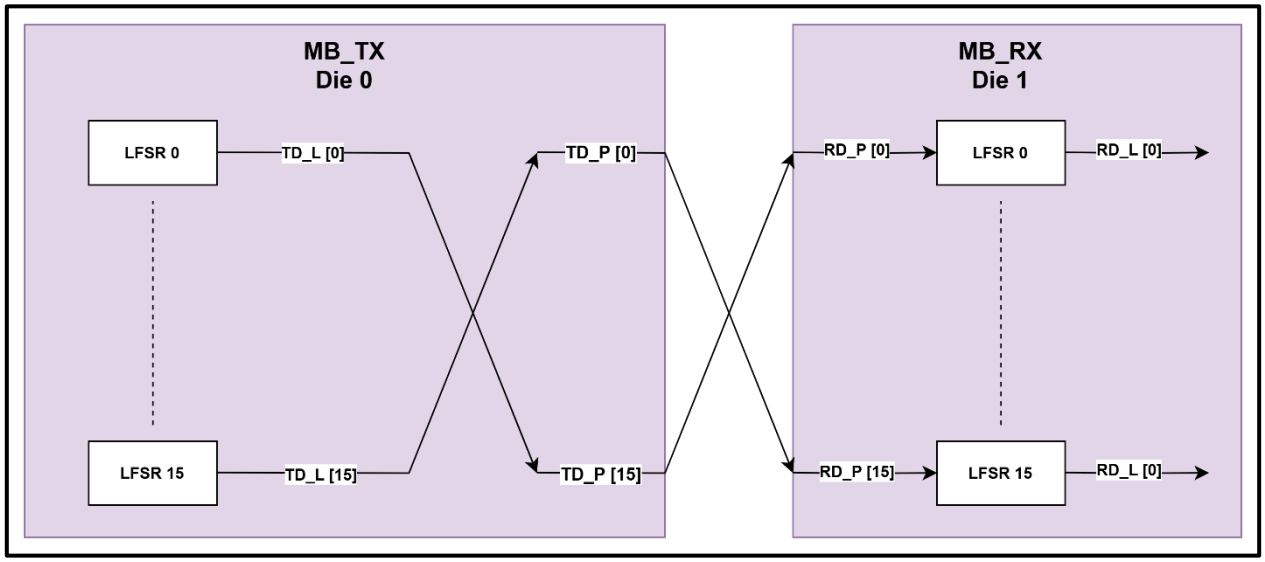


Figure 3-41: Lane Reversal feature mechanism

- Finally, the following figure shows clearly the difference between applying Lane Reversal and without applying it:

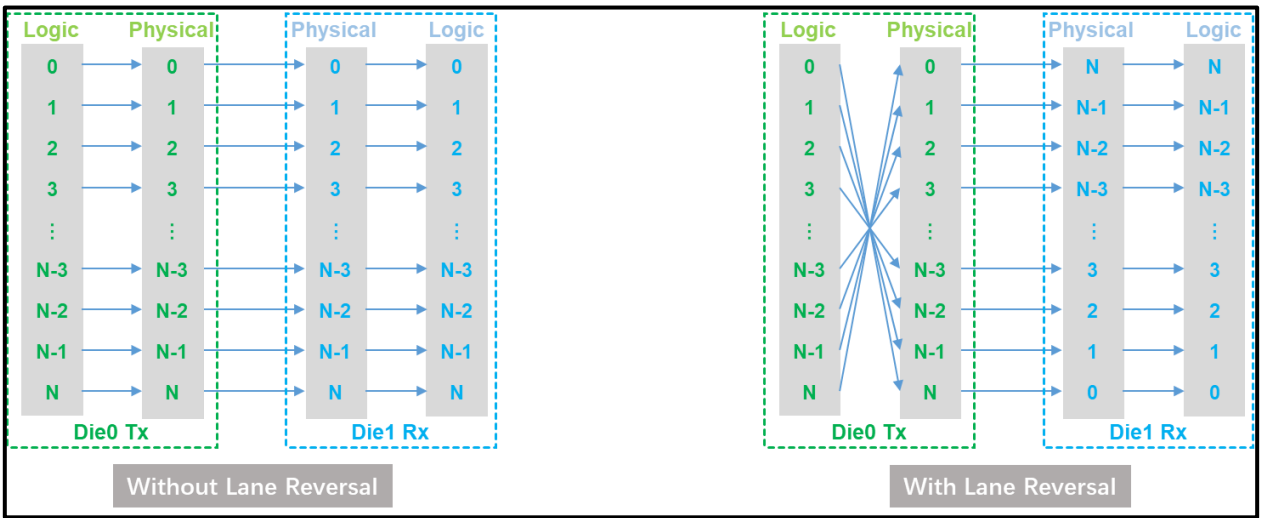


Figure 3-42: Normal and Reversed lanes connections.

Top Architecture

- The following figure will show the overall top architecture of the Mainband Transmitter as a black box with only its interface inputs/outputs signals with the outside world:

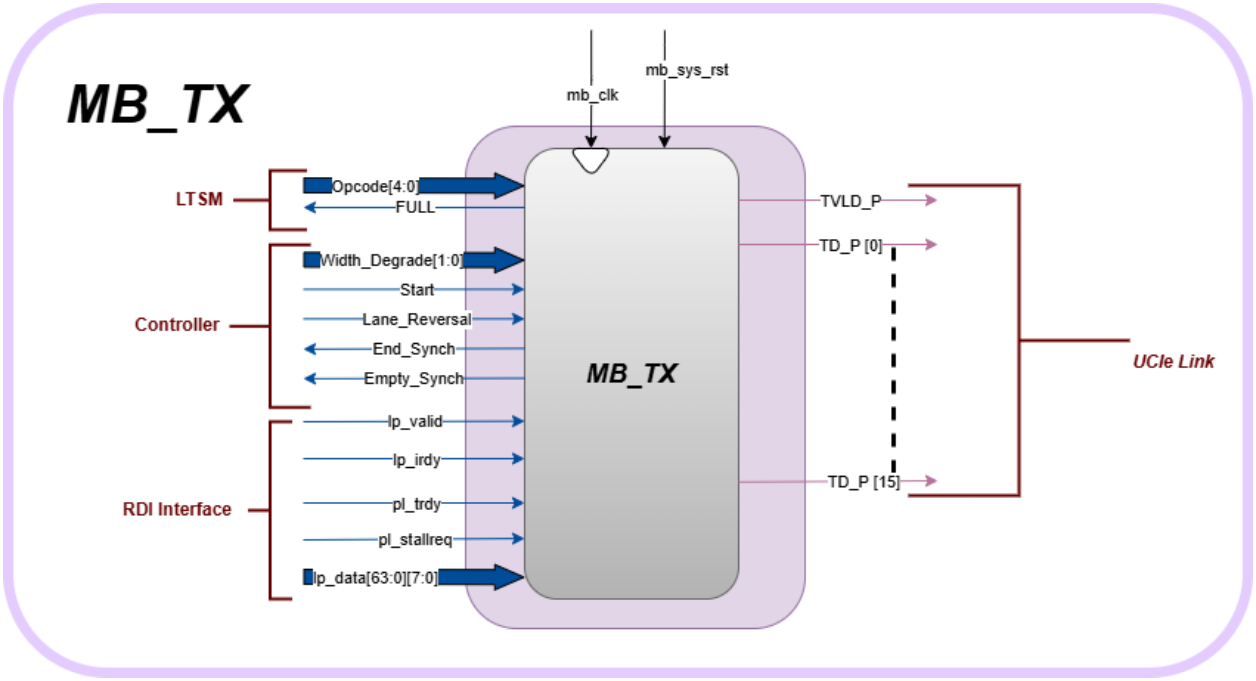


Figure 3-43: Main-Band Transmitter Top Architecture

Signals Interface

Table 3-71: Main-Band Transmitter Signals Interface (Sheet 1 of 2)

Port	Direction	Width	Description
MB_CLK	Input	1-bit	Main Band clock domain signal.
lclk		1-bit	System clock domain signal.
RST_n		1-bit	Asynchronous active-low reset synchronized with the Main Band clock domain.
lclk_RST		1-bit	Asynchronous active-low reset synchronized with the lclk domain.
Opcode		5-bits	Operation code coming from the LTSM block & indicates the current state of the LTSM.
Width_Degrade		2-bits	Number of active lanes encoding coming from the Controller.
Start		1-bit	A pulse signal coming from the Controller to trigger the MB_TX to start different tests.
Lane_Reversal		1-bit	A level signal indicates wether lane reversal is enabled or not coming from Controller.
lp_valid		1-bit	Adapter to Physical Layer indication that data is valid on the corresponding lp_data bytes.
lp_irdy		1-bit	Adapter to Physical Layer signal indication that the Adapter has data to send. This must be asserted if lp_valid is asserted and the Adapter wants the Physical Layer to sample the data.
pl_trdy		1-bit	A signal from Physical to Adapter Layer & indicates the the Physical Layer is ready to accept data.
pl_stallreq		1-bit	Physical Layer request to Adapter to align Transmitter at Flit boundary and not send any new Flits to prepare for state transition. It is asserted from the Controller Block.
lp_data		8x64	Array containing 64 bytes of input Adapter to Physical Layer data, coming through the RDI interface.
End_Synch	output	1-bit	A pulse signal synchronized to the lclk domain going to the Controller Block and indicates the end of a specific test or finishing the transmission of data packet.

Table 3-72: Main-Band Transmitter Signals Interface (Sheet 1 of 2)

Port	Direction	Width	Description
Empty_Synch	Output	1-bit	The ANDing result of the empty signal of both FIFOs synchronized to the lclk domain going to the Controller.
FULL		1-bit	The full signal of the active operation FIFO going to the LTSM to wether allow the adapter to write in the physical layer FIFO or not.
TVLD_P		1-bit	Physical Lane for Valid Transmitter on the UCIe Link. It indicates valid data presence on lanes.
TD_P		16-bits	Physical Lane for serial Data Transmitter on the UCIe Link of 16 different lanes.

Semi-Detailed Architecture

- Here, in this sub-section, we will zoom in a little bit on the above top architecture to have a closer view and to just know the main module blocks used for implementing the MB_TX without going deep into many details:

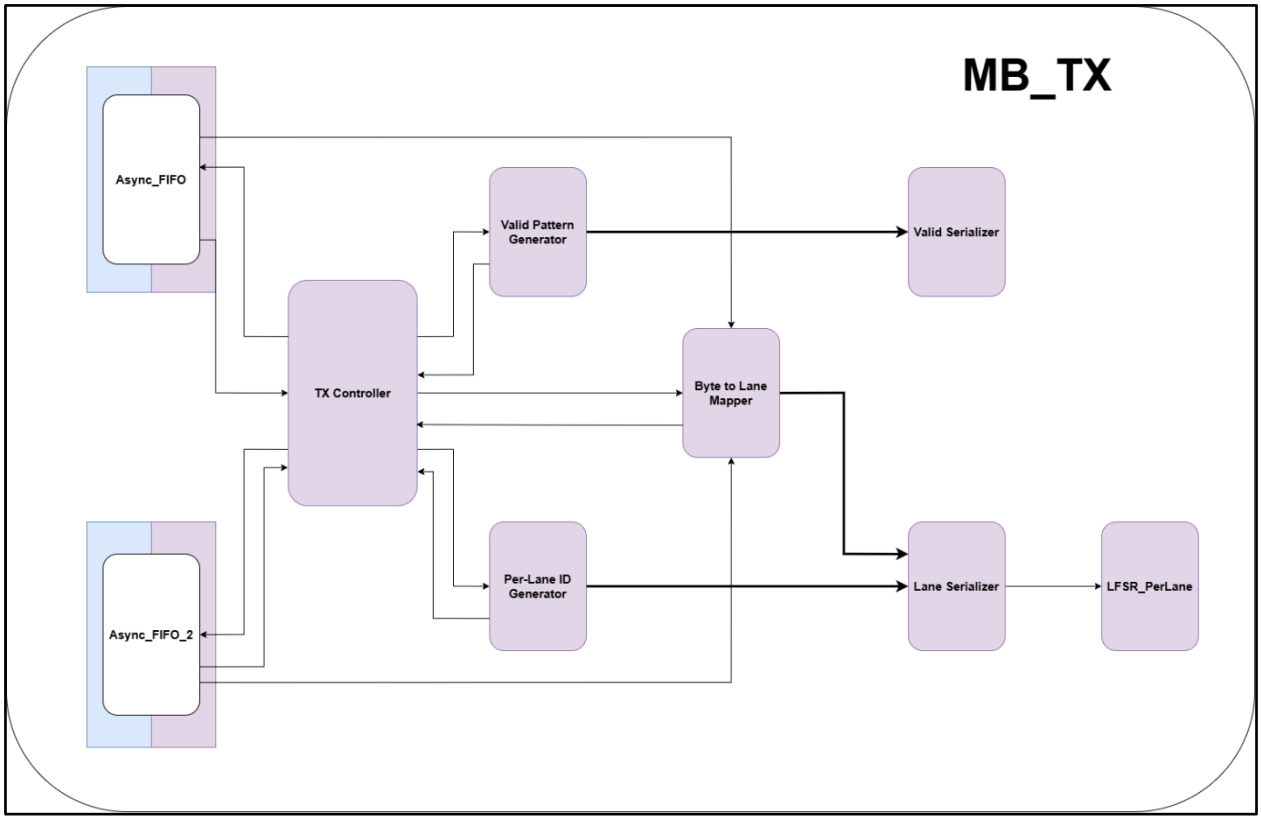


Figure 3-44: Main-Band Transmitter Semi-Detailed Architecture

Detailed Architecture

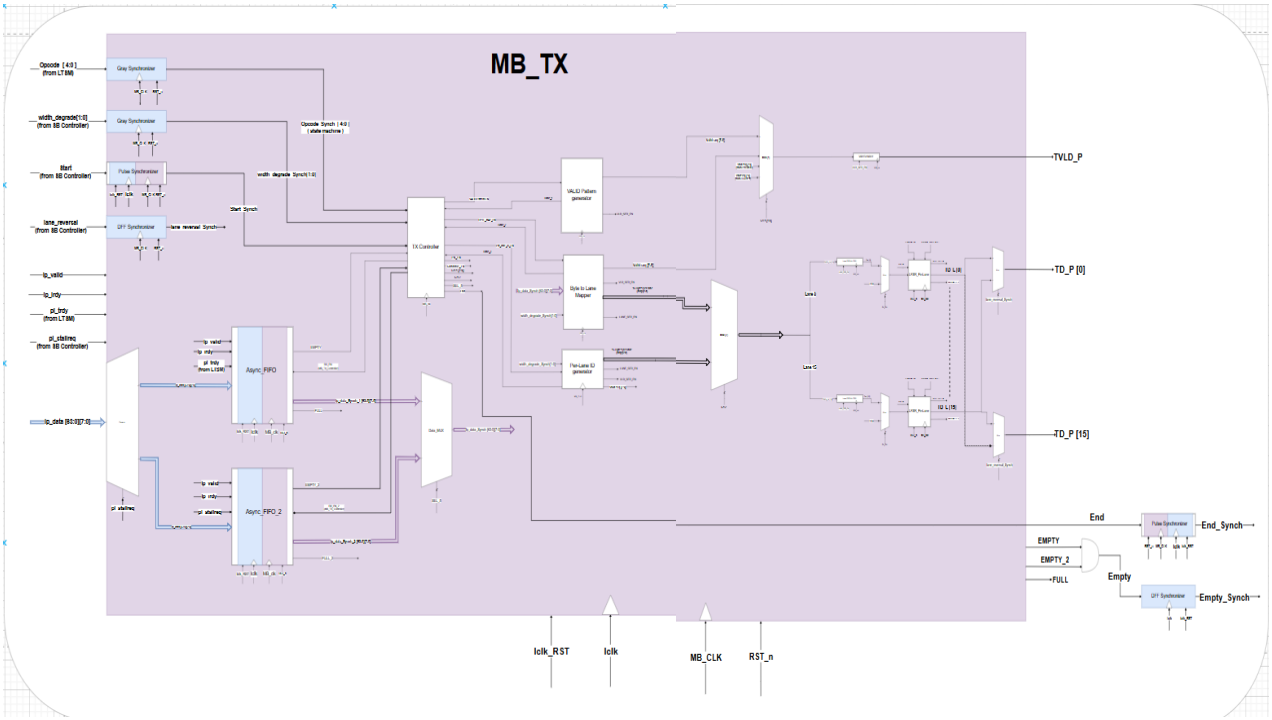


Figure 3-45: Main-Band Transmitter Detailed Architecture

Sub-Modules Description

Clocking and CDC:

- Today's chips have many clocks. Different parts of the circuit operate on different clocks. It is also common to have situations, where, the same portion of the circuit might perform multiple operations – sometimes on different clocks. Hence, it is very common to have data generated from one clock being consumed by some other clock.
- When data generated by a clock is captured by another clock which is asynchronous to the clock which generated the data, it is called Clock Domain Crossing (CDC).
- One of the biggest problems due to CDC is metastability, which means that the data capture is not reliable. Not reliable means, it might capture the intended value, or, it might capture the unintended value. Worse, it might not capture any value – and might go metastable and this could cause setup/hold time violations.
- So, Synchronizers are used to solve the above problems.
- There are different types of Synchronizers approaches based on the signal type (Pulse, Level or Bus) that needs to be synchronized.
- For Mainband Transmitter part, it is dealing with signals from **lclk** domain only, so different synchronizers types were needed for synchronizing different signals types which be discussed below.
- In the following part, we will be discussing the used Synchronizers approaches needed for different input signals.

Gray Synchronizer

Block Functionality:

- In digital systems involving multiple clock domains, transferring data safely across asynchronous boundaries is a critical challenge. Without proper synchronization, metastability can occur, leading to unpredictable behavior and potential system failure. One widely adopted solution to mitigate such issues is the use of Gray code in combination with multi-stage synchronizers.
- The **Gray Code Synchronizer** module presented in this work addresses the problem of transferring multi-bit binary data between two different clock domains. The design operates under the clock domain of the receiver (MB_CLK) and ensures robust data synchronization through a series of transformation and synchronization stages.
- The operation of the module begins by converting the incoming binary data into Gray code. Gray code, unlike binary, changes only one bit at a time between successive values, thereby reducing the probability of sampling errors due to partial transitions. This conversion is crucial when dealing with multi-bit signals that can exhibit glitches during simultaneous bit toggling in standard binary representations.
- Once the data is converted to Gray coded, it is passed through a two-stage flip-flop synchronizer.
- This double-flopping technique is a well-established practice in clock domain crossing (CDC) designs to minimize the likelihood of metastable events propagating through the system.
- Following synchronization, the Gray-coded signal is converted back into binary using a reverse Gray-to-binary algorithm. This step ensures that the data is restored to its original binary form before being utilized by downstream logic within the destination clock domain.
- For the Mainband Transmitter part, this approach (Gray Synchronizer) was used for synchronizing the interface data bus signals: **Opcode & Width_Degrade** mentioned above in MB_TX Signals Interface in ***Tables 3-71 & 3-72*** as they are multi-bit data buses coming from the **lclk** domain which can't be directly synchronized by the normal double flip-flop synchronizer and had to be gray encoded first.

Block Architecture:

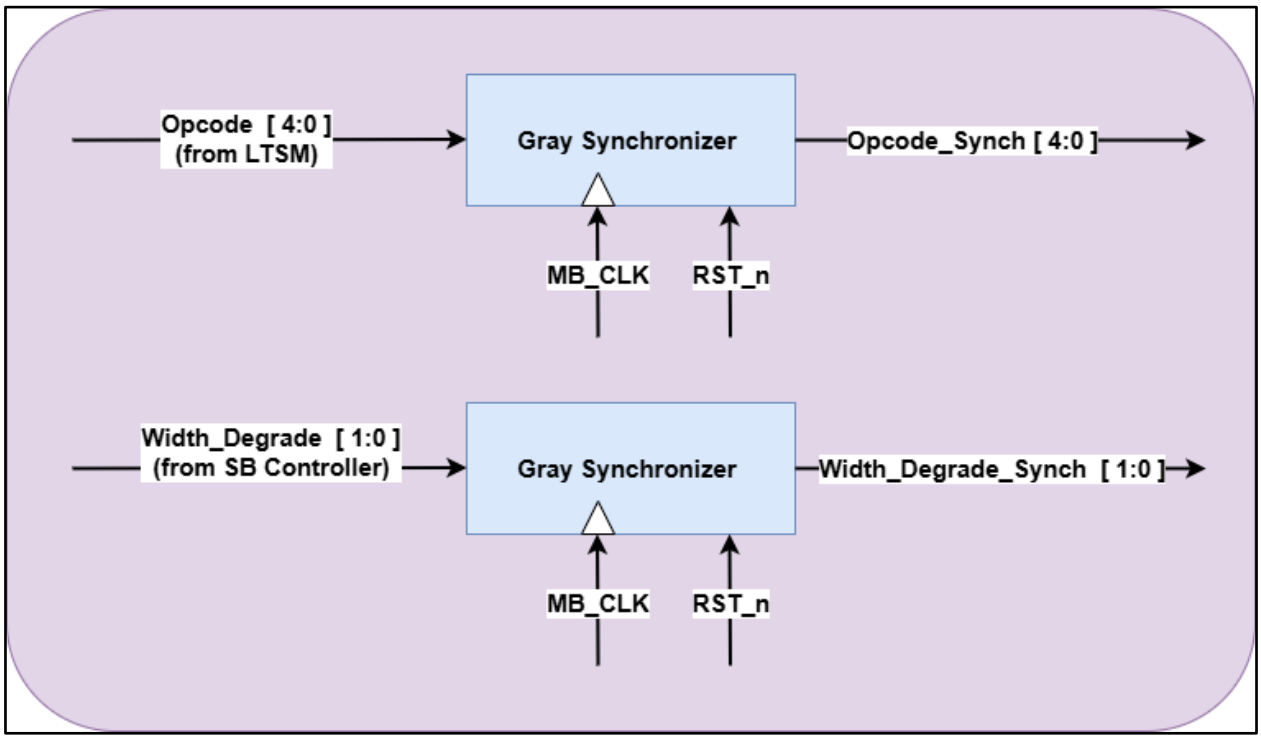


Figure 3-46: MB_TX Gray Synchronizer Block Architecture

Block Signals:

Table 3-73: MB_TX Gray Synchronizer Signals

Port	Direction	Width	Description
MB_CLK	Input	1-bit	Destination Main Band clock domain.
RST_n		1-bit	Asynchronous active-low reset synchronized with the destination Main Band clock domain.
Async_Data (Opcode or Width_Degrade)		Parametrized (DATA_SIZE)	Binary data bus input.
Sync_Data (Opcode_Synch or Width_Degrade_Synch)	Output	Parametrized (DATA_SIZE)	Synchronized Binary data bus output.

Block Parameters:

Table 3-74: MB_TX Gray Synchronizer Parameters

Parameter	Width	Description
DATA_SIZE	2-bits or 5-bits	Defines the data bus width of data needed to be synchronized.

Pulse Synchronizer

Block Functionality:

- A pulse generated in one clock domain typically lasts only a single clock cycle, and due to differing clock frequencies and phase relationships, such pulses can easily be missed or incorrectly interpreted in the destination domain. The **Pulse_Synchronizer** module is designed to safely and reliably transfer such single-cycle pulse signals between asynchronous clock domains.

- The **Pulse Synchronizer** block is responsible for transferring single-cycle pulse signals from a source clock domain (src_CLK) to a destination clock domain (dst_CLK). In systems with multiple asynchronous clock domains, directly passing such pulses can result in data loss due to timing mismatches or metastability. This block ensures robust and reliable transfer using a toggle-based edge detection synchronization mechanism combined with double flip-flop synchronization — a robust approach for minimizing the effects of metastability.

- The operation begins in the source clock domain. A toggle signal is generated in response to a rising edge of the pulse (src_pulse). The toggle changes state (from 0 to 1 or 1 to 0) only when a new pulse is detected, as determined by comparing the current and previous values of the input pulse. This approach ensures that even if the pulse is narrow, its occurrence is encoded into a persistent change in state that is easier to detect and synchronize.

- The generated toggle signal is then transferred into the destination domain via a **two-stage flip-flop synchronizer**. This technique ensures metastability containment and allows the destination domain to receive a clean and reliable version of the toggle signal. Once synchronized, the destination logic compares the current and previous values of the toggle signal to detect an edge (i.e., a change in state), and generates a one-cycle output pulse (dst_pulse) as an acknowledgment of the original pulse from the source domain.

- The final output, dst_pulse, is guaranteed to be: **Synchronized** with the destination clock domain, **Glitch-free**, and **Exactly one clock cycle in width**, regardless of the clock frequencies or phase offsets between the source and destination domains.

- Key benefits of the Pulse_Synchronizer module include:
  - **Reliable transfer of narrow pulses** across clock domains without loss or duplication.
  - **Minimized metastability** through the use of double flip-flop synchronizers.
  - **Deterministic pulse generation** in the destination domain, ensuring compatibility with state machines or sequential logic.
  - **Scalability and reusability**, as the module supports generic clock domains and can be integrated into larger systems requiring pulse-based handshaking.

- For the Mainband Transmitter part, this approach (Pulse Synchronizer) was used for synchronizing the interface pulse signal **"Start"** mentioned above in MB_TX Signals Interface in ***Tables 3-71 & 3-72*** which comes from **lclk** domain, it was also used for synchronizing the **"End"** output signal which is a pulse signal too coming from the **TX_Controller** block (wich will be discussed later on this section) inside the **MB_TX** part which operates with the **MB_CLK** and going to the **SB Controller** which works on **lclk**.

Block Architecture:

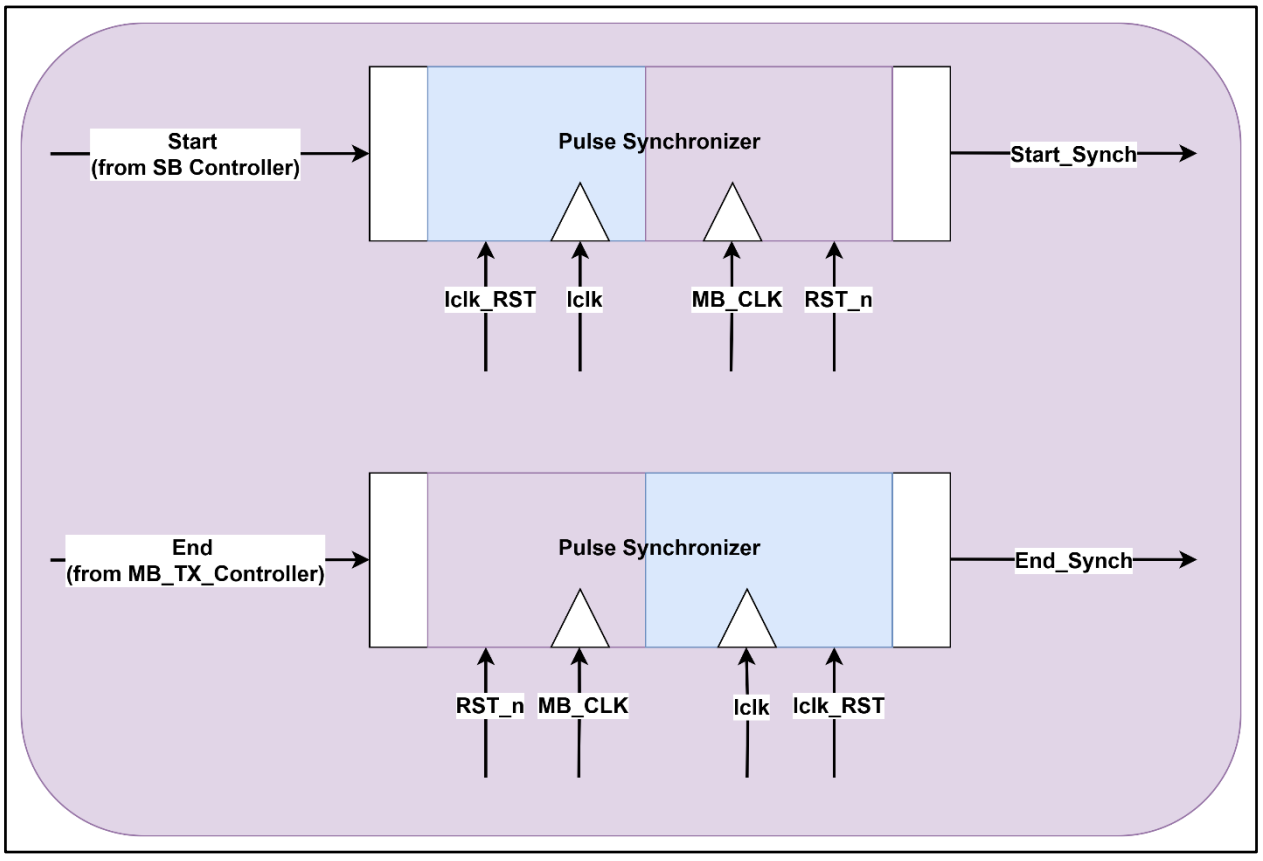


Figure 3-47: MB_TX Pulse Synchronizer Block Architecture

Block Signals:

Table 3-75: MB_TX Pulse Synchronizer Signals

Port	Direction	Width	Description
src_CLK (lclk or MB_CLK)	Input	1-bit	Source clock domain.
src_RST (lclk_RST or RST_n)		1-bit	Asynchronous active-low source reset synchronized with the source clock domain.
src_pulse (Start or End)		1-bit	Input pulse signal.
dst_CLK (MB_CLK or lclk)	Input	1-bit	Destination clock domain.
dst_RST (RST_n or lclk_RST)		1-bit	Asynchronous active-low destination reset synchronized with the destination clock domain.
dst_pulse (Start_Synch or End_Synch)	Output	1-bit	Output pulse signal synchronized with the destination clock domain.

Double Flop Synchronizer

Block Functionality:

- This is one of the simplest synchronizers used for synchronizing single-bit level signals.

- In ***Figure 3-48*** There is a CDC on flop F1. Hence, F1's output could go metastable. This output is fed directly to another flop F2– which is driven by the same clock as F1. F1 is also called destination flop. The second flop F2 is called the synchronizing flop. Now, the output of this second flop can be used for normal device operations. This technique of synchronization is called the Double Flop Synchronization Scheme.

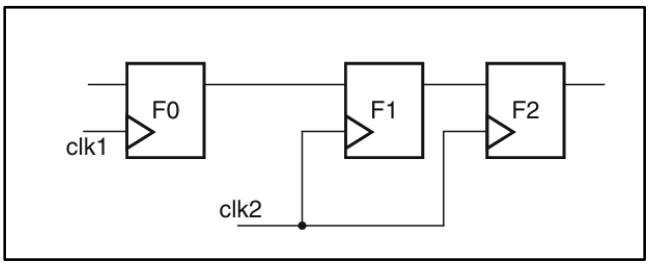


Figure 348-: DFF Synchronizer Diagram

- The fundamental concept behind such a scheme is that by the time of the next clock edge, the output of F1 should hopefully come to a stable state. And, F2 will be able to capture this stable state, which would now be visible to the rest of the design.
- There is usually no logic between F1 and F2. This ensures that the delay between F1 and F2 is very low. This allows that even if F1 settles close to the next clock edge, F2 gets the settled value.
- This synchronizer module addresses the issue by implementing a **two-stage flip-flop pipeline** in the destination clock domain (CLK). The objective is to safely capture and stabilize the asynchronous input (UnSynch_Input) and produce a clean, glitch-free output (Synch_Output), synchronized with the destination clock.
- The synchronizer consists of two sequential D flip-flops:

- **Stage 1** (UnSynch_Input_reg): Captures the potentially asynchronous input signal on the rising edge of the destination clock (CLK). At this point, the signal might still be metastable.
- **Stage 2** (UnSynch_Input_reg_2): On the next clock edge, the signal from the first stage is sampled again. By this time, the signal has had an entire clock period to resolve any metastability.
- **Synchronized Output**: The output Synch_Output is assigned the value of the second stage, ensuring that only stable, metastability-free signals are propagated into the destination logic.

- The reset (RST) is active-low and ensures that the flip-flops are initialized to a known logic level during system startup.
- The importance of the Double Flip-Flop Synchronizer lies in its ability to provide:
  - **Metastability Immunity**: While metastability cannot be completely eliminated, this circuit significantly reduces the probability of its propagation by giving the signal sufficient time to settle.
  - **Safe Cross-Domain Communication**: Ensures that asynchronous inputs are properly aligned with the internal system clock, avoiding timing violations.
  - **Glitch-Free Signal Integration**: Prevents hazards and spurious transitions that can result from unsynchronized signal sampling.
  - **Simplicity and Efficiency**: The structure is minimal in hardware resources and can be easily replicated across many inputs or control signals.
- For the Mainband Transmitter part, this approach (Double Flip/Flop Synchronizer) was used for synchronizing the interface input level signal **"Lane_Reversal"** mentioned above in MB_TX Signals Interface ***Tables 3-71 & 3-72*** which comes from **lclk** domain, it was also used for synchronizing the **"Empty"** output signal which is a level signal too coming from both **FIFOs (the ANDing of their Empty flags)** blocks (wich will be discussed later on this section) inside the **MB_TX** part which operates with the **MB_CLK** and going to the **SB Controller** which works on **lclk**.

Block Architecture:

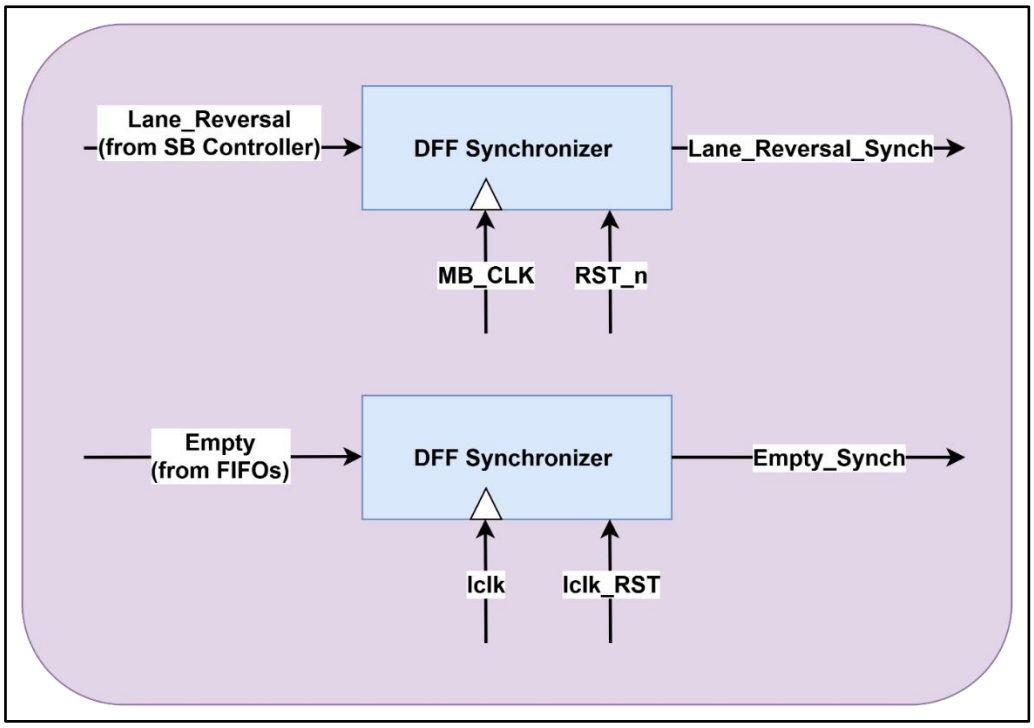


Figure 3-49: MB_TX DFF Synchronizer Block Architecture

Block Signals:

Table 3-76: MB_TX DFF Synchronizer Signals

Port	Direction	Width	Description
CLK (MB_CLK or lclk)	Input	1-bit	Destination clock domain.
RST (RST_n or lclk_RST)		1-bit	Asynchronous active-low reset synchronized with the destination clock domain.
UnSynch_Input (Lane_Reversal or Empty)		1-bit	Level input signal.
Synch_Output (Lane_Reversal_Synch or Empty_Synch)	Output	1-bit	Synchronized level signal output.

Asynchronous FIFOs

Block Functionality:

- Data Loss can also be prevented using FIFO (First In First Out) based mechanism. FIFO based mechanism is very useful for the following situation: The data launch is in bursts. The data launched in the bursts keep getting stored in the FIFO. While, the launch side becomes quite, the capture side keeps picking up the data stored in the FIFO.
- FIFO is helpful in such cases, because, it acts as an elastic buffer. So, for pairs of clocks which are very close in frequency, the launch clock can keep launching data– which gets stored in the FIFO, and, at each capture clock, the next data from the FIFO can be captured.
- If the launch frequency is slightly higher than the capture frequency, then, after several cycles, the FIFO might become FULL. At this stage, the launch has to be stopped. Within these cycles (of stopped launch), the capture clock will read a few more locations of the FIFO– making additional space in the FIFO– to resume the launch again & so on. And the vice versa, if the capture frequency is slightly higher than the launch frequency, then, after several cycles, the FIFO might become EMPTY. At this stage, the capture has to be stopped for some cycles. Within these cycles (of stopped capture), the launch clock will write into a few more locations of the FIFO– making additional data available in the FIFO– to resume the capture again & so on.

FIFO Basics:

- A FIFO is a 2-port memory. It has two clocks, one for read and one for write. It also has two addresses, one for read and one for write. A read happens at the location specified by read address, and is triggered at read clock. Similarly, write happens at the location specified by write address, and is triggered at write clock. The read and write addresses are generated by the respective counters, so that each subsequent read or write happens at the subsequent locations. The read and write addresses are typically called read pointer and write pointer respectively.
- Since, the memory employed is a 2-port memory, with separate address and clocks for read and write, both read and write can be independent of each other.
- In the context of a FIFO's application in CDC situation, the launch clock is used for write clock. As a new data is launched, the same data is written into the FIFO, and, the write pointer is updated– for the next data to be written onto the next location. Similarly, capture clock is used for read clock. The data for capture is read from the FIFO, and the read pointer is updated– for the next data to be

read from the next location. For either of the pointers, as the end of the FIFO is reached, it can start over from the first location of the FIFO.

Full and Empty flags Generation:

- When the read pointer tends to cross over the write pointer– it indicates that there are no valid locations to be read, which means that the FIFO is Empty. Similarly, when the write pointer tends to cross over the read pointer– it indicates that there are no valid locations to be written into, which means that the FIFO is Full. Thus, Full and Empty signals can be generated by comparing the read and write pointers of the FIFO. However, both the read and the write pointers are operating at asynchronous clocks. Hence, comparing them directly could result in failure, due to Clock Domain Crossing.

- The Empty signal is used to control the read side of the circuitry. Thus, for generating the Empty signal, you should synchronize the write pointer to the read clock. Similarly, for generating the Full signal, you should synchronize the read pointer to the write clock. Since the read and write pointers are both address busses, and they both are going to be synchronized to write and read clocks respectively, so, you should use grey-counters to generate read and write pointers then synchronize them using a double flip-flop synchronizer. ***Figure 3-50*** shows the generation of Full and Empty signals.

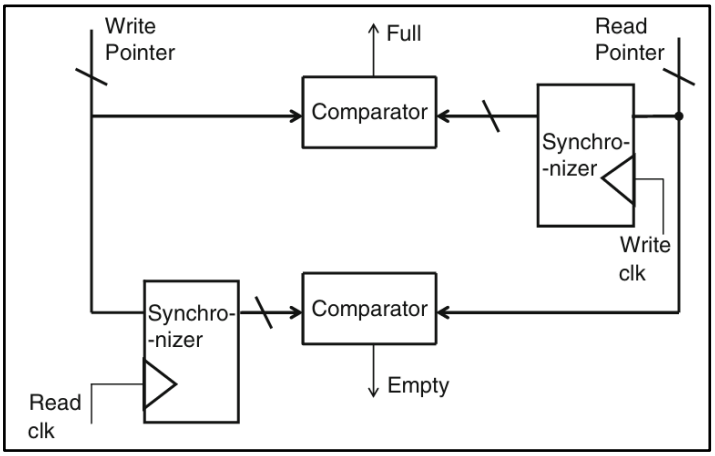


Figure 350-: FIFO FULL & EMPTY flags generation

- The schematic in ***Figure 3-50*** shows that for the generation of Full signal, the write pointer feeds in directly to the comparator. That means, the generation of the Full signal is without any delay. However, due to the delay (because of synchronization) on the read pointer side, the FIFO might take an extra cycle to de-assert the Full signal.

- Similarly, for the generation of Empty signal, the read pointer feeds in directly to the comparator. That means, the generation of the Empty signal is without any delay. However, due to the delay on the write pointer side, the FIFO might take an extra cycle to de-assert the Empty signal.

- So, effectively, the assertion of Full and Empty happens without any delay. But the de-assertion might involve an extra cycle. This means, there might be an additional wasted cycle; however, this additional wasted cycle is worth it– to prevent any failure.

Block Architecture:

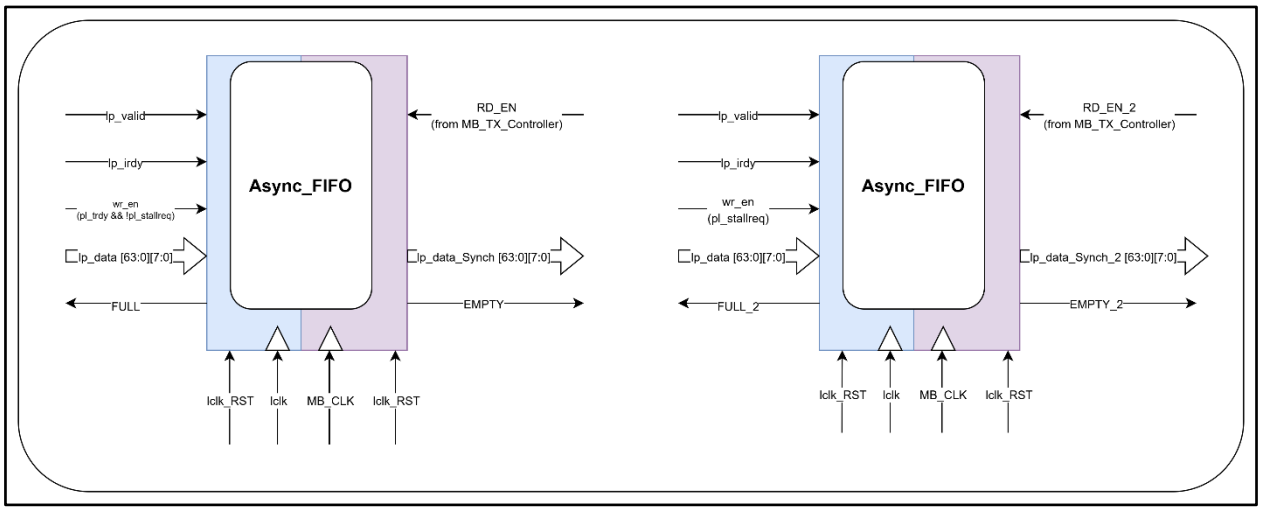


Figure 3-51: MB_TX Asynchronous FIFOs Block Architecture

Block Signals:

Table 3-77: MB_TX Asynchronous FIFO Signals (Sheet 1 of 2)

Port	Direction	Width	Description
lclk	Input	1-bit	Write clock domain.
lclk_RST		1-bit	Asynchronous active-low write domain reset.
MB_CLK		1-bit	Read clock domain.
rst_n		1-bit	Asynchronous active-low read domain reset.
lp_valid		1-bit	From RDI interface (defined above in ***Tables 3-71 & 3-72***)
lp_irdy		1-bit	From RDI interface (defined above in ***Tables 3-71 & 3-72***)
wr_en		1-bit	The ANDing result of both signals pl_trdy & !pl_stallreq (both are defined above in ***Tables 3-71 & 3-72***) for the first FIFO, while it is equal pl_stallreq signal for the second FIFO.
lp_data		64-bytes	Write data input (defined above in ***Tables 3-71 & 3-72***)
RD_EN		1-bit	Read enable

Table 3-78: MB_TX Asynchronous FIFO Signals (Sheet 2 of 2)

Port	Direction	Width	Description
lp_data_Synch	Output	64-bytes	Read data output.
FULL		1-bit	FIFO Full flag (defined above in ***Tables 3-71 & 3-72***)
EMPTY		1-bit	FIFO Empty flag.

Block Parameters:

Table 3-79: MB_TX Asynchronous FIFO Parameters

Parameter	Width	Description
DATA_WIDTH	64-bytes	Width of data in each FIFO location.
FIFO_DEPTH	4	FIFO depth (number of locations).

How it contributes to the UCIe PHY specs:

- The rationale for employing **two** FIFOs in the Mainband Transmitter part in the PHY Layer of UCIe is as follows:
  - The 1st FIFO is for storing the input **"lp_data"** array containing 64 bytes of input Adapter to Physical Layer data, coming through the RDI interface during the **ACTIVE** state of the **LTSM** (Link Training State Machine), to be further read and mapped to the lanes, then got serialized and scrambled till finishing the whole process and data is transmitted successfully through the serial **"TD_P"** lanes of the UCIe Link. This FIFO has a memory size of 64-Bytes per each place and with a depth of four which is the maximum flit size the Adapter may send to the Physical Layer (i.e. Maximum flit size is 256-Bytes which is equal to four chunks, where each chunk is 64-Bytes **"lp_data"**, so the Adapter may send the flit in four cycles, that's why the FIFO depth is four as this is the worst case).

  - While the 2nd FIFO is used for storing the "lp_data" bytes when transitioning from the **ACTIVE** state to **PHYRETRAIN** state, as it was mentioned in the UCIe specs that before this transition the **"Stallreq/Ack handshake Mechanism"** must be done first on the RDI which is mandatory for all FDI and RDI implementations and states that any data presented on the interface must be accepted by the physical layer when the **"pl_stallreq"** signal is asserted until the rising edge of **"lp_stallack"** signal and transmitting them all and any pending data through the mainband, then we can transition to the PHYRETRAIN state safely after finishing these transmissions, so we had to make sure that there will be an empty places with a maximum four places (as this is the worst case as stated above) when this transition is requested, that's why we had to put the 2nd FIFO to be ready and used only when this case happens.

  - So, finally the RDI must ensure that both FIFOs are empty and all data are transmitted before state transition.

- The way we handled the contribution between the two FIFOs and how we implemented and achieved the specs in our design is as follows:

- First of all, the input “lp_data” was passed through a De-multiplexer with the signal “pl_stallreq” connected to its selection line signal that indicates if the Stallreq/ack handshake is started or not.

- If this “pl_stallreq” is not asserted this means that we are still in the active operation and the “lp_data” will be passed and written in the 1st FIFO as shown in ***Figure 3-45.*** The Write Enable of this FIFO is the ANDing result of the signals: “lp_valid”, “lp_irdy”, “pl_trdy” and “!pl_stallreq”, knowing that the NOT of the “pl_stallreq” signal to make sure that data will be written in this FIFO in case of we are in the ACTIVE state only.

- Then, during the normal data transmission in ACTIVE state, the data will be read out form the 1st FIFO by the Read Enable control signal coming from the Maniband Transmitter Controller Block.

- If “pl_stallreq” is asserted, the input “lp_data” will be written in the 2nd FIFO, where its Write Enable signal is the ANDing result of the signals: “lp_valid”, “lp_irdy” and “pl_stallreq”, but it won’t be read till finishing transmitting the whole data remained in the 1st FIFO.

- Both FIFOs data out are then Multiplexed with a selection line signal coming from the Maniband Transmitter Controller Block as it is the block knows which FIFO is not Empty and can be read from, so decides the Multiplexer selection signal based on which FIFO we are currently reading from.

- Finally, the Multiplexer data output is then sent to the Byte-to-Lane Mapper Block to be further Processed and transmitted through the UCIe Link.

Other Reliability Concerns during synchronization

- Even though we have synchronized individual bits and busses, and, have taken care of data-loss, there could still be some reliability issues, which need to be guarded against. ***Figure 3-52*** shows a signal coming from domain clk1 and is needed at multiple places in domain clk2. Here, it has been synchronized multiple times in clk2 domain.

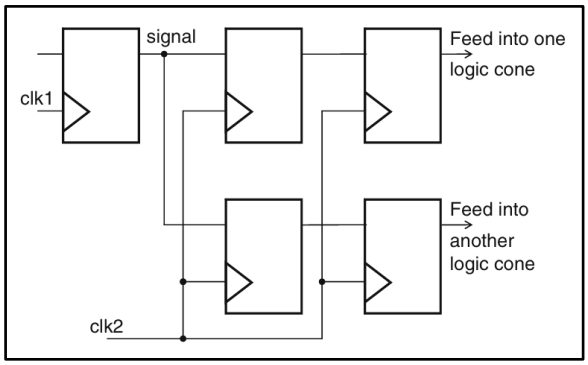


Figure 352-: Same signal synchronized twice independently

- As can be seen in ***Figure 3-53***, the same signal has stabilized to two different values in the two independent synchronizations. This might happen because once the f lops have entered into a metastable state, each of them will resolve independently which could be to two different logic levels. Thus, the same signal is being read differently in different parts of the destination domain. So, once again, there is a loss of reliability, even though, the signal is synchronized individually at all its points of usage.

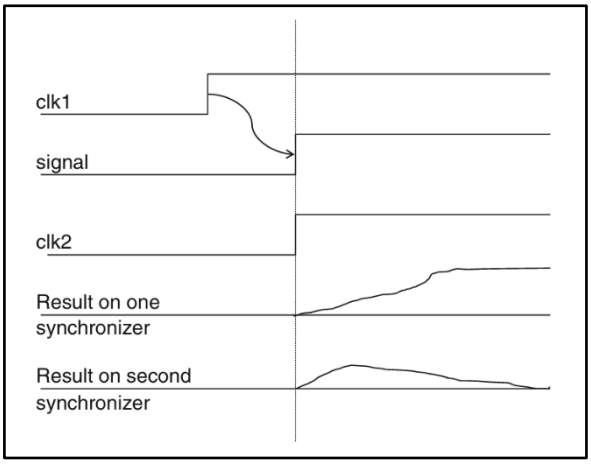


Figure 353-: Waveforms for same signal synchronized twice– independently

- So, it is very important that we should never synchronize the same signal more than once in the same destination domain. If the signal is required to be used at multiple places in the destination domain, we should synchronize it only once and then use it at as many places as required. ***Figure 3-54*** shows the right way.

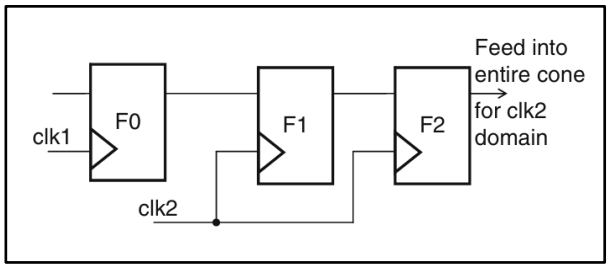


Figure 3-54: Synchronized signal only once

- An extension of this same concept shows that we should not allow to reconverge two or more signals synchronized independently. The resultant after the reconvergence could be totally different from the original combination. If a combination of multiple signals is needed, they should be combined together in the source domain, and the resultant output should be synchronized to the destination domain.

MB TX Controller:

Block Functionality:

- The Mainband Transmitter Controller (TX_Controller) module coordinates the transmission process of serialized data across different functional modes based on opcode and synchronization signals. It serves as the central finite state machine (FSM) or the main brain of the Mainband Transmitter part that controls the activation, sequencing, and mode selection for various data preparation blocks used in the physical layer of a high-speed communication system.
- Controller must only sample the data path through status signals to determine what to do next.
- Controller imposes what to do next on data path by control signals which go from the controller to the remaining blocks it interfaces with.
- This block core functionality is implemented through a multi-state **Moore FSM (Finite State Machine)**, which is the most common way used in implementing controllers in the digital design world.
- This FSM transitions between idle, training, lane configuration, test pattern, and active transmission states based on incoming control signals and opcode conditions.
- This block interfaces with several key components, including asynchronous FIFOs, LTSM (Link Training and Status Machine) synchronized signals, and data encoding/formatting blocks such as scramblers, training pattern generators, and byte-to-lane mappers.

- The controller operates synchronously with the mainband clock (MB_CLK) and reacts to decoded opcodes (Opcode_Synch), start signals from the Controller (Start_Synch), FIFO status flags (EMPTY, EMPTY_2), and data width configurations (Width_Degrade_Synch).
- The following includes its primary functional roles:
  - **State Transition and Control Logic**, the internal FSM manages five operational states:
    - IDLE: Default reset or wait state, where outputs are inactive.
    - VALTRAIN: Enables training pattern generation for link calibration.
    - PerLaneID: Activates per-lane training and mapping controls.
    - LFSR: Enables scramblers for test modes.
    - ACTIVE: Initiates actual data transmission through enabled byte mapping and optional scrambling.
  - Transitions between these states are driven by a combination of opcode values and handshaking signals indicating readiness (Start_Synch), FIFO availability, and operation completion (Done_1, Done_2, Done_3).
  - **Output Signal Control**, each state activates a specific set of control signals to direct subordinate modules:
    - VALTRAIN_EN, PerLaneID_EN, BYTE_MAP_EN: Enable corresponding functional blocks.
    - PN_EN, Scramble_EN: Control pseudorandom and scrambling operations.
    - SEL_1, SEL_2, SEL_5: Control multiplexers for selecting appropriate data paths.
    - RD_EN, RD_EN_2: Read-enable signals for two FIFO channels, depending on availability.
    - End: Signals the end of a particular transmission phase or a specific test back to the SB Controller.
  - **Counters and Timing Control**, the FSM employs internal counters (counter, LFSR_counter) to manage the duration of various states. For instance, the LFSR state includes a 13-bit counter to ensure a full 4K of LFSR pattern sequence before transitioning back to IDLE. These counters also help determine when to assert End, which signals the completion of the current operation.
- This controller ensures that all transmission phases, training, alignment, test patterns, and active data occur in a coordinated and orderly manner, avoiding timing conflicts or data corruption.
- It incorporates read control based on FIFO status (EMPTY, EMPTY_2), preventing underflows or invalid data access during transmission.
- It bridges the Link Training and Status Machine (LTSM) with physical layer blocks such as scramblers, byte mappers, and test pattern generators, enabling the full transmission stack to function cohesively.
- Finally, this block plays a vital role in managing the transmitter's behavioral modes in real-time, based on upper-layer commands and hardware status.

Block Architecture:

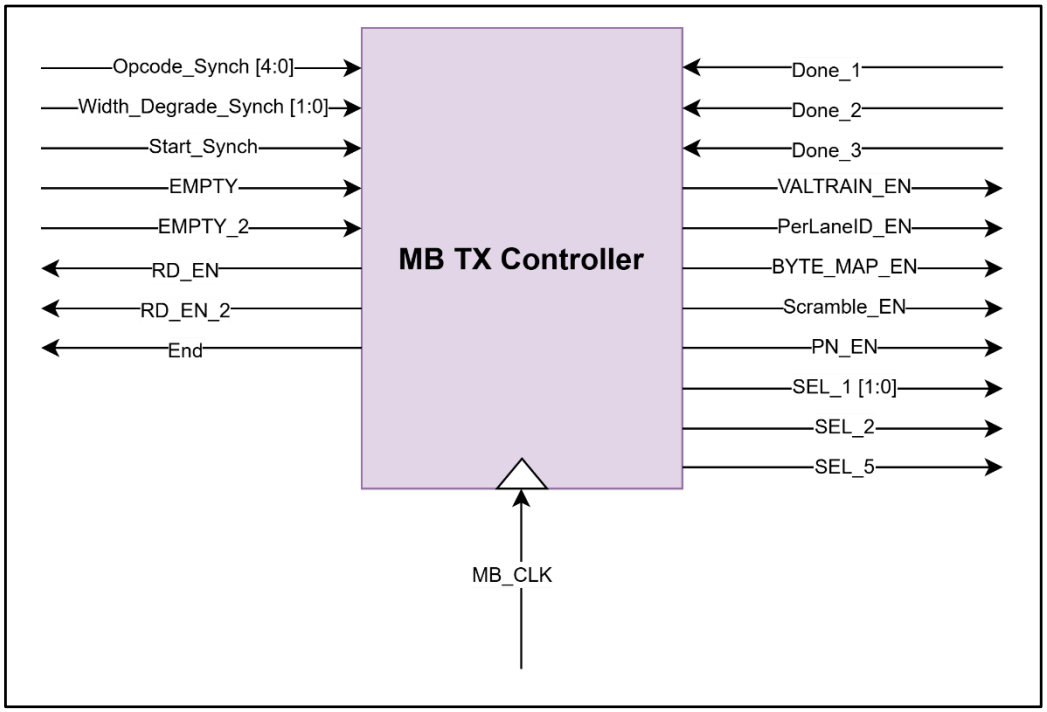


Figure 3-55: MB_TX Controller Block Architecture

Block Signals:

Table 3-80: MB_TX Controller Signals (Sheet 1 of 2)

Port	Direction	Width	Description
MB_CLK	Input	1-bit	Mainband clock domain.
Opcode_Synch		5-bits	Synchronized Operation code coming from the LTSM block & indicates the current state of the LTSM which determines which operation will be done through the MB_TX.
Width_Degrade_Synch		2-bits	Number of active lanes encoding coming from the Controller after being synchronized.
Start_Synch		1-bit	The synchronized pulse signal coming from the Controller to trigger the MB_TX to start different tests.
EMPTY		1-bit	Empty flag of the 1ST FIFO.
EMPTY_2		1-bit	Empty flag of the 2nd FIFO.
Done_1		1-bit	A done signal coming from the Valid Pattern Generator block indicating that the last byte of the test pattern is successfully sent on the Valid Lane.

Table 3-81: MB_TX Controller Signals (Sheet 2 of 2)

Port	Direction	Width	Description
Done_2	Input	1-bit	A done signal coming from the Byte-to-Lane Mapper block indicating that the last data bytes are mapped to the lanes and will be serialized.
Done_3		1-bit	A done signal coming from the Per-Lane ID Generator indicating that the last bytes of the PerLaneID test pattern is successfully sent on the Data Lanes and will be serialized.
RD_EN		1-bit	Read enable signal going to the 1st FIFO.
RD_EN_2		1-bit	Read enable signal going to the 2nd FIFO.
End		1-bit	A pulse signal indicating that the data or the tests had successfully serialized and sent through the UCIe Link after which the MB_TX Controller goes to the IDLE state, it's going to the controller to continue the remaining tests or messages sequences and to enable the MB to do further operations again by sending to it the "Start" signal.
VALTRAIN_EN	Output	1-bit	Signal enables the Valid Pattern Generator block to start the test.
PerLaneID_EN		1-bit	Signal enables the Per-Lane ID Generator block to start the test.
BYTE_MAP_EN		1-bit	Signal enables the Byte-to-Lane Mapper block to start mapping data into Lanes.
Scramble_EN		1-bit	Enable signal to allow data scrambling or not (in case of PerLane ID test).
PN_EN		1-bit	Enable signal to the LFSR block to indicate if it's normal transmission or a LFSR test pattern.
SEL_1		2-bits	Valid Mux selection to indicate the valid signal coming from which block.
SEL_2		1-bit	Lane Mux selection to indicate the lanes having mapped data bytes or Per-Lane ID pattern.
SEL_5		1-bit	Mux selection for the data output of the two FIFOs.

Valid Pattern Generator:

Block Functionality:

- The **Valid Pattern Generator** module (VALTRAIN_Generator) is a dedicated transmission block used to generate a predefined, repeatable training pattern used during link initialization, calibration, or alignment phases. It plays a crucial role in the transmitter pipeline by providing a known bit pattern to the serializer when VALTRAIN_EN is asserted, allowing the receiver to perform synchronization and signal integrity measurements before live data transmission begins.

- This module is typically activated during the **valid training phase** controlled by the **Mainband Transmitter Controller (TX_Controller)** and remains operational for a fixed number of iterations to ensure robust pattern transmission across Valid lane.

- It generates a valid pattern sequence which is a specific 8-bit pattern (8'b11110000), where this test pattern is used in functional check of the **Valid** signal.

- It generates a valid pattern sequence which is a specific 8-bit pattern (8'b11110000), where this test pattern is used in functional check of the **Valid** signal.

- This test is done in a predefined LTSM states by the UCIe specs document which are: "**REPAIRVAL**", "**VALVREF**", "**VALTRAINCENTER**" and "**VALTRAINVREF**".

- The generator operates synchronously with the main transmitter clock (MB_CLK) and responds to a control signal VALTRAIN_EN. When enabled, it executes the following sequence:

  - **Pattern Generation Logic**
    - The module generates a **fixed 8-bit pattern (8'b00001111)** on the output signal **Valid**.
    - This pattern is asserted only during specific byte boundaries controlled by an internal byte_count counter variable which ensures that the valid pattern is emitted in byte-aligned intervals and prevents overlapping or misaligned transmission.
    - If the byte_count is non-zero, the module suppresses pattern generation and continues counting to the next aligned position.
    - The VLD_SER_EN signal is asserted to enable serialization of the valid pattern to the physical layer.
    - A separate counter, iteration_count, counts up to **128 iterations (0 to 127)** which is the determined number of iterations in the UCIe specs document.
    - Upon completion the 128 iterations, the module sets the Done_1 signal high, indicating the completion of the training pattern generation phase.
    - If VALTRAIN_EN is de-asserted, all internal counters and output signals are reset, ensuring a clean and repeatable start on the next activation cycle.

Block Architecture:

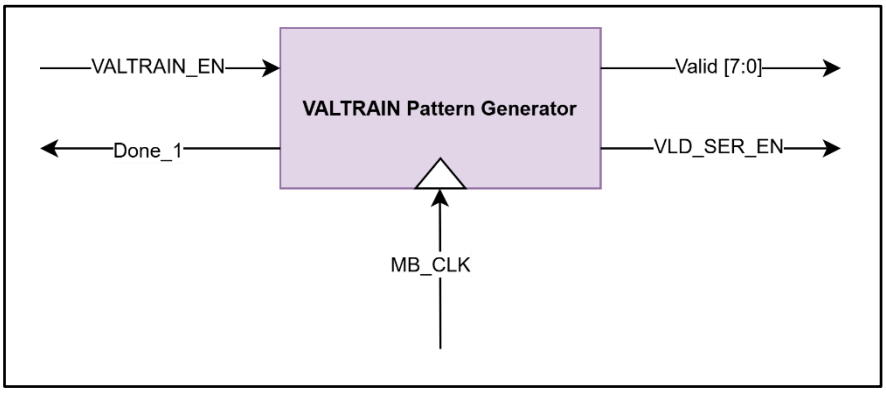


Figure 3-56: MB_TX VALTRAIN Pattern Generator Block Architecture

Block Signals:

Table 3-82: MB_TX VALTRAIN Pattern Generator Signals

Port	Direction	Width	Description
MB_CLK	Input	1-bit	Mainband clock domain.
VALTRAIN_EN		1-bit	Signal enables & controls the activation of the Valid Pattern Generator block to start the test, it is coming from the MB TX Controller
Valid	Output	8-bits	Valid test byte pattern sequence (8'b11110000).
VLD_SER_EN		1-bit	Serializer enable signal to serialize the Valid pattern and send it through the "TVLD_P" signal (defined above in ***Tables 3-71 & 3-72***) of the UCIe Link.
Done_1		1-bit	A done signal going to the MB TX Controller block indicating that the last byte of the test pattern is successfully sent on the Valid Lane.

Per-Lane ID Generator:

Block Functionality:

- Since each data lane often requires a unique identifier to ensure proper tracking and processing, so the **Per-Lane ID Generator** module is responsible for generating and broadcasting **unique & distinct 16-bit lane identifiers (IDs)** across each physical data lane, ensuring that the receiver can distinguish between lanes and verify proper connectivity and mapping during link training or diagnostics.

- This module is typically activated during the **Per-Lane Training (D2C)** phase, where identification and calibration of individual transmission lanes are required. The block works in close coordination with the **Mainband Transmitter Controller (TX_Controller)** and adapts dynamically to changes in link width or lane degradation.

- The block operates synchronously with the transmitter clock domain (MB_CLK) and is enabled when the PerLaneID_EN signal is asserted. Upon activation, the module generates, distributes, and serializes a pattern that uniquely identifies each of the 16 available lanes, depending on the negotiated link width specified by the Width_Degrade_Synch signal.

- Upon activation (PerLaneID_EN = 1), the module transitions through a deterministic process comprising the following key operational phases:
  - **Pattern Construction**
    - Each lane is assigned a unique 16-bit pattern of the format: {4'b1010, i[7:0], 4'b1010}, where i is the lane index from 0 to 15. This creates a framed identifier for each lane, making it easy to distinguish and detect on the receiver side. The framing bits (1010) provide clear boundaries for alignment, while the embedded lane index allows unambiguous identification.

- These IDs are used in sending the required 128 iterations of the Per-Lane ID Pattern, where this number of iterations are specified by the specs, to be used in the D2C point test that is needed in different LTSM states which are: "**REVERSALMB**" which detects whether Lane Reversal is needed or not and "**REPAIRMB**" which checks for functionality & indicates whether Width Degrade is performed or not.

- **Byte-Wise Transmission**
  - Since each pattern is 16 bits and the lane data interface is 8 bits wide, the transmission is split into two phases:

    First Cycle (Lower Byte):
    - The lower 8 bits [7:0] of each pattern are assigned to the corresponding Lanes[i].
    - The internal signal byte_position is used to track whether the current cycle is emitting the lower or upper byte.

    Second Cycle (Upper Byte):
    - The upper 8 bits [15:8] of each pattern are assigned to Lanes[i].

- **Link Width Adaptation**
  - The module supports dynamic adaptation to the current link width using the 2-bit input Width_Degrade_Synch, which defines which lanes are active:
    - 00: All lanes inactive — zeroed out.
    - 01: Lower 8 lanes active (Lanes[0–7]), upper lanes zeroed.
    - 10: Upper 8 lanes active (Lanes[8–15]), lower lanes zeroed.
    - 11: All 16 lanes active.
  - Depending on this signal, the module conditionally maps the generated patterns to only the enabled lanes while padding the others with zero. This allows proper operation even under link degradation, partial-lane fallback, or training scenarios.

- **Control and Timing**
  - The module maintains two counters:
    - byte_count: 3-bit counter to manage cycle boundaries and byte emission.
    - iteration_count: 8-bit counter that tracks how many 16-bit pattern transmissions have occurred.
  - The entire training sequence is repeated for **128 iterations**, after which the Done_3 signal is asserted to inform the system that the training phase is complete.
  - A one-bit flag is used to prevent further transmissions once training is completed, and internal state resets if PerLaneID_EN is deasserted.

- This operation ensures deterministic, synchronized, and flexible training of all active lanes, supporting both full and degraded link configurations. The module's behavior is aligned with the requirements of high-speed serial protocols, facilitating reliable lane identification and training during the PHY initialization stage.

- Finally, the PerLaneID_Generator plays a **fundamental role** in ensuring reliable, configurable, and verifiable multi-lane communication. Through flexible pattern generation, adaptive lane mapping, and seamless serializer interfacing, it forms a key component in the **initialization, training, and verification** stages of modern high-speed serial links.

- An examples and snapshots from the UCIe specs document of how the Per-lane ID Pattern is generated for each lane and transmitted through the UCIe Link are showed above in ***Figures 2-22, 2-23 and 2-24*** in main functionalities sub-chapter.

Block Architecture:

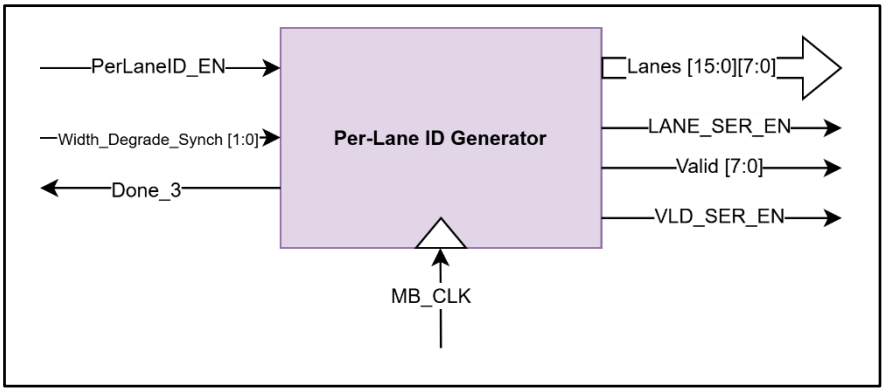


Figure 3-57: MB_TX Per-Lane ID Generator Block Architecture

Block Signals:

Table 3-83: MB_TX Per-Lane ID Generator Signals

Port	Direction	Width	Description
MB_CLK	Input	1-bit	Mainband clock domain.
PerLaneID_EN		1-bit	Enables the per-lane ID generation process coming from the TX Controller.
Width_Degrade_Synch		2-bits	Number of active lanes encoding coming from the Controller after being synchronized.
Lanes	Output	16-bytes	16 output lanes, each carrying an 8-bit ID segment.
LANE_SER_EN		1-bit	Enables the Lane Serializer to serialize each Lane.
Valid		8-bits	Indicates valid data presence on lanes.
VLD_SER_EN		1-bit	Enables the Valid Serializer to serialize the Valid signal.
Done_1		1-bit	A done signal completion after 128 iterations returned back to the MB TX Controller block indicating that the last bytes of the test pattern are successfully sent on the Lanes.

Byte-to-Lane Mapper:

Block Functionality:

- This block is responsible for mapping parallel input data bytes to the lanes in a width-adaptive system.
- It is designed to distribute bytes across multiple lanes for parallel processing or transmission.
- This SystemVerilog module provides a configurable way to map an incoming stream of bytes onto specific lanes while supporting **width degradation** mode.
- The **Byte_Mapper** module processes an array of 64-byte input data (lp_data_Synch) which is the data read out from the FIFOs and distributes the bytes across 16 lanes. Depending on the Width_Degrade_Synch signal, it selectively enables subsets of lanes and determines mapping strategy.
- The module dynamically maps the input bytes onto output lanes based on the 2-bit Width_Degrade_Synch signal, as follows:**2'b00 (Disabled Lanes)**: All lanes are disabled (i.e., mapped to zeros). This mode acts as a low-power or idle state.
  - **2'b01 (Half-width, lower lanes active)**: Only lanes [7:0] are active; [15:8] are zeroed. Eight bytes are mapped at a time.
  - **2'b10 (Half-width, upper lanes active)**: Lanes [15:8] are used while [7:0] are zeroed.
  - **2'b11 (Full-width)**: All sixteen lanes are active and carry 16 bytes per clock cycle.
- So, this flexible byte-to-lane allocation supports lane redundancy, link training fallback, or energy-saving modes as it Deactivates the unused lanes when operating in degraded width mode.
- The mapping process proceeds in chunks of 8 or 16 bytes depending on the active lanes encoding.
- This module contains control flags (flag and flag_2) to ensure that the data snapshot is captured once per cycle and processed only when the module is active. These flags also prevent repeated mapping of the same byte group or undefined updates.
- It handles enabling/disabling of serialization signals and completion status.
- It implements counter logic to control sequential processing over multiple cycles and to maintains the position of processed bytes.
- Its functional operation is as follows:
  - When BYTE_MAP_EN is asserted, the mapping process starts & it continuous mapping as long as the BYTE_MAP_EN signal is asserted.
  - The first cycle initializes internal signals and registers the input data.
  - Mapping progresses in groups of 8 or 16 bytes per cycle until all 64 bytes are processed.
  - Done_2 is asserted when the process completes (all 64 data bytes are successfully mapped into the 16 lanes).
  - In case of IDLE or RESET, the BYTE_MAP_EN is de-asserted so all outputs will be cleared.

Block Architecture:

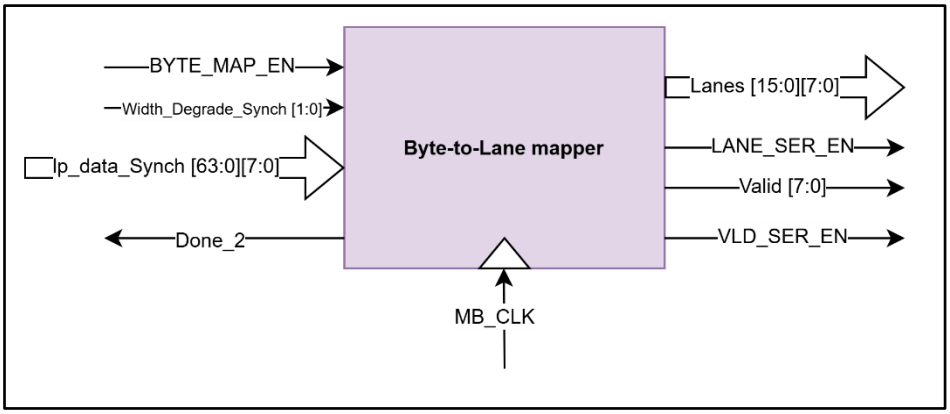


Figure 3-58: MB_TX Byte-to-Lane Mapper Block Architecture

Block Signals:

Table 3-84: MB_TX Byte-to-Lane Mapper Signals

Port	Direction	Width	Description
MB_CLK	Input	1-bit	Mainband clock domain.
BYTE_MAP_EN		1-bit	Enables the byte-to-lane mapping process coming from the TX Controller.
Width_Degrade_Synch		2-bits	Number of active lanes encoding coming from the Controller after being synchronized.
lp_data_Synch		64-bytes	Synchronized FIFO read data.
Lanes	Output	16-bytes	16 output lanes, each carrying an 8-bit mapped data byte.
LANE_SER_EN		1-bit	Enables the Lane Serializer to serialize each Lane.
Valid		8-bits	Indicates valid data presence on lanes.
VLD_SER_EN		1-bit	Enables the Valid Serializer to serialize the Valid signal.
Done_2		1-bit	Signals completion of mapping operation returned back to the TX Controller when all bytes are mapped.

Serializer:

Block Functionality:

- This module is used to convert parallel data byte into a serial data stream (parallel-to-serial converter).
- This module operates as follows:
  - A clock input MB_CLK for synchronous operation.
  - An 8-bit input DATA that holds the parallel byte of each data lane or valid lane to be serialized.
  - A control signal SER_EN that initiates the serialization process.
  - A serial data stream on SER_OUT, transmitting one bit per clock cycle.
  - It starts serializing starting from the LSB of the input byte first.
  - It shifts out the data bit by bit on each clock cycle when **SER_EN** is active.
- The data is right-shifted manually by reassigning each bit, creating a shift-register-like effect.
- Its logic ensures that the LSB is immediately sent without any delay or cycles waste, and the remaining bits are processed in following cycles in order to optimize in time as much as we can.
- The design uses simple sequential logic (flip-flops and shift operations), which is power-efficient and ideal for FPGA implementation.

Block Architecture:

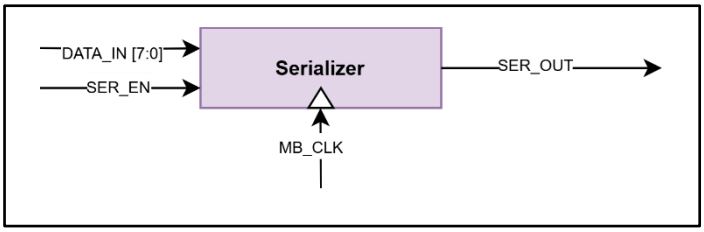


Figure 3-59: MB_TX Serializer Block Architecture

Block Signals:

Table 3-85: MB_TX Serializer Signals

Port	Direction	Width	Description
MB_CLK	Input	1-bit	Mainband clock domain.
DATA_IN		8-bits	Parallel input data to be serialized.
SER_EN		1-bit	Enable signal for the serializer operation.
SER_OUT	Output	1-bit	Serialized output bit.

Per-Lane LFSR:

Block Functionality:

- This block is used to implement a **Linear Feedback Shift Register (LFSR)** used for **per-lane active data scrambling and pseudo-random test pattern generation.**
- It plays a dual role: supporting both **data scrambling** during active transmission and **LFSR test pattern generation** for link integrity verification.
- The design is fully parameterizable to accommodate different LFSR widths and seed values, making it adaptable across various lane widths and communication protocols.
- The module operates synchronously with the mainband system clock (MB_CLK) and uses a reset signal (RST_n) for global initialization. Internally, it contains a shift register (LFSR_REG) of configurable width (WIDTH, default 23 bits) and a set of XOR tap points defined by the polynomial Tap_Points = 23'h210124. These tap points determine the feedback logic that drives the pseudo-random sequence generation.
- The module features the following functional behavior:
  - **State Initialization**: The LFSR is reset to a known seed value (Seed = 23'h1DBFBC) either during global reset or upon transition of the Opcode_Synch signal, which reflects changes in the Link Training and Status Machine (LTSM) state. This ensures a clean start of sequence generation for each distinct test/training phase.

  - **Scrambling Test Mode (PN_EN = 1)**: When enabled (EN = 1) and PN mode is active (PN_EN = 1), the module XORs the incoming data bit (Data_In) which is equal to 1'b0 with the most significant bit of the LFSR and outputs the result (Data_Out). This operation introduces pseudo-randomness to the data stream. This mode is enabled to generate the 4K iterations of the LFSR test used in Data-to-Clock (D2C) point tests at a pre-defined LTSM states by the UCIe specs, which are: **"DATAVREF"**, **"DATATRAINCENTER1"**, **"DATATRAINVREF"**, **"RXDESKEW"**, **"DATATRAINCENTER2"** and **"LINKSPEED"**.

  - **Input Data Lanes Mode (PN_EN = 0)**: When PN is inactive, the LFSR is either disabled and the input data is bypassed directly through the Link **without** scrambling or doing any operations on it, which is in the case of performing **Per-Lane ID** pattern test as stated in the specs, or it the input active data is scrambled with the same seed and tap points as that used in the LFSR test. This ensures compatibility with normal data paths when LFSR functionality is not required. As on overall:
    - In test mode (4K iterations of LFSR test pattern) with PN high, a registered (delayed) scrambled bit is output along with its Valid signal.
    - In active mode (active data transmission) with PN low, real-time scrambling is applied to the input data bit.
    - Otherwise (in case of performing the 128 iterations of the Per-Lane ID pattern test), the input bit is passed through as it is.

  - **Control and Validity Outputs**: The Valid and VLD_SER_EN signals are asserted during the 4K iterations of the LFSR test to indicate that the test is started which needs a valid signal to be sent along the test till it finishes all iterations.

- In general, the reason we do data scrambling is that to randomize the input data and it prevents or minimizes long sequences of identical bits (i.e.: long runs of 0s or 1s), which can lead to issues in clocking and synchronization.
- This block is instantiated 16 times for each lane after serializing them, where it takes the serializer output bit as an input to be scrambled (in case of active data transmission) or bypassed (in case of transmitting the Per-Lane ID pattern test) Or it takes an input zero bit through multiplexing it with the serializer lane output to be an input for performing the 4K LFSR test based on the Tap Points and implementation technique shown below in ***Figure 3-60***.

Block Architecture:

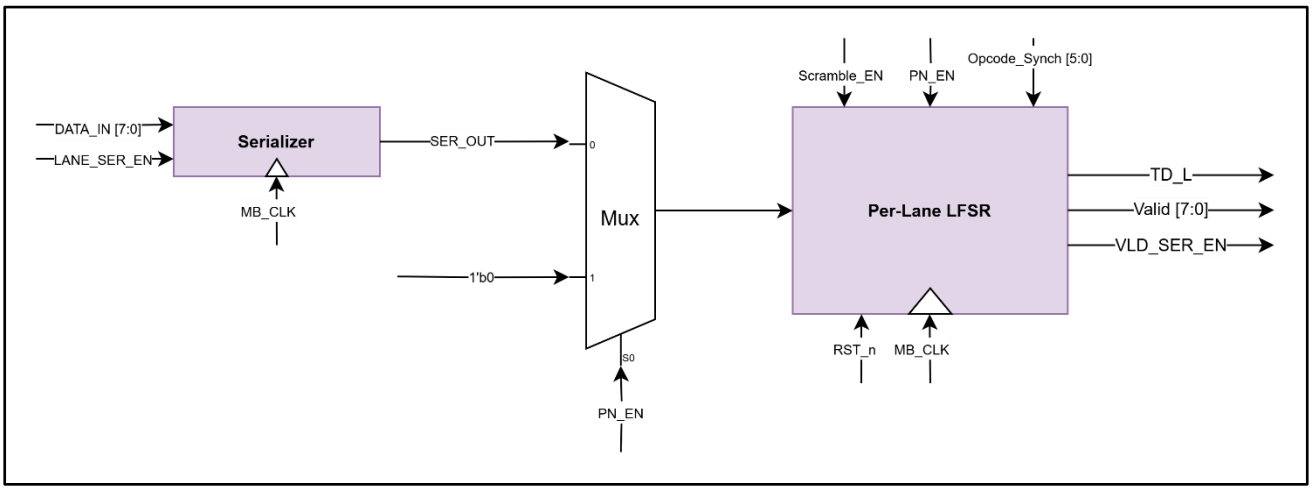


Figure 3-60: MB_TX Per-Lane LFSR Block Architecture

Block Signals:

Table 3-86: MB_TX Per-Lane LFSR Signals (Sheet 1 of 2)

Port	Direction	Width	Description
MB_CLK	Input	1-bit	Mainband clock domain.
RST_n		1-bit	Asynchronous active-low reset synchronized to the Main Band clock domain.
DATA_IN		8-bits	The input byte of each Lane.
LANE_SER_EN		1-bit	Number of active lanes encoding coming from the Controller after being synchronized.
Scramble_EN		1-bit	Enable signal to allow data scrambling or not (in case of PerLane ID test).
PN_EN		1-bit	Enable signal to indicate if it's normal transmission or a LFSR test pattern.

Table 3-87: MB_TX Per-Lane LFSR Signals (Sheet 2 of 2)

Port	Direction	Width	Description
Opcode_Synch	Input	5-bits	Synchronized Operation code coming from the LTSM block & indicates the current state of the LTSM which determines which operation will be done through the MB_TX. It is needed here to initialize the LFSR block to its initial Seed value when state transitions.
TD_L	Output	1-bit	Logical Lane for Data Transmit.
Valid		8-bits	Indicates valid data presence on lanes sent along with the LFSR test.
VLD_SER_EN		1-bit	Enables the Valid Serializer to serialize the Valid signal.

Block Parameters:

Table 3-88: MB_TX Per-Lane LFSR Parameters

Parameter	Width	Description
LFSR_WIDTH	23-bits	Defines the bit width of the LFSR.
LFSR_SEED	23-bits	Initial seed value for LFSR (unique for each lane).

Special features and optimization:

- The LFSR uses the same polynomial as PCIe: G(X)=X23 + X21 + X16 + X8 + X5 + X2 + 1.
- Each Transmitter is permitted to implement a separate LFSR for scrambling and pattern generation. Each Receiver is permitted to implement a separate LFSR using the same polynomial for de-scrambling and pattern comparison.
- The seed of the LFSR is Lane dependent, and based on the Logical Lane number and the seed value for Lane number is modulo 8 as shown in ***Figure 2-20*** mentioned above in main functionalities sub-chapter taken from the UCIe specs Where the Seed values of Lanes 8-15 are the same as that of Lanes 0-7 respectively.
- The LFSR implementation is shown in ***Figure 2-21*** mentioned above in main functionalities sub-chapter taken from the UCIe specs which is achieved by the Tap Points defined above.

3.4.2. Main band Receiver "MB_RX":

Block Functionality

During Training States (Bring-Up Phase):

- **Pattern Reception and Comparison:**
  - Receives known test patterns on the Valid Lane and Data Lanes.
  - Compares received patterns against expected values to assess link quality.
- **Reporting and Coordination:**
  - Sends comparison results to the Module Partner so that it can execute corrective measures aligned with the intended objective of each test *(1).*

During Active State (Post Bring-Up):

- **Data Handling:**
  - Receives scrambled data across active lanes.
  - Performs de-scrambling and deserialization.
- **Data Aggregation and Forwarding:**
  - Collect data into **64-byte** chunks.
  - Forwards chunks to the upper Die-to-Die (**D2D**) layer.
- **Error Signaling:**
  - Flags a **chunk-level error** whenever any Valid frame accompanying the transmitted data indicates corruption.

(1) The purposes of tests are diverse, but they generally fall into the following main categories:

Lane and Signal Integrity:

- Detect and apply repair to clock, track and Valid lanes and Detect Data Lane reversal.

Link Configuration and Optimization:

- Set up operational speed, Perform clock-to-data centering & Data-to-Clock training.

Sampling Accuracy:

- Optimize Receiver reference voltage (Vref) for Valid & Data sampling.

Link Validation and Reliability

- Perform width degradation (if needed), Check link stability at the operating data rate.

Top Architecture:

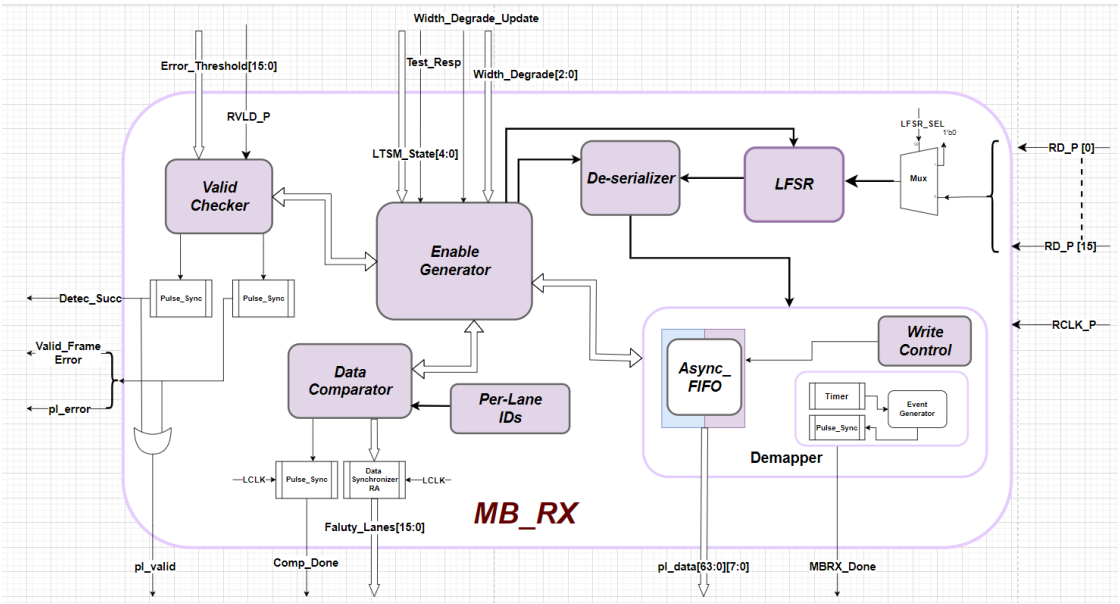


Figure 3-61: Main Band Receiver Top Architecture

The Main Band receiver consists of several blocks, some of which are involved in its functionality during the bring-up phase, others during the post bring-up phase, and some that operate in both.

Bring Up Phase Blocks:

- Data Comparator:

Post Bring Up Blocks:

- De-mapper
- De-serializer

Both:

- LFSR
- Valid Checker
- Enable Generator

Note: the de-serializer used here is used only for Serial-lane simulation purposes as in high-speed digital communication systems targeting very high data rates like 32 **Gbps**, normal (basic) serializers/de-serializers are **insufficient**, especially when **DDR (Double Data Rate) or multi-gigabit throughput is required**. Instead, the industry-standard solution implemented on FPGAs is to use dedicated high-speed **SERDES** (Serializer/De-serializer) transceiver blocks integrated within the FPGA fabric.

Signals Interface

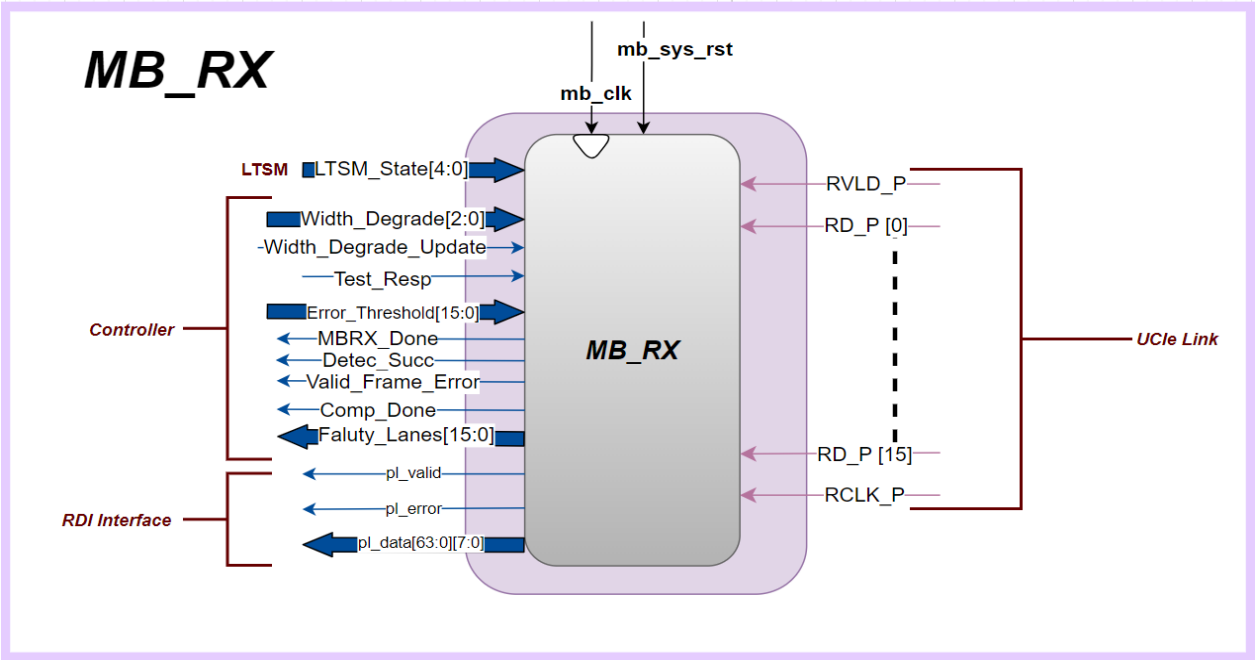


Figure 3-62: Main Band Receiver Signals Interface

Table 3-89: MBRX Top Level Interface Signals

Top-Level Interface Signals			
Signal	**Direction**	**Width**	**Description**
RVLD_P	**Input**	**1**	The valid lane is a dedicated serial lane defined by the protocol, used to transmit a valid pattern for each 16-byte group sent over the 16 data lanes (Standard Package)
RD_P [0], RD_P [1], ……., RD_P [15[	**Input**	**1**	16 data lanes that are dedicated to transmitting the training pattern during the bring-up phase and the main data during the post bring-up phase.
RCLK_P	**Input**	**1**	The timing source to which both the valid lane and data lanes are phase-aligned prior to main data transmission, ensuring synchronization and accurate sampling of valid data.
MB_CLK	**Input**	**1**	Main Band Clock is generated by PLL.
MB_SYS_RST	**Input**	**1**	Active Low Reset synchronized to the Main Band clock Domain.

Table 3-90: MBRX Top Level Interface Parameters

Parameters	**Width**
FSM_States	5
Main_Data_RDI	64 bytes (512 bit)
Degrade_Options	3
Num_Lanes	16
Error_Threshold_Length	16

Table 3-91: MBRX Inter Module Signals (sheet 1 of 2)

Inter-Module Signals				
Source /Destination	**signal**	**Direction**	**Width**	**Description**
LTSM	LTSM_State	Input	FSM_States	indicates whether we are in one of the Training States or in the Operational State (Active State).
RDI Interface	Pl_valid	Output	1	indicates that the Data on the pl_data bus is valid and should be sampled by the D2D layer.
	Pl_error	Output	1	indicates that there was an error in one of the 4 Valid Frames accompanied with the current Chunk (64 Byte) on the pl_data bus.
	Pl_data	Output	Main_Data_RDI	constitutes the Data received on the PHY Data lanes that is intended to be directed to the upper layer.

Table 3-92: MBRX Inter Module Signals (sheet 2 of 2)

Source /Destination	signal	Direction	Width	Description
Controller	Width_Degrade_ Update	Input	1	indicates that the value on the Width_Degrade bus is a new value that should be sampled.
	Width_Degrade	Input	Degrade_Options	indicates the current Operational Width the Receiver should adjust its modules to work according to it.
	Test_Resp	Input	1	indicates that the Response SB Message was sent and to start expecting to receive the Pattern soon.
	Error_Threshold	Input	Error_Threshold_ Length	carries the Threshold that represents the Criteria through which the Comparators decide whether the Lane was Operational or not.
	Detec_Succ	Output	1	indicates that the Valid Test Comparison ended successfully.
	Valid_Frame_ Error	Output	1	indicates that the Valid Test Comparison ended & failed.
	Comp_Done	Output	1	indicates that the Data test Comparison Ended.
	Faulty_Lanes	Output	Num_Lanes	carries the result of the Data Test where each bit with '1' indicates that the corresponding Data Lane is malfunction.
	MBRX_Done	Output	1	indicates that the Receiver finished Receiving Data from the Partner and the LTSM can change state.

Detailed Architecture:

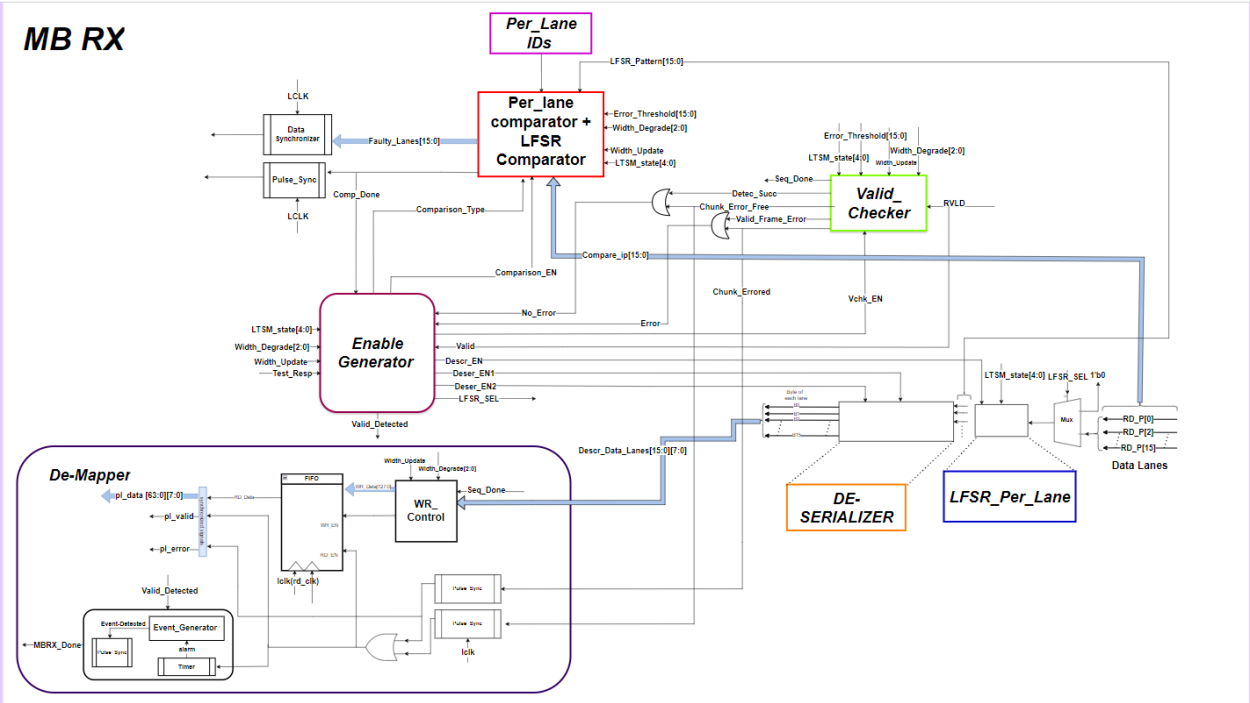


Figure 3-63: Main Band Receiver Detailed Architecture

Figure 3-63 shows all the Modules constituting the Main Band Receiver Submodule and all the Connections between them and the Upcoming Section would be Describing each Module Separately in Great Details

Submodules Description

1. Enable Generator

1.1 Block Functionality

The **Enable Generator** serves as a **centralized control unit** for the M**ain band receiver**. It operates in both the **Bring-Up phase** and the **Post-Bring-Up phase**, making it an essential part of the Receiver's system Control Architecture.

Responsibilities include:

- Generation of **Enable Signals** for Subordinate Blocks.
- **Control** and **Mode Selection signals** Generation Based on System State and Configurations.
- Monitoring of **Completion Signals** to Track various Blocks Status.

The **Enable Generator** primarily operates as a **combinational block.** However, to handle the reception of test patterns or data chunks reliably, it incorporates a **sequential element**—the **Chunk_Marker** register.

This register is necessary because the **valid pattern marking the start of a chunk is a combination of ones and zeros**, not a continuous stream of ones. As a result, a simple combinational or edge-based detection is not sufficient to hold the enable signal across the entire chunk duration.

The **de-assertion of Chunk_Marker** is controlled by the **logical OR of all "done" or termination signals** coming from various blocks.

During the Bring-Up phase, the Enable Generator holds off sampling the Valid_Lane signal until it receives a control signal from the system controller indicating that the sideband handshake—required prior to each test—has been fully completed. This delay accounts for the fact that the handshake spans multiple clock cycles, as the Main Band operates as the fastest block in the system.

This makes the Enable Generator the M**ain Controller** for orchestrating the sequence and state of various functional blocks in the receiver pipeline.

Block Architecture:

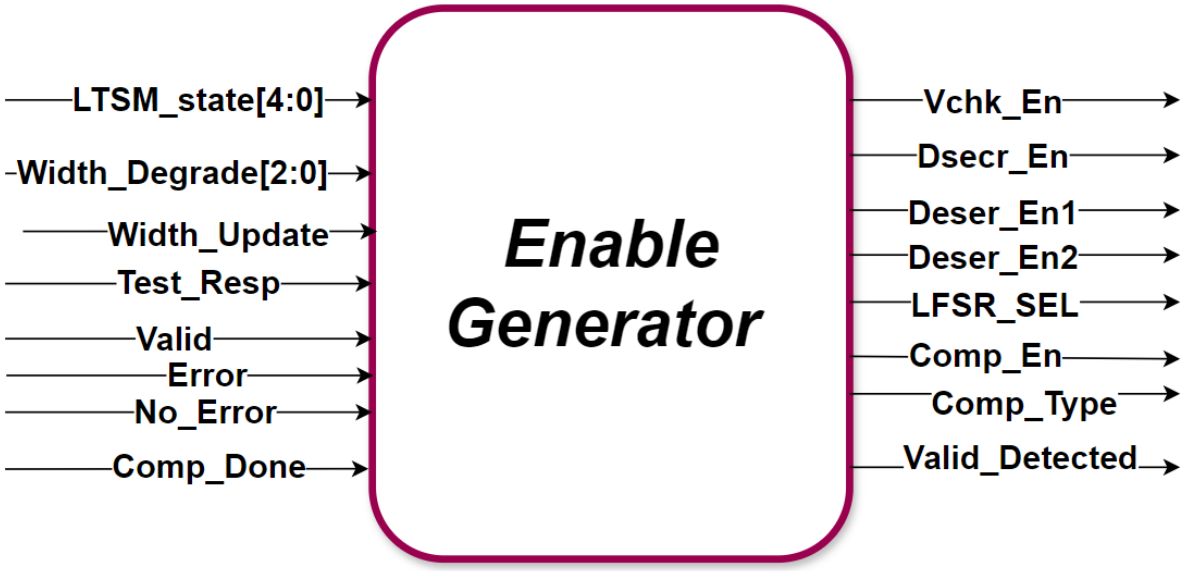


Figure 3-64: Main Band Receiver Enable Generator Architecture

Block Signals, Parameters & Configurability:

Table 3-93: MBRX Enable Generator Interface (sheet 1 of 2)

Enable Generator Interface				
Type	**Signal**	**Direction**	**Width**	**Description**
Enable	Vchk_En	Output	1	Enable signals for all other blocks of the Receiver.
	Descr_En			
	Comp_En			
	Deser_En1			
	Deser_En2			

Table 3-94: MBRX Enable Generator Interface (sheet 2 of 2)

Type	Signal	Direction	Width	Description
Control & Mode Select	Comp_Type	Output	1	Type of Test to be performed by the Comparator.
	LFSR_Sel	Output	1	Selection for the input of the LFSR Module.
Completion Signals	Error	Input	1	A signal that indicates both the Failure of the valid test and its end.
	No_Error	Input	1	A signal that indicates both the successful completion of the valid test and its end.
	Comp_Done	Input	1	Signal that marks the end of the Data Test.
Configurability	Valid	Input	1	The valid lane that is continuously sampled by the Generator.
	LTSM_State	Already Defined in **Error! Reference source not found.**		
	Width_Degrade_Update			
	Width_Degrade			
	Test_Resp			

Notes:

→ The Descrambler (LFSR) enable is only generated in ACTIVE state and Data Tests where the Pattern is already scrambled by the Transmitter.

→ We have 2 De-serializer Enables as we only activate the De-serializer Group of the Operational Lanes group.

Clocking & CDC:

- **Clocking**: The block operates within the Main Band (MB) clock domain.
- **CDC**: the block receives some Configurability signals from other Submodules, so some Synchronizers are deployed for this purpose.

✓ **LTSM_State bus**: this is the Main System FSM current State, so it has a special nature, and the deployment of a **gray synchronizer** would be the best fit

✓ **Width_Degrade_Update** signal **& Width_Degrade bus**: Since Width_Degrade is a multi-bit bus, **Enable Synchronization** is used to reliably transfer its data across clock domains. The level signal width_degrade_Update serves as a **synchronization trigger**, allowing safe transfer of the update event without risking metastability. By synchronizing this single

control signal with a level signal synchronizer ‘2FF synch’, the destination domain can reliably detect when the Width_Degrade bus data is valid and stable. Sampling the entire bus only when the synchronized update signal asserts guarantees that the data is captured atomically and consistently, avoiding partial or corrupted data. This approach is more efficient and robust than synchronizing each bit of the bus individually, which can lead to timing issues and data incoherence, providing a clean and reliable way to synchronize multi-bit data across asynchronous clock domains.

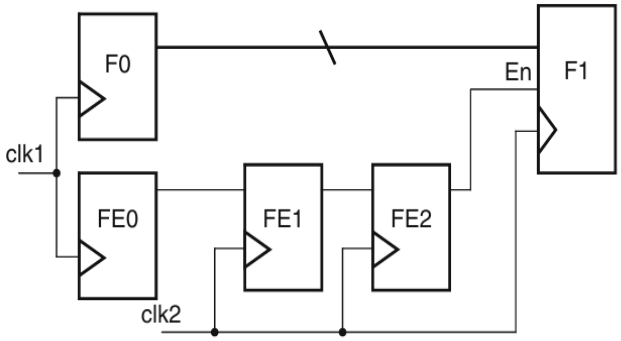


Figure 3-65: Bus Enable Synchronization

Special Features & Optimization:

Valid Lane Sampling

The Enable Generator deploys a dedicated **Positive-Edge Detector** on the Valid_Lane signal, as its rising transition serves as the **definitive trigger for initiating receiver operation**; protecting this critical event from potential glitches is essential to prevent the receiver from entering an undefined or unstable state.

Figure 3-66: Positive Edge detector for the Valid Lane

2. De-Mapper

Block Functionality:

The De-Mapper is one of the Main Band receiver components, active **exclusively** during the **Post Bring-Up phase**. It interfaces with the **RDI (Raw Data Interface)** to deliver data captured from the data lanes **Chunk by Chunk** where each chunk is 64 Bytes. Internally, it comprises several sub-blocks:

- A **Write Control** unit that manages writing of deserialized data into the FIFO.
- A **Gray-coded Asynchronous FIFO** for safe clock domain crossing.
- An **Event Generator** responsible for producing a key signal essential to **UCIe** functionality.
- **Toggle Synchronizers** to synchronize the deserialization-complete and valid frame error signals and uses their logical OR as the read enable for the FIFO.

The Write Control block handles data alignment and memory write operations based on the current lane configuration. Under normal operation, it writes a full 16-byte word—captured simultaneously from all 16 data lanes—into a single memory location. However, when **width degradation** is enabled (e.g., operating with only 8 active lanes), the block temporarily stores the first 8 bytes and completes the 16-byte word after two consecutive data lane captures before committing it to memory.

Data_Pending_Logic Contribution to the UCIe Specs.

3. The UCIe Module Partner on receiving the sideband message {LinkMgmt.RDI.Req.Retrain} must transition its RDI state to Retrain after completion of stall Req/Ack (**`pl_stallreq`**; **`lp_stallack`**) handshake on its RDI and there is no mainband data pending in the Receiver pipeline. After completion of stall Req/Ack handshake and transmitting any pending data over mainband, the UCIe Module Partner responds with {LinkMgmt.RDI.Rsp.Retrain}.
4. Once {LinkMgmt.RDI.Rsp.Retrain} is received and there is no mainband data pending in the receiver pipeline, the UCIe Module must transition its RDI to Retrain.

Figure 3-67: Data Pending in Receiver Pipeline Spec.

Because of the four bytes added by D2D Adapter, the alignment of the Flit does not always match the number of Lanes of the physical Link. The bytes added by D2D Adapter require the Adapter to shift the data arriving over FDI by four bytes for consecutive Flits transmitted over RDI. Data is always transferred in multiples of 256B (note that Retimer credits have a 256B data granularity). A mechanism to Pause the Data Stream is provided as a way to save power when the Link is idle. Before pausing the data stream, the data stream is terminated with a Pause of Data Stream (PDS) Flit

Figure 3-68: Transferring Data in Multiples of 256B in D2D.

The **Pending_Data_Logic** is a control module designed to detect when the receiver pipeline has been fully drained, ensuring that no data remains in flight before the system transitions to the **PHY Retrain** state. It plays a critical coordination role during the transition from active data reception to PHY reconfiguration, especially in the **Post Bring-Up phase**.

Functionality Overview:

- **Timer-Based Sampling**: Upon detection of a deserialization completion event, the module initiates a timer that delays for three clock cycles (MB domain).
- **Valid Lane Monitoring**: After the delay, it checks the Valid_Lane signal:
  - If Valid_Lane is still asserted, this implies that additional data is arriving, and the FIFO is not yet empty—so the timer restarts.
  - If Valid_Lane is de-asserted, it infers that the last chunk has passed through the pipeline, and no further data is pending.
- **Done Signaling**: Once the receiver is confirmed to be empty, the module triggers a toggle synchronizer that asserts **MBRX_Done**, indicating that the receiver has completed its operation safely and informs the main system FSM that it is safe to initiate PHY Retrain.

Transmitter-Side Assumption and D2D Nature:
This logic relies on a key assumption derived from the design of the **Main Band Transmitter**, which interfaces with the upper **D2D (Die-to-Die)** protocol layer. When D2D signals a request to enter PHY Retrain, it ensures that **any partially transmitted message is fully flushed** onto the interface. The TX does this by driving valid data continuously for **four consecutive clock cycles**, enough to transmit the worst-case message size of **256 bytes** (4 × 64 bytes).

Additionally, due to internal pipelining and TX FIFO behavior, the transmitter typically needs a **three-cycle interval** to prepare the next 64-byte chunk for transmission—even in the best-case

scenario where the FIFO is full. This predictable spacing allows the **Pending_Data_Logic** to infer, based on a quiet 3-cycle period, that the TX has finished and the RX can be safely marked as idle.

Top Architecture:

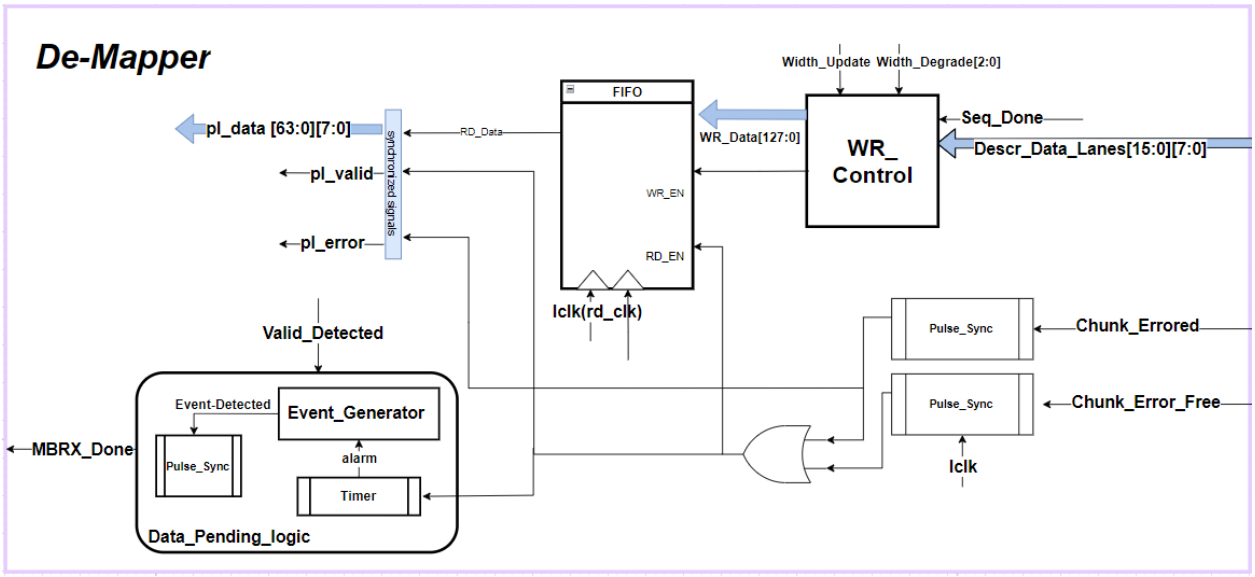


Figure 3-69: Main Band Receiver De-Mapper Architecture

Block Signals, Parameters & Configurability:

Table 3-95: MBRX De-Mapper Parameters

Parameters	Width
Main_Data_RDI	64 bytes (512 bit)
Num_Lanes	16
Frame_Length	8

Table 3-96: MBRX De-Mapper Interface (sheet 1 of 2)

De-Mapper Interface			
Signal	**Direction**	**Width**	**Description**
Width_Degrade	Already Defined in **Error! Reference source not found.**		
Width_Degrade_Update			
Seq_Done	Input	1	Indicates that the Deserialization of new 8 or 16 Bytes is done
Descr_Data_Lanes	Input	A packed array of 'Num_Lanes' entries each entry is 'Frame_Length' wide	The output of the De-serializer to be saved in the FIFO.
Valid _Detected	Input	1	The positive edge detector output that indicates the start of receiving a new Chunk on the data lanes (used by the Data_Pending_Logic)
Chunk_Errored	Input	1	Indicates that one of the valid frames of the current chunk was errored.

Table 3-97: MBRX De-Mapper Interface (sheet 2 of 2)

Signal	Direction	Width	Description
Chunk_Error_Free	Input	1	Indicates that all the valid frames of the current chunk Are errors free
Pl_valid	Already Defined in Table 3-89: MBRX Top Level Interface Signals		
Pl_data			
Pl_error			
MBRX_Done			

Clocking & CDC:

- **Clocking**: The block operates within the Main Band (MB) clock domain.
- **CDC: FIFO Depth Calculation for 32 Gbps DDR SERDES Receiver.**
- System context:
  - **Maximum data rate**: 32 Gbps (DDR)
  - SERDES clock frequency: 16 GHz (DDR halves the data rate clock)
  - System clock frequency: 2 GHz
  - Data width after deserialization: e.g., 64 bytes (512 bits) per system clock cycle
- Clock domain crossing and rate mismatch:
  The FIFO bridges the fast SERDES domain (16 GHz) and the system clock domain (2 GHz).
  The SERDES domain writes data at 8× the system clock frequency.
- FIFO depth baseline:
  The FIFO depth must at least cover the **ratio of write to read clock frequencies** to prevent overflow or underflow:
- FIFO depth $\approx \frac{\text{fwrite}}{\text{fread}} = \frac{16\ \text{GHz}}{2\ \text{GHz}} = 8$

Additional margin and practical considerations:

- Add 20–30% margin for clock jitter, latency, and burstiness.
- Round up to a power of two for FPGA IP compatibility.

✅ ***Minimum FIFO depth required: 16 entries***

3.LFSR

***3.1 Block* Functionality**

Why Scrambling is Used in UCIe
Scrambling in high-speed interfaces like UCIe addresses critical signal integrity challenges:

1. Reduce Electromagnetic Interference (EMI)
   a. Randomizes data patterns to prevent spectral peaks.
   b. Minimizes radiated noise from repetitive sequences (e.g., long strings of 0s/1s).
2. Mitigate Crosstalk and ISI
   a. Balances 0/1 distribution to minimize inter-symbol interference (ISI).
   b. reduces coupling between adjacent lanes.
3. Security
   a. Adds a layer of pattern obfuscation (though not cryptographic).

Implemented LFSR

- This Block Implements a **23-bit** Linear Feedback Shift Register (LFSR) for per-lane descrambling.
- Receives serial data bit-by-bit on each lane and outputs descrambled data by **XORing** the input with the LFSR's output bit.
- We use one with each Lane and are fed with **different Seed** depending on the Number of the Lane.
- The module resets the LFSR to the seed value whenever
  The Main System FSM (LTSM) state changes, ensuring synchronization with the transmitter.
- The enable (EN) signal controls when descrambling is active; when disabled, input data passes through unchanged.
- Not operational in all states, only in ACTIVE state ‘where Main Data is sent’ and in the states that use the Lane ID as a pattern for the Test.
- Each LFSR is packed with a **MUX** where it either inputs the data to be descrambled or inputs Zeros to serve as a **Scrambler** that generates LFSR pattern to be compared to the Received Pattern in some Data Tests in the Bring Up Phase.

3.2 Block Architecture

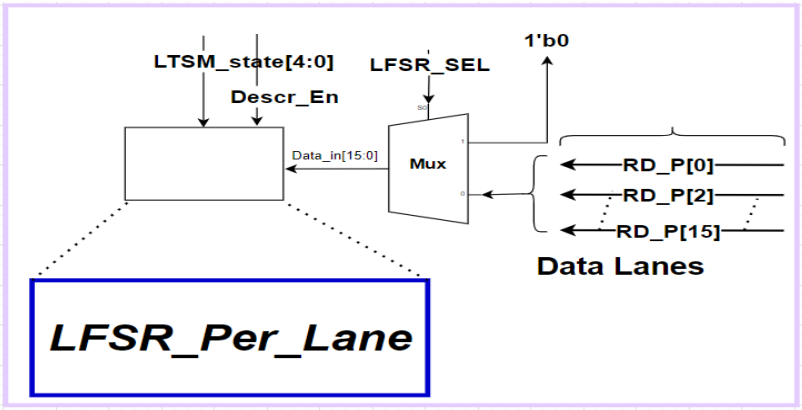


Figure 3-70: Main Band Receiver LFSR Architecture

3.5. Block Signals, Parameters & Configurability

Table 3-98: MBRX LFSR Parameters

Parameter	Width
FSM States	5

Table 3-99: MBRX LFSR Interface

LFSR Interface			
signal	**direction**	**width**	**Description**
LTSM_State	**Input**	**FSM_States**	Already Defined in Table2. But it is an input to this module to load the LFSR_Reg with the seed with each State change.
Descr_En	**Input**	**1**	Generated by the Enable Generator to indicate when this module is operational.
LFSR_Sel	**Input**	**1**	Selection signal to decide whether it is used as a scrambler or a descrambler.
Data_In	**Input**	**1**	Scrambled bit.
Data_Out	**Output**	**1**	De-scrambled bit.

4.Data Comparator

4.1 Block Functionality

The Data Comparator is a **Bring-Up–only diagnostic block** that **validates the physical integrity** of the Main Band receiver's data lanes before the link is released for **user traffic**. Positioned in line with the deserialized lane data, it orchestrates two self-test regimes—**Per-Lane-ID** and **LFSR**—and deterministically reports lane health to the Main Controller.

Functional Highlights

- **Dual Test Modes**
  - **Per-Lane-ID Test** – Confirms that each lane exclusively receives its own **16-bit signature pattern.** Success is declared when any lane observes 16 successive correct patterns within a 128-iteration window.
  - **LFSR Test** – Verifies bit-error-free transmission of a pseudo-random stream generated in the transmitter. Each lane is **seeded uniquely**; the test passes when **4096-unit** intervals are received error-free **(consecutive reception is not required).**
- **Mode Selection & Autonomy**
  A mode-select input from the Main Controller activates exactly one test at a time; the Comparator asserts a **test-active** flag that blocks any width-change commands until the current test concludes, **safeguarding the results** against undefined operating widths.
- **Width-Degradation Awareness**
  Only lanes marked **functional** by system configuration are evaluated. The Comparator *outputs* a 16-bit status vector—one bit per lane—containing pass/fail results **solely** for those active lanes. The vector is delivered to the Controller through a dedicated synchronizer.
- **Immediate Result Resolution**
  To ensure minimal latency in test evaluation, the Data Comparator performs a real-time assessment of the test outcome **during the very cycle** in which the final bit of the final test iteration is received. This is accomplished through combinational logic that simultaneously checks whether the success criteria have been met—**either by verifying that the required number of successful consecutive iterations has already been reached, or by confirming**

that the current bit completes the last needed correct iteration. This approach guarantees that the pass/fail result is ready without incurring additional clock cycle delays.

- **Synthesis-Friendly Counter Scheme**
  To avoid superfluous hardware, the Per-Lane-ID test employs a 5**-bit up-counter** that saturates at 16; counting beyond the success threshold provides no additional diagnostic value and would not shorten the remote transmitter's mandated test duration.
- **Error Threshold-Based Evaluation in LFSR Test**
  In the LFSR test mode, each lane's functionality is determined based on an **error threshold** value, which is communicated via a sideband handshake message and subsequently forwarded to the Data Comparator by the main controller. After processing 4096 Unit Intervals (UIs), a lane is deemed functional if the total number of detected errors remains **below the specified threshold**, thereby enabling tolerance to limited bit errors as per system configuration.

4.2 Block Architecture

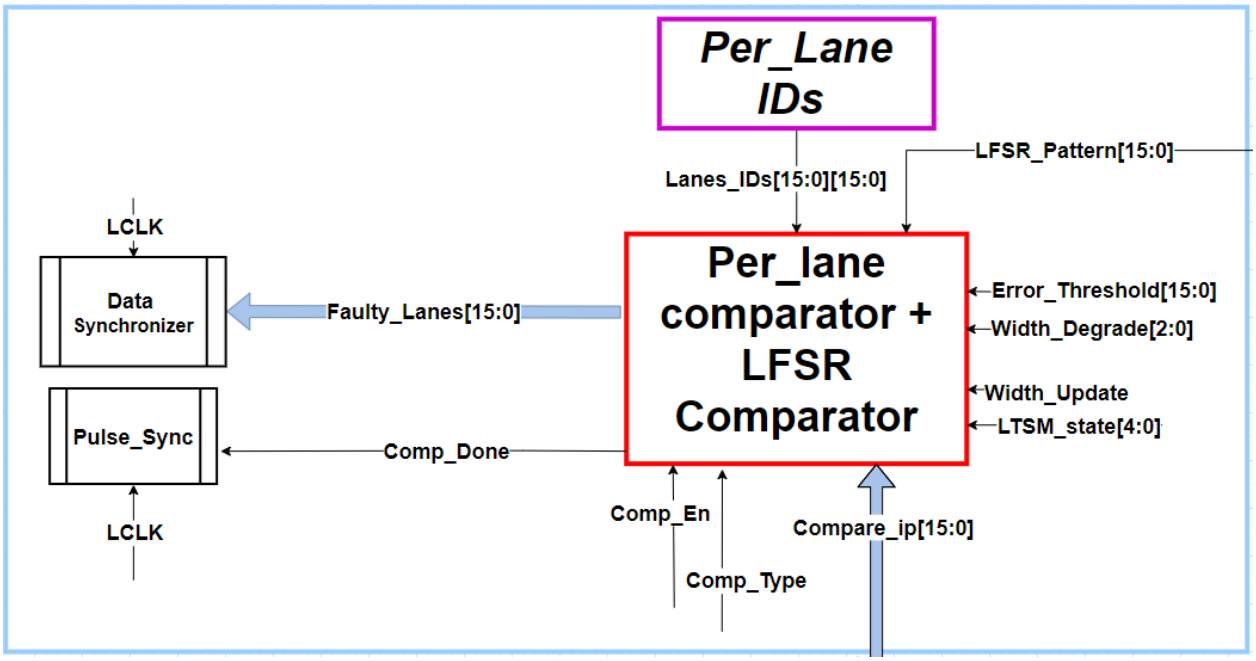


Figure 3-71: Main Band Receiver Data Comparator Architecture

4.3 Block Signals, Parameters & Configurability:

Table 3-100: MBRX Data Comparator Parameters

Parameter	Width
NUM_LANES	16

Table 3-101: MBRX Data Comparator Parameters

Data Comparator Interface			
Signal	**Direction**	**Width**	**Description**
Lanes_IDs	Input	A packed array of 'NUM_LANES' entries each is 'NUM_LANES' bit.	Comparison reference for the received pattern in "Per Lane ID" test that is Generated by the Per_Lane_IDs Module.
LFSR_Pattern	Input	NUM_LANES	Comparison reference for the received pattern in "LFSR" test that is Generated by the LFSR Module.
Compare_In	Input	NUM_LANES	The Actual Bits received on the Main Band Data lanes.
Comp_En	Input	1	An Enable level signal that is asserted throughout the Test.
Comp_Type	Input	1	A Mode selection signal that decides which Test to be performed "LFSR" or "Per Lane ID" Test.
LTSM_State	Already Defined in Table 3-89: MBRX Top Level Interface Signals		
Width_Degrade_Update			
Width_Degrade			
Error_ Threshold			
Comp_Done			
Faulty_Lanes			

Notes:

The Per_Lane_IDs module produces 16 unique, statically defined patterns—one per lane—that are continuously available at the output interface.

4.4 Clocking & CDC

- **Clocking**: The block operates within the Main Band (MB) clock domain
- **CDC: Synchronization of a Data Bus (Faulty Lanes) Between Two Clock Domains.**

Since Faulty_Lanes is a multi-bit (bus) then **Enable Synchronization** would be the best fit for this case where a normal **Pulse Synchronizer** (Toggle) is used for the Comp_Done signal and this signal represents the enable for the bus synchronization.

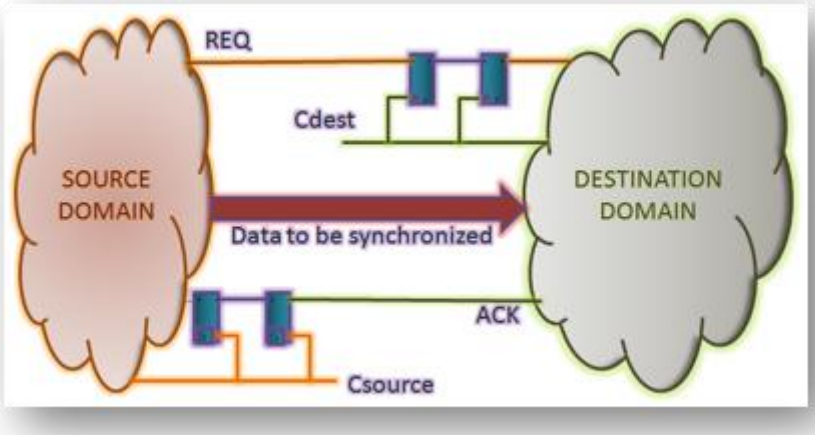


Figure 3-72: Req-Ack Enable Synchronization

4.5 Special Features & Optimization
Several Comparator Approaches
Since we have 2 Checkers (Valid & Data) we need to consider several options available.

1- Direct Counting Approach: the used one & fully defined in the Block Functionality Section.

2- Sliding Window Approach: it depends mainly on using a shift register that is 8 or 16 bits where a counter is incremented for each bit of the register if matches the required pattern.
Drawback: This approach is not well-suited to the nature of the test due to the shifting behavior of the shift register. It assumes bit counting begins immediately upon detecting the first match, which is incompatible with test scenarios that rely on fixed-position identifiers (IDs).

3- **FSM Approach**: This method relies on traversing multiple states to handle the test sequence—specifically, **collecting data for a single iteration**, **computing the error count**, and **reporting the result**. It is suited for cases where there are multiple clock cycles between iterations, such as when data for each iteration arrives every 16-clock cycle.

Drawback: This approach requires the use of data de-serializers to accumulate a full iteration before performing comparisons. However, this leads to **unnecessary power consumption**, especially considering that the specification mandates **bit-by-bit comparison at every Unit Interval (UI)**. Therefore, the FSM method introduces inefficiency by processing data in blocks instead of evaluating each UI in real time.

5.Valid Checker

5.1 Block Functionality

The **Valid Checker** is a streamlined counterpart of the Data Comparator that operates on the single Valid_Lane; for each data chunk it inspects the associated validity frames—**four under normal width or eight under width-degradation**—and asserts pass/fail signals depending on whether every bit in each frame matches the expected pattern 11110000.

5.2 Block Architecture

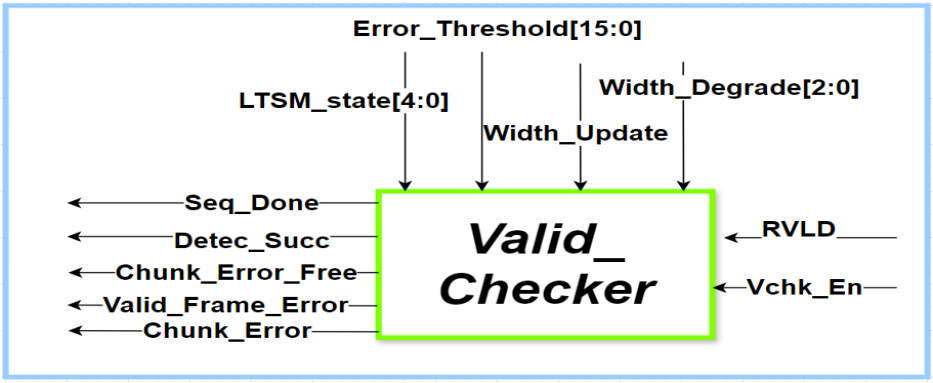


Figure 3-73: Main Band Receiver Valid Checker Architecture

5.3 Block Signals, Parameters & Configurability

Table 3-102: MBRX Valid Checker Interface

Valid Checker Interface			
Signal	**Direction**	**Width**	**Description**
Vchk_En	input	1	Asserted throughout the Valid test or the Chunk Reception
Seq_Done	output	1	Asserted each time a new byte on each lane is de-serialized
Chunk_Error	output	1	Current Data chunk has an error in one of the 4 valid frames accompanied
Chunk_Error_Free	output	1	Current Data chunk has no error in any of the 4 valid frames accompanied
Detec_Succ	Already Defined in Tables Table 3-89: MBRX Top Level Interface Signals & **Error! Reference source not found.**		
Valid_Frame_Error			
LTSM_State			
Width_Degrade			
Width_Degrade_ Update			
Error_Threshold			
RVLD_P			

5.4 Clocking & CDC

- **Clocking**: The block operates within the Main Band (MB) clock domain.
- **CDC:** The **Valid Checker** module generates two key pulse-based signals—**Detec_Succ** and **Valid_Frame_Error**—that serve **dual purposes** depending on the system state:
- **In Training states**, these signals are synchronized and forwarded to the **Controller**, indicating both the **completion** and the **result** of the pattern comparison process.
- **In ACTIVE state**, the same signals are **repurposed**: after synchronization, **Detec_Succ** maps to pl_valid and **Valid_Frame_Error** maps to pl_error. The **OR** of the two serves as the trigger for the pl_valid signal.

Because these signals are **asynchronous pulses** that are **infrequent and non-consecutive**, a **Toggle Synchronizer** is the most appropriate mechanism for reliably transferring them across clock domains. This ensures robust synchronization without risk of metastability or missed pulses.

5.5 Special Features & Optimization

Multi-Byte Pattern Validation for Enhanced Valid Frame Error Detection

The system receives a fixed synchronization pattern (11110000) once for each byte on each of the 16 lanes. Initially, frame validation relied on detecting this pattern at the byte level. However, this method is inherently limited, as it cannot detect errors occurring in the leading '1' bits of each valid frame. To overcome this and by utilizing the fact that Bytes related to same Chunk are sent back-to-back, the validation scheme has been extended to examine full 4-byte(chunk) sequences by matching concatenated patterns such as 1111000011110000.... This approach increases sensitivity to bit-level faults across the entire frame, including errors in the early bits that would otherwise go undetected.

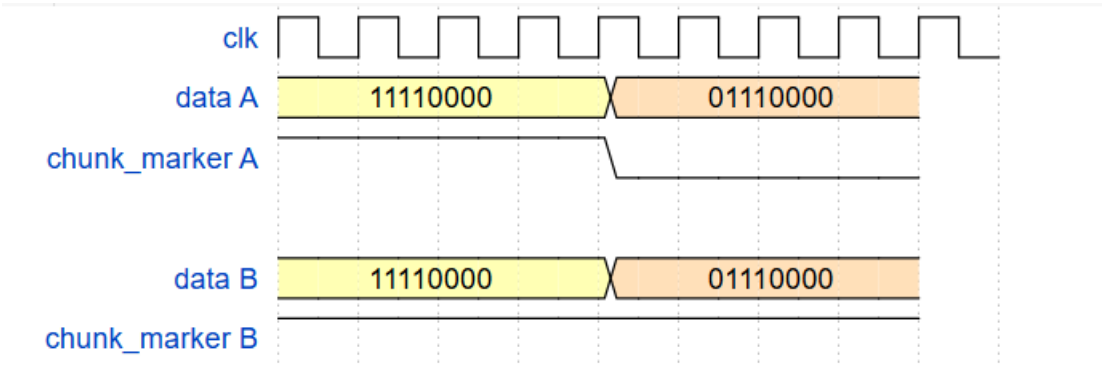


Figure 3-74: Chunk Marker Functionality Clarification

Chapter 4: UVM Basics:

The Universal Verification Methodology (UVM) is an open-source, standardized verification framework developed and maintained by the Accellera Systems Initiative.
Historically, UVM was born out of a need to consolidate competing methodologies in the industry. It evolved from two earlier methodologies: OVM (Open Verification Methodology) developed by Mentor Graphics, and VMM (Verification Methodology Manual) for System Verilog, promoted by Synopsys. These frameworks introduced many foundational verification concepts but lacked a unified standard. In 2009, Accellera launched a consolidation effort, and in 2011, the first release—UVM 1.0 EA (Early Adopter) was made public.
Since then, UVM has rapidly become the industry standard, adopted by virtually all major EDA vendors and verification teams worldwide. Its ongoing evolution led to formal standardization as IEEE 1800.2™ in 2017, under the title *IEEE Standard for Universal Verification Methodology Language Reference Manual*. Each revision has introduced enhancements in usability, configuration management, coverage integration, and object-oriented design, further solidifying UVM's role in modern functional verification.

4.1. Motivation for UVM:

As integrated circuits continue to grow in complexity, the verification of digital designs has become one of the most time-consuming and challenging phases of the development lifecycle. Traditional directed testing methodologies are no longer sufficient to meet the scalability, reusability, and comprehensive coverage demands of today's sophisticated system-on-chips (SoCs). This growing gap in verification efficiency and effectiveness served as the key motivation behind the development of the Universal Verification Methodology (UVM).

UVM emerged as a standardized framework to address these challenges by introducing a structured, reusable, and scalable approach to verification. Built on System Verilog and leveraging object-oriented programming principles, UVM enables engineers to abstract complex design interactions into transactions and separate concerns across different layers of the testbench .
A major strength of UVM lies in its support for constrained-random stimulus generation, functional coverage-driven verification, and self-checking testbenches key techniques that help achieve higher bug detection rates and improved verification closure. Additionally, UVM incorporates powerful mechanisms like TLM (Transaction-Level Modeling) for flexible communication between components, factory overrides for customization, and configuration databases for environment control—further enhancing flexibility and reuse.

By establishing a common methodology adopted industry-wide, UVM fosters collaboration, simplifies the integration of third-party IP, and ensures compatibility across EDA tools. As a result, it significantly reduces verification effort, improves productivity, and accelerates time-to-market—making it the standard for modern digital verification flows.

4.2. UVM Testbench:

The Universal Verification Methodology (UVM) testbench architecture is designed to provide a modular, reusable, and scalable framework for verifying complex digital designs. As depicted in the diagram, the UVM testbench is organized hierarchically, with clear separation of roles and responsibilities among its components.

At the top of the hierarchy, the test layer defines the verification scenario and controls the overall execution. The environment (env) encapsulates multiple verification agents and shared components, coordinating their operation. Each agent is responsible for a specific interface or protocol and is composed of three primary components: the driver (which applies stimulus to the Design Under Test), the sequencer (which generates and sequences transactions), and the monitor (which observes and collects data from the DUT interface). This modular agent structure allows for easy reuse and extension across different projects and verification environments.

A scoreboard is also integrated into the environment, serving as a central component for checking and comparing expected versus actual DUT behavior, thus ensuring functional correctness. The connections between agents and the scoreboard facilitate comprehensive coverage and robust checking mechanisms.

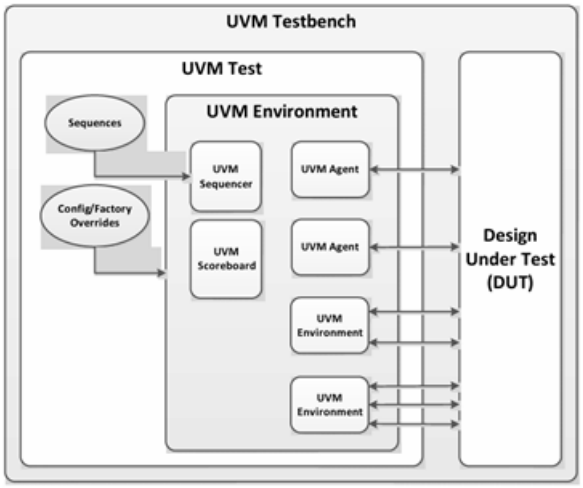


Figure 4-1: Typical UVM Testbench Architecture

4.2.1. Test

The **UVM Test** is the top-level component in a UVM testbench that orchestrates the entire verification process. Its primary responsibilities include configuring the verification environment, instantiating the environment and other verification components, and initiating stimulus generation by starting sequences. Essentially, the test acts as the main controller that sets up the context and runs the verification scenario.
More specifically, the test performs the following functions:

- **Instantiates the top-level environment**: It creates the environment component, which groups together all verification agents, scoreboards, and other subcomponents targeting the Design Under Test (DUT).

- **Configures the environment**: Using UVM's configuration database or factory overrides, the test sets parameters and options that control the behavior of agents and other components without modifying their source code. This allows flexible customization for different test scenarios.

- **Applies stimulus by invoking sequences**: The test triggers sequences—objects that generate stimulus transactions—on the environment's sequencers. These sequences define the specific test scenarios and drive the DUT inputs indirectly through the agents.

- **Manages test flow and reporting**: It controls the execution phases of the testbench and manages the collection and reporting of verification results.

This hierarchical organization allows multiple tests to extend a base test class, each configuring the environment differently or selecting different sequences to run, thus promoting reusability and scalability of the verification environment.

4.2.2. Environment (env)

The UVM Environment (env) is a key static component in the UVM testbench hierarchy that acts as a container for organizing verification components such as agents, scoreboards, and sub-environments. It defines the overall testbench topology and manages the configuration and interaction of its contained components. By grouping related verification elements into a cohesive unit, the environment promotes modularity, scalability, and reusability across different verification levels and projects. This hierarchical structure enables efficient management of complex verification scenarios and supports flexible customization without modifying component source code.

4.2.3. Agent

The **UVM Agent** is responsible for managing protocol-specific verification elements for a particular DUT interface. It consists mainly of three components—Sequencer, Driver, and Monitor—each playing a distinct role in stimulus generation, application, and observation.

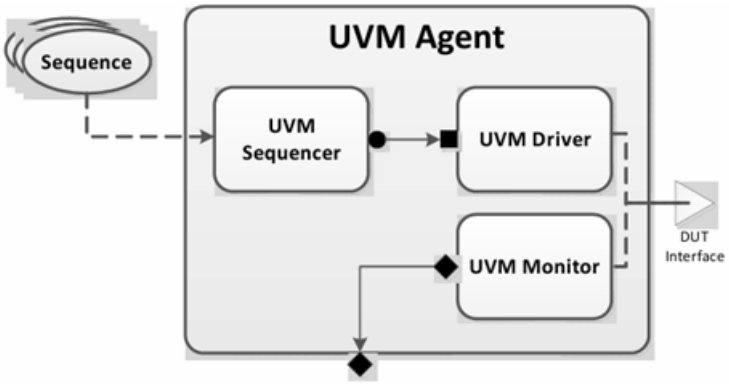


Figure 4-2: UVM Agent components

4.2.3.1 Sequencer

The **UVM Sequencer** controls the flow of transactions (called sequence items) from one or more sequences to the driver. It acts as an arbiter that manages multiple stimulus generators and ensures orderly delivery of transactions to the driver. The sequencer communicates with sequences, which define the stimulus behavior, and with the driver, which applies the stimulus to the DUT. It is responsible for the following:

- Manages transaction flow by granting permission to sequences to send transactions.
- Supports reusability by allowing sequences to be reused across different drivers and tests.

- Uses Transaction-Level Modeling (TLM) interfaces to communicate with both sequences and the driver.
- Ensures synchronization so that the driver is ready before passing a transaction.

In practice, sequences create transaction objects (sequence items) representing stimulus packets, and the sequencer schedules and forwards these transactions to the driver.

4.2.3.2 Driver

The **UVM Driver** receives transactions from the sequencer and translates these high-level transactions into pin-level signals that directly stimulate the DUT interface.
It acts as a bridge between the abstract transaction level and the physical DUT signals.
The driver implements the protocol-specific signal toggling and timing required to apply stimulus.
Communication with the sequencer is done through TLM ports, typically using a *seq_item_port* to pull transactions from the sequencer.
The driver is responsible for the actual stimulus generation on the DUT pins, converting transaction data into signal waveforms.

4.2.3.3 Monitor

The **UVM Monitor** passively observes the DUT interface signals without driving them. Its role is to capture the DUT's behavior and convert pin-level activity back into transaction-level data.
It collects transactions from the DUT interface for analysis, checking, and coverage purposes.
The monitor sends observed transactions to scoreboards and coverage collectors.
It supports protocol checking by verifying that the DUT signals conform to protocol rules.
Since it is passive, it does not interfere with the DUT operation.
The monitor provides critical feedback for functional verification by observing the DUT outputs and internal signals**.**

4.2.3.4 Scoreboard

The **UVM scoreboard** is a verification component that ensures the functional correctness of the Design Under Test (DUT) by comparing actual DUT outputs with expected results. It receives transaction data from monitors through TLM analysis ports and uses a reference model or golden data to generate expected outcomes. The scoreboard then performs automated comparisons to detect mismatches, reporting errors when discrepancies occur. Scoreboards can be designed to handle in-order outputs, where the DUT response order matches the stimulus order, or out-of-order outputs, where responses may arrive asynchronously and require associative storage for matching transactions. This component plays a critical role in verifying DUT behavior throughout simulation

4.2.4. UVM Configuration Object

The **UVM configuration** object is a key mechanism used to manage and share configuration parameters across different components of a testbench in a flexible and reusable manner. It allows testbench components such as drivers, monitors, and agents to access common configuration settings without tight coupling.

4.2.5. Coverage Collector

The **UVM coverage collector** tracks functional coverage metrics to measure how thoroughly the verification environment exercises the DUT's functionality. By collecting data on which scenarios, transactions, or conditions have been tested, it helps identify coverage holes and guides the creation of additional tests to improve verification completeness.
Functional coverage is a key metric in coverage-driven verification methodologies, providing quantitative feedback on verification progress and confidence in the DUT's correctness.

These rewritten points summarize the key roles and mechanisms of the scoreboard and coverage collector in a UVM testbench, supported by the detailed explanations and examples from the search results.

4.2.6. UVM Class Library

The **UVM Class Library** is a comprehensive System Verilog library that provides the foundational building blocks for developing well-structured, reusable, and scalable verification components and test environments. It standardizes the creation, configuration, communication, and execution of verification elements, enabling verification engineers to build robust testbenches efficiently.
Key Features and Components of the UVM Class Library

Core Base Classes
The library's core consists of base classes such as uvm_object (the root for all data and transaction classes) and uvm_component (the root for hierarchical verification components). These classes provide essential methods for object creation, copying, comparison, printing, and randomization, forming the backbone of UVM testbench elements.

Component Hierarchy and Phasing
Components derived from uvm_component represent entities like drivers, monitors, scoreboards, and environments. The library defines a standardized phased execution model (build, connect, run, etc.) that governs component lifecycle and synchronization, ensuring orderly simulation flow.

Transaction-Level Modeling (TLM)
UVM includes TLM interfaces and communication mechanisms that facilitate modular and reusable data exchange between components. This abstraction allows components to interact through transactions rather than low-level signals, improving testbench flexibility.

Configuration Database
The configuration and resource classes provide a centralized database for storing and retrieving configuration parameters at build and runtime. This enables flexible customization of verification components without modifying their source code.

Factory Mechanism
The factory classes support dynamic creation and substitution of components and objects, allowing users to override default implementations globally or locally. This enhances reusability and adaptability of verification environments.

Reporting and Messaging
The reporting classes enable consistent, configurable message logging, filtering, and error handling throughout the testbench, aiding debugging and verification status monitoring.

Utility Classes

The library also includes container classes (queues, pools), synchronization primitives (events, barriers), and policy classes (for printing, comparing, recording) that simplify common tasks in verification.

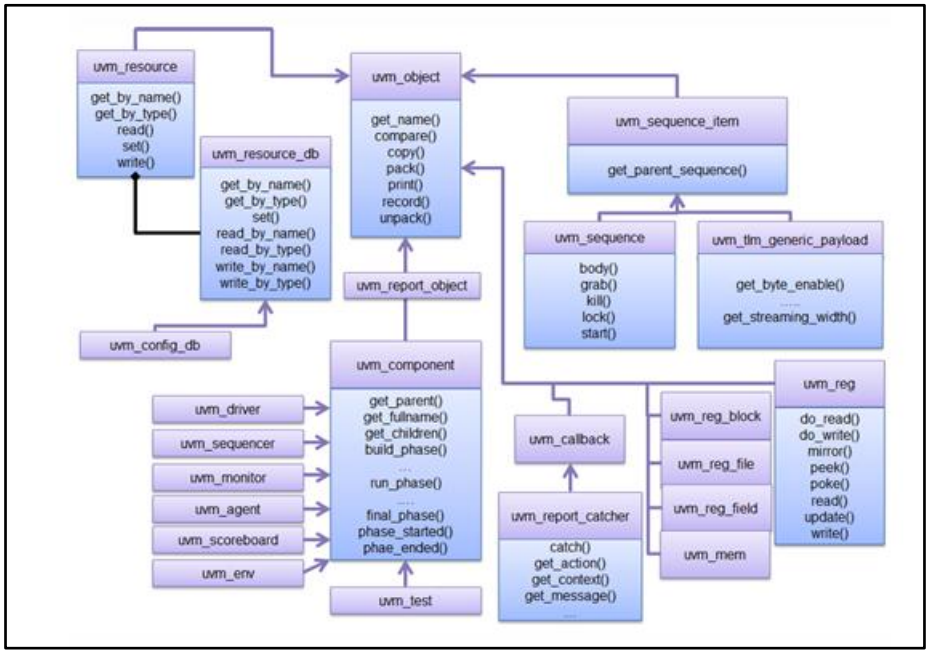


Figure 4-3: UVM Class Diagram

4.2.7. UVM phases

The Universal Verification Methodology (UVM) defines a structured sequence of simulation phases to control the execution order of testbench components. These phases ensure proper initialization, stimulus generation, checking, and cleanup during verification.

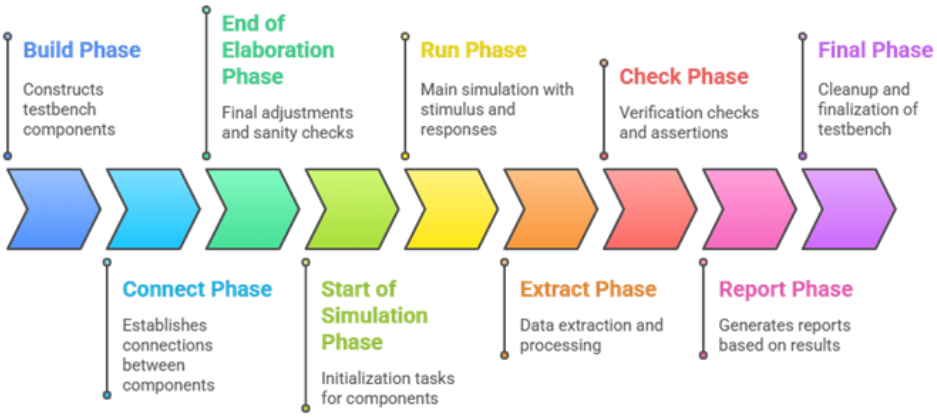


Figure 4-4: UVM phases diagram

Key UVM Phases

1. **Build Phase:**
   - Constructs the testbench components and sets up the environment hierarchy.
   - Components are created but not yet connected or started.

2. **Connect Phase:**
   - Establishes connections between components, such as TLM ports and exports.
   - Ensures communication paths are ready before simulation starts.

3. **End of Elaboration Phase**
   - Final adjustments after all components are built and connected.
   - Used for final configuration and sanity checks.

4. **Start of Simulation Phase**
   - Initialization tasks that require all components to be ready.
   - Often used to start sequences or stimulus generators.

5. **Run Phase**
   - The main simulation phase where stimulus is applied and responses are checked.
   - Often used to start sequences or stimulus generators.

6. **Extract Phase**
   - Data extraction and processing after the run phase.
   - Useful for coverage and scoreboarding.

7. **Check Phase**
   - Verification checks and assertions are evaluated.
   - Helps in determining pass/fail status.

8. **Report Phase**
   - Generates reports based on the verification results.
   - Summarizes coverage and errors.

9. **Final Phase**
   - Cleanup and finalization of the testbench.
   - Frees resources and prepares for simulation end.

4.2.8. Basic UVM Connections in Testbench Architecture

UVM Connection Mechanisms for Reusable and Scalable Verification Environments
The Universal Verification Methodology (UVM) implements standardized connection mechanisms that enable interoperability among testbench components, forming the foundation for reusable and scalable verification environments. These mechanisms facilitate precise communication while maintaining component independence:

Core Connection Mechanisms

Transaction-Level Communication (TLM)
TLM ports and exports enable decoupled communication between components like sequencers, drivers, and monitors. By abstracting low-level signal details, TLM promotes component reuse across diverse verification scenarios while maintaining protocol accuracy.

Broadcast Communication
Analysis ports and exports allow monitors to disseminate transaction data to multiple subscribers (scoreboards, coverage collectors) simultaneously, enabling parallel data processing and efficient event propagation.

Configuration Management
The uvm_config_db serves as a centralized resource database for distributing configuration objects, virtual interfaces, and parameters across the component hierarchy at runtime. This eliminates hard-coded dependencies and supports dynamic, scenario-specific configuration.

Hardware Integration
Virtual interfaces bridge the UVM testbench with RTL implementations, allowing testbench components to access and manipulate the DUT's signal-level interfaces while maintaining abstraction from RTL details

Flow Control Protocols
Blocking and non-blocking interfaces define transaction flow control between sequencers and drivers, ensuring proper synchronization and timing alignment during protocol execution.

Sequence-Item-Driver-Sequencer Communication Architecture
UVM establishes a structured communication paradigm between its core stimulus-generation components:

- **Sequence Items** encapsulate transaction data (e.g., memory operations) generated by sequences executing on the sequencer.
- **Sequencers** manage arbitration among competing sequences and dispatch transactions to drivers via TLM interfaces.
- **Drivers** translate abstract transactions into protocol-specific signal activity applied directly to the DUT interface.

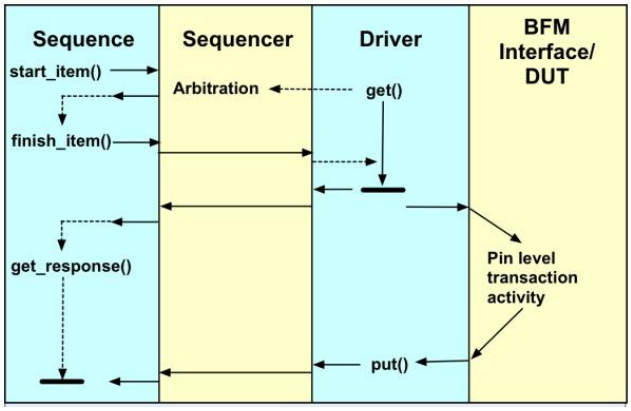


Figure 4-5: Sequencer driver diagram

4.3. Guidelines for Robust Verification of Complex Systems Using UVM

In this section, we explore practical guidelines and best practices for using UVM to build robust verification environments, helping engineers deliver reliable hardware with greater confidence and less effort.

4.3.1. Handling multiclock domain in UVM

Verification of modules with two clock domains requires careful coordination to ensure proper operation across asynchronous boundaries. In such environments, both the driver and sequencer components in the testbench must be designed to operate in synchronization with their respective clocks. Specifically, the driver should generate stimulus aligned with the clock domain it interfaces with, while the sequencer must schedule and dispatch transactions according to the timing of that domain.

4.3.2. Interface Modeling and TB Arch. for Dual-Clock Domain Verification

When verifying a Design Under Test (DUT) that incorporates two distinct clock domains, it is essential to adopt an effective interface modeling strategy to ensure accurate and efficient verification. Two primary approaches are commonly employed: the use of separate interfaces for each clock domain, or the implementation of separate modports within a single interface to represent the individual domains. The selection between these approaches is primarily influenced by the signal configuration of the DUT.

In cases where the DUT exhibits both input and output signals within each clock domain, the recommended practice is to instantiate two distinct agents. Each agent should include its own dedicated driver and monitor, synchronized to the respective clock domain. This separation facilitates precise stimulus generation and response monitoring tailored to each clock's timing requirements, thereby enhancing the clarity and robustness of the verification process.

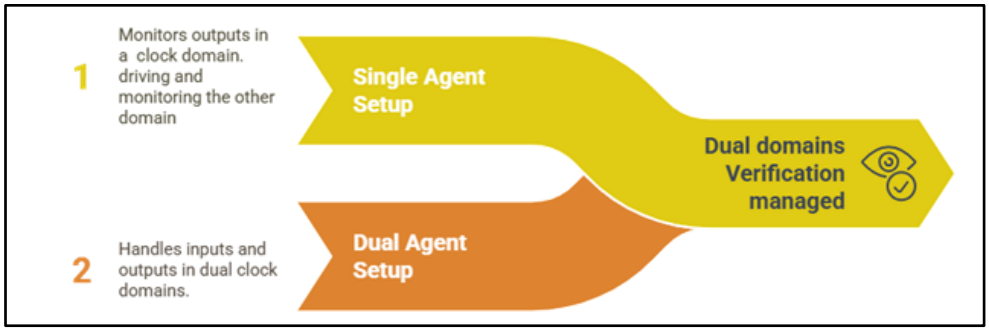


Figure 4-6: diagram illustrating the tb architecture

Alternatively, if one of the interfaces is output-only—for instance, a read-only interface—verification can be streamlined by employing a single agent that contains two monitors, each aligned with one of the clock domains. Since no stimulus generation is necessary for the output-only interface, the absence of a driver reduces complexity without compromising verification coverage.
This adaptable testbench architecture enables effective synchronization and comprehensive verification across multiple clock domains, while optimizing resource utilization based on the DUT's interface characteristics. Such flexibility is crucial for managing the complexities inherent in modern multi-clock designs and achieving thorough verification results.

4.3.3. Managing Process Synchronization in UVM Testbench

Purpose of Synchronizers

Testbench Coordination:
Within UVM testbenches, synchronizers are used to orchestrate the actions of different verification components, some of which may operate at varying simulation times or require strict sequencing to maintain simulation accuracy.

Implementing synchronization mechanisms in testbenches:
Several synchronization constructs are commonly used in UVM environments to coordinate the execution of parallel processes and manage resource access:

Event-Based Synchronization:
The uvm_event class allows components to wait for or trigger specific events, enabling coordination between concurrent processes. For instance, stimulus generation can be delayed until all components have completed initialization.

Barriers:
UVM barriers provide a mechanism to block multiple processes until all have reached a common synchronization point. This is useful, for example, to ensure that a test phase does not begin until all agents are fully prepared.

Semaphores:
Semaphores control access to shared resources by limiting the number of processes that can access them concurrently. They also serve as synchronization tools by managing entry to critical code sections, ensuring orderly execution.

Custom Synchronizer Objects:
In clock domain crossing (CDC) scenarios, verification engineers often implement custom synchronizer models that mimic the behavior of hardware synchronizers. These models enable accurate simulation of CDC paths and help validate the design's robustness in handling cross-clock-domain communication.

Chapter 5: Physical Layer Verification:

5.1. Link Training State Machine "LTSM":

5.1.1. Interface Signals:

As previously discussed in chapter 4 in the LTSM section, the LTSM block serves as a central control unit that interfaces with all major components of the Logical PHY layer, including the Controller, Mainband, and Sideband blocks. Since the LTSM governs the overall layer operating state, its output signals are all driven in the local clock domain (Lclk). It is therefore the responsibility of each connected block to determine whether synchronization is required when interfacing with LTSM-driven signals, depending on its own operating clock domain.

5.1.2. UVM Testbench Architecture:

One of the biggest challenges we faced while verifying the LTSM block was dealing with its complex and heavily branched state machine, as seen in Fig. 3-5. The sheer number of possible transitions makes it clear that simply applying flat, state-independent probabilities to drive control signals like `sb_exit` or `trainerror_hs` isn't enough. For instance, giving `sb_exit` a fixed 60% probability often caused the FSM to fly through states without hitting any of the timeouts or retry conditions—cases that are critical for coverage.

To get more meaningful behavior, we moved away from static probabilities and instead used **conditional probabilities**—tied directly to the FSM's current state (`cs`). This made the sequence more context-aware and helped us control the flow of transitions more deliberately.

A good example of this is how we handled `trainerror_hs`. We wanted to inject training errors in *all* states to exercise error-handling and recovery logic. But some states, like `SBINIT_State`, show up more often—especially early on—so injecting errors with equal probability across all states just meant we were stressing the early states far too much. Instead, we lowered the error injection probability in those frequent early states and gradually increased it as the FSM progressed deeper, helping us explore a broader and more balanced set of error scenarios.

We also made sure certain control signals were only active during relevant states. Some signals just don't make sense unless the FSM is in a specific context, so we added state-based constraints to prevent them from toggling randomly. This avoided invalid transitions and reduced noise in the simulation.

In the end, conditioning signal behavior on the current FSM state turned out to be essential. It allowed us to generate deeper, more realistic scenarios, helped hit corner cases, and ultimately improved both coverage and simulation efficiency.

Now in order to implement conditional probability based on the LTSM's current state—specifically the output signals `cs`, `mbinit_cs` & `mbtrain_cs` —we needed a mechanism for the sequence to *observe* output signals before generating the next set of inputs. This is non-trivial because `cs` is an output from the DUT, not an input to the sequence, and standard UVM sequences are inherently input-driven and stateless between items.

We explored several approaches to solve this. One idea was to have the sequence pull the DUT interface from the UVM configuration database, then sample the required output signals using helper functions (e.g., sampling, triggering events, and setting class variables). While technically feasible, this method was both cumbersome and **non-LRM-compliant**, as it violated the abstraction and encapsulation rules expected of UVM sequences.

A more elegant and compliant solution was inspired by a technique described in a paper by Cummings and Gass, which proposed using a **reactive stimulus generation architecture**. In this approach, stimulus is generated in response to observed DUT outputs—achieved by shifting stimulus logic into reactive components like monitors or predictors, which can legally observe outputs and influence future sequence behavior through transactions or shared state.

This methodology, achieved by simply adding extra connections between the Driver, Sequencer & Sequence, and an extra RSP item going in the opposite direction of the normal ITM item containing the stimulus, allowed us to base stimulus probabilities on actual DUT behavior in a clean, scalable way, fully within UVM best practices.

The LTSM UVM architecture was built around a fundamental principle: **smart stimulus drives smarter coverage**. Rather than relying on a large set of static tests or scenario-based sequences, the environment was architected to generate context-aware, self-adaptive stimulus that evolves in response to the DUT's state. At the heart of this setup is a reactive stimulus framework, which allows the testbench to observe key output signals from the LTSM block—such as the current FSM state—and feed them back into the sequence layer. This enables the sequence to shape future transactions based on what the DUT has already done.

What makes this approach particularly powerful is that it transforms the sequence into a **state-sensitive generator**, not just a packet pusher. Stimulus isn't blindly random—it's probabilistic, but conditioned on the current protocol context. The sequence constantly recalculates its behavior to try to "exhaust" all reachable paths, transitions, and edge cases over time, effectively stress-testing the design using probabilities defined by the verification engineer. This method drastically reduces the need for hardcoded paths and allows the DUT to drive the test as much as the test drives the DUT.

This isn't just functional—it's efficient. The architecture is intentionally minimal in components but maximized in behavioral depth, designed to scale naturally with the protocol's complexity while maintaining tight alignment with UVM best practices.

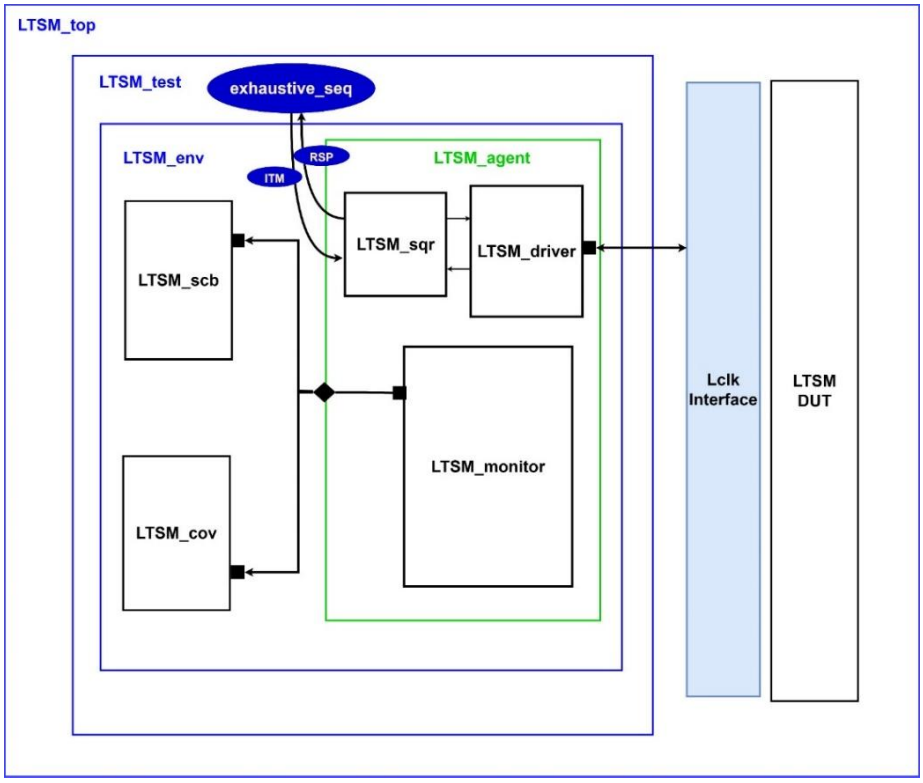


Figure 5-1: UVM architecture used for the LTSM environment.

5.1.3. Test Plan:

The test plan for the LTSM was designed to develop an exhaustive sequence that covers all possible event paths within the long and intertwined state machine, ensuring all scenarios outlined in the following table are exercised.

5.1.3.1 Test Cases

The following table contains several test cases that ensure all possible functionalities are excersied:

Table 5-1: LTSM Verification Plan (sheet 1 of 4).

Test Case Label	Test Case Description	Functional Coverage	Functionality Check
RESET	Randomized reset occurring with low random probability to ensure the system acts correctly	–	Checked by golden model
LP_STATE_REQ_TRANS	First method of exiting RESET via an adapter request	Covered by a dedicated transitional bin named LP_STATE_REQ_TRANS	Checked by golden model
SW_exit_RESET	Second method of exiting RESET via a SW request	–	Checked by golden model
SB_exit_RESET	Third and final method of exiting RESET via remote module request	–	Checked by golden model
MB_OP_CODE	Checking that the Mainband gets the correct opcodes all the time	–	Checked by an Assertion
PL_TRDY	Ensuring the pl_trdy flag is correctly de asserted whenever a stall ack is received	–	Checked by golden model
PHY_IN_RETRAIN	Ensuring the Flag is asserted while the LTSM is in Retrain	–	Checked by golden model
ACTIVE_FLAG	Ensuring the Flag is asserted while the LTSM is in ACTIVE	–	Checked by an Assertion
CURRENT_STATE_COVERAGE_1	LTSM transition from RESET to SBINIT	Covered by a dedicated transitional bin under the CURRENT_STATE_COVERAGE coverpoint	Checked by golden model
CURRENT_STATE_COVERAGE_2	LTSM transition from SBINIT to MBINIT	Covered by a dedicated transitional bin under the CURRENT_STATE_COVERAGE coverpoint	Checked by golden model
CURRENT_STATE_COVERAGE_3	LTSM transition from SBINIT to TRAINERROR	Covered by a dedicated transitional bin under the CURRENT_STATE_COVERAGE coverpoint	Checked by golden model
CURRENT_STATE_COVERAGE_4	LTSM transition from MBINIT to MBTRAIN	Covered by a dedicated transitional bin under the CURRENT_STATE_COVERAGE coverpoint	Checked by golden model
CURRENT_STATE_COVERAGE_5	LTSM transition from MBINIT to TRAINERROR	Covered by a dedicated transitional bin under the CURRENT_STATE_COVERAGE coverpoint	Checked by golden model

Table 5-2: LTSM Verification Plan (sheet 2 of 4).

Test Case Label	Test Case Description	Functional Coverage	Functionality Check
CURRENT_STATE_COVERAGE_6	LTSM transition from MBTRAIN to LINKINIT	Covered by a dedicated transitional bin under the CURRENT_STATE_COVERAGE coverpoint	Checked by golden model
CURRENT_STATE_COVERAGE_7	LTSM transition from MBTRAIN to LINKINIT	Covered by a dedicated transitional bin under the CURRENT_STATE_COVERAGE coverpoint	Checked by golden model
CURRENT_STATE_COVERAGE_8	LTSM transition from LINKINIT to ACTIVE	Covered by a dedicated transitional bin under the CURRENT_STATE_COVERAGE coverpoint	Checked by golden model
CURRENT_STATE_COVERAGE_9	LTSM transition from LINKINIT to TRAINERROR	Covered by a dedicated transitional bin under the CURRENT_STATE_COVERAGE coverpoint	Checked by golden model
CURRENT_STATE_COVERAGE_10	LTSM transition from ACTIVE to TRAINERROR	Covered by a dedicated transitional bin under the CURRENT_STATE_COVERAGE coverpoint	Checked by golden model
CURRENT_STATE_COVERAGE_11	LTSM transition from TRAINERROR to RESET	Covered by a dedicated transitional bin under the CURRENT_STATE_COVERAGE coverpoint	Checked by golden model
CURRENT_STATE_COVERAGE_12	LTSM transition from PHYRETRAIN to MBTRAIN	Covered by a dedicated transitional bin under the CURRENT_STATE_COVERAGE coverpoint	Checked by golden model
CURRENT_STATE_COVERAGE_13	LTSM transition from PHYRETRAIN to TRAINERROR	Covered by a dedicated transitional bin under the CURRENT_STATE_COVERAGE coverpoint	Checked by golden model
MBINIT_STATE_COVERAGE_1	MBINIT: PARAM to CAL	Covered by a dedicated transitional bin under the MBINIT_STATE_COVERAGE coverpoint	Checked by golden model
MBINIT_STATE_COVERAGE_2	MBINIT: CAL to REPAIRCLK	Covered by a dedicated transitional bin under the MBINIT_STATE_COVERAGE coverpoint	Checked by golden model
MBINIT_STATE_COVERAGE_3	MBINIT: REPAIRCLK to REPAIRVAL	Covered by a dedicated transitional bin under the MBINIT_STATE_COVERAGE coverpoint	Checked by golden model
MBINIT_STATE_COVERAGE_4	MBINIT: REPAIRVAL to REVERSALMB	Covered by a dedicated transitional bin under the MBINIT_STATE_COVERAGE coverpoint	Checked by golden model

Table 5-3: LTSM Verification Plan (sheet 3 of 4).

Test Case Label	Test Case Description	Functional Coverage	Functionality Check
MBINIT_STATE_COVERAGE_5	MBINIT: REVERSALMB to REPAIRMB	Covered by a dedicated transitional bin under the MBINIT_STATE_COVERAGE coverpoint	Checked by golden model
MBTRAIN_STATE_COVERAGE_1	MBTRAIN: VALVREF to DATAVREF	Covered by a dedicated transitional bin under the MBTRAIN_STATE_COVERAGE coverpoint	Checked by golden model
MBTRAIN_STATE_COVERAGE_2	MBTRAIN: DATAVREF to SPEEDIDLE	Covered by a dedicated transitional bin under the MBTRAIN_STATE_COVERAGE coverpoint	Checked by golden model
MBTRAIN_STATE_COVERAGE_3	MBTRAIN: SPEEDIDLE to TXSELFCAL	Covered by a dedicated transitional bin under the MBTRAIN_STATE_COVERAGE coverpoint	Checked by golden model
MBTRAIN_STATE_COVERAGE_4	MBTRAIN: TXSELFCAL to RXCLKCAL	Covered by a dedicated transitional bin under the MBTRAIN_STATE_COVERAGE coverpoint	Checked by golden model
MBTRAIN_STATE_COVERAGE_5	MBTRAIN: RXCLKCAL to VALTRAINCENTER	Covered by a dedicated transitional bin under the MBTRAIN_STATE_COVERAGE coverpoint	Checked by golden model
MBTRAIN_STATE_COVERAGE_6	MBTRAIN: VALTRAINCENTER to VALTRAINVREF	Covered by a dedicated transitional bin under the MBTRAIN_STATE_COVERAGE coverpoint	Checked by golden model
MBTRAIN_STATE_COVERAGE_7	MBTRAIN: VALTRAINVREF to DATATRAINCENTER1	Covered by a dedicated transitional bin under the MBTRAIN_STATE_COVERAGE coverpoint	Checked by golden model
MBTRAIN_STATE_COVERAGE_8	MBTRAIN: DATATRAINCENTER1 to DATATRAINVREF	Covered by a dedicated transitional bin under the MBTRAIN_STATE_COVERAGE coverpoint	Checked by golden model
MBTRAIN_STATE_COVERAGE_9	MBTRAIN: DATATRAINVREF to RXDESKEW	Covered by a dedicated transitional bin under the MBTRAIN_STATE_COVERAGE coverpoint	Checked by golden model
MBTRAIN_STATE_COVERAGE_10	MBTRAIN: RXDESKEW to DATATRAINCENTER2	Covered by a dedicated transitional bin under the MBTRAIN_STATE_COVERAGE coverpoint	Checked by golden model
MBTRAIN_STATE_COVERAGE_11	MBTRAIN: DATATRAINCENTER2 to LINKSPEED	Covered by a dedicated transitional bin under the MBTRAIN_STATE_COVERAGE coverpoint	Checked by golden model

Table 5-4: LTSM Verification Plan (sheet 4 of 4).

Test Case Label	Test Case Description	Functional Coverage	Functionality Check
MBTRAIN_STATE_COVERAGE_12	MBTRAIN: LINKSPEED to REPAIR	Covered by a dedicated transitional bin under the MBTRAIN_STATE_COVERAGE coverpoint	Checked by golden model
MBTRAIN_STATE_COVERAGE_13	MBTRAIN: LINKSPEED to SPEEDIDLE	Covered by a dedicated transitional bin under the MBTRAIN_STATE_COVERAGE coverpoint	Checked by golden model
STATE_COVERAGE_ALL	LTSM state presence: RESET, SBINIT, MBINIT, MBTRAIN, LINKINIT, ACTIVE, TRAINERROR...	Covered by a dedicated bin to each state	Checked by golden model

5.1.3.2 Test Sequence

After fully utilizing the reactive stimulus methodology, a single test sequence was optimized enough to generate a wide range of meaningful LTSM scenarios, including timeout cases, retraining sequences, and external error scenarios—all without requiring multiple handcrafted tests. By adjusting signal probabilities based on the FSM's current state, the sequence could naturally explore deep and rarely visited states while still maintaining valid protocol behavior. This not only improved functional coverage significantly but also simplified regression by reducing the number of tests needed to hit key scenarios.

5.1.4. Results:

5.1.4.1 Bugs Found & Fixed

1. During verification, we identified a signal contention issue involving the mbtrain_ns (next state) signal. Specifically, mbtrain_ns was being driven by two separate always blocks within the design: one within the main MBTRAIN FSM logic and another within a parallel conditional block related to PHY retrain handling (phyretrain). This caused inconsistent behavior during simulation—most notably, when the PHY requested a retrain, it attempted to set mbtrain_ns to REPAIR, but this assignment was immediately overridden by the default state logic in the main FSM block, which assigned mbtrain_ns to VALVREF. As a result, the FSM failed to enter the intended REPAIR state during retrain conditions, breaking the protocol flow and exposing a priority conflict between independent control paths. This bug highlights the need for centralized or coordinated next-state assignment in designs with multiple control sources to avoid race conditions and unintended state overrides. This can be noticed in the next figure where the "3'b100" retrain encoding should lead to the Repair mbtrain state but due to the racing it led to datavref which is the assigned next state in the first always block.

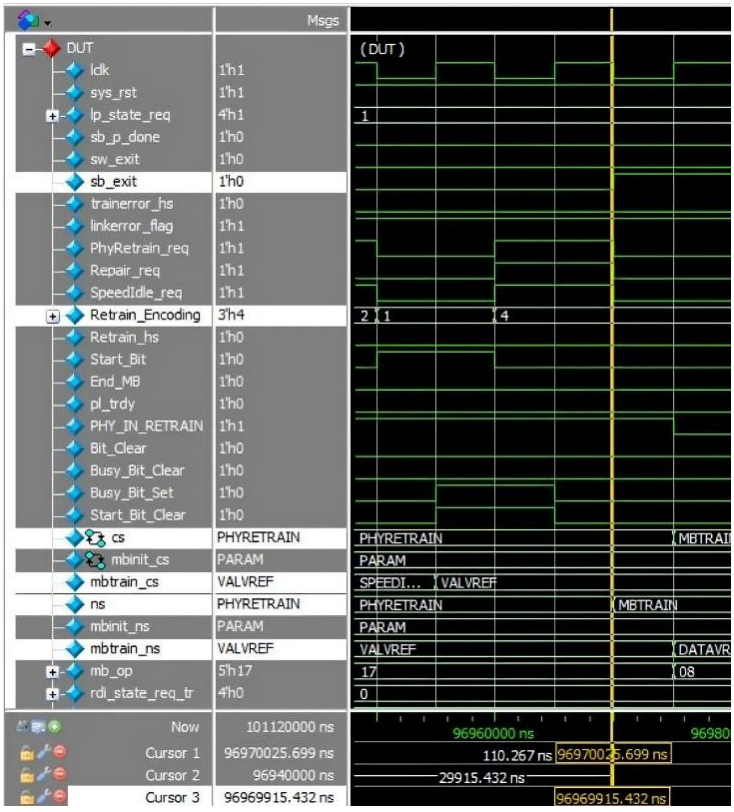


Figure 5-2: Bug (1) MB Rx Verification

2. The timeout timer didn't return to 0 after reaching timeout occurrence.

Figure 5-3: state_timer didnt return to zero after timeout

3. Some outputs were not reset.

4. When entering RESET state, the timer was reset to 1 not 0.

5. Adapter_exit was not de-asserted even after hitting a hard rst. That can cause unintentional RESET exit.

6. if the lp_state transition happens before the 4ms timer, the current state won't change to sb_exit as the transition is not registered.

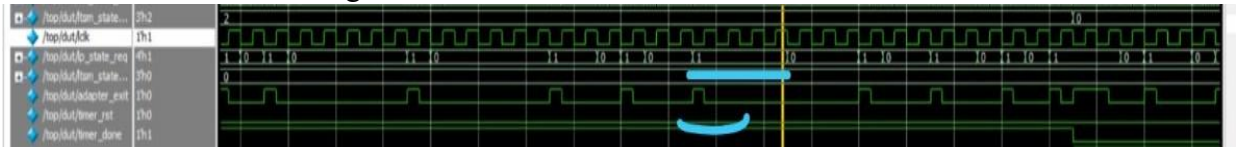


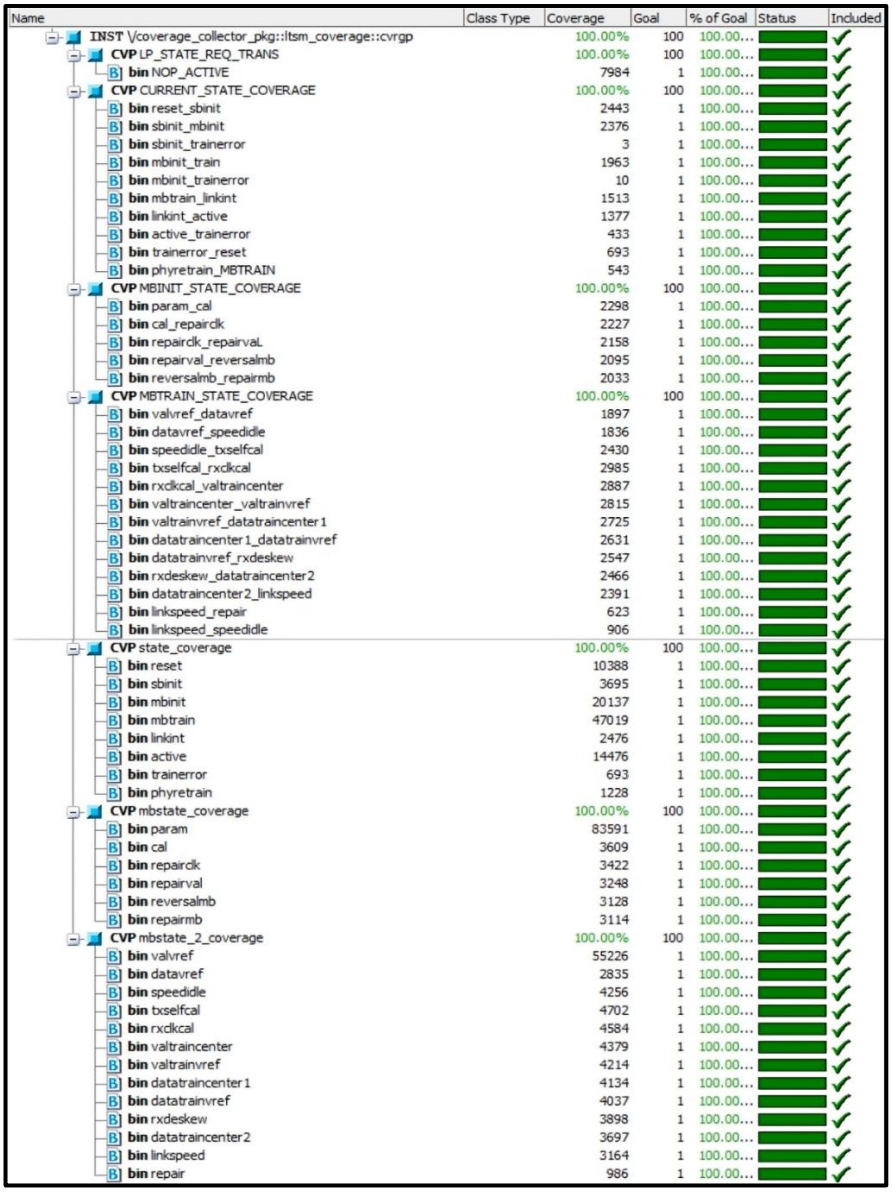
Figure 5-4: adapter exit not registered

7. Some internal signals were latched.

8. sb_exit was not registered so after the reset- 4ms-timeout occur, the LTSM did not enter sb_init state.

5.1.4.2 Coverage Achieved

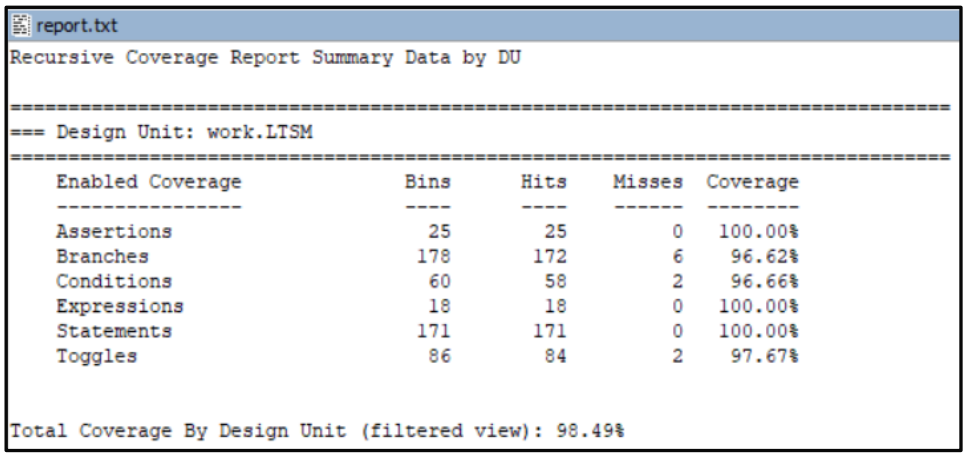
- Functional Coverage:
  It's important to notice the number of simulation cycles required in order to hit all required bins is very high. Therefore, for the LTSM sequence we ran 10,000 cycleswith 8 cycles timeout for shorter simulation time.



Name	Class Type	Coverage	Goal	% of Goal	Status	Included
INST \/coverage_collector_pkg::ltsm_coverage::cvrgp		100.00%	100	100.00...		
CVP LP_STATE_REQ_TRANS		100.00%	100	100.00...		
bin NOP_ACTIVE		7984	1	100.00...		
CVP CURRENT_STATE_COVERAGE		100.00%	100	100.00...		
bin reset_sbinit		2443	1	100.00...		
bin sbinit_mbinit		2376	1	100.00...		
bin sbinit_trainerror		3	1	100.00...		
bin mbinit_train		1963	1	100.00...		
bin mbinit_trainerror		10	1	100.00...		
bin mbtrain_linkint		1513	1	100.00...		
bin linkint_active		1377	1	100.00...		
bin active_trainerror		433	1	100.00...		
bin trainerror_reset		693	1	100.00...		
bin phyretrain_MBTRAIN		543	1	100.00...		
CVP MBINIT_STATE_COVERAGE		100.00%	100	100.00...		
bin param_cal		2298	1	100.00...		
bin cal_repairclk		2227	1	100.00...		
bin repairclk_repairvaL		2158	1	100.00...		
bin repairval_reversalmb		2095	1	100.00...		
bin reversalmb_repairmb		2033	1	100.00...		
CVP MBTRAIN_STATE_COVERAGE		100.00%	100	100.00...		
bin valvref_datavref		1897	1	100.00...		
bin datavref_speedidle		1836	1	100.00...		
bin speedidle_txselfcal		2430	1	100.00...		
bin txselfcal_rxclkcal		2985	1	100.00...		
bin rxclkcal_valtraincenter		2887	1	100.00...		
bin valtraincenter_valtrainvref		2815	1	100.00...		
bin valtrainvref_datatraincenter1		2725	1	100.00...		
bin datatraincenter1_datatrainvref		2631	1	100.00...		
bin datatrainvref_rxdeskew		2547	1	100.00...		
bin rxdeskew_datatraincenter2		2466	1	100.00...		
bin datatraincenter2_linkspeed		2391	1	100.00...		
bin linkspeed_repair		623	1	100.00...		
bin linkspeed_speedidle		906	1	100.00...		
CVP state_coverage		100.00%	100	100.00...		
bin reset		10388	1	100.00...		
bin sbinit		3695	1	100.00...		
bin mbinit		20137	1	100.00...		
bin mbtrain		47019	1	100.00...		
bin linkint		2476	1	100.00...		
bin active		14476	1	100.00...		
bin trainerror		693	1	100.00...		
bin phyretrain		1228	1	100.00...		
CVP mbstate_coverage		100.00%	100	100.00...		
bin param		83591	1	100.00...		
bin cal		3609	1	100.00...		
bin repairclk		3422	1	100.00...		
bin repairval		3248	1	100.00...		
bin reversalmb		3128	1	100.00...		
bin repairmb		3114	1	100.00...		
CVP mbstate_2_coverage		100.00%	100	100.00...		
bin valvref		55226	1	100.00...		
bin datavref		2835	1	100.00...		
bin speedidle		4256	1	100.00...		
bin txselfcal		4702	1	100.00...		
bin rxclkcal		4584	1	100.00...		
bin valtraincenter		4379	1	100.00...		
bin valtrainvref		4214	1	100.00...		
bin datatraincenter1		4134	1	100.00...		
bin datatrainvref		4037	1	100.00...		
bin rxdeskew		3898	1	100.00...		
bin datatraincenter2		3697	1	100.00...		
bin linkspeed		3164	1	100.00...		
bin repair		986	1	100.00...		

Figure 5-5: LTSM Functional Coverage Report

- Code Coverage:
  The coverage report above shows the final results for the LTSM block after applying the reactive stimulus methodology and refining sequence behavior based on FSM state awareness. We achieved **98.49% total coverage**, with **100% assertion, statement, and expression coverage**, indicating full activation of all intended functionality and checks. Branch and condition coverage reached **96.62%** and **96.66%** respectively, confirming that the majority of decision logic paths were exercised, including corner cases such as error handling and timeouts. **Toggle coverage** also reached a strong **97.67%**, validating signal-level activity across the design. The remaining uncovered bins were analyzed and found to correspond to unreachable or specification-excluded paths.



report.txt

```
Recursive Coverage Report Summary Data by DU

================================================================================
=== Design Unit: work.LTSM
================================================================================
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Assertions                      25        25         0   100.00%
    Branches                       178       172         6    96.62%
    Conditions                      60        58         2    96.66%
    Expressions                     18        18         0   100.00%
    Statements                     171       171         0   100.00%
    Toggles                         86        84         2    97.67%


Total Coverage By Design Unit (filtered view): 98.49%
```

Figure 5-6: LTSM Code Coverage

5.2. Controller:

5.2.1. RDI:

5.2.1.1 Interface Signals:

The RDI module serves as the bridge between the physical and adapter layers in the UCIe protocol stack. Its responsibilities span multiple protocol-critical functions, including state management, handshake sequencing, error detection, and packet processing. From a verification perspective, this module exposes a wide range of interface signals that reflect both its internal FSM and its interactions with surrounding layers.

Key signals include state indicators tied to the RDI state machine (`Nop`, `Active`, `Retrain`, `LinkReset`, etc.), as well as handshake control lines such as `stall_req/ack`, `wake_req/ack`, and `clk_req/ack`. These signals were monitored and constrained carefully during verification to ensure accurate sequencing and protocol compliance. In addition, dedicated error reporting signals and packet-related input/output interfaces were validated through reactive checks and assertions to confirm correct behavior under both nominal and corner-case conditions.

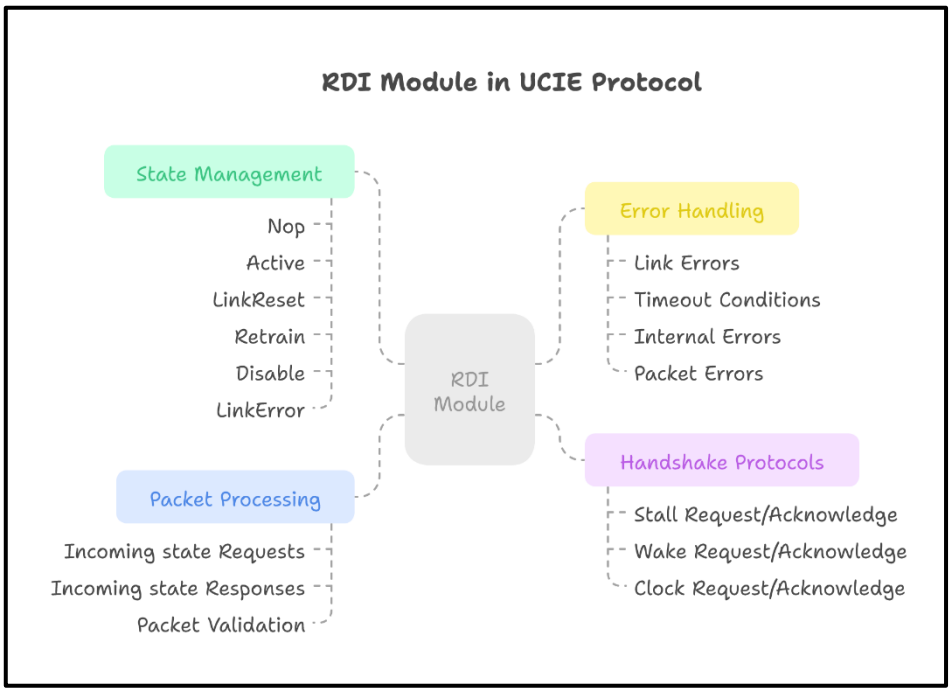


Figure 5-7: RDI responsibilities.

Building the RDI item constraints required a comprehensive emulation of correct system-level behavior, rooted primarily in the values of the `cs` (current state) output from the LTSM block. To generate realistic and protocol-compliant stimulus, the sequence also had to accurately reflect the behavior of several critical signals from the Adapter layer, such as `lp_state_req` and `lp_linkerror`, in addition to handshake signals like `wake_req/ack`, `stall_req/ack`, and `clk_req/ack`. These signals had to follow specific timing rules and protocol conditions dictated by the UCIe spec, and therefore could not be generated independently without considering system context.

On top of this, the stimulus had to simulate valid `data_in` packet contents to represent inputs from the remote RDI (partner die), which included encoding legal and illegal messages according to the current link state. Special attention was paid to ensure that the sequence maintained correct packet sequences, but with a small, controlled probability of error injected into the stimulus. This was key to validating the RDI module's packet error detection mechanisms and its ability to handle unexpected or malformed sequences.

A few notable constraint cases include:

- The Adapter layer is **not allowed** to issue a request to transition to the `Active` state immediately after asserting `lp_linkerror`. Such a transition violates protocol and must be filtered at the sequence level.
- Once `lp_linkerror` is asserted by the Adapter, it must **remain high** until the PHY layer (through LTSM) acknowledges it by transitioning into the `LinkError` or `TrainError` state. The sequence constraints ensure this condition is respected.
- Several Mainband signals, such as `mb_tx_done`, `mb_rx_done`, and `pl_error`, are **pulsed events**. Because these are single-cycle signals, the RDI module must be capable of **latching** or registering them. Our constraints accounted for this by ensuring their pulses occurred at protocol-valid times and didn't span multiple cycles.
- To inject controlled errors into the system, we introduced a randomized variable `response_type`. This variable determined the type of `data_in` packet the RDI would receive, allowing it to be either:

- ✓ A correctly formatted response based on the current operating point (e.g., state transition acknowledgment).
- ✓ An all-zero message, which was useful for delaying link transitions or simulating a "no-response" state.
- ✓ An incorrect response message intended to test how the RDI module reacts to invalid conditions or out-of-order events.

These constraints formed the backbone of a highly reactive and realistic sequence environment that not only verified protocol compliance but also stressed the RDI's ability to handle asynchronous, incorrect, and edge-case behavior in a controlled and measurable way.

5.2.1.2 UVM Testbench Architecture:

Given the functional scope outlined in the RDI block's respective section in the Design chapter, we determined that the verification environment did not require a radically new architecture beyond what had already been established in the LTSM environment. The existing setup—centered around a reactive stimulus methodology—proved not only sufficient, but highly effective. This approach feeds DUT output signals back into the sequence layer, enabling the stimulus to adapt in real time based on the system's current state and behavior.

By integrating this feedback loop directly into the stimulus generation process, the sequence becomes **self-exhausting**—meaning it continuously explores new stimulus permutations on its own, guided by dynamic probabilities defined by the verification engineer. This design ensures broad protocol exploration while maintaining stimulus relevance and avoiding unrealistic transitions. It's an elegant way to offload the complexity of corner-case generation to a smart, context-aware system that evolves with the DUT state. Rather than relying on a large number of handcrafted tests, this architecture pushes the design to its limits within a unified and scalable framework.

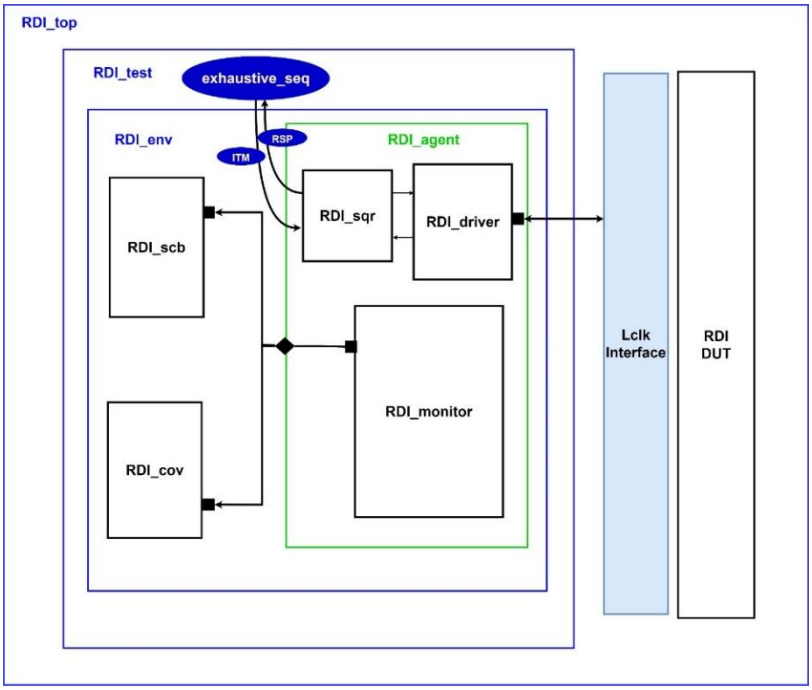


Figure 5-8: UVM architecutre used for the RDI environment.

5.2.1.3 Test Plan:

5.2.1.3.1 Test Cases:

The test cases in the following table include both legal and illegal functionalities of the RDI block and ensure the block reacts accordingly.

Table 5-5: RDI Verification Plan(sheet 1 of 7).

Test Case Label	Test Case Description	Functional Coverage	Functionality Check
SYS_RST	Randomized reset occurring with low random probability to ensure the system acts correctly	__	Checked by golden model
STALL_HS	Complete correct STALL Handshake mechanism	__	Checked by golden model
WAKE_HS	Complete correct WAKE Handshake mechanism	__	Checked by golden model
CLK_HS	Complete correct CLK Handshake mechanism	__	Checked by golden model
MB_DONE	Mainband done is correctly registered, acted upon & reset.	__	Checked by golden model
PL_ERROR	Pl_error signal is correctly registered, acted upon & reset.	__	Checked by golden model
LTSM_TIMEOUT	The RDI requests Linkerror when LTSM timeouts occur.	__	Checked by golden model
NOP_ACTIVE	Transition from `Nop` to `Active`	Covered in the coverpoint named pl_state_transition under a bin with the same test case label	Checked by golden model

Table 5-6: RDI Verification Plan(sheet 2 of 7).

Test Case Label	Test Case Description	Functional Coverage	Functionality Check
NOP_LINKRESET	Transition from `Nop` to `LinkReset`	Covered in the coverpoint named pl_state_transition under a bin with the same test case label	Checked by golden model
NOP_LINKERROR	Transition from `Nop` to `LinkError`	Covered in the coverpoint named pl_state_transition under a bin with the same test case label	Checked by golden model
NOP_DISABLE	Transition from `Nop` to `Disable`	Covered in the coverpoint named pl_state_transition under a bin with the same test case label	Checked by golden model
ACTIVE_RETRAIN	Transition from `Active` to `Retrain`	Covered in the coverpoint named pl_state_transition under a bin with the same test case label	Checked by golden model
ACTIVE_NOP	Reset path: `Active` to `Nop`	Covered in the coverpoint named pl_state_transition under a bin with the same test case label	Checked by golden model
ACTIVE_LINKERROR	Error recovery: `Active` to `LinkError`	Covered in the coverpoint named pl_state_transition under a bin with the same test case label	Checked by golden model
ACTIVE_LINKRESET	Transition from `Active` to `LinkReset`	Covered in the coverpoint named pl_state_transition under a bin with the same test case label	Checked by golden model

Table 5-7: RDI Verification Plan(sheet 3 of 7).

Test Case Label	Test Case Description	Functional Coverage	Functionality Check
ACTIVE_DISABLE	Transition from `Active` to `Disable`	Covered in the coverpoint named pl_state_transition under a bin with the same test case label	Checked by golden model
RETRAIN_ACTIVE	Successful retrain: `Retrain` to `Active`	Covered in the coverpoint named pl_state_transition under a bin with the same test case label	Checked by golden model
RETRAIN_LINKERROR	Retrain fails or interrupted by error	Covered in the coverpoint named pl_state_transition under a bin with the same test case label	Checked by golden model
RETRAIN_LINKRESET	Reset issued during retrain	Covered in the coverpoint named pl_state_transition under a bin with the same test case label	Checked by golden model
RETRAIN_DISABLE	Retrain aborted or interrupted with disable	Covered in the coverpoint named pl_state_transition under a bin with the same test case label	Checked by golden model
LINKRESET_DISABLE	Link reset ends with disable	Covered in the coverpoint named pl_state_transition under a bin with the same test case label	Checked by golden model
LINKRESET_NOP	Recovery path: reset to `Nop`	Covered in the coverpoint named pl_state_transition under a bin with the same test case label	Checked by golden model

Table 5-8: RDI Verification Plan(sheet 4 of 7).

Test Case Label	Test Case Description	Functional Coverage	Functionality Check
LINKRESET_LINKERROR	Error occurs during or after reset	Covered in the coverpoint named pl_state_transition under a bin with the same test case label	Checked by golden model
DISABLE_LINKERROR	Error in `Disable` state	Covered in the coverpoint named pl_state_transition under a bin with the same test case label	Checked by golden model
DISABLE_NOP	Link recovers from disable to `Nop`	Covered in the coverpoint named pl_state_transition under a bin with the same test case label	Checked by golden model
LINKERROR_NOP	Recovery from `LinkError` to `Nop`	Covered in the coverpoint named pl_state_transition under a bin with the same test case label	Checked by golden model
LINKRESET_RETRAIN	Illegal: reset going to retrain	Covered in the coverpoint named pl_state_transition under a bin with the same test case label	Checked by golden model
DISABLE_LINKRESET	Illegal: disable transitioning to reset	Covered in the coverpoint named pl_state_transition under a bin with the same test case label	Checked by golden model
LINKERROR_RETRAIN	Illegal: retrain from `LinkError`	Covered in the coverpoint named pl_state_transition under a bin with the same test case label	Checked by golden model

Table 5-9: RDI Verification Plan(sheet 5 of 7).

Test Case Label	Test Case Description	Functional Coverage	Functionality Check
LINKERROR_DISABLE	Illegal: `LinkError` to `Disable`	Covered in the coverpoint named pl_state_transition under a bin with the same test case label	Checked by golden model
LINKERROR_LINKRESET	Illegal: `LinkError` to `LinkReset`	Covered in the coverpoint named pl_state_transition under a bin with the same test case label	Checked by golden model
LINKERROR_ACTIVE	Illegal: `LinkError` to `Active`	Covered in the coverpoint named pl_state_transition under a bin with the same test case label	Checked by golden model
RDI_states	RDI enters each state	Under the coverpoint named pl_state_cp	Checked by golden model
Sb_exit_rdi	`sb_exit` is functioning correctly to transition ltsm from reset to sb_init	Under the coverpoint named sb_exit_cp	Checked by golden model
rdi_over_ltsm_states	The emulated fsm reaches all states. This is necessary as the rdi acts differently in each state for different handshakes and functionalities	Under the coverpoint named cs_cp	—
active_req	`data_out` = request for `Active`	Covered in the coverpoint named msg_cp under a bin with the same test case label	Checked by golden model

Table 5-10: RDI Verification Plan(sheet 6 of 7).

Test Case Label	Test Case Description	Functional Coverage	Functionality Check
active_rsp	`data_out` = response confirming `Active`	Covered in the coverpoint named msg_cp under a bin with the same test case label	Checked by golden model
linkerror_req	`data_out` = request for `LinkError`	Covered in the coverpoint named msg_cp under a bin with the same test case label	Checked by golden model
linkerror_rsp	`data_out` = response confirming `LinkError`	Covered in the coverpoint named msg_cp under a bin with the same test case label	Checked by golden model
retrain_req	`data_out` = request for `Retrain`	Covered in the coverpoint named msg_cp under a bin with the same test case label	Checked by golden model
retrain_rsp	`data_out` = response confirming `Retrain`	Covered in the coverpoint named msg_cp under a bin with the same test case label	Checked by golden model
linkreset_req	`data_out` = request for `LinkReset`	Covered in the coverpoint named msg_cp under a bin with the same test case label	Checked by golden model
linkreset_rsp	`data_out` = response confirming `LinkReset`	Covered in the coverpoint named msg_cp under a bin with the same test case label	Checked by golden model

Table 5-11: RDI Verification Plan(sheet 7 of 7).

Test Case Label	Test Case Description	Functional Coverage	Functionality Check
disable_req	`data_out` = request for `Disable`	Covered in the coverpoint named msg_cp under a bin with the same test case label	Checked by golden model
disable_rsp	`data_out` = response confirming `Disable`	Covered in the coverpoint named msg_cp under a bin with the same test case label	Checked by golden model

5.2.1.3.2 Test Sequence:

Similar to the sequence developed for the LTSM verification environment, the RDI sequence was also built entirely around the reactive stimulus methodology described earlier. However, due to the simpler FSM structure in the RDI block, the emphasis shifted away from dynamic conditional probabilities and toward more tightly defined constraints. While LTSM relied heavily on the state-dependent generation of control signals, the RDI environment presented a different challenge—**a significantly larger number of input signals** that had to be carefully constrained to maintain protocol validity. As a result, the sequence focused more on statically modeling legal input combinations and enforcing strict timing and value constraints, ensuring comprehensive functional exploration without relying on deeply nested probability logic.

5.2.1.4 Results:

5.2.1.4.1 Bugs Found & Fixed

1. **Incorrect Retrain Exit Behavior (FSM Transition Violation)**: The correct protocol behavior for exiting the `Retrain` state requires the `lp_state_req` input to transition from `Nop` to `Active`. This requirement (UCIe Spec, Page 499, Paragraph 1) was not enforced in the design, allowing invalid transitions and premature exits from `Retrain`.
2. **Incorrect Latching Behavior of Timeout Timers**: Both the 8 ms standard timeout timer and the 16 ms `LinkError` residency timer were latched, leading to incorrect behavior. In the next waveform the second timer (the state 8ms timer should return to zero after timeout and entering linkerror).

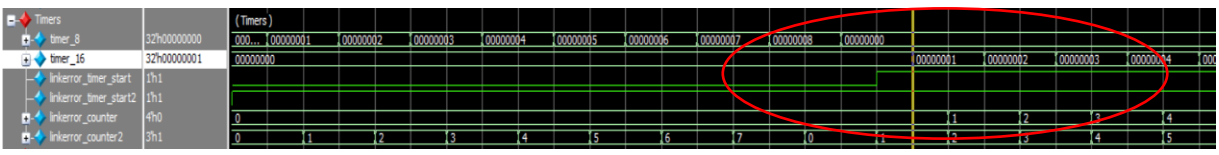


Figure 5-9: Timer 2 start signal latched

3. **Incorrect timers sizing:** The timers mentioned above were initially given wrong number of bits, this were noticed as the second timer kept counting and then re-wrapped causing a hanging in the test case.
4. **Racing Condition between state assignment / response generation always blocks**
   Response generation logic was affected by premature zeroing of output signals in one of the `always` blocks. This led to valid responses being unintentionally cancelled, due to a race condition between state logic and transaction outputs. The issue was resolved by conditioning the zeroing logic with additional checks and eventually merging both blocks into a unified always block, ensuring deterministic behavior.
5. **Simultaneous Request/Response Conflict Scenario**
   A race condition was also observed when say for example the Adapter asserted `lp_state_req = LinkReset` on the same edge a sideband RDI `LinkReset` entry request arrived from the remote side. The split handling of responses and new requests across separate `always` blocks led to conflicting assignments. The correct behavior is to **respond** to the incoming request, as it is time-critical. This was resolved by merging the logic into a single always block as discussed above with proper conditions.

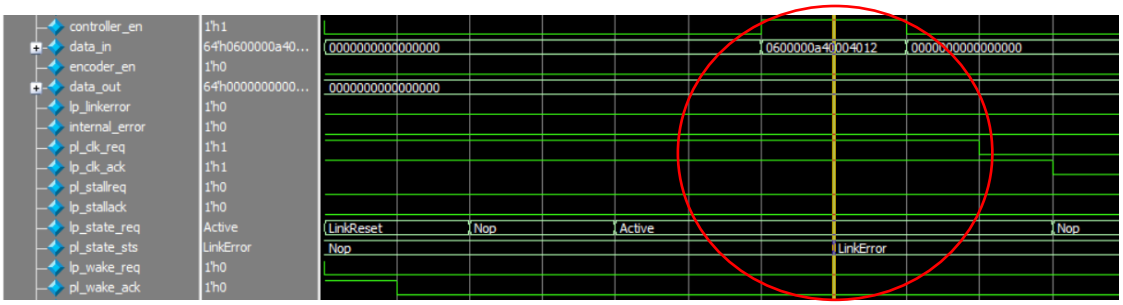


Figure 5-10: Racing between 2 different response generating always blocks

6. **Latched output signals**
   Some output signals, specifically `data_out` & `encoder_en` were latched, this would lead to complete system failure as the arbiter (the block storing all sideband signals from module, module partner & RDI) can neglect other messages from module or module partner (as the RDI has higher priorities than both in sideband messages) leading to failure.
7. **One Tx FSM For All Requests**
   If a certain Req is sent, if a higher request condition became true after the RDI won't send since it's waiting for the response to the initial req. Leading to very delayed and wrong error handling.

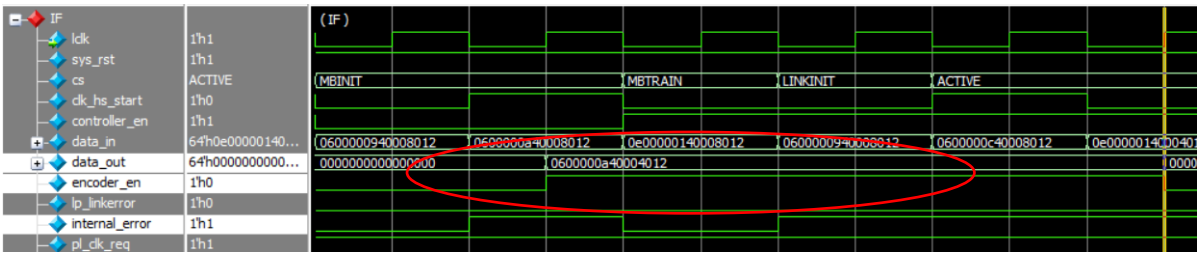


Figure 5-11: Certain outputs were Latched

5.2.1.4.2 Coverage Achieved

- Code Coverage

```
Recursive Coverage Report Summary Data by DU

================================================================================
=== Design Unit: work.rdi_interface
================================================================================
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                       312       262        50    83.97%
    Conditions                     179       125        54    69.83%
    Expressions                     49        46         3    93.87%
    FSM States                       4         4         0   100.00%
    FSM Transitions                  6         6         0   100.00%
    Statements                     523       470        53    89.86%
    Toggles                        176       174         2    98.86%

Total Coverage By Design Unit (filtered view): 90.91%
```

Figure 5-12: RDI Code Coverage

- Functional Coverage

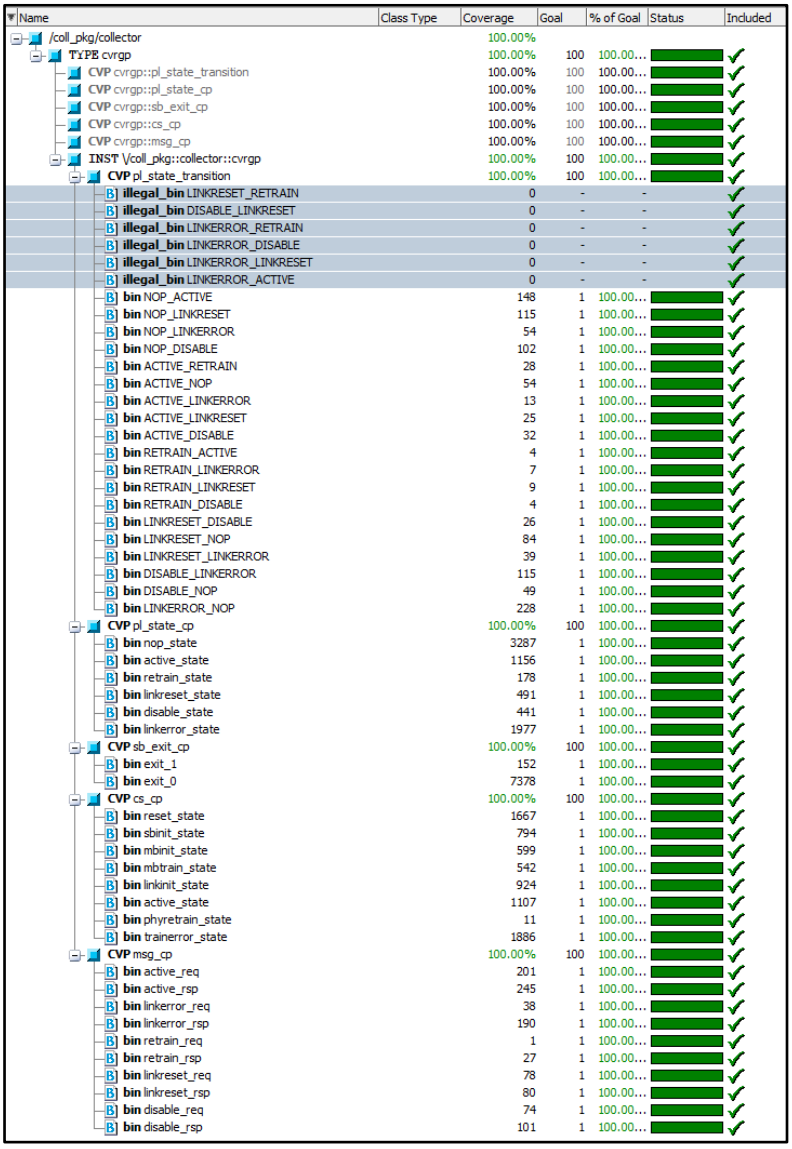


Figure 5-13: RDI Functional Coverage

5.2.2. Phy_Module:

5.2.2.1 Interface with Blocks

PHY module serves as the primary block responsible for initiating handshakes with the LTSM (Link Training State Machine) during the training states. It also manages the initiation of retraining process in the event of any errors on its local physical layer. All control logic within the controller operates at the LCLK clock speed. The diagram below illustrates the key blocks with which the PHY-module interacts, highlighting its central and critical role within the system. This interaction map clearly demonstrates how the PHY-module serves as a vital interface, coordinating communication and control between multiple functional units.

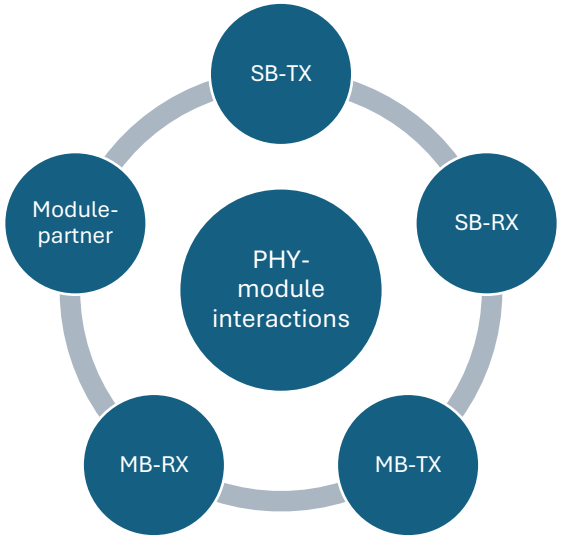


Figure 5-14: the blocks PHY_module interacts with

While the design section provides a detailed description of the block interface signals and their clock domain assignments, this verification study focuses on the implications of these interfaces for testbench architecture and verification methodology. Coverage analysis targets handshake sequences and timing-related corner cases, ensuring comprehensive validation of the interface's functional and temporal behavior.

5.2.2.2 UVM Testbench Architecture

Since our design operates within a single clock domain, only one agent comprising a single driver and monitor is necessary. Because our UVM block primarily focuses on verifying handshakes during the training states, employing reactive stimulus is essential similar to the LTSM environment. This ensures that the DUT is driven responsively and accurately, adapting stimulus generation based on the DUT's behavior to reflect real operational conditions and protocol requirements, where the architecture is shown below.

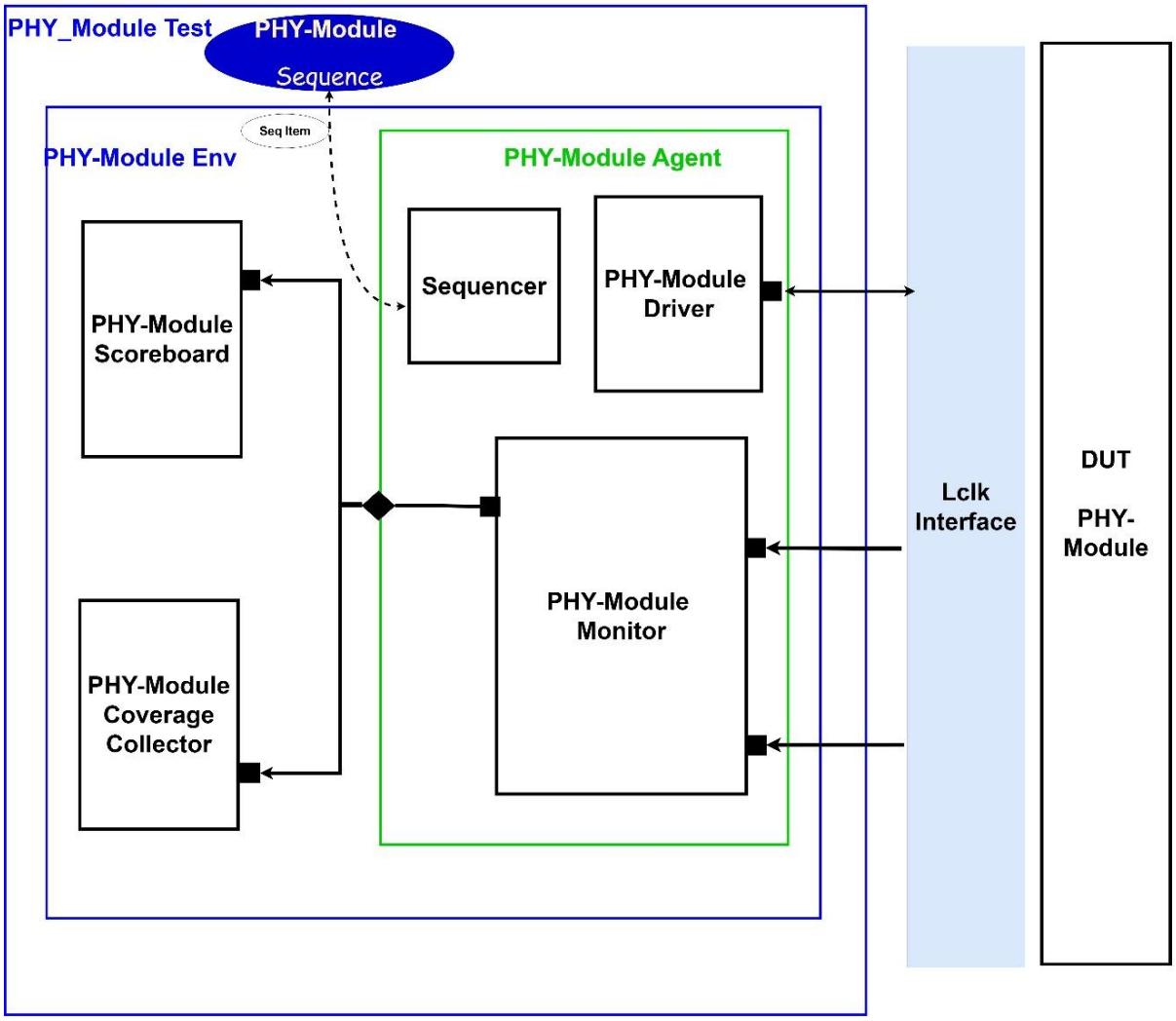


Figure 5-15: PHY-module UVM Testbench Architecture

5.2.2.3 Test plan

5.2.2.3.1 Test cases

Table 5-12: Module Verification Plan (sheet 1 of 4).

Test case Label	Test case description	Functional coverage	Functionality check
Reset_SBINIT_1	*Incorrect order of receiving signals of sb_o_detected and pattern done will be neglected*	Covered by Sb_o_detected bin and Pattern done bin	Checked by reference model
SBINIT_path_sel	path_sel is asserted as long as the ltsm_state is sbinit	Covered by path_sel bin	Checked by reference model
MBINIT_HS	the block is capable of completing the MBINIT handshaks for all of its substates	Covered by covering data_in and data_out_m req and resp bins	Checked by reference model

Table 5-13: Module Verification Plan (sheet 2 of 4).

Test case Label	Test case description	Functional coverage	Functionality check
MBINIT_HS_out_of order	The block will send train error entry request whenever it receives unexpected message even if its correct	Covered by covering data_in and data_out_m req and resp bins	Checked by reference model
MEM_PARAM	In param sub state the memory address and the memory data are correct	—	Checked by reference model
MBINIT_VALVREF	The corruption of test result bit will lead to sending train error entry request	Covered by data_in and data_out_m req and resp bins	Checked by reference model
TEST_RESP	TEST_RESP signal in all the states where the mainband rx is expected to start a test	Covered by bin for test_resp	Checked by reference model
MBINT_LANE_REVERSAL	Detect if there's a need for reversal in mainband if the number of faulty lanes is more than the half a train error entry req is sent	Covered by resp bins	Checked by reference model
Start_mb_send	Signal is asserted whenever a lfsr clear error response is received in any f TX_D_TO_C_point_test or RX_D_TO_Cpoint_test	Covered by start_mb bin	Checked by reference model
End_mb_wait	Before sending the result of the TX_D_TO_C_point_test The module waits for the assertion of the end_mb to make sure that the result is calculated	Covered by end_mb bin	Checked by reference model

Table 5-14: Module Verification Plan (sheet 3 of 4).

Test case Label	Test case description	Functional coverage	Functionality check
Apply_degrade	Depending on the result of the test of the faulty lanes number the width degrade encoding is sent to the mainband to take action on their lanes		Checked by reference model
MBTRAIN_HS	All the handshakes of the MBTRAIN substates work correctly	Covered by req bins and resps bins on data_in and data_out_m	Checked by reference model
MBTRAIN_HS_DEST	Errors in destination even if the parity is correct would be detected and a train err entry req would be send	———————	Checked by reference model
MBINIT_HS_DEST	Errors in destination even if the parity is correct would be detected and a train err entry req would be send	———————	Checked by reference model
MBTRAIN_HS_src	Errors in source even if the parity is correct would be detected and a train error entry req would be send	———————	Checked by reference model
MBINIT_HS_src	Errors in destination even if the parity is correct would be detected and a train err entry req would be send	———————	Checked by reference model
MBTRAIN__OUT_Of_order	Receiving messages of the same substate in uncorrect order will be detected and a train error entry request will be send	———————	Checked by reference model
MBINIT__OUT_Of_order	Receiving messages of the same substate in uncorrect order will be detected and a train error entry request will be send	———————	Checked by reference model

Table 5-15: Module Verification Plan(sheet 4 of 4).

Test case Label	Test case description	Functional coverage	Functionality check
PHYRETRAIN_HS	The handshake between the 2 dies is handled correctly	Covered by resp and req bins of data_in and data_out_m	Checked by reference model
Decoder_mode	Encoding is sent correctly depending on the current state of the ltsm to sbtx	_____	Checked by reference model
Pl_cfg_encoding	The pl_cfg_encofing send to the adapter layer depending on the current width is correct	Covered by pl_cfg bins	Checked by reference model

5.2.2.3.2 Test Sequences

Stimulus generation

The sequence uses a struct to represent data_in and randomize message fields where the stimulus is created to act as a module partner responding the module requests in another words "The stimulus emulates the behavior of a die by responding to handshake interactions and providing corresponding outputs,thereby enabling realistic protocol-level verification". The message formats are found in chapter 3 which where we used reactive stimulus and the handshakes are as shown below

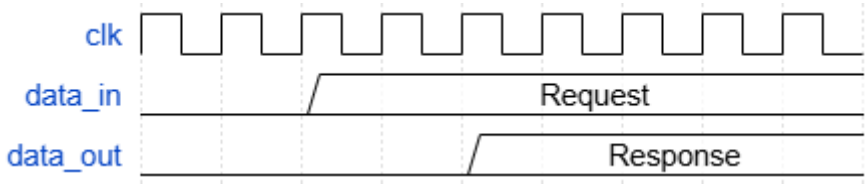


Figure 5-16: handshake representation

The sequences generated

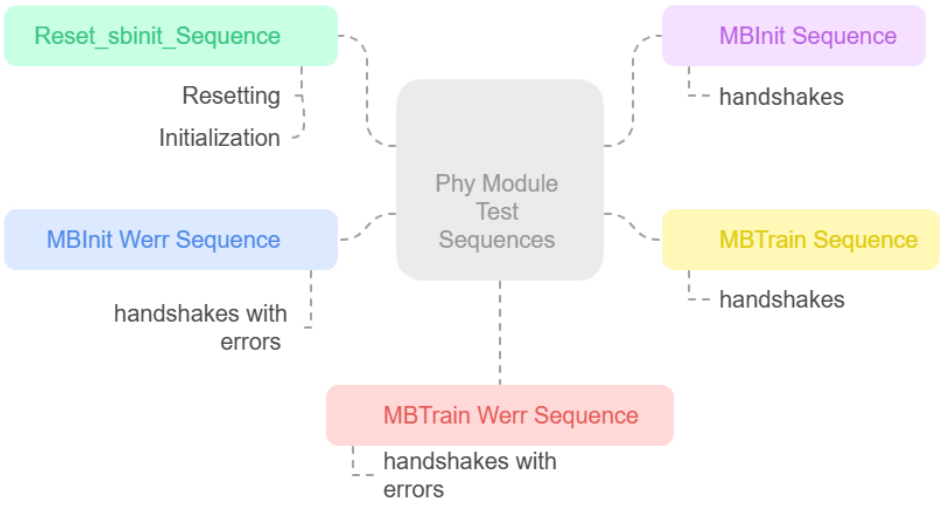


Figure 5-17: diagram illustrating PHy-sequences

The sequences shown in Figure 3 are crafted to thoroughly test key functions of the PHY module controller by simulating a wide range of operating scenarios. Each sequence focuses on a specific aspect of the controller's functionality, helping ensure that it performs reliably under both normal and error conditions.

- The Reset_sbinit_Sequence tests the controller's ability to handle reset and initialization. it confirms that the controller interacts with the pattern detector of the side band receiver and how it would trigger the LTSM to start the training process. it also handel the handshakes required to exit the phyretrain state in case of any error.
- The MBInit_Sequence and MBTrain_Sequence are aimed at validating the handshake processes that are critical during PHY initialization and training. The MBInit_Sequence checks the exchange of parameters and initial handshakes required to set up communication, while the MBTrain_Sequence focuses on the training phase handshakes — ensuring proper calibration and alignment of clock and data signals handshakes, as and it takes action depending on the result received.
- To test error handling, the MBInit_Werr_Sequence and MBTrain_Werr_Sequence introduce faults during handshake operations. As sending incorrect destination or source, sending out of order message response of the same substate,These sequences are important for verifying that the controller can detect and respond appropriately to errors, such as protocol violations or abnormal link behavior.

Together, these sequences provide comprehensive coverage by testing both expected and unexpected conditions. This systematic approach helps uncover potential design issues early and ensures that the PHY controller behaves correctly and reliably in real-world situations.

5.2.2.4 Results

5.2.2.4.1 Bugs

- **Bug0**

The first bug involved the module remaining in the same state for an extended period, which is a realistic scenario. In this situation, the module incorrectly sent the same request multiple times. However, the correct behavior should have been to send the request only once and then wait for the corresponding response before issuing any further requests. The following figure can be more clarifying

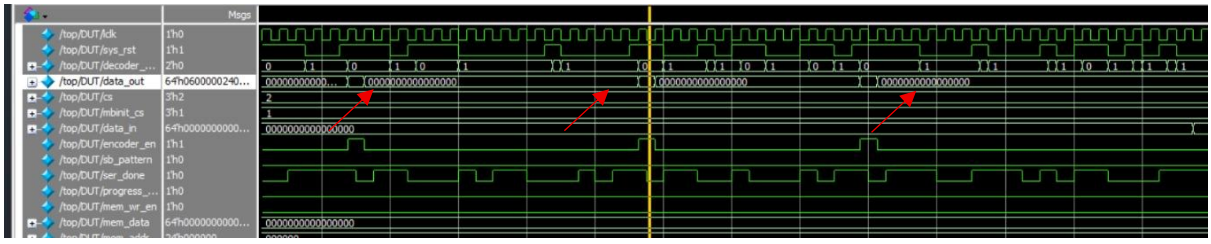


Figure 5-18: PHY-module bug0

- **Bug1**

The second bug occurred during the first request cycle. The design contains an internal state machine that indicates when it is the first cycle to send a request. However, although the state transitioned to indicate that the request should be sent, the request was not actually issued. Despite this, the internal state advanced to the next state, which waits for the response starting from cycle 1. This premature state transition without sending the request leads to a protocol mismatch and incorrect behavior.

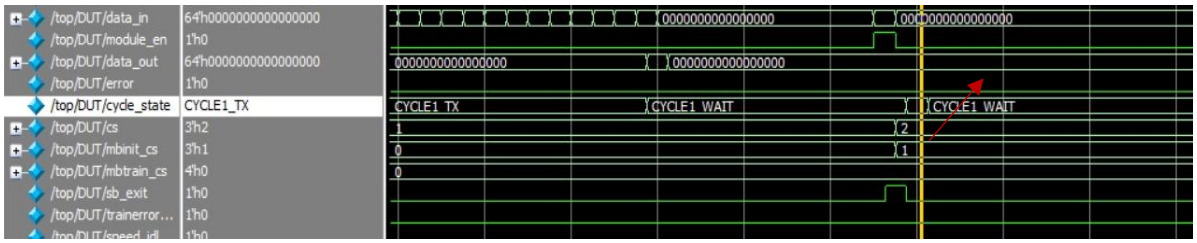


Figure 5-19: PHY-module bug 1

- **Bug2**

The message sent in the data_out is unexpected at this state where another request should be sent

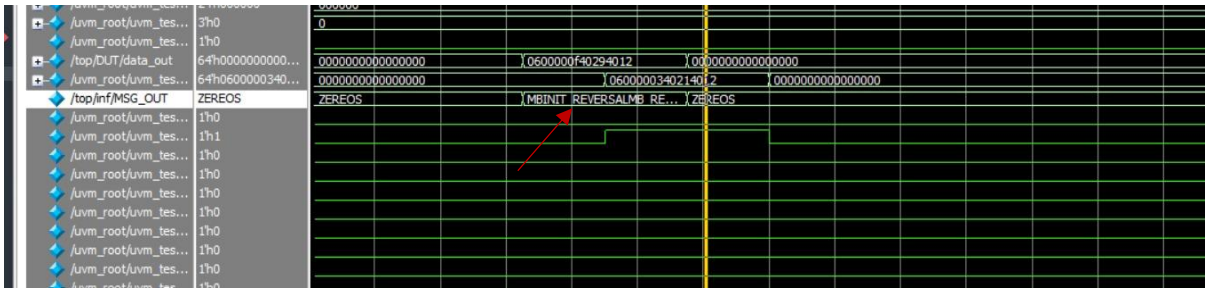


Figure 5-20: PHY-module bug1

- **Bug3**

The cycle error signal is asserted when the state machine enters a default or unexpected state, which serves as a debugging indicator. Upon asserting this signal, the design becomes unresponsive and hangs at cycle 8, effectively halting further operation. This behavior highlights a critical fault in the state machine's transition logic, requiring thorough investigation to identify and resolve the underlying cause of the deadlock.

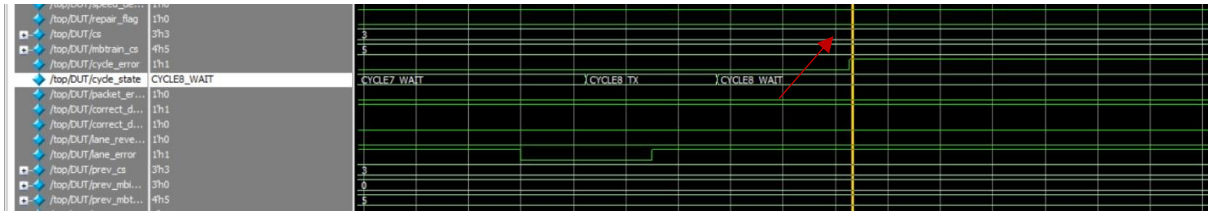


Figure 5-21: PHY-module bug-3

- **Bug4**

When the sys_rst signal is asserted, the path_sel signal is incorrectly set to zero. According to the design specification, path_sel should be set to one during both the reset and sbinit states in order to enable the pattern detector. Additionally, the decoder_mode signal is set to zero whenever sys_rst is asserted, whereas it is intended to be zero during normal operation and one specifically in the RESET state. This misbehavior results in improper configuration of the pattern detector and decoder during system reset.

5.2.2.4.2 Coverage results

Code coverage

The verification of a 3,000-line RTL implementation featuring a hierarchical state machine architecture has revealed a notably low condition coverage of 24.17% . This is particularly concerning given the design's complexity, which includes multiple nested sub-state machines governed by conditional logic that either:

- Advances the system to the next state
- Triggers a full system reset upon failure.

To address these gaps and ensure thorough validation of the control logic, complementary formal verification techniques are recommended. These provide exhaustive analysis by mathematically proving the correctness of all possible state transitions and condition outcomes, offering a more robust approach to verifying complex, safety-critical designs.

```
================================================================================
=== Design Unit: work.PHY_module
================================================================================
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                       728       567       161    77.88%
    Conditions                      91        22        69    24.17%
    Expressions                      3         3         0   100.00%
    Statements                    1783      1457       326    81.71%
    Toggles                        378       288        90    76.19%


Total Coverage By Design Unit (filtered view): 71.99%
```

Figure 5-22: PHY-module coverage

Functional coverage

The functional coverage results confirm that the design is performing as intended, with all training-related messages successfully triggered and observed throughout simulation. Each message type was exercised under the appropriate protocol conditions, achieving full coverage across all defined coverpoints and cross-coverage bins. This demonstrates that the verification environment effectively stimulates the design across the expected functional scenarios, particularly in terms of message sequencing and protocol compliance.

It is worth noting, however, that while functional coverage is complete, the code coverage metrics—especially condition and branch coverage—are relatively lower. This discrepancy is expected due to the use of dynamic, scenario-based verification rather than exhaustive static testing of all internal paths. In particular, hierarchical state machine transitions, which may require specific corner-case conditions to activate, are not always exercised through high-level stimulus alone. Despite this, the functional coverage provides strong assurance that the externally visible behaviors of the design have been thoroughly validated. Attached below the functional coverage snippets

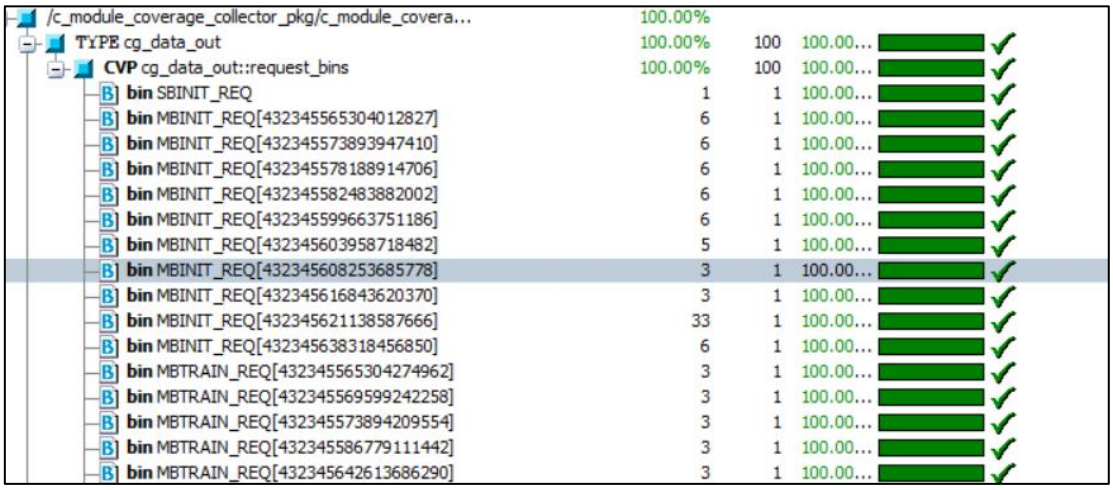


Figure 5-23: phy-module functional coverage 0

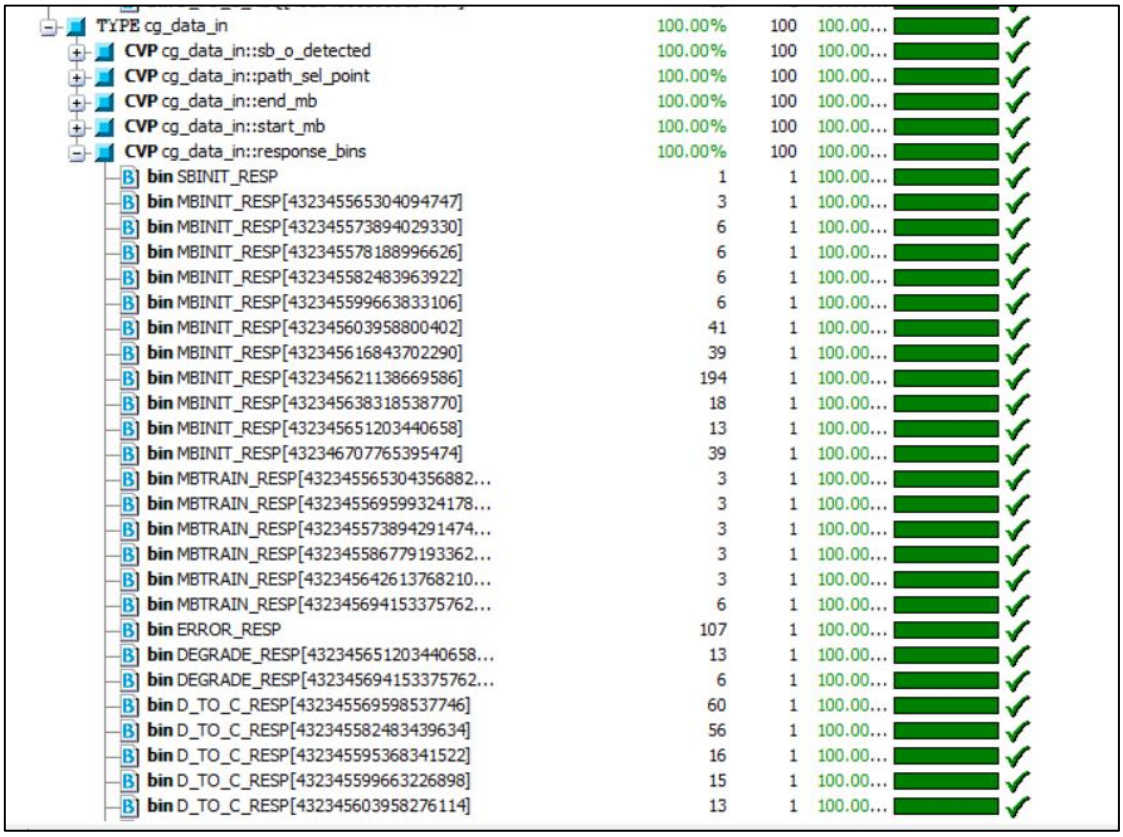


Figure 5-24: PHY-module functional coverage 1

5.2.3. Phy_Module_Partner:

5.2.3.1. Interface with other blocks

The Module Partner is responsible for processing handshake responses received from the remote die during training states. It also oversees the initiation of the retraining process upon detection of errors on the remote die by receiving the train error entry request and responding to it. As illustrated in the following diagram, the Module Partner interacts exclusively with the Module and Mainband Transceiver to obtain the outcomes of handshake-related tests, and to initiate or terminate the test sequences. All control logic within the controller operates synchronously with the LCLK clock domain.

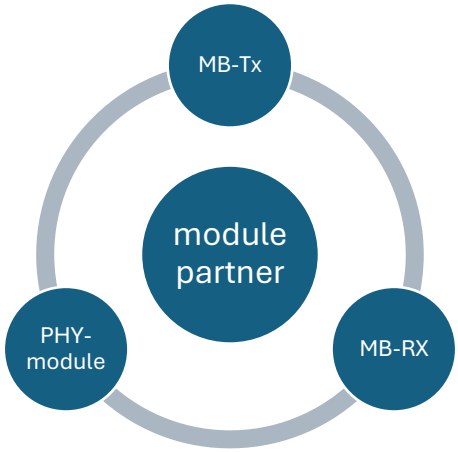


Figure 5-25: PHY-module-partner interaction with other block

While the design section details the interface signals and their respective clock domain assignments, the present verification study examines the implications of these interfaces on the testbench architecture and verification methodology. Coverage analysis is directed toward handshake sequences and timing-related corner cases to ensure rigorous validation of the interface's functional correctness and temporal integrity.

5.2.3.2. UVM Testbench Architecture

The architecture implemented in this environment follows the standard UVM testbench structure, consisting of a single agent that encapsulates both a driver and a monitor. This agent communicates with the Device Under Test (DUT), providing stimulus through the driver and collecting functional coverage and scoreboard data through the monitor. All DUT logic, including the driver interactions and monitored responses, operates synchronously with the lclk clock domain. This setup mirrors the architecture previously discussed for the module, ensuring consistency in design and verification methodology while maintaining modularity and ease of reuse across different verification scenarios. The simplicity of using a single agent also allows for focused testing of the interface under investigation without unnecessary architectural complexity.

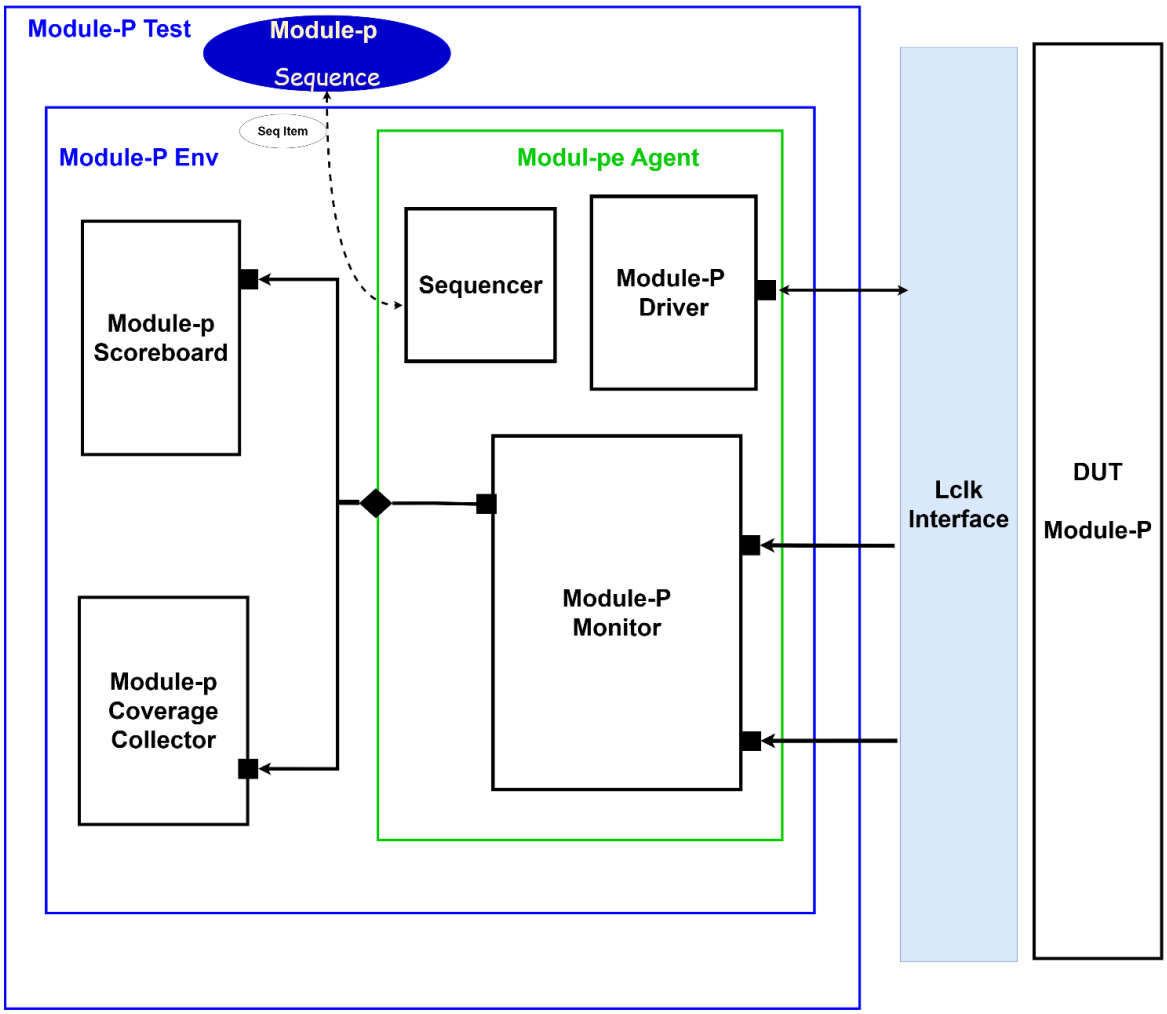


Figure 5-26: Module partner UVM Testbench Architecture

5.2.3.3. Test plan

5.2.3.3.1 Test cases

Table 5-16: Partner Verification Plan (sheet 1 of 3).

Test case Label	Test case description	Functional coverage	Functionality check
MBINIT_HS	the block is capable of completing the MBINIT handshaks for all of it's substates	Covered by covering data_in and data_out_mp req and resp bins	Checked by reference model
MBINIT_HS_out_of order	The block will send train error entry request whenever it receives unexpected message even if its correct	Covered by covering data_in and data_out_mp req and resp bins	Checked by reference model

Table 5-17: Partner Verification Plan (sheet 2 of 3).

Test case Label	Test case description	Functional coverage	Functionality check
MBINIT_VALVREF	**The corruption of test result bit will lead to sending train error entry request**	**Covered by data_in and data_out_m req and resp bins** **-**	**Checked by reference model**
TEST_RESP	**TEST_RESP signal in all the states where the mainband rx is expected to start a test**	**Covered by bin for test_resp**	**Checked by reference model**
Start_mb	Signal is asserted whenever a lfsr clear error response is received in any f TX_D_TO_C_point_test or RX_D_T_Cpoint_test	Covered by start_mb bin	Checked by reference model
End_mb_wait	**Before sending the result of the RX_D_TO_C_point_test** **The module waits for the assertion of the end_mb to make sure that the result is calculated**	**Covered by end_mb bin**	**Checked by reference model**
MBTRAIN_HS	**All the handshakes of the MBTRAIN substates work correctly**	**Covered by req bins and resps bins on data_in and data_out_m**	**Checked by reference model**
MBTRAIN_HS_DEST	**Errors in destination even if the parity is correct would be detected and a train err entry req would be send**	_____	**Checked by reference model**

Table 5-18: Partner Verification Plan (sheet 3 of 3).

Test case Label	Test case description	Functional coverage	Functionality check
MBINIT_HS_DEST	**Errors in destination even if the parity is correct would be detected and a train err entry req would be send**	_____	**Checked by reference model**
MBTRAIN_HS_src	**Errors in source even if the parity is correct would be detected and a train error entry req would be send**	_____	**Checked by reference model**
MBINIT_HS_src	**Errors in destination even if the parity is correct would be detected and a train err entry req would be send**	_____	**Checked by reference model**
PHYRETRAIN_HS	The handshake between the 2 dies is handled correctly	Covered by resp and req bins of data_in and data_out_m	Checked by reference model

5.2.3.3.2 Test sequences

The stimulus operates based on a configurable sequence item, which allows control over the distribution of received training error entry requests, invalid messages, and idle conditions (i.e., no messages). It also ensures the generation of appropriate outgoing requests in response to these scenarios.

The unified sequence is designed to comprehensively validate the PHY controller by emulating a wide range of initialization, training, and error-handling scenarios, all configured through a flexible sequence item. This item governs the distribution and type of requests sent—such as training handshakes, and error injection—allowing fine-grained control over the verification environment. The sequence also verifies the controller's ability to manage the required handshakes for safely exiting the phyretrain state in response to detected errors.

The same sequence further exercises the controller's handling of parameter exchange and training-phase handshakes, ensuring correct behavior during both MBInit and MBTrain phases. It verifies clock and data alignment procedures and ensures that appropriate actions are taken based on training outcomes.
To assess robustness, the sequence is also capable of injecting protocol-level errors—including incorrect source or destination IDs. These scenarios validate the controller's ability to detect protocol violations and initiate suitable error-handling or recovery procedures.
By encapsulating all test conditions—both standard and fault-induced—within a single configurable structure, this sequence provides thorough functional and temporal coverage of the PHY controller, supporting early identification of design flaws and ensuring reliable operation under real-world conditions.

5.2.3.4. Results

5.2.3.4.1. Bugs

- **Bug-0**

As illustrated in the following figure, the Partner sends a request with opcode '1B', which indicates a message containing data. However, the associated data is not transmitted afterward. This behavior results from the deassertion of the flag responsible for enabling data transmission prior to the actual data being sent.

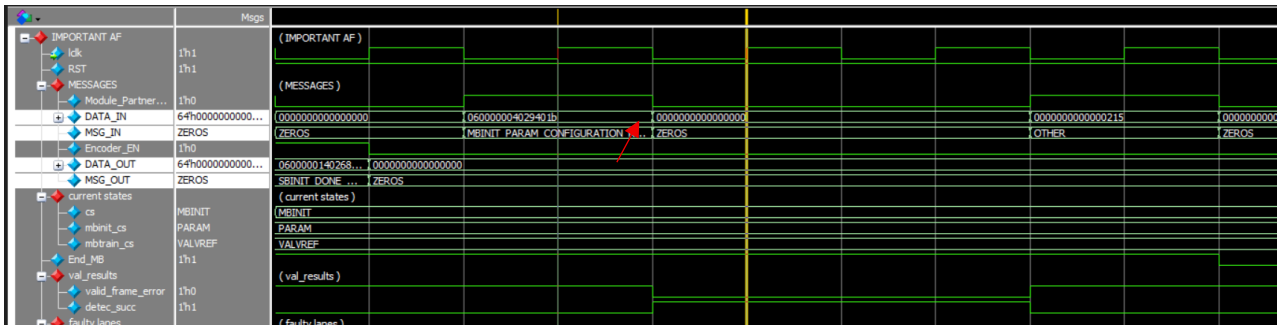


Figure 5-27: Module partner bug-0

- **Bug-1**

The implementation of the PHY retrain encoding resolution logic contained a critical sequential design flaw. The logic incorrectly utilized a registered storage approach for resolving the phyretrain encoding value, resulting in a one-cycle latency between value determination and transmission.

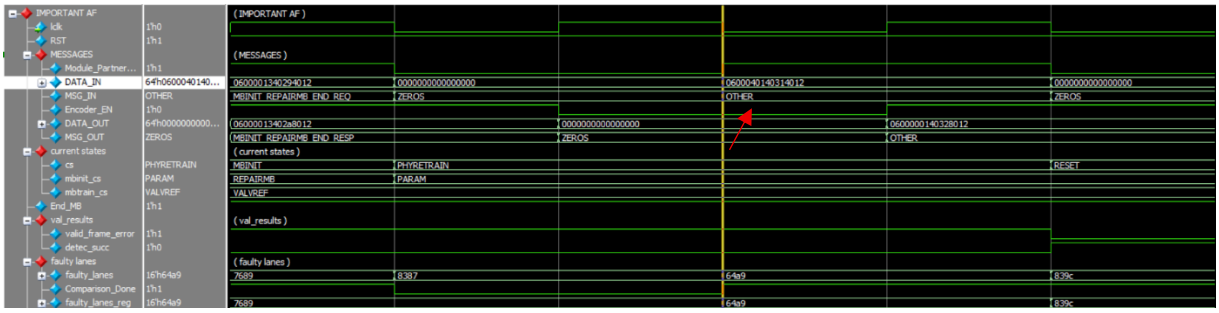


Figure 5-28: Module partner bug-1

- **Bug-2**

In the event of entering the reset state—whether following a training error or any soft reset condition—the signals data_out and encoder_enable were not properly cleared to zero. This improper reset behavior causes data to remain asserted on the bus, resulting in the unintended repeated transmission of stale data. Such persistence of non-zero values during reset leads to data corruption and violates protocol correctness by causing erroneous multiple transmissions.

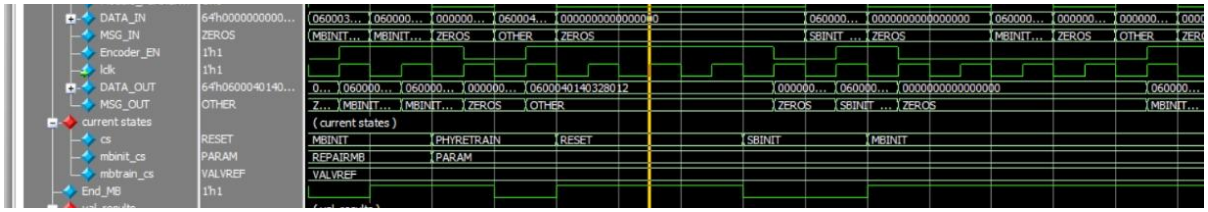


Figure 5-29: Module partner bug-2

- **Bug-3**

The speed idle request is currently transmitted as a level signal, whereas the Link Training State Machine (LTSM) is designed to detect and respond to this request as a pulse signal. This mismatch in signal type can lead to improper recognition of the idle request by the LTSSM, potentially causing timing and protocol synchronization issues.

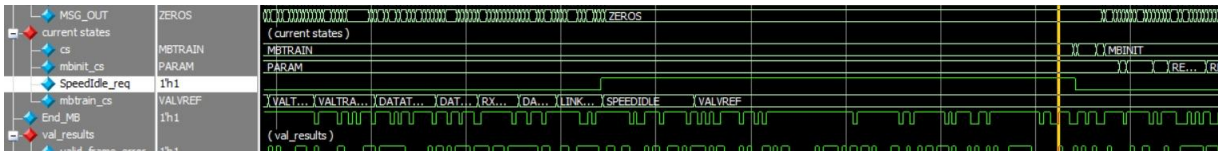


Figure 5-30: Module partner bug3

- **Bug-4**

The source and destination fields were not properly validated, and as a result, if the destination is set to D2D, the receiver fails to detect the message. Additionally, the MSG_INFO field is not being checked

5.2.3.4.2. Coverage

- **Code coverage**

The observed low code coverage— largely attributed to the presence of complex, multi-field conditional expressions, such as:
These types of compound conditions require that all subconditions evaluate to true simultaneously for the corresponding branch or code path to be taken. When such logic is evaluated in simulation, the probability of all these fields aligning correctly with random or even constrained stimulus is extremely low—especially when the conditions are highly specific and bit-accurate.

```
================================================================================
=== Design Unit: work.Module_Partner
================================================================================
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                       252       166        86    65.87%
    Conditions                     545       307       238    56.33%
    Expressions                      3         3         0   100.00%
    Statements                    1430       845       585    59.09%
    Toggles                        710       500       210    70.42%


Total Coverage By Design Unit (filtered view): 70.32%
```

Figure 5-31: Module partner code coverage

- **Functional coverage**

The functional coverage report demonstrates excellent performance, with all bins achieving 100.00% coverage . This indicates that the testbench is effectively stimulating the design to exercise all defined scenarios. Even if the code coverage percentage is not very high as we expected the high functional coverage ensure the functionality of the module being tested

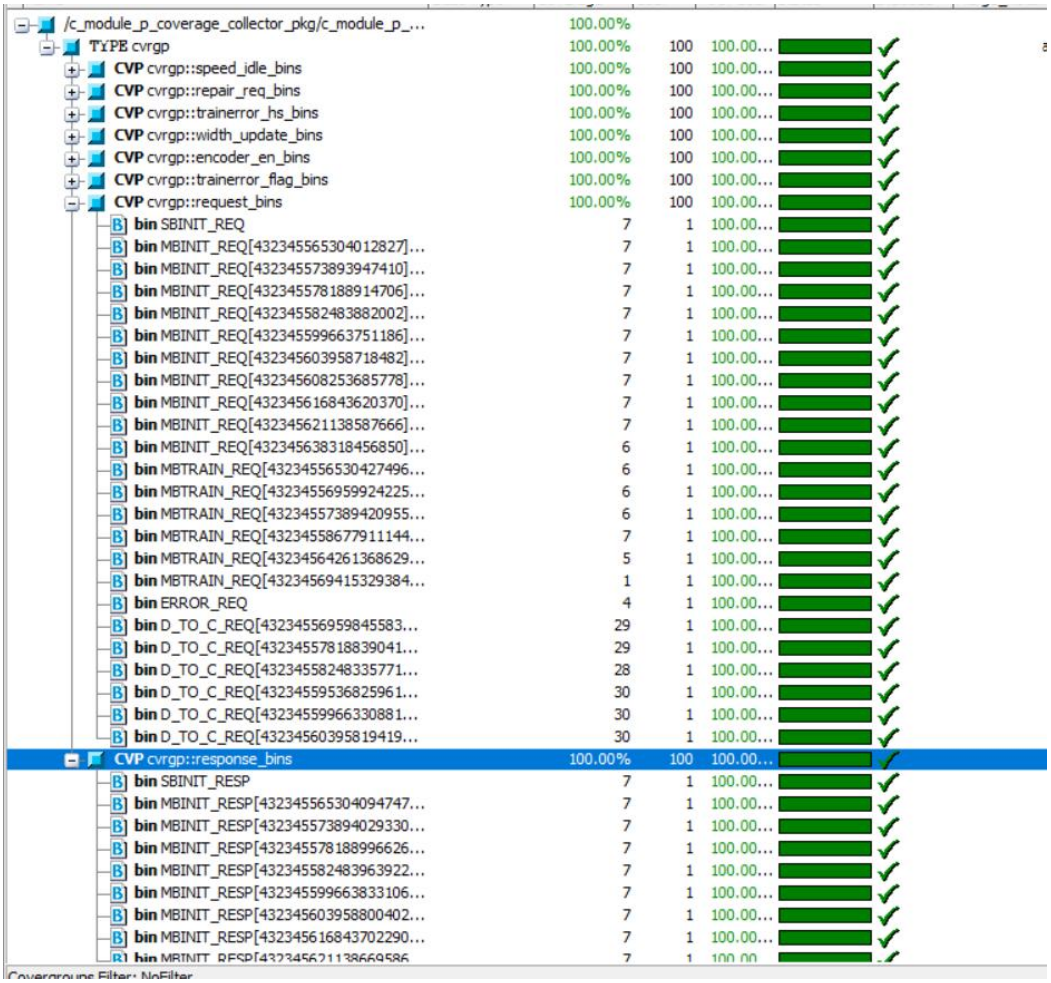


Figure 5-32: Module partner functional coverage

5.3. Sideband "SB":

5.3.1. Sideband Transmitter "SB_TX":

5.3.1.1 Interface Signals

The SB Tx block operates across two clock domains: **LCLK Domain** and **SBCLK Domain**, interfacing with **D2D**, **UIRB**, and the **PHY Controller**. As detailed in Chapter 3, certain SB Tx inputs/outputs follow strictly **UCIe specification formats**. To address this, the UVM testbench incorporates specialized techniques, including protocol-aware sequences and constrained randomization, enabling precise driving of the SB Tx block, exercising accurately the design under all expected legal operation and edge conditions.

Stimulus Generation:

- **UIRB Responses**: this imitates the UIRB space and its respWonse while handling read and write requests, the Virtual interface is accessed using virtual sequencers. Upon request initiation, the UIRB_Done flag is asserted within **1 to 5 LCLK cycles**, with the UIRB_Resp signal carrying the corresponding data in the case of read operations as shown in the example figure below.

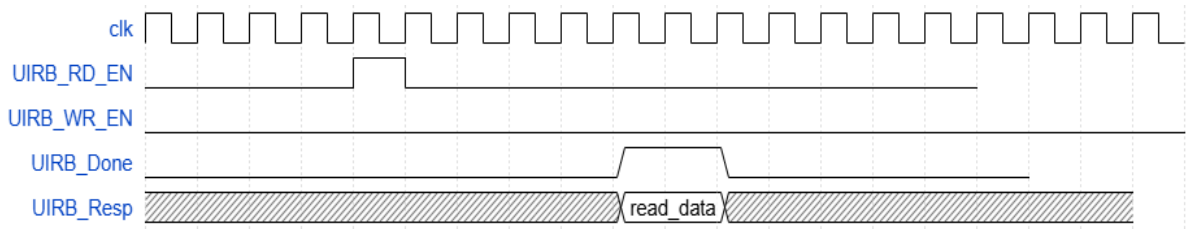


Figure 5-33: UIRB Stimulus Generation Example

- **Encoder Messaging**: Encoder messages are selectively generated to emulate realistic controller behavior, with or without data payloads. The Encoder_Mess_Valid signal is randomized using **weighted distribution constraints**, allowing fine control over message flow and traffic pacing and the phy_data_marker also matches the type of transaction (with or without data) just like this figure below as it should kept asserted for Messages with data from controller.

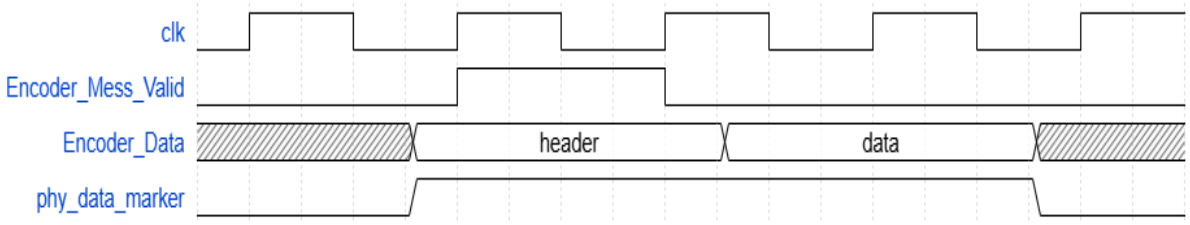


Figure 5-34: Encoder Message Stimulus Generation Example

- **RDI Transactions**: Complete RDI transaction is randomized at once, ensuring compliance with field formatting and parity rules. These transactions are then applied to the lp_cfg input at different phases depending on transaction type (with or without header). Additionally, **RDI credit counters** which control downstream data flow from D2D to PHY are actively monitored, and the probability of asserting lp_cfg_vld is increased as credits accumulate, supporting a more realistic flow control scenario. As seen in the figure down below where the credit release flag (lp_cfg_crd) increases the simulated downstream credits and also different transaction phases is transmitted through pl_cfg correspondingly according to its type.

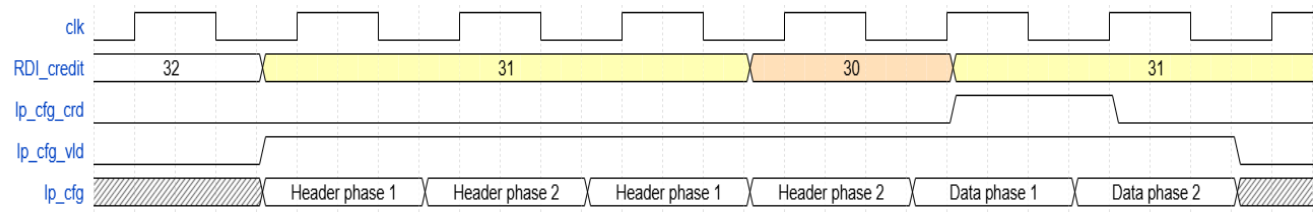


Figure 5-35: RDI Transitions Stimulus Generation Example

- Send Pattern Flag is asserted only at RESET_SBINIT state and it is kept high until pattern detected to simulate the expected correct operation of SBINIT and SB_OUT_OF_RST flow.

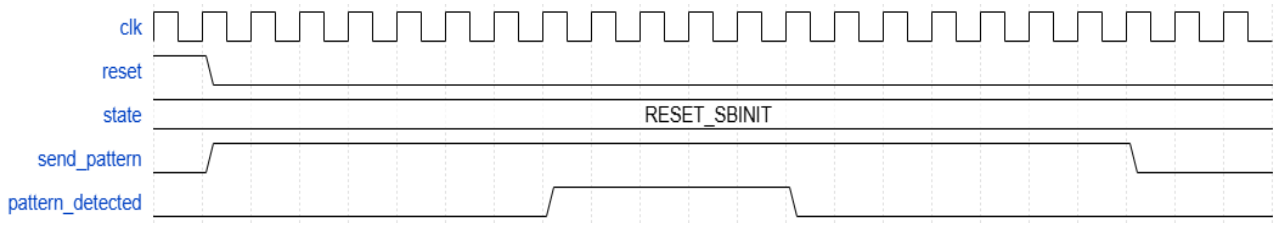


Figure 5-36: Control Flags Stimulus Generation Example

5.3.1.2 UVM Testbench Architecture

Before presenting the final UVM architecture, it is important to first identify the specific requirements of the testbench, in order to determine its essential components for accurate verification:

- The testbench must drive and observe both SB and LCLK domains, in order to react with the design during testing to mark a sequence endpoint using a flag or respond dynamically to a certain UIRB requests.

- The **sideband outputs**, whether clock or data, must be sampled on the **positive edge of the SB clock**. This introduces a potential **sampling race condition** that the testbench must account for using proper synchronization techniques.

From these requirements, we conclude that beyond the standard UVM testbench components the following adjustments are necessary:

- **Dual agent architecture**: Similar to previous multi-domain environments, two distinct agents will be used each dedicated to one clock domain (SB and LCLK) to independently drive and monitor their corresponding interfaces.

- **Clocking block usage in the SB agent monitor**: This enables explicit control over sampling times for SB Tx output data and clock, making sure that signal sampling occurs only after the design has completed state transitions and the outputs have stabilized, avoiding race conditions or incorrect captures.

With these adjustments in place, the resulting UVM testbench architecture is illustrated in the figure below.

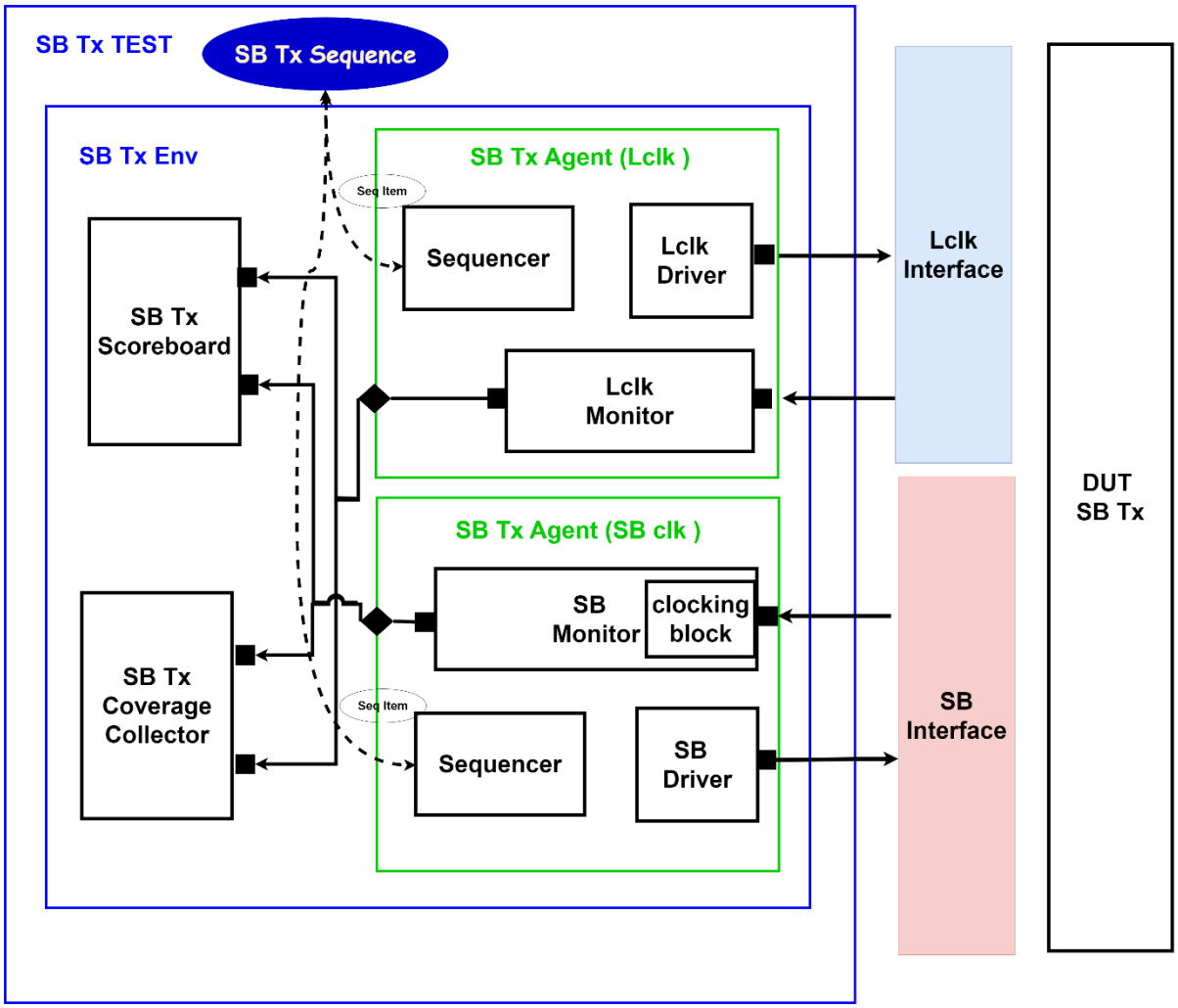


Figure 5-37: Final SB Tx UVM Architecture

5.3.1.3 Test Plan:

Given the SB Tx functionality, a comprehensive set of test cases has been planned to cover the majority of its operational scenarios. To effectively stimulate the SB Tx and validate these cases, a series of dedicated sequences has been developed. These sequences are crafted to generate precise and reactive stimulus, guiding the DUT through its various functional paths and ensuring thorough verification coverage.

5.3.1.3.1 Test Cases:

Table 5-19: SB Tx Verification Plan (sheet 1 of 5)

Test Case Label	Test Case Description	Functional Coverage	Functionality Check
RESET_CASE_SBTX	At low RST_n all outputs should be zero	Reached by Reset Sequence	Covered by a single bin to cover RST_n signal
SBINIT_RDI_READ1	If state = SBINIT for any local RAR read from the RDI it is discarded and its credit is returned	Reached by SBINIT_RDI_READ Sequence	Covered by cross coverage bin between lp_cfg_vld, local dst_id, state and opcodes
SBINIT_RDI_READ2	If state = SBINIT for any Remote RAR read from the RDI it is responded by Unsupported request completion and RDI credit return. This includes 2-phase, 4-phase transactions and a mixed sequences of consecutive transactions	Reached by SBINIT_RDI_READ Sequence	Covered by cross coverage bin between lp_cfg_vld transitional bins and state bins
SBINIT_RDI_READ3	If state = SBINIT for any Remote message only a local credit is returned	Reached by SBINIT_RDI_READ Sequence	Covered by cross coverage bin between lp_cfg_vld bin , opcode and state bins
SBINIT_ENC_READ1	If state = SBINIT and the pattern is not successfully completed, Any Encoder Message should be discarded	Reached by SBINIT_ENC_READ sequence	Covered by a simple cross coverage bin between state and enc_msg_vld just to make sure it happened once

Table 5-20: SB Tx Verification Plan (sheet 2 of 5)

Test Case Label	Test Case Description	Functional Coverage	Functionality Check
SBINIT_ENC_READ1	If state = SBINIT and the pattern is not successfully completed, Any Encoder Message should be discarded	Reached by SBINIT_ENC_READ sequence when ran before SBINIT_PATTERN_SUCC sequence	Covered by a simple cross coverage bin between state and enc_msg_vld just to make sure it happened once
SBINIT_ENC_READ2	If state = SBINIT and the pattern is successfully completed, Any Encoder Message should be read, re-decoded and eventually sent on the UCIe Link in the 64UI -32UI Format correctly	Reached by SBINIT_ENC_READ sequence when ran after SBINIT_PATTERN_SUCC sequence	Covered by a simple coverage bin between state and enc_msg_vld and tx_clk_sb to verify that the message is sent on the UCIe Link
SBINIT_RDI_ENC_READ1	This test case combines SBINIT_RDI_READ and SBINIT_ENC_READ simultaneously to make sure that the design can handle both cases together	Reached by SBINIT_RDI_ENC_READ Sequence	Covered by a cross coverage bin between these two cases coverage bins
SBINIT_PATTERN_FAIL1	While State = SBINIT , if the send_pattern flag is asserted the SBINIT pattern should be transmitted continuously alternating between on and off each 1ms until the 8ms timer fires and the controller de-asserts the send+pattern flag	Reached by SBINIT_PATTERN_FAIL Sequence	Covered by a single bin to cover lp_vfg_vld signal and a transition bin
SBINIT_PATTERN_FAIL2	While State = SBINIT , the rdi remote RAR and encoder messages behavior mentioned above should still hold in this case	Reached by SBINIT_PATTERN_FAIL Sequence	Covered by the same coverage bin mentioned above but crossed with send_pattern flag

Table 5-21: SB Tx Verification Plan (sheet 3 of 5)

Test Case Label	Test Case Description	Functional Coverage	Functionality Check
SBINIT_PATTERN_SUCC	While State = SBINIT , if the send_pattern flag is asserted the SBINIT pattern should be transmitted continuously alternating between on and off each 1ms until the pattern_detected flag is high then the design should transmit the pattern for 4 more iterations and then assert the pattern_done flag	Reached by SBINIT_PATTERN_SUCC Sequence	Covered by two coverage bins one for the pattern detected crossed with state and the other coverage bin is for pattern_done also crossed with state
SBINIT_OUT_OF_RESET	After Pattern is successfully detected the encoder message (SB_OUT_OF_RST) is read and sent on the UCIe link continuously until encoder_msg_vld is de-asserted	Reached by SBINIT_PATTERN_SUCC Sequence	Covered by a cross coverage between send_pattern and pattern_detected
SBINIT_FLOW	This Test Case Contains the test cases SBINIT_PATTERN_SUC, SBINIT_OUT_OF_RESET performed sequentially without reset in between to emulate the full successful SBINIT FLOW	Reached by SBINIT_PATTERN_SUCC_OUT_OF_RST Sequence	Requires no additional Coverage
ACTIVE_REMOTE_RDI1	In ACTIVE State the design should read the rdi phases when lp_cfg_vld is asserted and direct it to the UCIe link in the agreed upon format	Reached by ACTIVE_REMOTE_RDI Sequence	Covered by a cross coverage bin between lp_cfg_vld , state and txclk
ACTIVE_REMOTE_RDI_2	In ACTIVE State after reading the rdi phases and checking the parity bits if the transaction consumes credit, it should assert pl_cfg_crd once it is being transmitted	Reached by ACTIVE_REMOTE_RDI Sequence	Covered by a cross coverage bin between state, txclk and pl_cfg_crd

Table 5-22: SB Tx Verification Plan (sheet 4 of 5)

Test Case Label	Test Case Description	Functional Coverage	Functionality Check
ACTIVE_REMOTE_RDI3	Upon Reading from the RDI interface if the current transaction is found to be a Message it should be prioritized and sent first before the rest of saved transactions in the fifo	Reached by ACTIVE_REMOTE_RDI Sequence	Covered by a cross coverage bin between state,lp_cfg_vld and opcode of transaction
ACTIVE_LOCAL_RDI1	During ACTIVE STATE if the recieved RAR was local the design should send to UIRB a coressponding request correctly	Reached by ACTIVE_LOCAL_RDI Sequence	covered by coverage bins for UIRB_RD_EN and UIRB_WR_EN
ACTIVE_LOCAL_RDI2	Upon Waiting the UIRB response, once it arrives and UIRB done is asserted a completion message should be transmitted and comp_msg_vld must be asserted and RDI Credit must be returned as well	Reached by ACTIVE_LOCAL_RDI Sequence	covered by cross coverage bin between UIRB_RD_EN , UIRB_WR_EN, state and comp_msg_vld
ACTIVE_LOCAL_RDI3	Upon Reading from the RDI interface if the current transaction is found to be a Message it should be prioritized and served at first	Reached by ACTIVE_LOCAL_RDI Sequence	Covered by a cross coverage bin between state,lp_cfg_vld and opcode of transaction
ACTIVE_REMOTE_PHY	At Active state, if enc_msg_vld is asserted the encoder message is read (typically read before fifo1) and re-decoded then transmitted correctly	Reached by ACTIVE_REMOTE_PHY Sequence	Covered by a cross coverage bin between state,enc_msg_vld

Table 5-23: SB Tx Verification Plan (sheet 5 of 5)

Test Case Label	Test Case Description	Functional Coverage	Functionality Check
ACTIVE_REMOTE_PHY_RDI	This Test Case Represents the combination of reading from both rdi and encode simultaneously , where the design should correctly transmit transactions without any ordering errors or confusing the transactions	Reached by ACTIVE_REMOTE_PHY_RDI Sequence	covered by coverage bin of a cross coverage bin between state,lp_cfg_vld and enc_msg_vld which shows that this simultaneous operations occurred
ACTIVE_TRANSITION1	At Active state while transmitting SB transactions if the progress_stop flag is high, SB Tx should only send Controller messages and stop sending D2D RDI transactions to the remote partner	Reached by ACTIVE_TRANSITION Sequence	Covered by a coverage bin to ensure that the stop_progress flag is raised at least once through test
ACTIVE_CTRL_PARITY_ERROR	At Active state, if the rdi phases read from interface contains a false control parity error the design should raise the UIE flag in a pulse style	Reached by ACTIVE_CTRL_PARITY_ERROR Sequence	Requires only a coverage bin for UIE
ACTIVE_DATA_PARITY_ERROR	At Active state, if the rdi phases read from interface contains a false data parity error the design should raise the UIE flag in a pulse style	Reached by ACTIVE_DATA_PARITY_ERROR Sequence	Requires only a coverage bin for UIE

5.3.1.3.2 Test Sequences:

To Target the test cases listed the following sequences were developed to reach these cases with correct and legal stimulus. Each sequence from down below will **inherit** the reset sequence as a parent sequence and by default at the beginning of each sequence it will start the reset sequence by using **super** method unless told otherwise and also each one will be handed the virtual interface handler to sample the interface and respond dynamically as mentioned earlier in this sub chapter.

Test sequences are **categorized** according its **objective** and **state of operation** as following:

a) IDLE sequences
b) Pattern sequences
c) Active sequences
d) Error sequences

A) IDLE Sequences:

This group of sequences test the SB Tx while in reset state before it starts transmitting pattern where it should respond to RDI transactions in a specific way depending on its opcode and destination. This group contains the following sequences:

1. **SBTX_RESET**: activates reset to ensure the SB Tx remains in the reset or idle state for one transaction to establish a known initial state.

2. **SBINIT_RDI_READ**: Emulates the SB Tx in RESET, responding to remote SB RAR as UR and Local access requests it returns credit to D2D. It does not send an SBINIT pattern or receive encoder messages.

3. **SBINIT_RDI_ENC:** Verifies the SB Tx will not transmit any encoder messages and will ignore it until the pattern is sent and received asserting the pattern_detected flag.

4. **SBINIT_RDI_ENC_READ:** Combines the two previous sequences properties to test if the SB Tx can handle both cases simultaneously as it should.

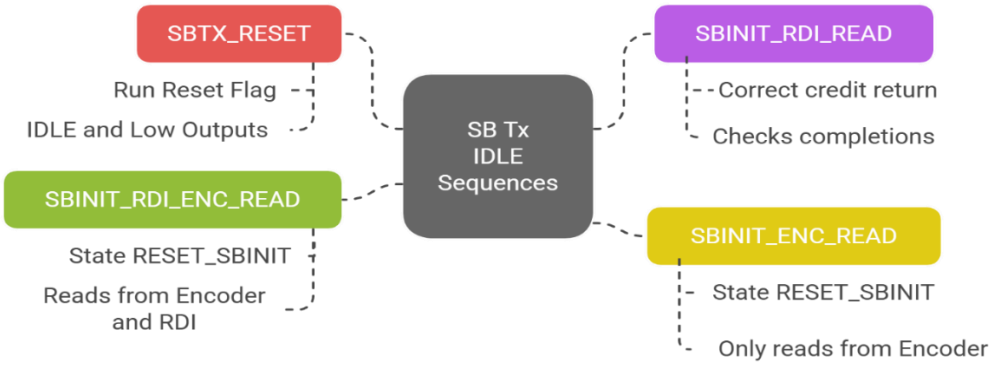


Figure 5-38: SB Tx IDLE Sequences

B) Pattern Sequences:

This group of sequences test the SB Tx while in SBINIT state and starts sending SBINIT pattern and the SB_OUT_OF_RST message next in case of success with all its scenarios.

5. **SBINIT_PATTERN_FAIL**: Practices the case where the SBINIT pattern is not detected by SB Rx so the pattern_detected flag stays low until timeout and state transitions to TRAINERROR.

6. **SBINIT_PATTERN_SUCC**: Emulates the successful transmission of SBINIT pattern where the pattern_detected flag is asserted after asserting send_pattern by a random number of cycles.

7. **SBINIT_OUT_OF_RST:** Simulates the SBINIT out-of-reset message transmission after the pattern detection.

8. **SBINIT_PATTERN_SUCC_OUT_OF_RST:** Combines the two previous sequences and runs them sequentially to emulate the full SBINIT flow.

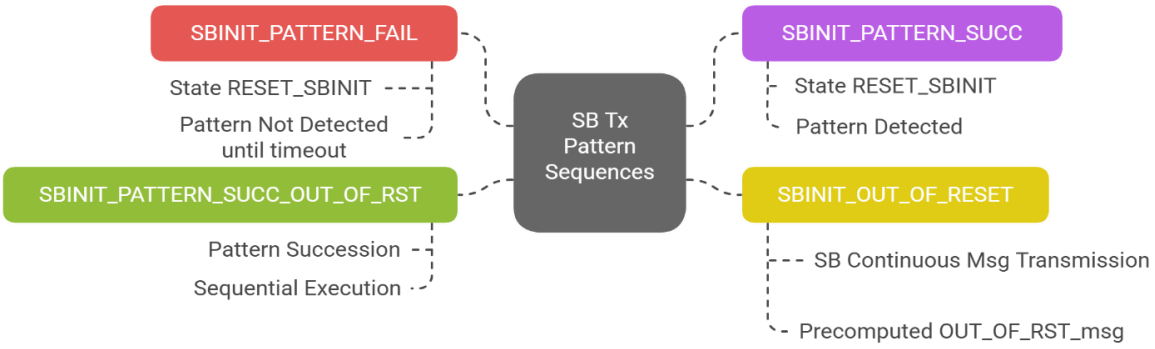


Figure 5-39: SB Tx Pattern Sequences

C) **Active Sequences:**

This group of sequences test the active operation of the SB Tx where it should read RDI transactions and controller messages and handles them accurately as mentioned in the block design chapter.

9. **ACTIVE_REMOTE_RDI**: SB Tx only reads from D2D RDI any remote transactions, where it transmits it with any insertion after checking data integrity using parity bits in the header.

10. **ACTIVE_LOCAL_RDI**: Tests SB Tx handling local RAR where it should send a corresponding request to UIRB and sends completion upon receiving the response.

11. **ACTIVE_REMOTE_PHY:** Simulates the SB Tx receiving transactions only from the controller and their transmission.

12. **ACTIVE_REMOTE_PYH_RDI:** Combines all the transactions processing at once where the transactions on RDI could be of any destination and opcode, simultaneously reading from the controller to test message prioritization.

13. **ACTIVE_TRANSITION:** Emulates the Transition from ACTIVE to TRAINERROR as the stop_progress flag is asserted so the transmissions should stop until it de-asserts.

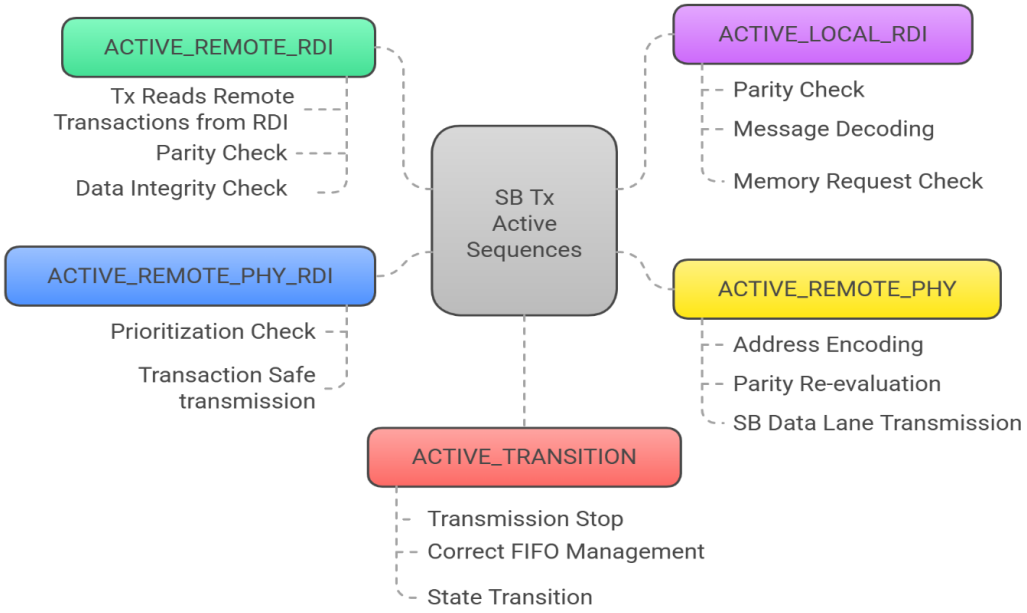


Figure 5-40: SB Tx Active Sequences

D) Error Sequences:

This group of sequences induces parity errors in RDI transactions whether it is control or data parity errors the SB Tx should detect those errors and asserts the UIE.

14. ACTIVE_CTRL_PARITY_ERROR: Induces control parity errors in RDI transactions.

15. ACTIVE_DATA_PARITY_ERROR: Induces data parity errors in RDI transactions.

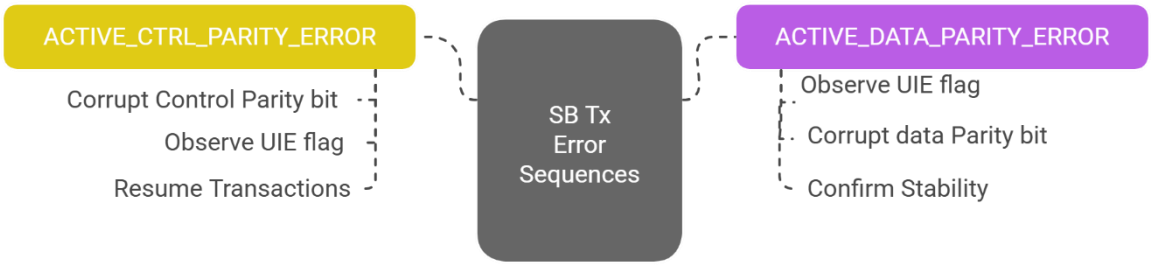


Figure 5-41: SB Tx Error Sequences

5.3.1.4 Results:

5.3.1.4.1 Bugs Found & Solved:

Bug (1): During SBINIT_RDI_READ sequence the 4-phase transaction is handled as two 2-phase transactions where design sends two UR completions and releases two credit units for it, which on the long run would cause Overflowing the D2D layer with unnecessary completions and also as a result of double credit release the D2D will also overflow the PHY Layer thinking it had more credits.

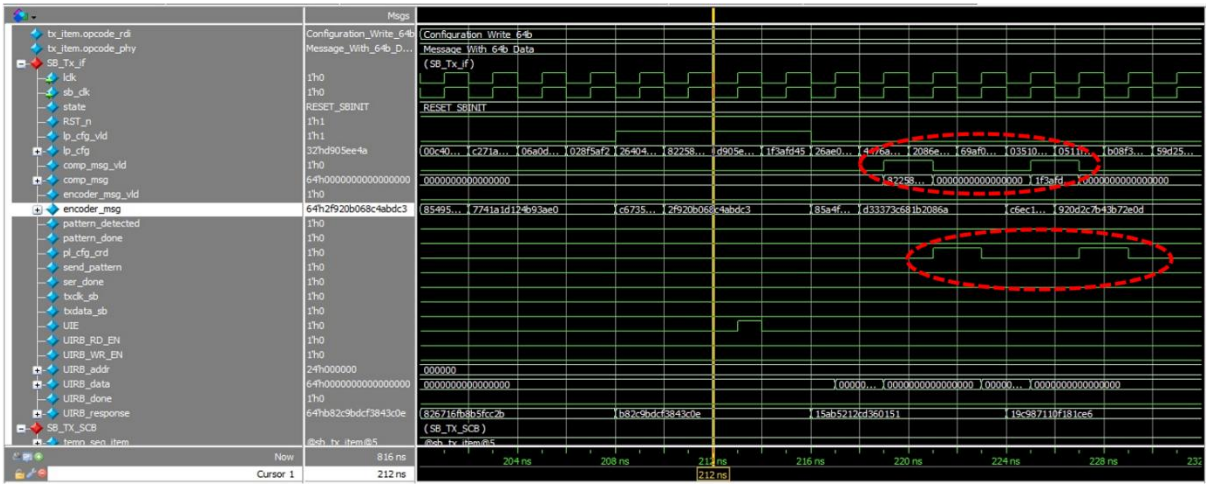


Figure 5-42: Bug (1) in SB Tx Verification

Bug (2): During SBINIT_PATTERN_SUCC sequence while transmitting SBINIT pattern (101010…) the data is transmitted earlier than clock by one clock cycle this will cause complete failure at remote die receiver to detect the pattern which will kill any chance of link establishment.

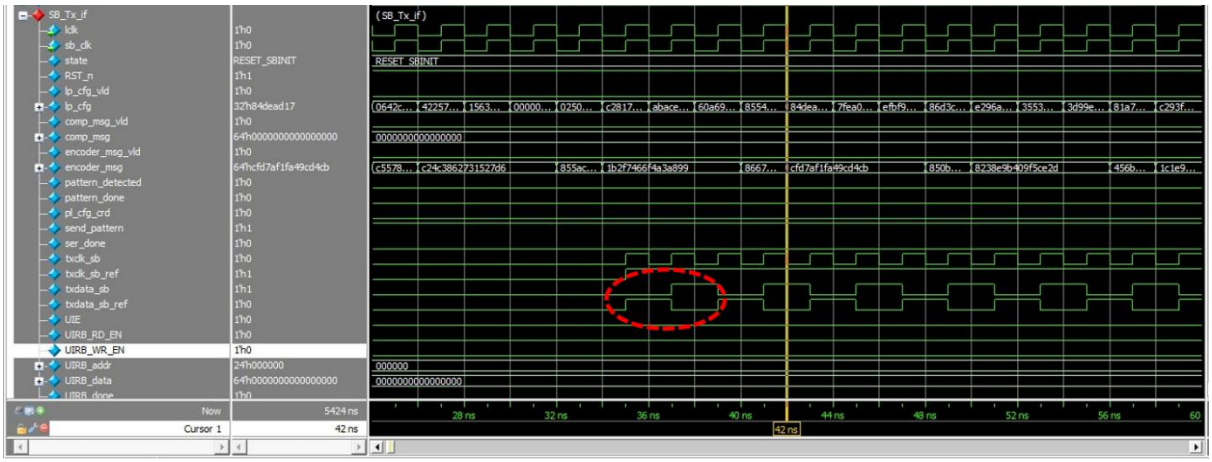


Figure 5-43: Bug (2) in SB Tx Verification

Bug (3): During SBINIT_PATTERN_SUCC sequence while transmitting SBINIT pattern the pattern was found to be 32 UI Active not 64 which is wrong in concept as this will lead to the same problem as Bug (2).

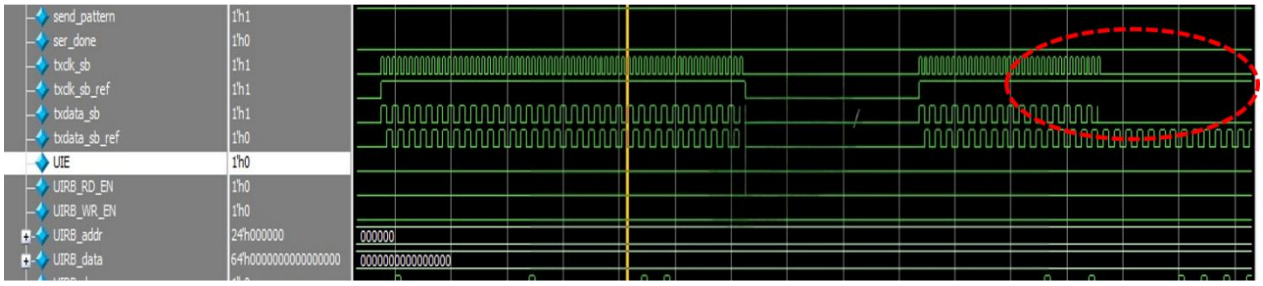


Figure 5-44:Bug (3) in SB Tx Verification

Bug (4): Wrong Completions was sent for a message transaction during SBINIT_RDI_READ sequence.

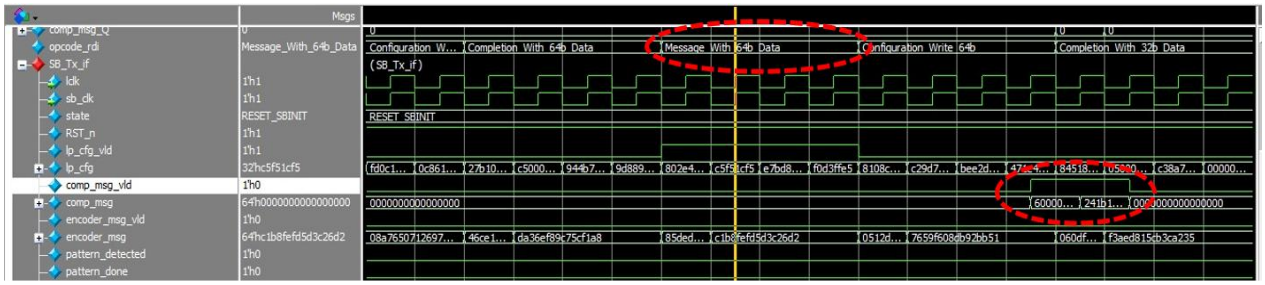


Figure 5-45: Bug (4) in SB Tx Verification

Bug (5): Completion message valid is asserted for only half cycle risking sampling faults where completion will not make it to the D2D and a request timeout will occur.

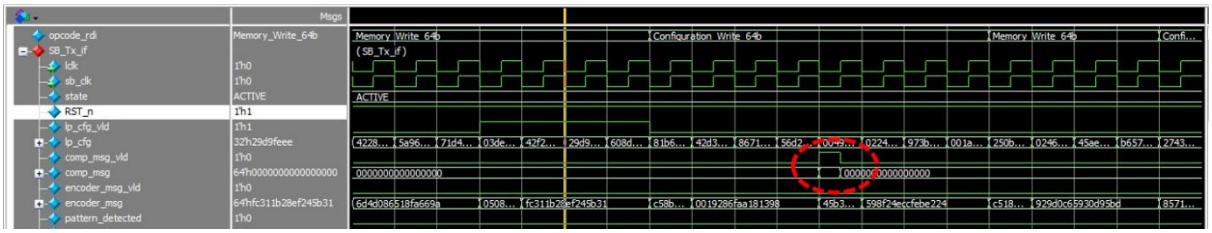


Figure 5-46: Bug (5) in SB Tx Verification

Bug (6): While in ACTIVE the data output on SB lane found to be faulty after debugging it was found that Transactions were written twice in the FIFO causing replication of transactions consuming E2E credits it doesn't have.

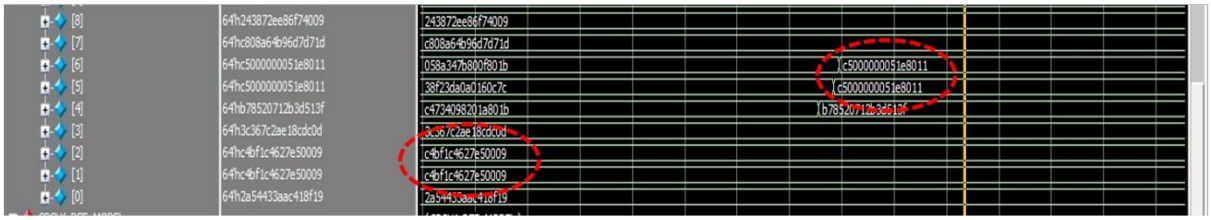


Figure 5-47: Bug (6) in SB Tx Verification

Bug (7): In Case of RDI error the SB Tx wrongly activates the clock latch which causes random sampling with unknown results at SB Rx at remote die. Also, there was no credit return which will eventually cause the downstream traffic to die as the credits will run out.

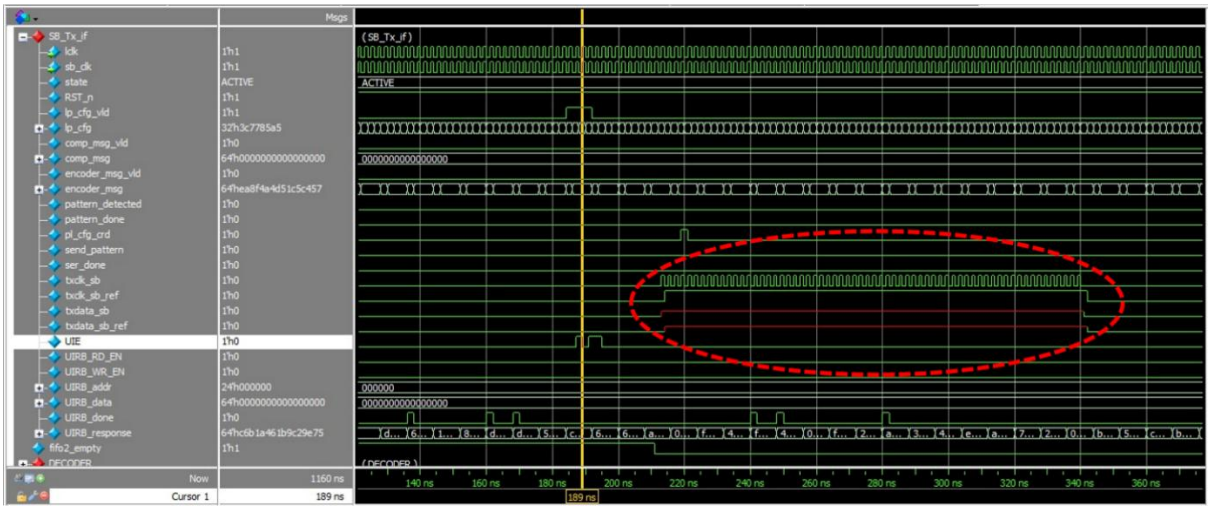


Figure 5-48: Bug (7) in SB Tx Verification

5.3.1.4.2 Coverage Achieved:

- **Code Coverage:**

This coverage report shows that the **SB Tx** design has been thoroughly verified. FSM state coverage is complete, and most FSM transitions were exercised, indicating strong state machine validation. High branch, expression, and statement coverage confirms wide functional exploration. Condition coverage is slightly lower due to design **safeguards and default cases** that are rarely triggered under valid operation. These are intentional checks that protect against illegal states and don't affect normal functionality. Overall, the environment effectively covers nearly all practical behaviors of the design.

```
================================================================================
=== Design Unit: work.SB_TX_Top
================================================================================
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                       222       208        14    93.69%
    Conditions                      19        14         5    73.68%
    Expressions                     45        38         7    84.44%
    FSM States                      12        12         0   100.00%
    FSM Transitions                 22        19         3    86.36%
    Statements                     312       306         6    98.07%


Total Coverage By Design Unit (filtered view): 89.37%
```

Figure 5-49: SB Tx Code Coverage

- **Functional Coverage:**

The following functional coverage report shows an excellent overall result with approximately 99% coverage achieved. All key coverpoints such as **states**, **controller message opcodes**, **RDI opcodes**, **progress_stop** and **send_pattern** and all state transitions have been fully hit. The only uncovered bin lies within the **Opcodes_PHY** cross coverage, which is due to an unsupported opcode combination that is not expected to occur under normal or valid operation. This bin was intentionally left unexercised as the design does not support it, and generating stimulus for it would not reflect a realistic usage scenario.

Name	Coverage	Goal	% of Goal	Status	Included
/sb_tx_coverage/sb_tx_coverage	99.07%				
TYPE tx_cg	99.07%	100	99.07%		
INST \/sb_tx_coverage::sb_tx_coverage...	99.07%	100	99.07%		
CVP tx_cg::{#cov_item_Lclk.lp_cfg_vld__...	100.00%	100	100.00...		
CVP tx_cg::States	100.00%	100	100.00...		
CVP tx_cg::serial_done	100.00%	100	100.00...		
CVP tx_cg::serial_data	100.00%	100	100.00...		
CVP tx_cg::reset	100.00%	100	100.00...		
CVP tx_cg::reg_acc_req	100.00%	100	100.00...		
CVP tx_cg::rdi_read	100.00%	100	100.00...		
CVP tx_cg::Prog_stop_cp	100.00%	100	100.00...		
CVP tx_cg::Phy_send_cp	100.00%	100	100.00...		
CVP tx_cg::Phy_data_cp	100.00%	100	100.00...		
CVP tx_cg::pattern_done	100.00%	100	100.00...		
CVP tx_cg::Opcodes_rdi	100.00%	100	100.00...		
CVP tx_cg::Opcodes_phy	100.00%	100	100.00...		
CVP tx_cg::init_pattern	100.00%	100	100.00...		
CVP tx_cg::error	100.00%	100	100.00...		
CVP tx_cg::encoder_read	100.00%	100	100.00...		
CVP tx_cg::dest	100.00%	100	100.00...		
CVP tx_cg::completion	100.00%	100	100.00...		
CVP tx_cg::comp_msg	100.00%	100	100.00...		
CROSS tx_cg::SBINIT_PAT	100.00%	100	100.00...		
CROSS tx_cg::Sb_frame_flag	100.00%	100	100.00...		
CROSS tx_cg::rdi_and_encoder_reads	100.00%	100	100.00...		
CROSS tx_cg::RAR_process	100.00%	100	100.00...		
CROSS tx_cg::Parity_bits	100.00%	100	100.00...		
CROSS tx_cg::Opcodes_RDI	100.00%	100	100.00...		
CROSS tx_cg::Opcodes_PHY	75.00%	100	75.00%		
CROSS tx_cg::Destination	100.00%	100	100.00...		

Figure 5-50: SB Tx Functional Coverage

5.3.2. Sideband Receiver “SB_RX”:

5.3.2.1 Sideband Receiver interaction with blocks

The Sideband Receiver is responsible for handling all handshake messages associated with both the PHY layer and the Adapter layer, in addition to Register Access Requests (RAR) destined for the protocol layer. This broad scope of functionality makes it one of the more complex components within the system.

The receiver operates on its local sideband clock (sb_clk) while receiving messages that originate from the remote die's sideband clock domain. Where the potential phase or frequency differences between these asynchronous clock domains is handled at the front end . Once synchronization is achieved, the messages are forwarded to the controller and upper protocol layers, which operate on the primary logic clock (LCLK). This cross-domain coordination is essential for maintaining data integrity and ensuring reliable communication across dies.

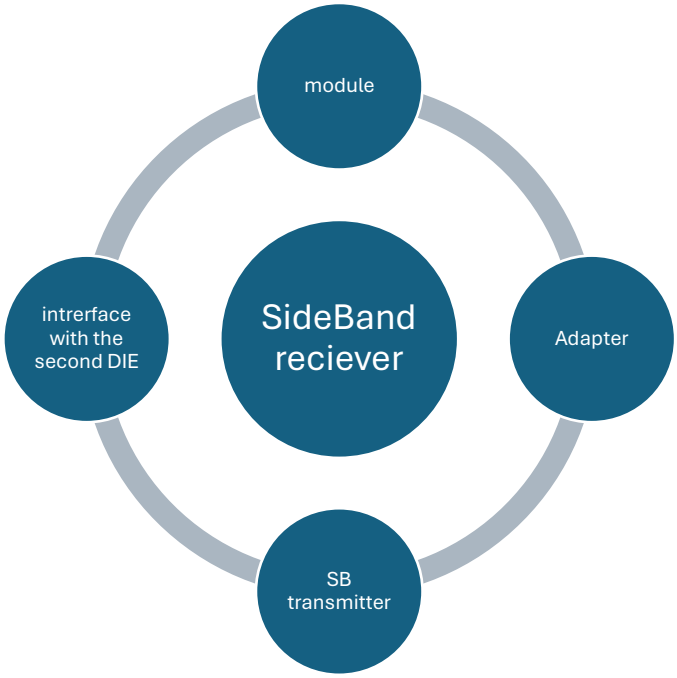


Figure 5-51: Sideband Receiver interaction with other block

5.3.2.2 UVM Testbench Architecture

Given that the design spans two asynchronous clock domains, several architectural strategies were explored to effectively manage clock domain crossing (CDC) within the verification environment. The first approach involved connecting two sequencers to a single driver, where each sequencer represented a different domain. While this allowed centralized control, it introduced complexity in driver logic and synchronization.

The second approach utilized a single sequencer connected to two separate drivers, each dedicated to a specific clock domain. These drivers communicated via a shared TLM port to coordinate data

exchange. Although this configuration improved modularity, it proved challenging to manage timing accuracy and data consistency across domains. The figure below clarify it more
A third approach was based on the use of a virtual sequencer, which coordinated multiple domain-specific sequencers and drivers under a unified control layer., this solution was not ultimately adopted, as there's no coordination required here berween the two agents

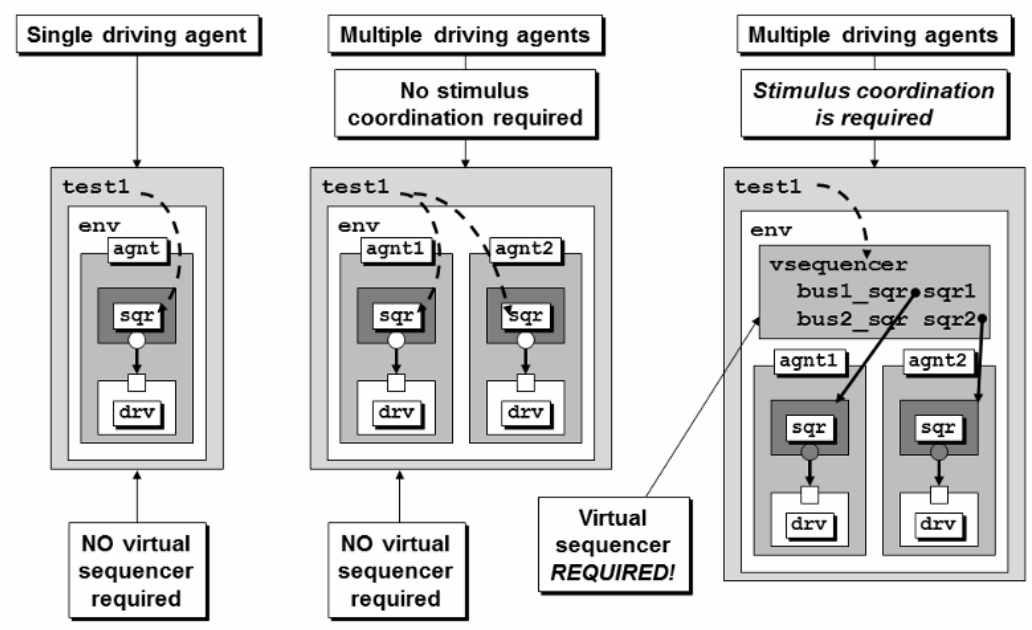


Figure 5-52: architectures with and without virtual sequence

Instead, the final and adopted solution implements dedicated agents for each clock domain, each encapsulating its own sequencer, driver, and monitor. Specifically, the design is partitioned into two verification interfaces: one aligned with the LCLK domain, responsible for driving and monitoring controller-side operations, and another aligned with the SB_CLK domain, which handles the reception of sideband messages. This approach ensures that stimulus generation and data collection are tightly synchronized with their respective clock domains, thereby enabling accurate and reliable verification of multi-clock interactions without the complexity of shared control layers. In addition to we used reactive stimulus which is explained in the 'LTSM' section for handling the clock gating handshake in stimulus. Attached below the final architecture implemented

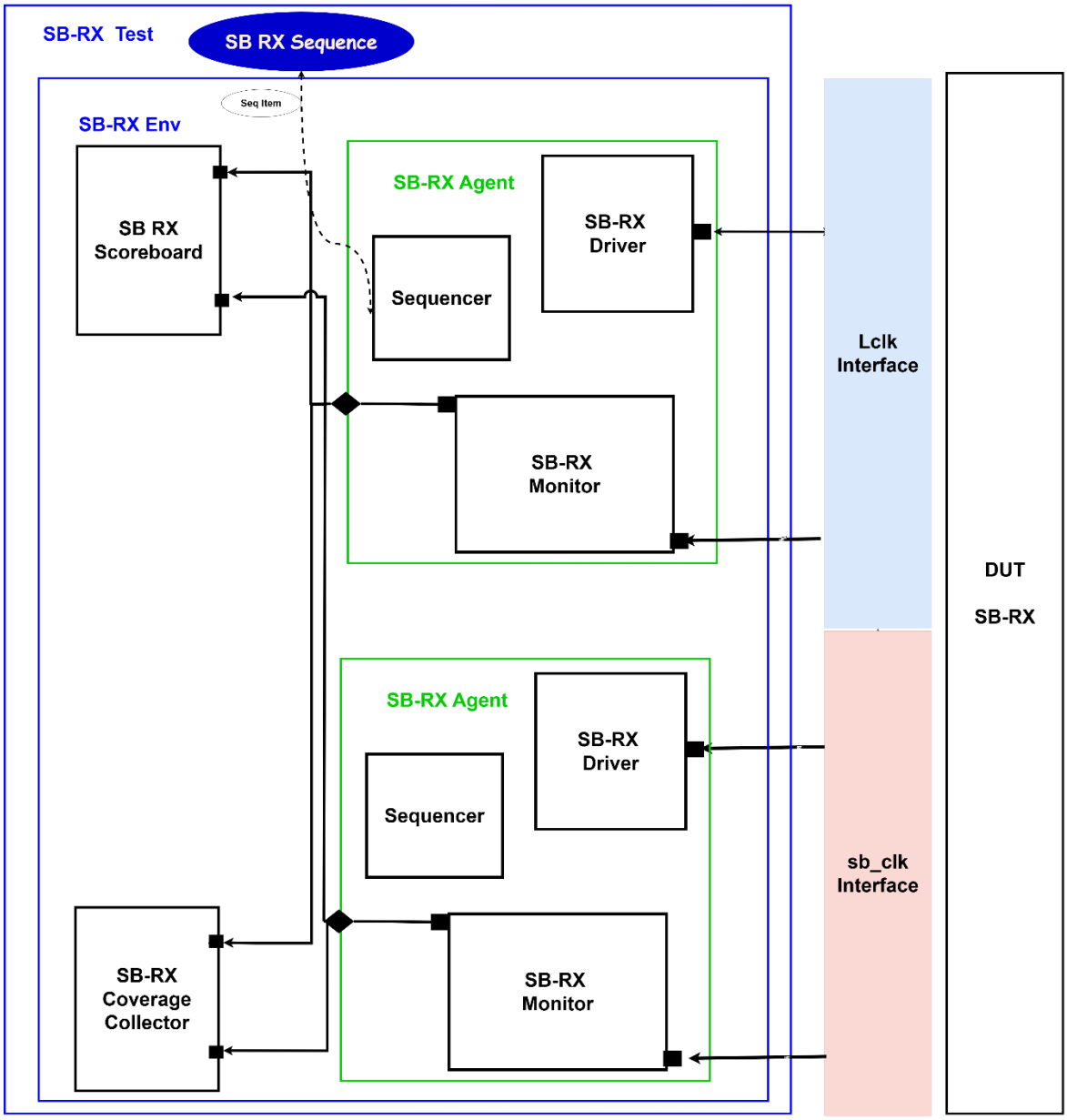


Figure 5-53: sbrx-uvm environment

5.3.2.3 Test plan

5.3.2.3.1 Test Cases

Table 5-24: SB Rx Verification Plan (sheet 1 of 3)

Test Case Label	Test Case Description	Functional Coverage	Functionality Check
sbinit_pattern consecutive	a pattern detcted flag is raised after 2 *64 alternating ui	__	Checked by golden model
sbinit_pattern non_consecutive	a pattern detcted flag is raised after 2 *64 alternating ui	___	Checked by golden model
sbinit_out_of_reset	after receiving sbinit_out_of_reset message a flag is asserted	__	Checked by golden model
Sbinit_out_of_reset _parity	receiving sbinit_out_of_reset message with parity error flag is not assertrd asserted	__	Checked by golden model
module related msgs	check that module__en is asserted and dest=00 whenever it receives a handshake request in training states	Covered by cp_module_en bin And Cross coverage bins msg_no_data_ modulee msg_data_mod ule	Checked by Golden model
Module_partner _related_mssgs	check that module_partner_en is asserted and dest=00 whenever it receives a handshake response in training states	Covered by cp_modulea_pa rtner_en bin And Cross coverage bins msg_no_data_ modulee msg_data_mod ule	_
RDI_related_ms sgs	check that controller en is asserted and dest 00 whenever it receives RDI message	Covered by cp_controller_e n bin And cross coverage bins msg_no_data_c ontroller	Checked by golden model

Table 5-25: SB Rx Verification Plan (sheet 2 of 3)

Test Case Label	Test Case Description	Functional Coverage	Functionality check
completions_mssgs	Check that the destination is equal to 11 whenever a completion is received from the remote die or received locally	—	Checked by golden model
module related msgs_w_parity_err	No data is out and the receiver assert the error signal to the module to send a train error entry request	___	
Module_partner_related_mssgs w_parity_err	No data is out and the receiver assert the error signal to the module to send a train error entry request	___	Checked by Golden model
RDI_related_mssgs w_parity_err	No data is out and the receiver assert the error signal to the module to send a train error entry request		Checked by golden model
mismatched_opcode_and_message code	both valid message code and opcode but message is meaningless error flag is asserted	____	Checked by golden model
Reserved_message_codes	If the die received a reserved message code error flag is asserted	___	Checked by golden model
Reserved_opcode	If the die received a reserved opcode error flag is asserted	___	Checked by golden model
Vendor_defined_mssgs	If a vendor defined messages is received the decoder will skip it and won't assert an error	___	Checked by golden model
priority testing_1	if a completion is recieved from TX and message priority goes to message	____	Checked by golden model

Table 5-26: SB Rx Verification Plan (sheet 3 of 3)

Test Case Label	Test Case Description	Functional Coverage	Functionality Check
Priority_testing _2	if a completion is recieved from TX and RAR priority goes to RAR	——	Checked by golden model
Available credit_1	If available credit is equal to one the packet_handler reads data from the full fifo	——	Checked by golden model
Available credit_2	If no available credit the packet handler mustn't read from any fifo	——	Checked by golden model
Lp_clk_ack_hs_ comp	The handshake must be done before the pl_data is changed to it's new value	——	Checked by golden model
Lp_clk_ack_hs_ comp_wdata	It is required that the handshake sequence be completed prior to updating pl_data with its next value. Following this, two packets are transferred sequentially through the pl_data signal.	___	Checked by golden model
Active_comp	No handshake is required to update the pl_data with its next value	——	Checked by golden model
Active_comp_w data	No handshake is required to update the pl_data with its next value.the, two packets are transferred sequentially through the pl_data signal.	——	Checked by golden model
Async_fifo_writ e till_full	The full flag must be asserted till a read signal come	___	Checked by golden model
Async_fifo_read _till empty	The empty flag must be asserted till a write signal come	___	Checked by golden model
Wrapping_fifo	Fifo has the ability to wrap	—	Checked by golden model

5.3.2.3.2 Test Sequences

Stimulus generated

Based on the message formats described in Chapter 3, the sequence item was implemented as a structure data type, where the relevant fields of the message are randomized according to the verification objectives. Specifically, we focus only on fields that influence the behavior of the receiver. For instance, fields such as msginfo and the actual data payload are not considered critical in this context. This is because, as long as the data parity is valid, the receiver does not distinguish whether the data itself carries meaningful content. This holds true even for locally terminated completions that are forwarded to the adapter layer without further processing.
As such, our primary interest lies in the DP,CP, miscode and msgopcode fields. These fields determine how the receiver interprets the message and which internal block should handle it. Once identified, the receiver proceeds to evaluate the message's source and destination fields to ensure it is routed appropriately.

Where all these message being randomized through the stimulus are serialized as an input for the RXDATA-SB to emulate the behavior of the second die

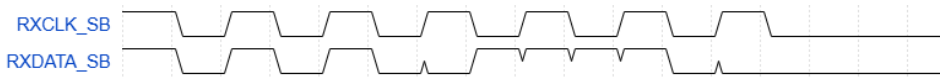


Figure 5-54: sideband-rx stimulus generation

We used rective stimulus for clock gating handshakes where after the DUT assert the pl_clk_req the lp_clk_ack must be asserted even if the phy_lyer does'nt support clock gating

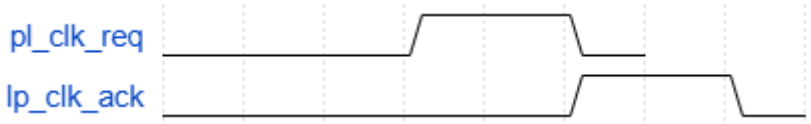


Figure 5-55: clock gating handshakes

Sideband clk-domain sequences

The figure below contain some of the sequences we generated such as reset_sbinit_sequence in which we exit from reset and receive sbinit_pattern initialization where the die must detect 2 sbinit pattern consecutive or non consecutive to exit from reset to sbinit and it also contain the sbinit_out_of _reset message correct, with parity err , with dest err.

completion message sequence which emulate the behavior of the remote die sending a completion that the phy layer must bypass to the adapter where the com pletion messages doesn't consume credit so no need for availability of credits at the adapter to send it .

RAR message which is sent by the remote die also and bypassed by the phy-layer to the adapter layer as the adapter layer contain the mailbox mechanism which responsible for the remote die register access requests.

Module related sequence is a sequence which emulate the behavior of the module partner on the remote die as we send all the responses received from the remote die to the module as the module is responsible for sending the requests and waits for the responses
Module partner sequence is a sequence in which we send the training requests which is sent by module to module partner an check if it goes to module as designed or not.
Controller sequence is a sequence which generates RDI messages to the controller.

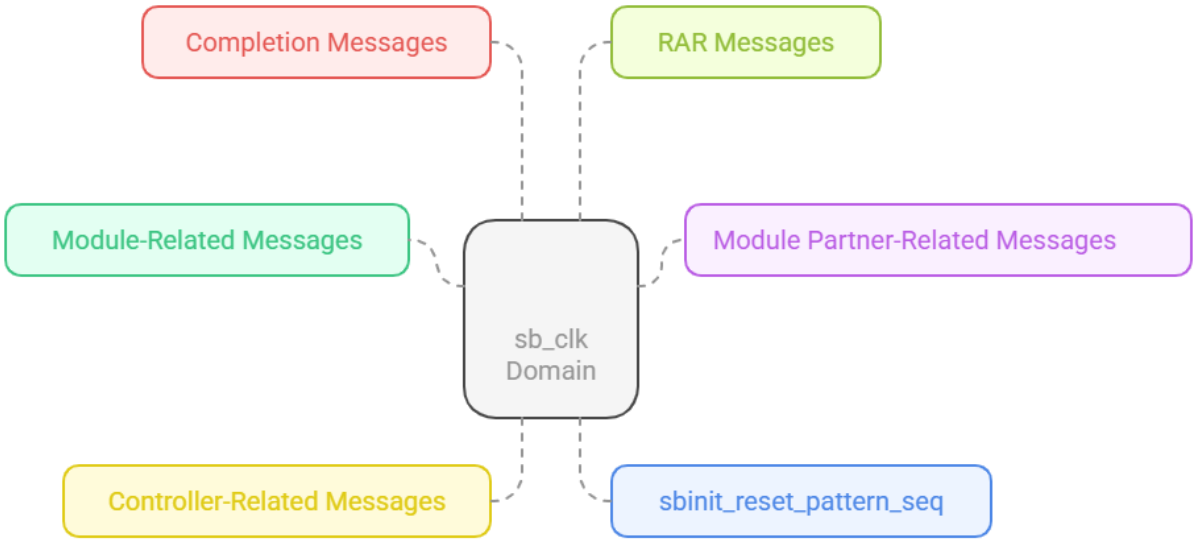


Figure 5-56: sb_clk message sequences

Moreover we generated the same stimulus but with randomizing all the possible errors which could happen preventing the data from being sent to its block correctly such as data parity error and control parity error mismatched opcode with message code , vendor defined message and MPM requests which must be skipped by the decoder

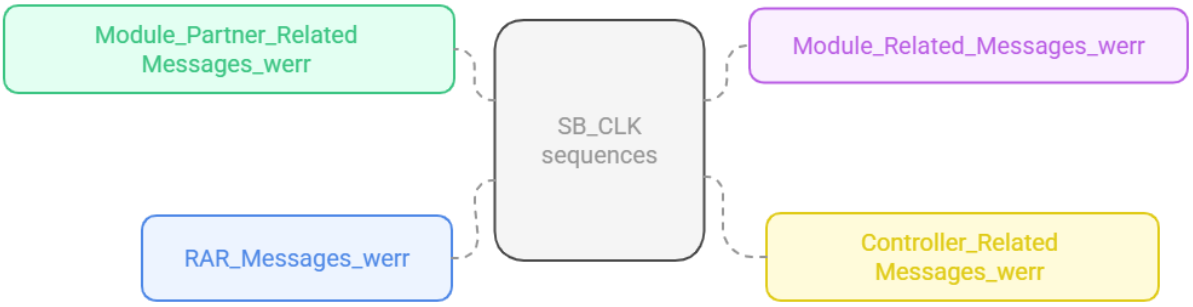


Figure 5-57: sb sequences with injited errors

Lclk domain-sequences

The figure below shows the sequences needed for the Lclk domain interface driving

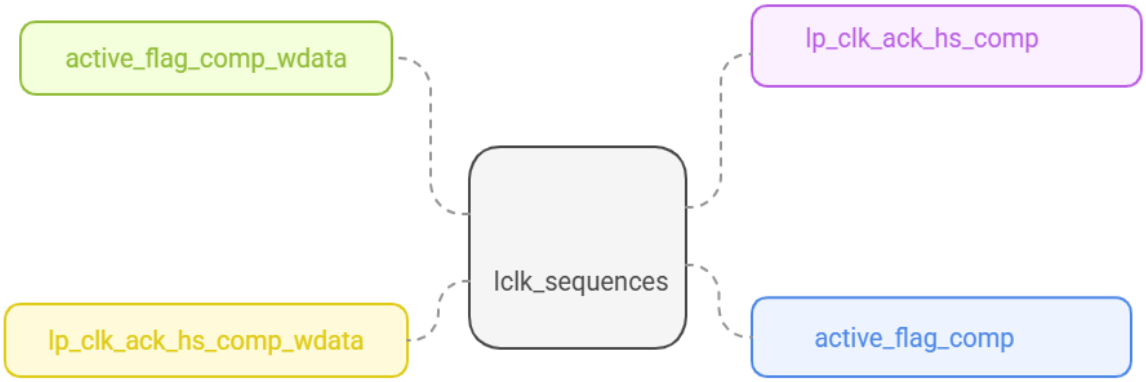


Figure 5-58: lclk domain sequences

These sequences simulate the 2 modes of which we send data from PHY to adapter where At active state we don't have 2 do the clock gating handshake before transmission so if active flag is asserted no handshakes is needed in any another state clock gating handshake is needed , we also generated local completions which is being sent from sideban tx configuration space accessing mechanism to be bypassed to the adapter.where we have two types of completion with and without data.

To ensure the integrity of all interface interactions, different combinations of sequences are executed concurrently on the two interfaces, allowing comprehensive validation of their behavior under various communication scenarios.

5.3.2.4 Results

5.3.2.4.1 Bugs Found & Fixed

- **Bug0**

The pattern detector is unable to detect nonconsecutive patterns where specs mentioned that detecting 2 64 ui pattern is the condition for exiting reset where they can be consecutive or nonconsecutive patterns

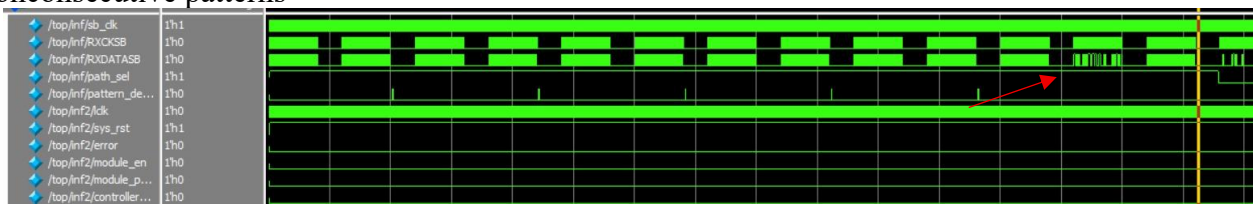


Figure 5-59: side band Rx bug-0

- **Bug1**

Following the clock gating handshake, only the first 32 bits of the data are transmittedto the adapter, while the remaining 32 bits are not forwarded, resulting in incomplete data delivery.

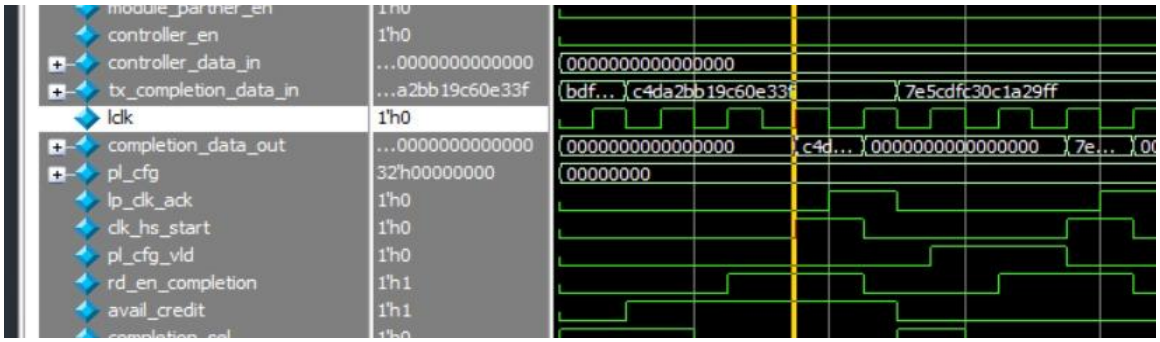


Figure 5-60: side band Rx bug-1

- **Bug2**

The decoder reading from the asynchronous FIFO is not properly synchronized with the timing of data availability on the bus, leading it to sample outdated or stale values instead of the most recent data.

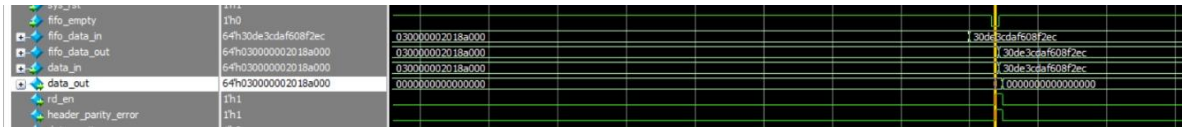


Figure 5-61: side band Rx bug-2

- **Bug3**

The completion with data is not fully captured, as only the header is read while the associated data payload is inadvertently ignored.

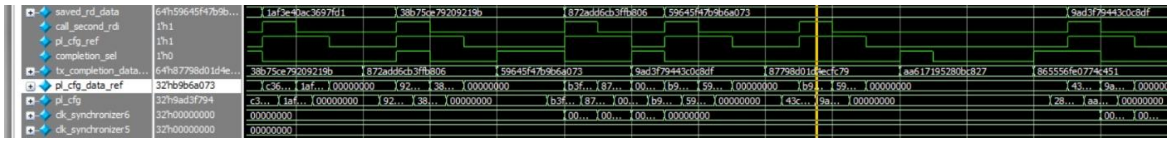


Figure 5-62: side band Rx bug-3

- **Bug4**

The data_vld signal is incorrectly asserted as a level signal rather than a pulse. Since it is used to trigger data writes into the asynchronous FIFO, maintaining it high for 32 cycles results in the same data being written repeatedly, leading to 32 unintended write operations. For correct behavior, data_vld must be a single-cycle pulse aligned with valid data.

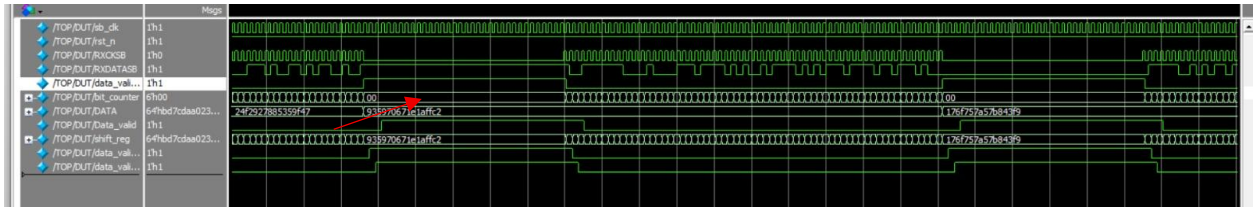


Figure 5-63: sideband Rx bug-4

5.3.2.4.2 Coverage

Code coverage

The overall coverage for the design unit SB_RX_TOP is reported at 90.71%, indicating a strong and well-stimulated verification environment. Most individual coverage metrics are high, with full coverage achieved for both FSM states and transitions (100%), and near-complete toggle (99.57%) and statement (92.88%) coverage. These results demonstrate that the major functional paths and control structures in the design have been effectively exercised.

The only notably lower metric is condition coverage at 70.45%, which is attributed to a known scenario where the condition (write_en && !full rd_en) (read_en && !empty) is structurally prevented from occurring. In this design, Since this combination is not intended to happen by design, its absence in simulation is expected and does not indicate a coverage hole.

The expression coverage is also low due to full flag expression which will never happen
Where we have these conditions and expression repeated on the all the buffers and fifos.

Therefore, the unhit condition does not reflect a functional gap but rather a structurally unreachable combination. Excluding such unreachable conditions from the coverage analysis—either manually or via pragmas or exclusion directives—can help refine the coverage metrics to better represent true verification completeness.

```
================================================================================
=== Design Unit: work.SB_RX_TOP
================================================================================
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                       175       158        17    90.28%
    Conditions                      44        31        13    70.45%
    Expressions                     22        18         4    81.81%
    FSM States                       4         4         0   100.00%
    FSM Transitions                  6         6         0   100.00%
    Statements                     506       470        36    92.88%
    Toggles                       5582      5558        24    99.57%


Total Coverage By Design Unit (filtered view): 90.71%
```

Figure 5-64: sideband Rx code coverage

Functional coverage

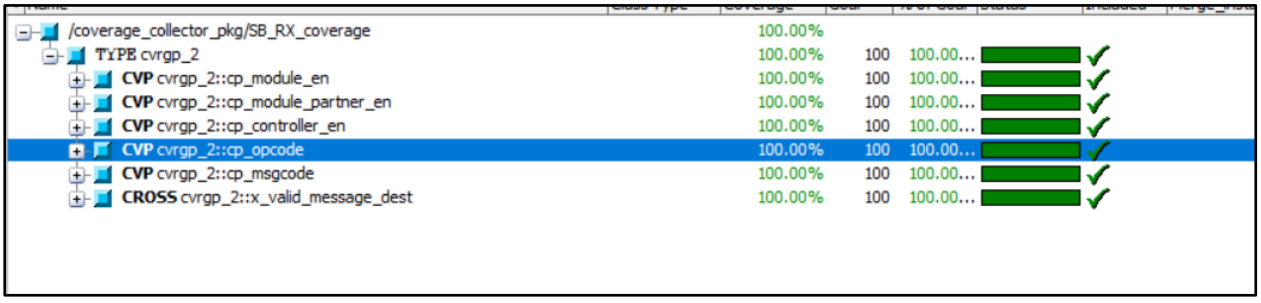


Figure 5-65: sideband RX functional coverage

The functional coverage results for SB_RX_coverage demonstrate comprehensive stimulus quality and coverage completeness across all defined coverpoints and cross coverage metrics. Each coverpoint—capturing critical protocol attributes such as module enable states, opcodes, and message codes—achieved 100% bin coverage, confirming that all expected functional scenarios were exercised. Moreover, the cross-coverage metric ix_valid_message_dest, which evaluates the interaction between multiple parameters, also reached full coverage, indicating a well-distributed and thorough stimulus space.

These results strongly support the conclusion that the verification environment effectively covers the intended functional behavior and protocol permutations, and that no major stimulus gaps exist within the monitored signal domains.

5.4. Mainband "MB":

5.4.1. Mainband Transmitter "MB_TX":

5.4.1.1 Interface Signals

The MB Tx block operates across two clock domains: Lclk and MB clk. In the block diagram, signal paths are color-coded for clarity. All input signals originate from the Lclk domain, while the output signals are driven from both the Lclk and MB clk domain.

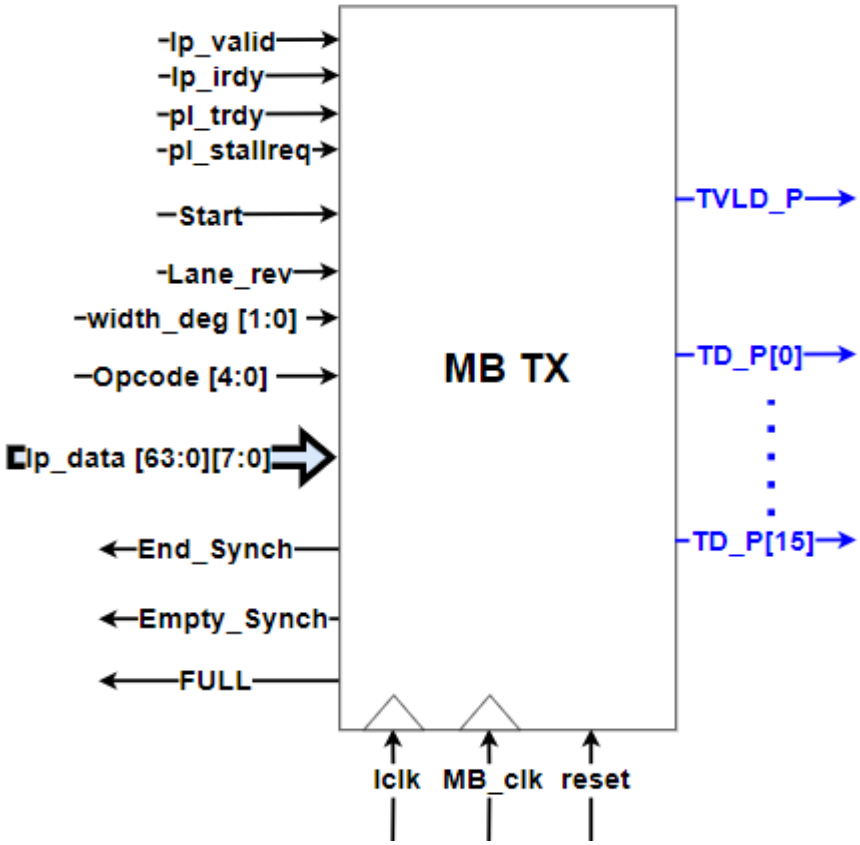


Figure 5-66:Abstract MB Tx Interface

5.4.1.2 UVM Testbench Architecture

Based on the functionality outlined in the design chapter and considering the multi-clock domain nature of the interface, the MB Tx testbench includes several key differences compared to a standard UVM testbench:

- **Dual-Domain Monitors:**

The testbench incorporates dedicated monitors for each clock domain to accurately capture and analyze output activity. These monitors are responsible for observing domain-specific signals and forwarding the collected data to the scoreboard, ensuring that output verification is aligned with the originating clock domain.

- **Clock-Domain Aware Scoreboard and coverage collector**:

The scoreboard is designed to receive both clock domain sequence items and to be aware of the source clock domain for each transaction, enabling precise comparison of expected and actual outputs across both the Lclk and MB clk domains. This ensures that domain-specific behaviors such as timing, latency, or synchronization are correctly validated. The same goes for the functional coverage collector which samples transactions from separate clock domain transactions and utilizes any cross coverage across these domains to correctly assess the verification process coverage.

All that said, the final testbench architecture is as shown in the below diagram dealing with different interfaces each one corresponds to a clock domain.

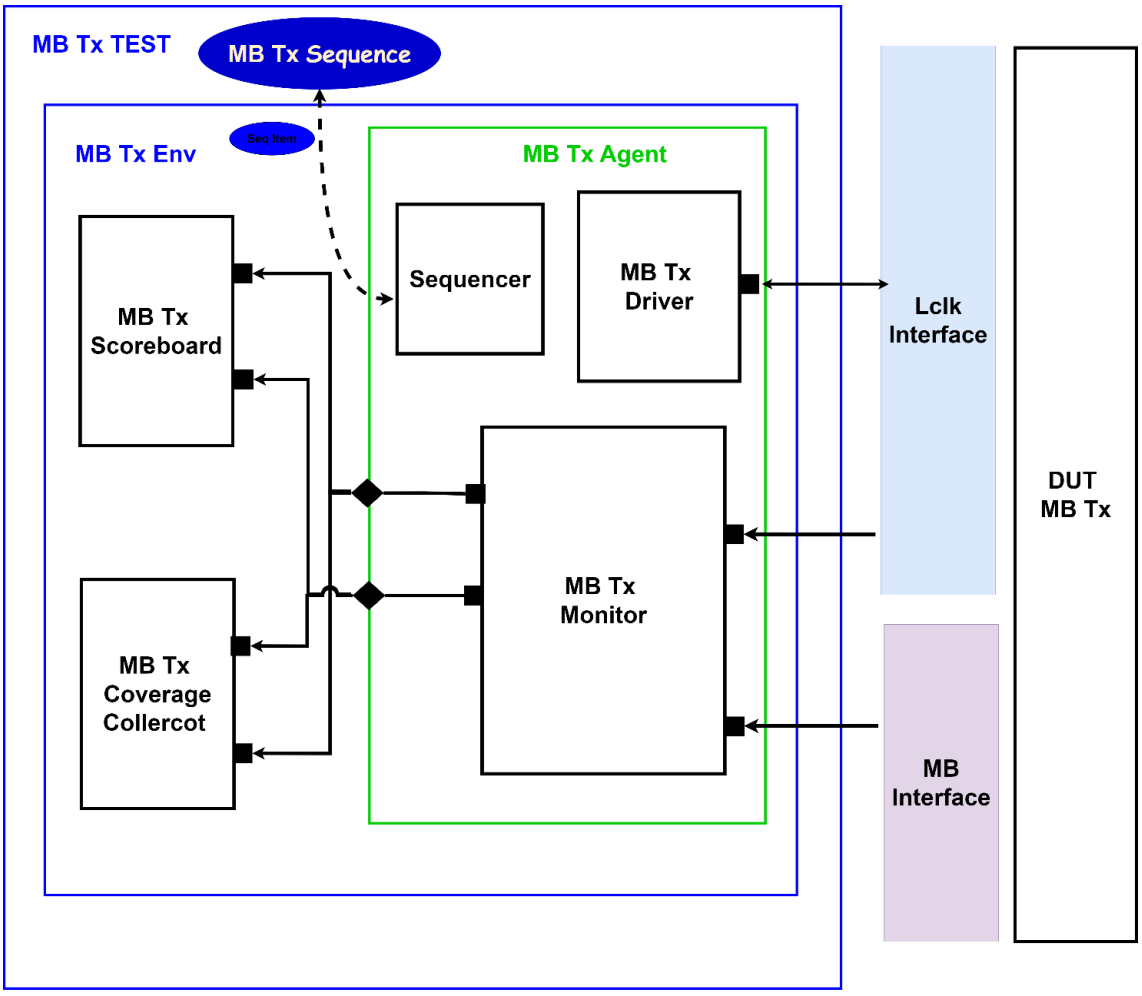


Figure 5-67: MB Tx UVM Testbench Architecture

5.4.1.3 Test Plan:

Given the MB Tx functionality A test cases list covering most of the MB Tx functionality is planned and then to effectively drive the MB Tx through these targeted scenarios, a set of dedicated sequences has been developed. to generate the stimulus needed to nudge the MB Tx through all these test cases. These sequences are designed to generate the specific stimulus required to guide the DUT through these test cases.

5.4.1.3.1 Test Cases:

Table 5-27: MB Tx Verification Plan (Sheet 1 of 4).

Test Case Label	Test Case Description	Functional Coverage	Functionality Check
RESET_CASE_MBTX	At Reset State the data and valid tracks are kept low and all the outputs should be low	Reached by Reset Sequence	Covered by a coverage bin that ensures that the RESET state is reached at least once
VAL_SEQ_UNACTIVE	At any Valid training State if the start signal is low all tracks should stay the same	Reached by Bring-up flow Sequence	Covered by a cross-coverage bin between any Valid training state and low start
VAL_SEQ	At any Valid training State and when start is asserted the valid tracks should have 128 iterations of valid training sequence and data tracks are kept low and the end of this process the END_LTSM is high	Reached by Bring-up flow Sequence	Covered by a cross-coverage bin between any Valid training state and high start
D2C_PER_LANE_UNACTIVE	at D2C_PER_LANE states if the start signal is low the tracks are kept low	Reached by Bring-up flow Sequence	Covered by a cross-coverage bin between any state requiring D2C Per-lane test and low start
D2C_PER_LANE	at D2C_PER_LANE states in case of high start signal each data lane should have its per-lane pattern assigned to it in sync with the correct valid frame	Reached by Bring-up flow Sequence	Covered by cross-coverage bin between any state requiring D2C Per-lane test and high start

Table 5-28: MB Tx Verification Plan (Sheet 2 of 4).

Test Case Label	Test Case Description	Functional Coverage	Functionality Check
D2C_PER_LANE_Width_Degraded	at D2C_PER_LANE states and high start signal if the width degrade is activated the working lanes alone should have the per-lane pattern as expected and the deactivated lanes should hold low	Reached by Width Reversal Sequence	Covered by a cross-coverage bin between any state requiring D2C Per-lane test, high start and width_degrade encoding that is not "11"
D2C_PER_LANE_Reversed	at D2C_PER_LANE opcode and high start signal the per-lane data pattern is sent in reversed connectivity to the case above to check for reversed connection	Reached by Width Reversal Sequence	Covered by a cross-coverage bin between any state requiring D2C Per-lane test, high start and high lane reverse flag
D2C_LFSR_UNACTIVE	at D2C_LFSR opcode if the start signal is low all the lanes are kept low as the process hasn't been triggered yet	Reached by Bring-up flow Sequence	Covered by a cross-coverage bin between any state requiring D2C LFSR test and low start
D2C_LFSR	at D2C_LFSR opcode and high start signal data lanes should have its LFSR pattern assigned to it in the correct phase also the valid lane outputs the correct valid frame	Reached by Bring-up flow Sequence	Covered by a cross-coverage bin between any state requiring D2C LFSR test and high start
D2C_LFSR_Reversed	at D2C_LFSR opcode and high start signal data lanes should have its LFSR pattern assigned to it in the correct phase also the valid lane outputs the correct valid frame	Reached by Width Reversal Sequence	Covered by a cross-coverage bin between any state requiring D2C LFSR test, high start and high lane reverse flag
D2C_PER_LANE_Width_Degraded	at D2C_PER_LANE states and high start signal if the width degrade is activated the working lanes alone should have the per-lane pattern as expected and the deactivated lanes should hold low	Reached by Width Reversal Sequence	Covered by a cross-coverage bin between any state requiring D2C LFSR test, high start and width_degrade encoding that is not "11"

Table 5-29: MB Tx Verification Plan (Sheet 3 of 4).

Test Case Label	Test Case Description	Functional Coverage	Functionality Check
ACTIVE	at ACTIVE state and with high lp_valid, lp_irdy and pl_trdy and low lane reversal and with no width degradation the data lanes should be assigned the corresponding data bits after scrambling and the valid framing is synchronized with it	Reached by Bring-up, Width Reversal and RETRAIN_Flow Sequences	Covered by a cross-coverage bin between ACTIVE state and high control flags (lp_valid, lp_irdy, pl_trdy)
ACTIVE_not_ready	In case of ACTIVE state and the same inputs as the case above but pl_trdy is low the data shouldn't be read and the valid tracks are kept low	Reached by Bring-up flow Sequence	Covered by a cross-coverage bin between ACTIVE state and low pl_trdy flag
ACTIVE_IDLE1	at ACTIVE state and if the same inputs as ACTIVE case above but lp_irdy is low so there is no data in the downstream and the valid tracks are kept low	Reached by Bring-up flow Sequence	Covered by a cross-coverage bin between ACTIVE state and low lp_irdy flag
ACTIVE_IDLE2	In case the state = ACTIVE and the same inputs as ACTIVE case above but lp_valid is low so there is no data in the downstream and the valid tracks are kept low	Reached by Bring-up flow Sequence	Covered by a cross-coverage bin between ACTIVE state and low lp_valid flag
ACTIVE_DEGRADED1	at ACTIVE state after correct data registration, if width degradation =01 the 8 Least significant lanes should be assigned the corresponding data bits and the valid framing with it	Reached by Width Reversal Sequence	Covered by a cross-coverage bin between ACTIVE state, high control flags (lp_valid, lp_irdy, pl_trdy) and width_degrade = 01
ACTIVE_DEGRADED2	at ACTIVE state after correct data registration, if width degradation =10 the 8 Most significant lanes should be assigned the corresponding data bits and the valid framing with it	Reached by randomized stimulus	Covered by a cross-coverage bin between ACTIVE state, high control flags (lp_valid, lp_irdy, pl_trdy) and width_degrade = 10

Table 5-30: MB Tx Verification Plan (Sheet 4 of 4).

Test Case Label	Test Case Description	Functional Coverage	Functionality Check
ACTIVE_REVERSE	at ACTIVE state after correct data registration, if lane reverse flag is high the physical lanes should have the data bytes mapped in reverse during transmission	Reached by Width Reversal Sequence	Covered by a cross-coverage bin between ACTIVE state, high control flags (lp_valid, lp_irdy, pl_trdy) and high lane reverse
ACTIVE_TRANS	at ACTIVE state in case the pl_stallreq is asserted regardless of the pl_trdy the design should be able to register the data chunks correctly at high lp_irdy & lp_valid and assert end_synch when done transmitting those chunks	Reached by RETRAIN_Flow Sequence	Covered by a cross-coverage bin between ACTIVE state, high control flags (lp_valid, lp_irdy) and high pl_stallreq flag

5.4.1.3.2 Test Sequences:

As illustrated in the figure below each sequence from the following targets a distinct operational flow—including its edge cases—to ensure robust and exhaustive verification.

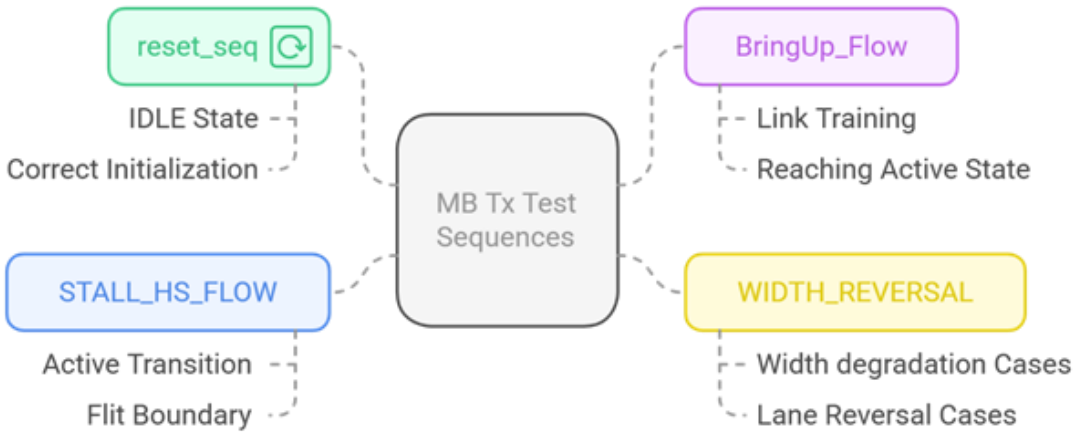


Figure 5-68:MB Tx verification Sequences

1. Reset Sequence

- **Objective:** Emulates the reset state of the design, where all outputs are driven to their default values, and ensures the DUT enters a known initial state at the start of the test.

- **Targeted Functional Checks:**
  - Reset assertions and de-assertions
  - Tried out Reset State Opcode functionality and ensured correct initialization.

2. Bring-Up Flow

- **Objective:** Emulates the complete bring-up flow passing by each state in LTSM waiting for "END_Synch" signal to indicate that the test corresponding to the state is done.

- **Targeted Functional Checks:**
  - Every Test (Valid, LFSR, Per-lane Id) is tried out more than once with full operating width and no lane reversal detected.
  - Each State correct response to the control signals (Start, Lane_Rev, width_degrade).
  - Correct Assertion for End_Synch flag

3. Width Reversal Cases

- **Objective:** Confirm correct behavior under width degradation and lane reversal configurations for both tests and active data transmission.

- **Targeted Functional Checks:**
  - All lane reversal and width degradation combinations are practiced across different states.
  - Checked all combinations of data control flags (pl_trdy, lp_valid, lp_irdy) to ensure it only registers data only in correct condition.
  - Verified Correct data byte mapping.
  - Ensured perfect valid and data alignment during transmission.
  - Tested LFSR Scrambling throughout tests and data transfer.

4. RETRAIN_FLOW

- **Objective:** Models PHY transition from active where the MB Tx should be able to receive the rest of the flit chunks from D2D after pl_stallreq flag is asserted until the assertion of lp_stallack.

- **Targeted Functional Checks:**
  - Verified correct data registration of the remaining flit segments and ensured accurate data transfer.
  - Ensured right handling of pl_stallreq flag.

5.4.1.4 Results:

5.4.1.4.1 Bugs Found & Solved:

Bug (1): End flag is asserted and de-asserted before the next positive edge clock it as it marks the ending of any test leading to state timeout which causes LTSM block missing this pulse and infinitely waiting for and PHY transitions to TRAINERROR state.

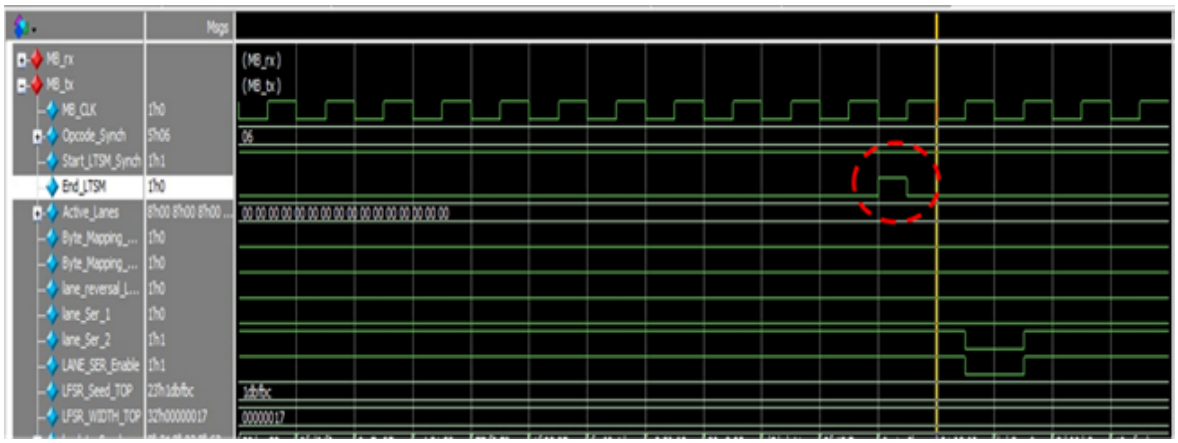


Figure 5-69: Bug (1) in MB Tx Verification

Bug (2): Valid frame is asserted at the end of the Per-lane ID test while asserting end flag, which indicates that the design wrongly restarts the Per-Lane ID test after finishing it which could cause confusion at MB Rx receiving the same previous

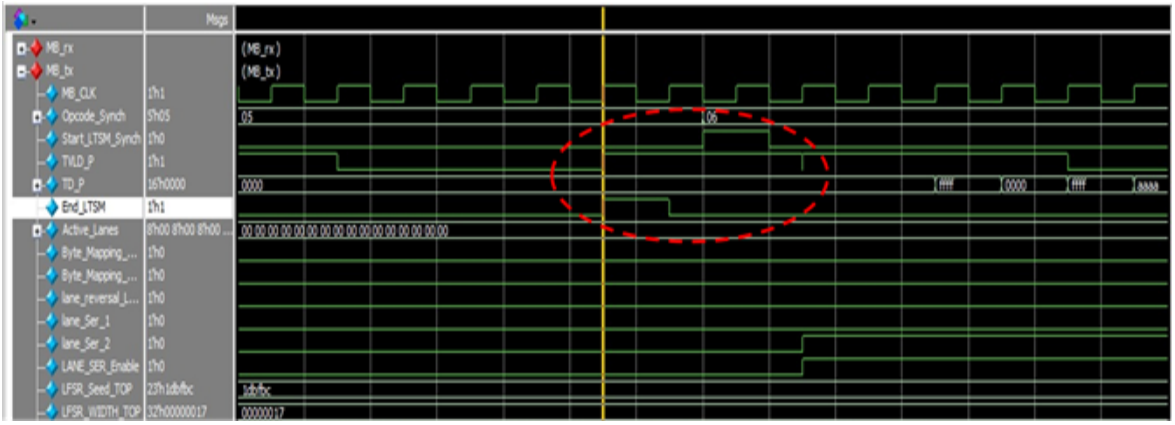


Figure 5-70: Bug (2) in MB Tx Verification

Bug (3): In the Per-Lane ID pattern case, the valid frame is asserted one clock cycle earlier than expected. This results in an incorrect test failure at the MB Rx, even though the links are fully operational, potentially hindering the link establishment phase.

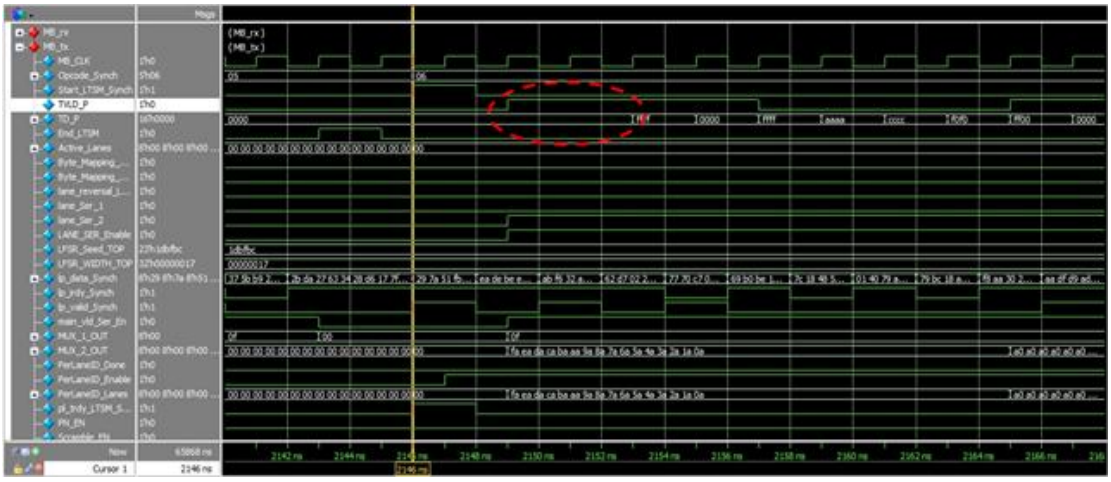


Figure 5-71: Bug (3) in MB Tx Verification

Bug (4): During Active State after the chunk is transferred the valid frame is incorrectly raised this leads to incorrect data being received by the other die, resulting in a corrupted flit that can be no longer trusted.

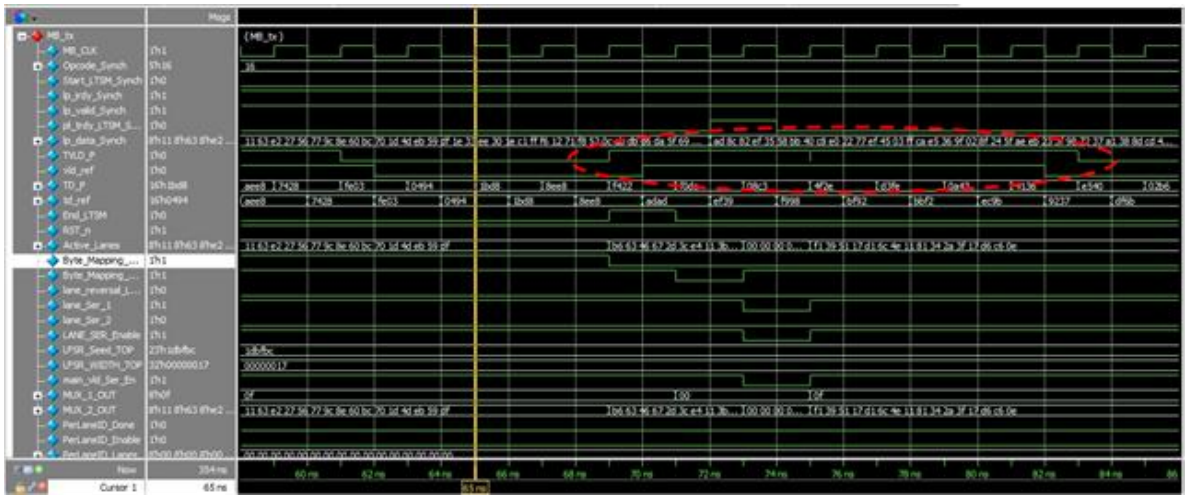


Figure 5-72: Bug (4) in MB Tx Verification

Bug (5): Scrambling was incorrectly synchronized with the data sent , leading to a de-scrambling mismatch at MB Rx and overall data corruption throughout all active transmissions

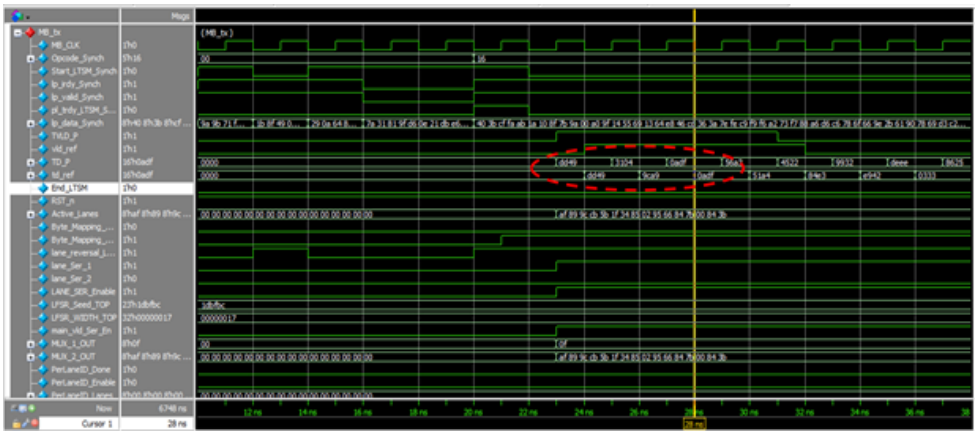


Figure 5-73: Bug (5) in MB Tx Verification

5.4.1.4.2 Coverage Achieved:

- **Code Coverage:**

This coverage summary indicates that this verification environment explored and tested most of the design with 100% coverage of FSM states and transitions, also high expression, toggle, branch and statement coverage. While condition coverage was lower at 85.52%, highlighting some uncovered logical paths, this was mainly due to default conditions that are never triggered during normal operation, precautionary checks such as if (read_en && empty) to guard against invalid FIFO reads, and design-time safety clauses that are unlikely to activate under constrained or valid stimulus.

```
================================================================================
=== Design Unit: work.MB_TX_TOP
================================================================================
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                       197       189         8    95.93%
    Conditions                      76        65        11    85.52%
    Expressions                     85        83         2    97.64%
    FSM States                       5         5         0   100.00%
    FSM Transitions                  8         8         0   100.00%
    Statements                     441       418        23    94.78%
    Toggles                      17650     17332       318    98.19%

Total Coverage By Design Unit (filtered view): 96.01%
```

Figure 5-74: Final MB Tx Code Coverage

- **Functional Coverage:**

All Coverage bins mentioned in the plan are implemented in one main cover group. Reaching 100% functional coverage with 0 illegal bins failures indicating that the stimulus is exhaustive and correctly applied.

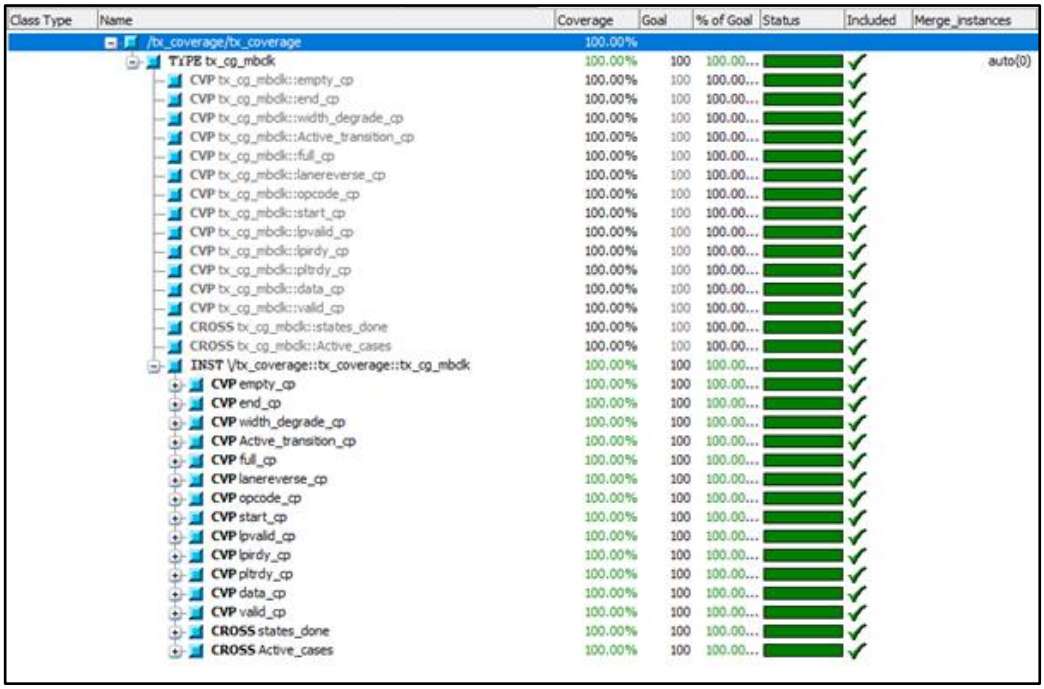


Figure 5-75: Final MB Tx Functional Coverage

5.4.2. Mainband Receiver "MB_RX":

5.4.2.1 Interface Signals

The **Mainband Receiver (MBRX) block** defines the communication between the remote **Transmitter (MBTX)** logic and the rest of the local PHY layer, primarily driven by two clock domains: `lclk` and `mbclk`. Most of the input signals—such as `ltsm_state`, `test_resp`, `error_threshold`, `width update`, and `width degrade`, are driven in the `lclk` domain, as they reflect system-level control and operating context. However, `RD_P` and `RVLD_P`, which represent the mainband data and its validity, are asserted in the `mbclk` domain, corresponding to the physical data path clock.

Several constraints govern the proper use and behavior of these signals. For instance, `test_resp` is a **pulse-based trigger** indicating that the RX should enable its comparing logic for sampling this must be correctly timed and then registered within the RX. The `ltsm_state` input carries the current operating point of the PHY link as determined by the LTSM FSM; based on its value, the RX is expected to selectively enable or disable certain logic blocks to match expected protocol behavior. The `error_threshold` input, which was fixed to 10 errors per test during testing, defines how many errors can be tolerated before reporting test failure for the respective lane. When `width_update` is asserted, it signals the RX to register the value of `width_degrade` (From Module / Module Partner), which reflects the currently degraded link width negotiated between ends.

All output signals from the RX, such as `pl_valid`, `pl_error`, `MB_RX_Done`, and all other flag outputs originate in the `mbclk` domain but are **synchronized to** `lclk` for consumption by the control and monitoring logic. This synchronization process is critical for reliable operation across clock domains and **must** be explicitly verified in the test plan through assertion-based and functional checking.

5.4.2.2 UVM Test Architecture

The UVM architecture for the **Mainband Receiver (MB Rx)** block was designed to reflect the nature of the block itself—**a straightforward datapath processor** rather than a complex state machine or an interface between two protocol layers. As such, the testbench does **not require a reactive stimulus methodology** like those used for more dynamic control blocks such as the LTSM or RDI.

Multiple sequences are need here as there are several unique functionalities in the Rx as we will discuss later in the Test Plan.

One of the key architectural features here is the **dual analysis port setup** on the **MB Rx Monitor**, which allows it to track activity on both the `mbclk` and `lclk` domains. This was essential because the **Scoreboard** operates primarily in the `mbclk` domain, where it checks for correctness of data processing and compares DUT output behavior with expected results. However, since all outputs from the MB Rx (such as `MB_RX_Done`, `pl_valid_syn`, etc.) are synchronized back into the `lclk` domain, the scoreboard also performs timing checks to ensure that these outputs are generated at the correct cycle after synchronization—preventing false positives caused by CDC latency.

On the other hand, the **Coverage Collector** is only connected to the **`lclk` domain**. This is intentional, as functional coverage only needs to observe final, stable signal behavior post-synchronization. Capturing in the `mbclk` domain could lead to miscounted bins or misleading coverage results due to intermediate or metastable states, so filtering through the synchronized domain ensures clean, meaningful coverage statistics.

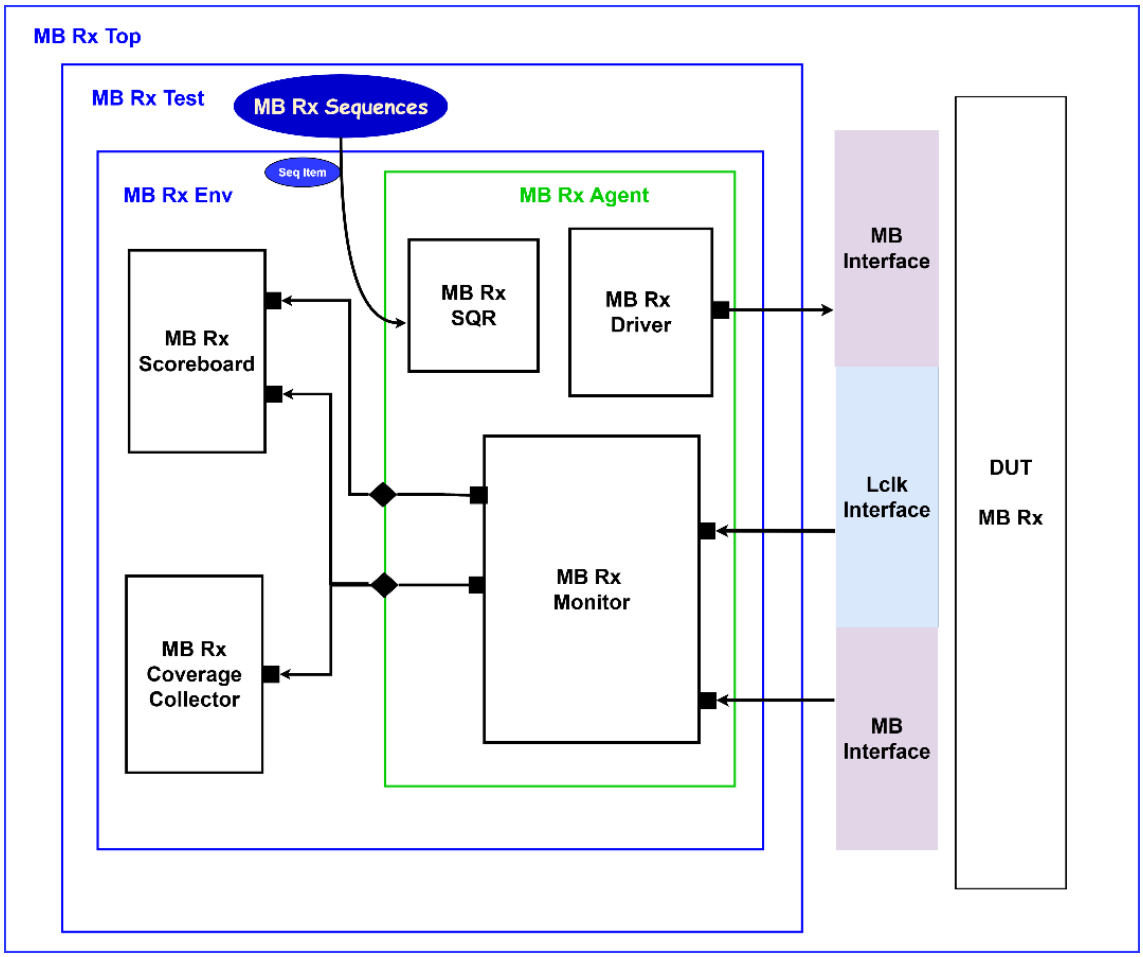
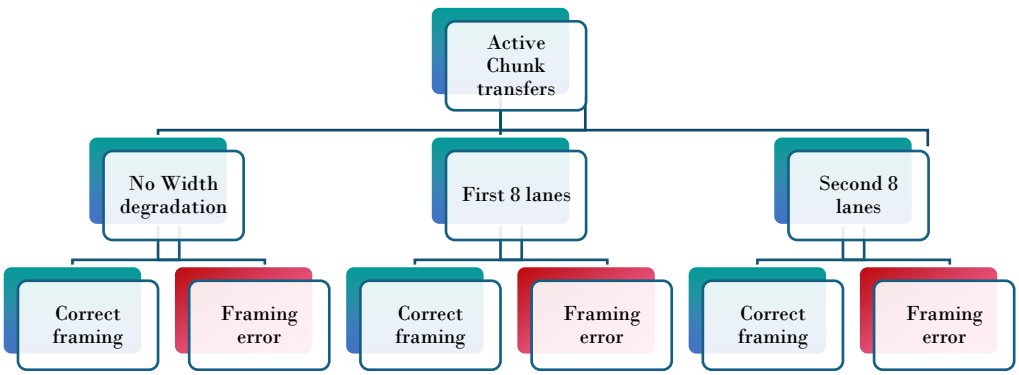


Figure 5-76: UVM Architecture followed for Mainband Receiver Verification Environment

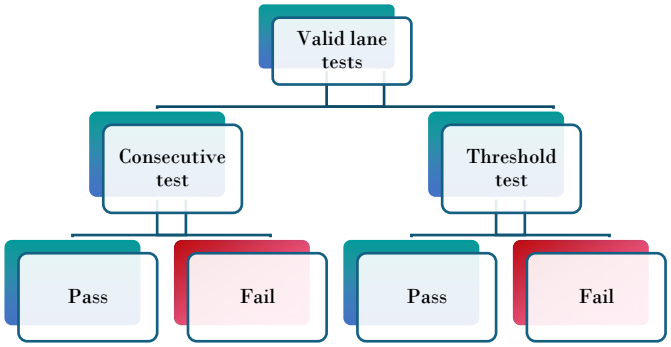
5.4.2.3 Test Plan

The test plan for the Mainband Rx is to simply check correct behavior across all five possible functionalities. This includes correct lane failure detection and reporting. Therefore, the main test cases are declared by the following diagrams:

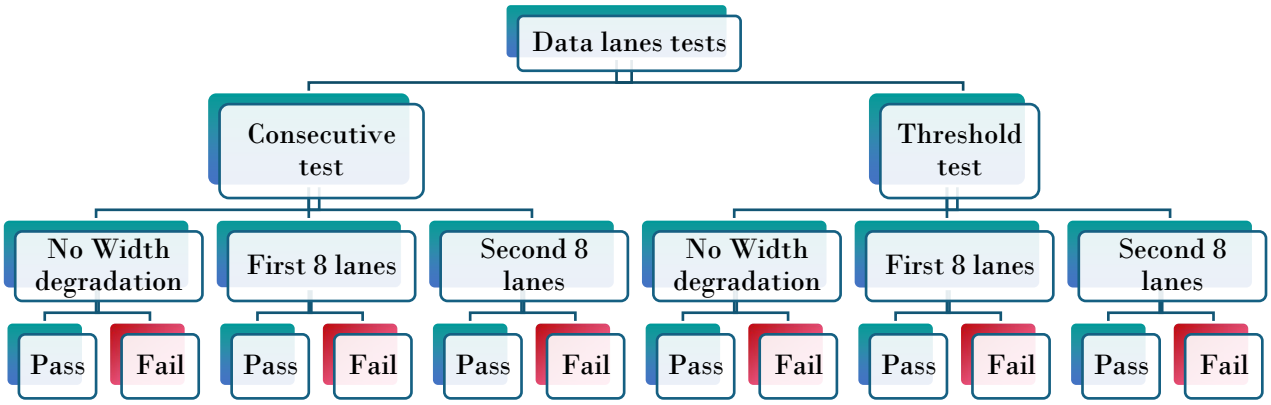
1. **Active State Related Test Cases**



2. **Valid Lane Tests Related Test Cases**



3. **Data Lanes Tests Related Test Cases**



5.4.2.3.1 Test Cases

Table 5-31: MB Rx Verification Plan.

Test Case Label	Test Case Description	Functional Coverage	Functionality Check
RST_STATE_1	Ensuring Correct Reset behavior	—	Golden model based
ACTIVE_FULL_WIDTH_2	Ensuring the ACTIVE state operation is correct in the FIRST_8 mode	Covered by Coverpoints ltsm_state, width_degrade_cross, MB_RX_done,pl_valid & pl_error	Golden model based
ACTIVE_HALF_WIDTH_3	Ensuring the ACTIVE state operation is correct in the FIRST_8 mode	Covered by Coverpoints ltsm_state, width_degrade_cross, MB_RX_done,pl_valid & pl_error	Golden model based
ACTIVE_HALF_WIDTH_4	Ensuring the ACTIVE state operation is correct in the SEC_8 mode	Covered by Coverpoints ltsm_state, width_degrade_cross, MB_RX_done,pl_valid & pl_error	Golden model based
VALID_16_CONSEC_5	Ensuring the first type of valid lane test is correct (16 out of 16 passing condition)	Covered by Coverpoints ltsm_state, width_degrade_cross, detec_succ,frame_err	Golden model based
VALID_16_CONSEC_6	Ensuring the second type of valid lane test is correct (upper more than the threshold set is failing condition)	Covered by Coverpoints ltsm_state, width_degrade_cross, detec_succ,frame_err	Golden model based
DATA_ID_TEST_7	Ensuring the first type of data lanes test is correct (FIRST_8 id iteration is passing condition)	Covered by Coverpoints ltsm_state, width_degrade_cross, comp_done & faulty lanes	Golden model based
DATA_ID_TEST_8	Ensuring the second type of data lanes test is correct (more than the threshold set is failing condition)	Covered by Coverpoints ltsm_state, width_degrade_cross, comp_done & faulty lanes	Golden model based

Test Case Label	Test Case Description	Functional Coverage	Functionality Check
DATA_ID_TEST_9	Ensuring the first type of data lanes test is correct (16 out of 16 passing condition) with FIRST_8 mode	Covered by Coverpoints ltsm_state, width_degrade_cross, comp_done & faulty lanes	Golden model based
DATA_ID_TEST_10	Ensuring the second type of data lanes test is correct (more than the threshold set is failing condition) with FIRST_8 mode	Covered by Coverpoints ltsm_state, width_degrade_cross, comp_done & faulty lanes	Golden model based
DATA_ID_TEST_11	Ensuring the first type of data lanes test is correct (16 out of 16 passing condition) with SEC_8 mode	Covered by Coverpoints ltsm_state, width_degrade_cross, comp_done & faulty lanes	Golden model based
DATA_ID_TEST_12	Ensuring the second type of data lanes test is correct (more than the threshold set is failing condition) with SEC_8 mode	Covered by Coverpoints ltsm_state, width_degrade_cross, comp_done & faulty lanes	Golden model based
ACTIVE_FULL_WIDTH_13	Ensuring the ACTIVE state operation is correct in the FIRST_8 mode (error detection)	Covered by Coverpoints ltsm_state, width_degrade_cross, MB_RX_done,pl_valid & pl_error	Golden model based
ACTIVE_HALF_WIDTH_14	Ensuring the ACTIVE state operation is correct in the FIRST_8 mode (error detection)	Covered by Coverpoints ltsm_state, width_degrade_cross, MB_RX_done,pl_valid & pl_error	Golden model based
ACTIVE_HALF_WIDTH_15	Ensuring the ACTIVE state operation is correct in the SEC_8 mode (error detection)	Covered by Coverpoints ltsm_state, width_degrade_cross, MB_RX_done,pl_valid & pl_error	Golden model based

Test Case Label	Test Case Description	Functional Coverage	Functionality Check
VALID_16_CONSEC_17	Ensuring the second type of valid lane test is correct (upper more than the threshold set is failing condition) (failing detection)	Covered by Coverpoints ltsm_state, width_degrade_cross, detec_succ,frame_err	Golden model based
VALID_16_CONSEC_16	Ensuring the first type of valid lane test is correct (16 out of 16 passing condition) (failing detection)	Covered by Coverpoints ltsm_state, width_degrade_cross, detec_succ,frame_err	Golden model based
DATA_ID_TEST_18	Ensuring the first type of data lanes test is correct (FIRST_8 id iteration is passing condition) (failing detection)	Covered by Coverpoints ltsm_state, width_degrade_cross, comp_done & faulty lanes	Golden model based
DATA_ID_TEST_19	Ensuring the second type of data lanes test is correct (more than the threshold set is failing condition) (failing detection)	Covered by Coverpoints ltsm_state, width_degrade_cross, comp_done & faulty lanes	Golden model based
DATA_ID_TEST_20	Ensuring the first type of data lanes test is correct (16 out of 16 passing condition) with FIRST_8 mode (failing detection)	Covered by Coverpoints ltsm_state, width_degrade_cross, comp_done & faulty lanes	Golden model based
DATA_ID_TEST_21	Ensuring the second type of data lanes test is correct (more than the threshold set is failing condition) with FIRST_8 mode (failing detection)	Covered by Coverpoints ltsm_state, width_degrade_cross, comp_done & faulty lanes	Golden model based

Test Case Label	Test Case Description	Functional Coverage	Functionality Check
DATA_ID_TEST_22	Ensuring the first type of data lanes test is correct (16 out of 16 passing condition) with SEC_8 mode (failing detection)	Covered by Coverpoints ltsm_state, width_degrade_cross, comp_done & faulty lanes	Golden model based
DATA_ID_TEST_23	Ensuring the second type of data lanes test is correct (more than the threshold set is failing condition) with SEC_8 mode (failing detection)	Covered by Coverpoints ltsm_state, width_degrade_cross, comp_done & faulty lanes	Golden model based
SYNC_24	Ensuring pl_valid & pl_error are correctly synced	__	Golden model based & Assertion based
SYNC_25	Ensuring comp_done correctly synced	__	Golden model based & Assertion based
SYNC_26	Ensuring MB_EX_done is correctly synced	__	Golden model based & Assertion based
SYNC_27	Ensuring detec_succ & VFE are correctly synced	__	Golden model based & Assertion based

5.4.2.3.2 Test Sequences

The **configurability** of the Mainband Rx sequences is a critical aspect of the verification strategy. Each sequence is parameterized to allow flexible control from the test layer, enabling a wide range of corner cases, stress conditions, and functional scenarios to be exercised without modifying the underlying sequence logic. This approach enhances reuse, improves coverage, and ensures targeted stimulus generation across different operating modes.

1. **Reset Sequence**
   Ensures the complete reset of the Mainband Rx logic, verifying that all internal state and outputs return to default values after asserting reset. This acts as a baseline check before other test scenarios.

2. **Width Update Sequence**
   A configurable sequence that applies different width configurations to the receiver. It simulates the width negotiation process and ensures the receiver correctly registers and responds to `width_update` and `width_degrade` signals.

3. **Active State Sequence**
   Designed to exercise the receiver under active data transfer conditions. The sequence is configurable in terms of the number of chunks transmitted and the probability of valid frame errors, allowing both normal and stress-case scenarios to be verified.

4. **Valid Consecutive Test**
   Targets validation of consecutive correct data lane behavior. The sequence introduces valid data patterns with a configurable probability of introducing errors, allowing robustness testing of error detection mechanisms.

5. **Valid Threshold Test**
   Similar to the consecutive test, this sequence focuses on evaluating the receiver's ability to detect when the number of valid lanes drops below an acceptable threshold. Error probability is tunable to explore both boundary and fault cases.

6. **Data Lanes Consecutive Test**
   Validates correct data delivery on a per-lane basis across consecutive cycles. Configurable error injection allows testing of RX resilience and logic under intermittent or lane-specific faults.

7. **Data Lanes Threshold Test**
   Evaluates the receiver's ability to respond correctly when the number of valid data lanes drops below the required operational threshold. The error pattern and rate are configurable to simulate various degradation conditions.

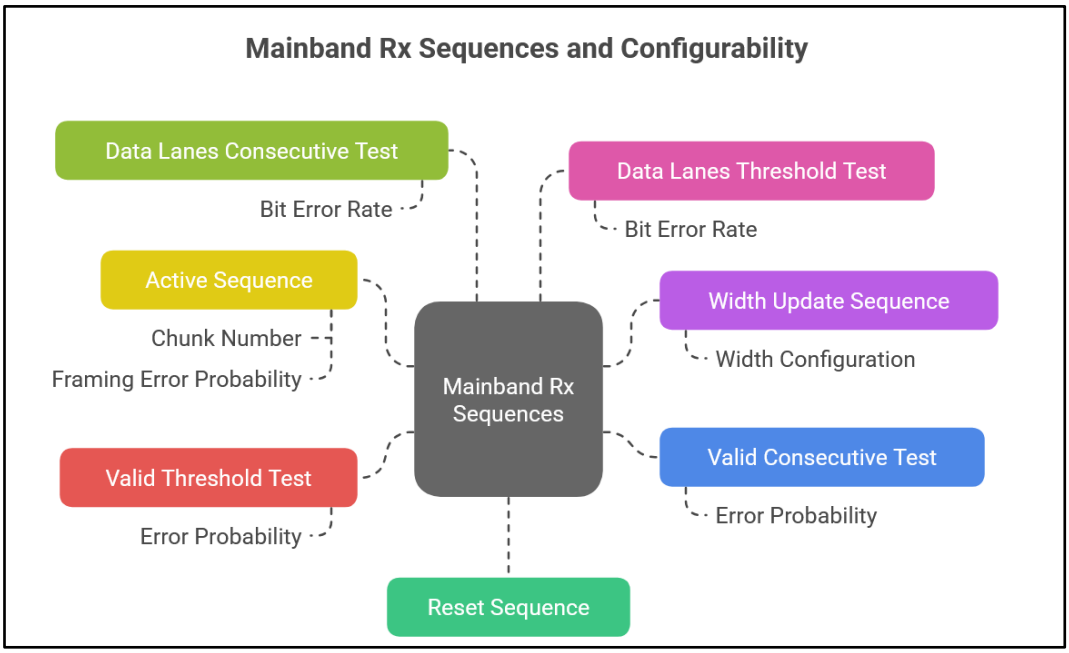


Figure 5-77: All MB Rx sequences used in the verification environment

5.4.2.4 Results

5.4.2.4.1 Bugs Found & Fixed

1. Some flags were latched, some of them internal and others were external on the interface including but not limited to Detec_succ & Valid_Frame_Error “both signals imply success or failure of a valid test”.
2. The FIFO read pointer was being incremented by 1 slot instead of 4, even though the receiver demapper writes four slots at once.

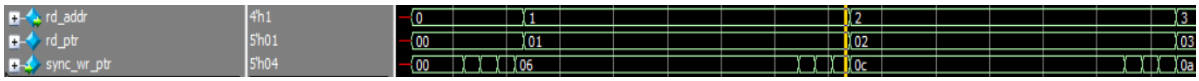


Figure 5-78: Bug (2) in MB Rx Verification

3. The Descrambler used in both Mainband Tx and Mainband Rx was sequential leading to an accumulating one cycle delay that would lead to correct first frame receival but all later frames would be incorrect.
4. Valid frame error detection was checked in the last bit on an asynchronous flag, meaning that if the first 7 bits were wrong and the last bit was correct the frame would be flagged correct even though it is wrong.

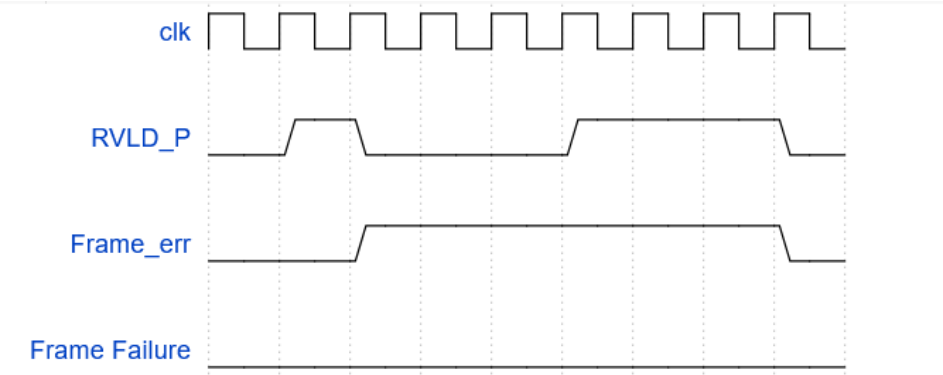


Figure 5-79: Bug (4) in MB Rx Verification

Instead, the result should be dependent on all bits of the frame like in the figure below:

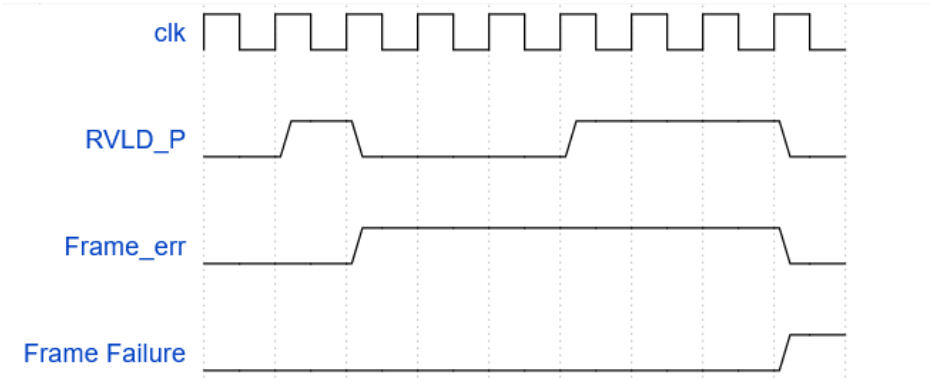
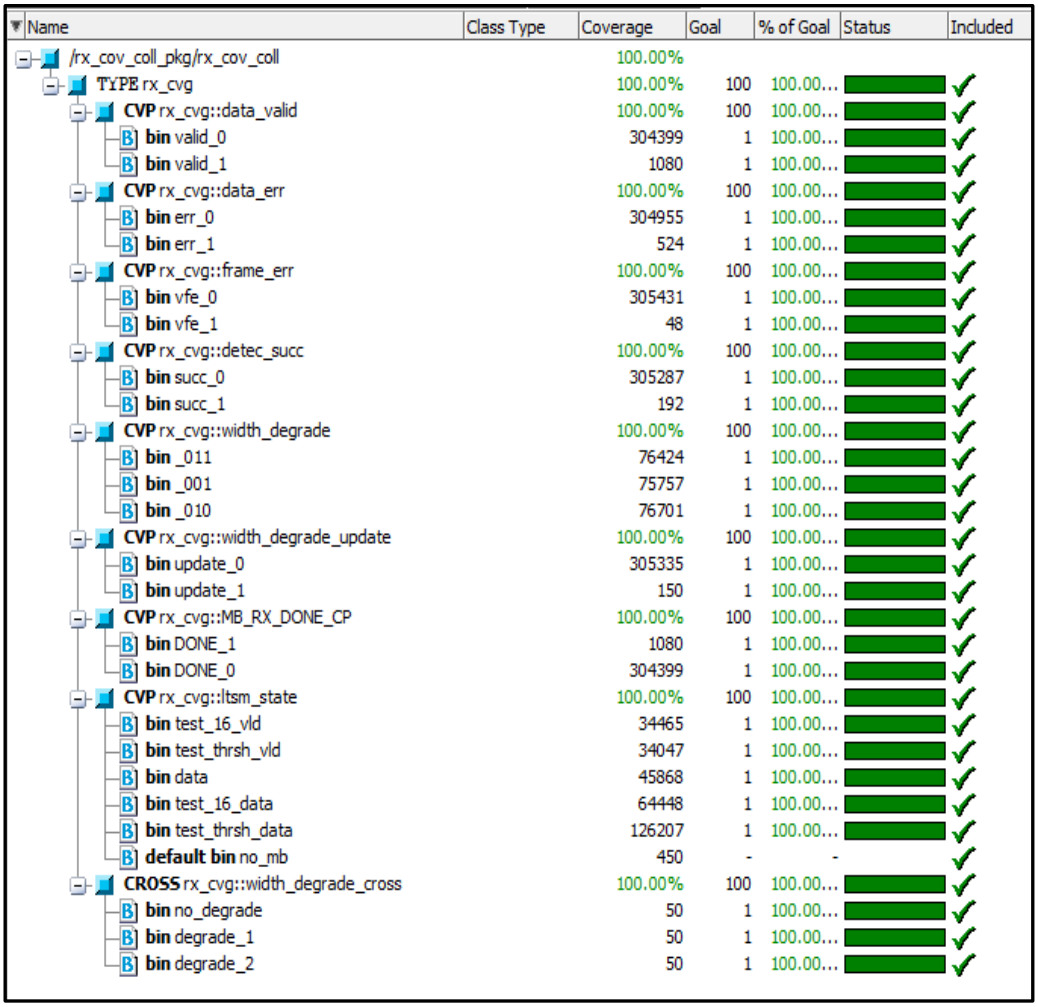


Figure 5-80: Bug (4) fixed in MB Rx Verification

5.4.2.4.2 Coverage Achieved

- Functional Coverage: All functional bins were hit.



Name	Class Type	Coverage	Goal	% of Goal	Status	Included
/rx_cov_coll_pkg/rx_cov_coll		100.00%				
TYPE rx_cvg		100.00%	100	100.00...		
CVP rx_cvg::data_valid		100.00%	100	100.00...		
bin valid_0		304399	1	100.00...		
bin valid_1		1080	1	100.00...		
CVP rx_cvg::data_err		100.00%	100	100.00...		
bin err_0		304955	1	100.00...		
bin err_1		524	1	100.00...		
CVP rx_cvg::frame_err		100.00%	100	100.00...		
bin vfe_0		305431	1	100.00...		
bin vfe_1		48	1	100.00...		
CVP rx_cvg::detec_succ		100.00%	100	100.00...		
bin succ_0		305287	1	100.00...		
bin succ_1		192	1	100.00...		
CVP rx_cvg::width_degrade		100.00%	100	100.00...		
bin _011		76424	1	100.00...		
bin _001		75757	1	100.00...		
bin _010		76701	1	100.00...		
CVP rx_cvg::width_degrade_update		100.00%	100	100.00...		
bin update_0		305335	1	100.00...		
bin update_1		150	1	100.00...		
CVP rx_cvg::MB_RX_DONE_CP		100.00%	100	100.00...		
bin DONE_1		1080	1	100.00...		
bin DONE_0		304399	1	100.00...		
CVP rx_cvg::ltsm_state		100.00%	100	100.00...		
bin test_16_vld		34465	1	100.00...		
bin test_thrsh_vld		34047	1	100.00...		
bin data		45868	1	100.00...		
bin test_16_data		64448	1	100.00...		
bin test_thrsh_data		126207	1	100.00...		
default bin no_mb		450	-	-		
CROSS rx_cvg::width_degrade_cross		100.00%	100	100.00...		
bin no_degrade		50	1	100.00...		
bin degrade_1		50	1	100.00...		
bin degrade_2		50	1	100.00...		

Figure 5-81: MB Rx Functional Coverage

- Code Coverage

```
================================================================================
=== Design Unit: work.MB_RX
================================================================================
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                       240       219        21    91.25%
    Conditions                      45        42         3    93.33%
    Expressions                     62        57         5    91.93%
    Statements                     339       334         5    98.52%
    Toggles                       8042      7354       688    91.44%


Total Coverage By Design Unit (filtered view): 93.29%
```

Figure 5-82: MB Rx Code Coverage

Chapter 6: Sub-Integration Phases:

6.1. Sub-Integration Motive:

While our UVM testbenches are designed to be as accurate and thorough as possible, certain functional issues may still arise especially those not caught during block level verification this is mainly due to the limited scope of these environments as they only predict the outputs depending on the block inputs, so there are functional errors that could originate from broader system level interactions that these environments will not catch. For example:

1. **Miscommunication between blocks:** A common example is when one block asserts control flag as a pulse, while the receiving block is expecting a level signal. Such mismatches may go unnoticed in the isolated block level environment but this would cause a series integration problem and an overall functionality bug. Also, another example is different control fields encodings.

2. **Uncovered corner-case scenarios:** Some rare or edge-case behaviors may not be explicitly exercised during block level testing, leading to gaps in validation that only surface during full system or top-level simulation.

3. **Incorrect architectural assumptions at system level:** In some cases, the architecture itself may make assumptions that don't hold true in actual operation. For example, assuming certain handshake signals will never overlap or that some sequences are mutually exclusive, while in reality timing variations or external inputs may lead to unforeseen concurrency. These design-time misinterpretations can cause failures that are only observable when all blocks operate together.

To address these issues and the exponential verification scope expansion of the Top PHY Verification testbench the integration will take place in smaller steps to accommodate this complexity and ease the integration process by finding early sub-integration bugs.

6.2. Mainband Integration:

This Sub-integration step is done by integrating the Mainband Tx and Rx blocks to validate their interoperability across all Mainband tests and to ensure seamless data exchange and proper protocol handling across the UCIe link based on the UCIe 2.0 specs. The two modules will be instantiated to communicate with each other directly and are configured under identical conditions such as the same lane width degrade and the same operational state to prevent any functional mismatches and false negatives during testing.

6.2.1. Integration Steps:

To achieve these synchronized operational conditions, enabling effective stimulus driving for both MB Tx and Rx in this MB Top environment, several modifications were made by benefiting from the existing MB Tx and Rx environments. These steps were made to ensure synchronization and co-ordination between the two modules.

This integration process involved:

1. Turning MB Rx Agent passive using configuration object.
2. Modifying MB Tx to drive both MB Tx and Rx as a simple & easy synchronization technique.
3. Creation of a combined interface that holds both modules' inputs and outputs.
4. Connect MB Tx outputs to the MB Rx inputs in the interface
5. Added assertions at the interface to provide the Overall data transmission checks.
6. Add error inducing parameters to the combined interface to be driven by separate sequences to test for error detection and test fails scenarios
7. Develop those error inducing sequences to turn on the interface parameters during testing.

After applying those steps, the final MB Environment Architecture is as shown below.

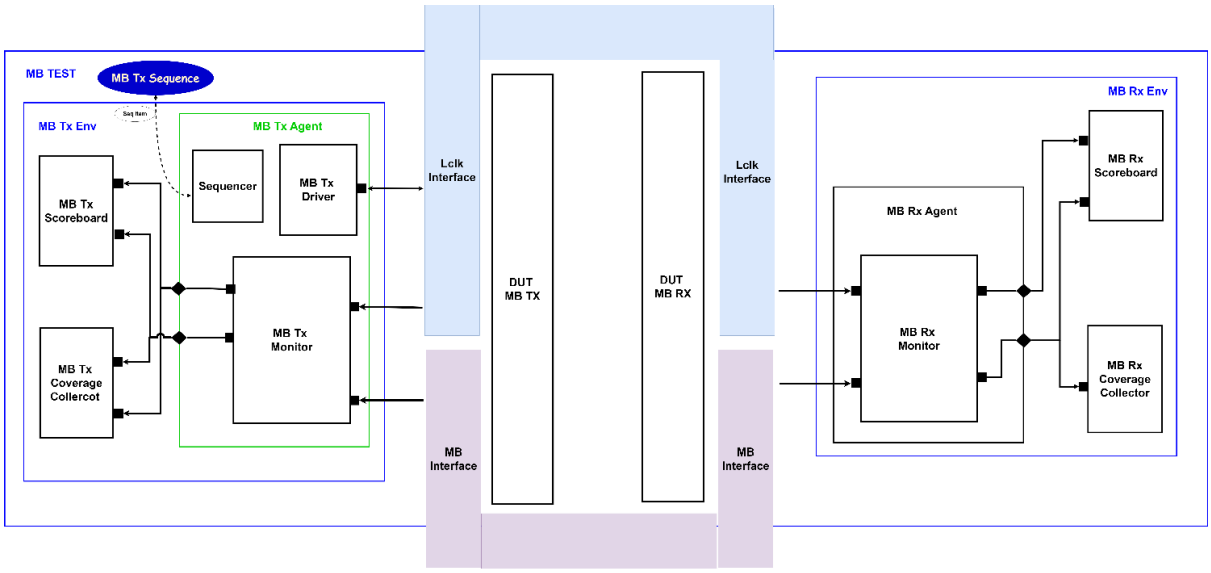


Figure 6-1: MB Top Environment

6.2.2. Bugs Found and Solved:

Only one architectural bug was found in this sub-integration test detected by the added assertion in interface. This bug appeared specifically in Lane Reversal scenario where the lane reversal logic was wrongly implemented before the LFSR (scrambler) in the MB_TX (Die 0) block as shown in the figure below. This led to scrambling process mismatch between MB Tx and MB Rx causing corruption in the deserialized output at the receiver where the original data couldn't be retrieved correctly. This wrong architecture was later fixed and the correct architecture is already shown in the design chapter in figure (75).

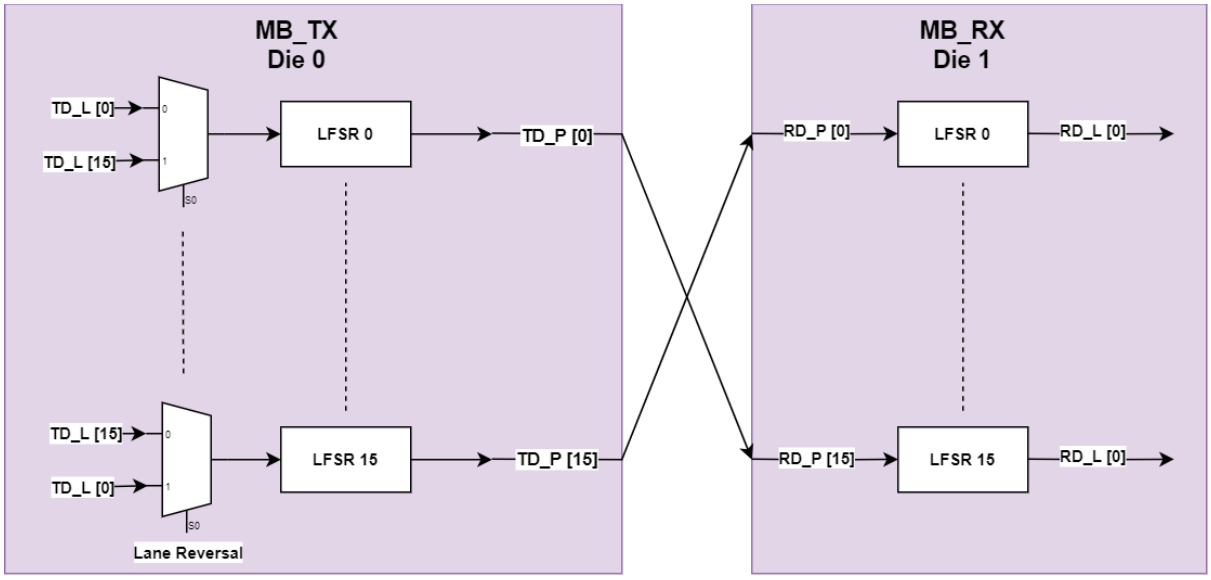


Figure 6-2: Wrong Lane Reversal Architecture

6.2.3. Simulation results

A. Code Coverage: As shown in those code coverage summaries reusing the same MB Tx stimulus generation resulted in exactly the same separate code coverage is as in the block level environment, while the MB Rx code coverage decreased since its UCIe link is now derived by MB Tx outputs which is more limited than freely driving it using stimulus generation as done in the MB Rx environment. Finally, the combined coverage achieved 92% coverage which is within a reasonable range.

```
Total Coverage By Instance (filtered view): 94.29%
```

Figure 6-3: Separate MB Tx Code Coverage

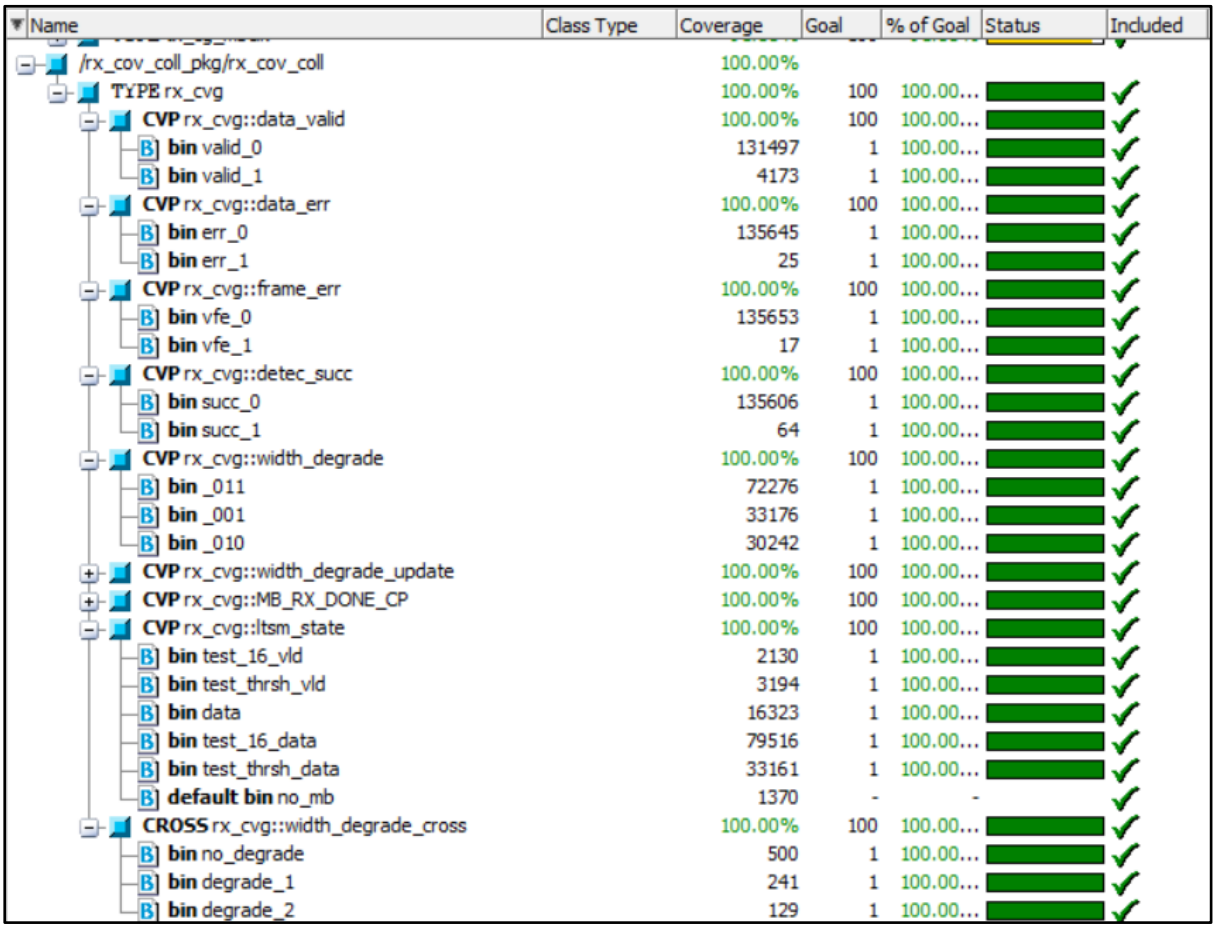
```
Total Coverage By Instance (filtered view): 91.55%
```

Figure 6-4: Separate MB Tx Code Coverage

```
Total Coverage By Instance (filtered view): 92.83%
```

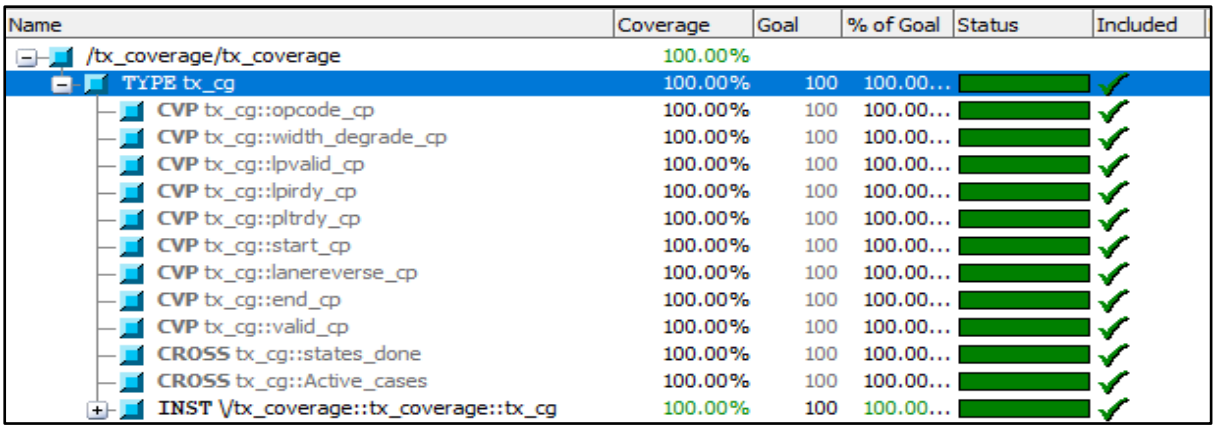
Figure 6-5: Total MB Code Coverage

B. Functional coverage: while the code coverage showed slight reduction the functional coverage remained constant assuring that all the meaningful cases were also covered in this integration test.



Name	Class Type	Coverage	Goal	% of Goal	Status	Included
/rx_cov_coll_pkg/rx_cov_coll		100.00%				
TYPE rx_cvg		100.00%	100	100.00...		
CVP rx_cvg::data_valid		100.00%	100	100.00...		
bin valid_0		131497	1	100.00...		
bin valid_1		4173	1	100.00...		
CVP rx_cvg::data_err		100.00%	100	100.00...		
bin err_0		135645	1	100.00...		
bin err_1		25	1	100.00...		
CVP rx_cvg::frame_err		100.00%	100	100.00...		
bin vfe_0		135653	1	100.00...		
bin vfe_1		17	1	100.00...		
CVP rx_cvg::detec_succ		100.00%	100	100.00...		
bin succ_0		135606	1	100.00...		
bin succ_1		64	1	100.00...		
CVP rx_cvg::width_degrade		100.00%	100	100.00...		
bin _011		72276	1	100.00...		
bin _001		33176	1	100.00...		
bin _010		30242	1	100.00...		
CVP rx_cvg::width_degrade_update		100.00%	100	100.00...		
CVP rx_cvg::MB_RX_DONE_CP		100.00%	100	100.00...		
CVP rx_cvg::ltsm_state		100.00%	100	100.00...		
bin test_16_vld		2130	1	100.00...		
bin test_thrsh_vld		3194	1	100.00...		
bin data		16323	1	100.00...		
bin test_16_data		79516	1	100.00...		
bin test_thrsh_data		33161	1	100.00...		
default bin no_mb		1370	-	-		
CROSS rx_cvg::width_degrade_cross		100.00%	100	100.00...		
bin no_degrade		500	1	100.00...		
bin degrade_1		241	1	100.00...		
bin degrade_2		129	1	100.00...		

Figure 6-4: Separate MB Rx Functional Coverage



Name	Coverage	Goal	% of Goal	Status	Included
/tx_coverage/tx_coverage	100.00%				
TYPE tx_cg	100.00%	100	100.00...		
CVP tx_cg::opcode_cp	100.00%	100	100.00...		
CVP tx_cg::width_degrade_cp	100.00%	100	100.00...		
CVP tx_cg::lpvalid_cp	100.00%	100	100.00...		
CVP tx_cg::lpirdy_cp	100.00%	100	100.00...		
CVP tx_cg::pltrdy_cp	100.00%	100	100.00...		
CVP tx_cg::start_cp	100.00%	100	100.00...		
CVP tx_cg::lanereverse_cp	100.00%	100	100.00...		
CVP tx_cg::end_cp	100.00%	100	100.00...		
CVP tx_cg::valid_cp	100.00%	100	100.00...		
CROSS tx_cg::states_done	100.00%	100	100.00...		
CROSS tx_cg::Active_cases	100.00%	100	100.00...		
INST \/tx_coverage::tx_coverage::tx_cg	100.00%	100	100.00...		

Figure 6-5: Separate MB Rx Functional Coverage

6.3. Sideband Integration:

Similar to Mainband integration this step involved integration between SB Tx and SB Rx to test for the compatibility between them and overall correct functionality which includes correct detection of SBINIT pattern and SB_OUT_OF_RESET message and receiving of both completions and UCIe transactions in parallel.

6.3.1. Integration Steps:

This integration setup is a little different than MB one, where each local environment has two agents one for each clock domain as specified earlier. The SB Tx and Rx are instantiated as in the figure below where the pattern_detected flag in SB Tx is driven by SB Rx, while the data & clock outputs from SB Tx is connected to the inputs of SB Rx. This eliminates the need for the active SB agents in both of the local environments. The remaining challenge is how to synchronize the SB Tx and Rx sequences to operate efficiently in parallel providing smooth configurations continuity. This can be simply done by running the sequences with similar operational status in parallel using fork join where the sequence that finishes first awaits the other one without any operational transition just reacting to the module's requests and outputs.

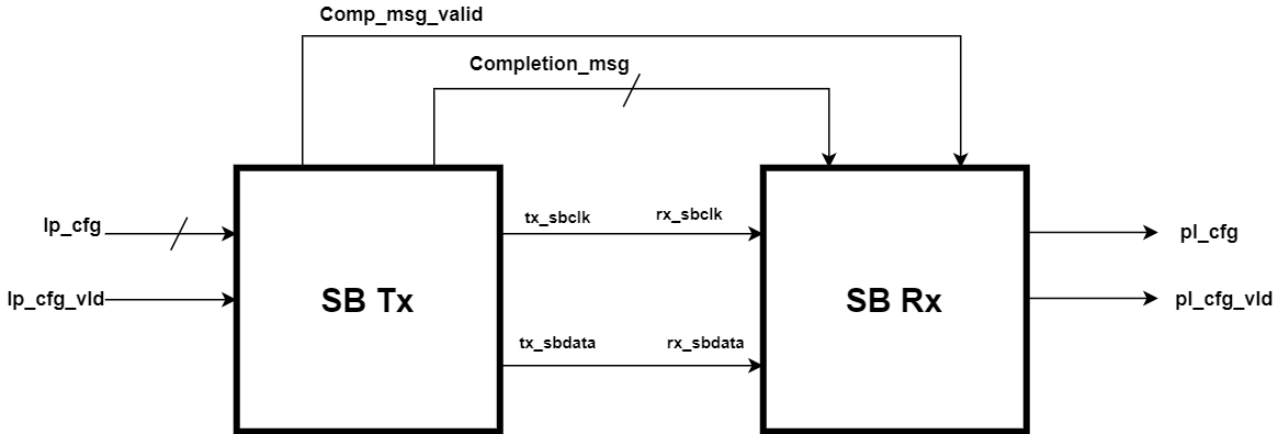


Figure 6-8: SB Test instantiations

This integration process involved:

A. Turning SB-Agents in SB Rx & Tx passive using configuration object.
B. Modifying Sequences to only react upon finishing its run task until the other sequence activates flag indicating it has also finished its functionality.
C. Creation of a combined interface that holds both modules' inputs and outputs.
D. Connect the signals mentioned above in the combined interface.
E. Add a shared queue to help check the overall transaction transmissions
F. Add error inducing parameters to the combined interface to be driven by separate sequences to test for error detection and test fails scenarios
G. Develop those error inducing sequences to turn on the interface parameters during testing.

After doing those steps accurately the final targeted testbench architecture is like this:

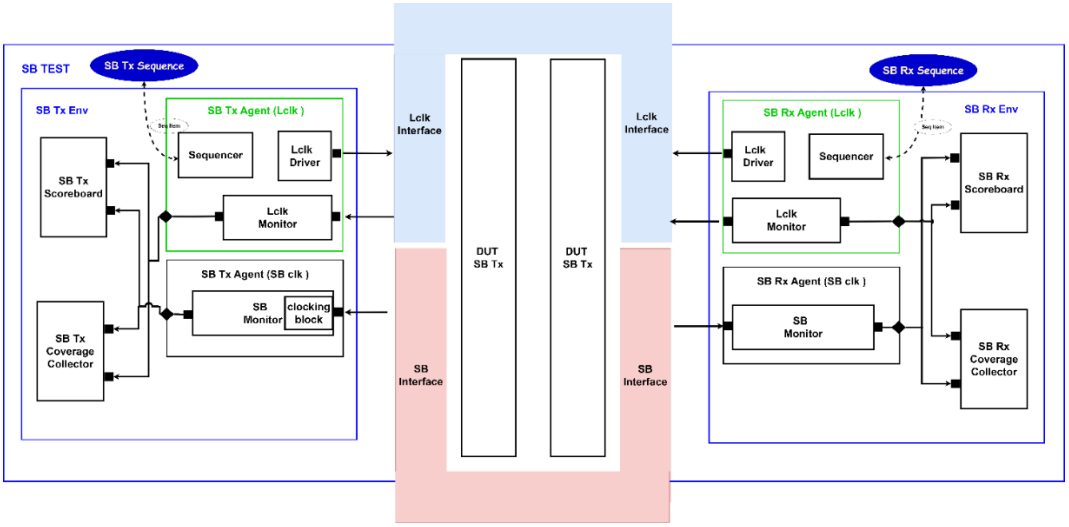


Figure 6-6:SB Top Environment

6.3.2. Bugs Found and Solved

After applying the modifications and steps needed the simulation was up and the bugs found were:

1. **Bug (1):** The expected SB_OUT_OF_RST message at Rx didn't match the message sent by Tx this resulted from misunderstanding in some of the message fields. This bug leads to continuous transmission of SB_OUT_OF_RST message from Tx and the Rx keeps misinterpreting it as a data transaction until time out occurs and the link establishment fails at the beginning.

2. **Bug (2):** This issue arises during an RDI transaction when the header phase has already been sent to the D2D layer, and the SB Rx is still in the process of deserializing the corresponding data phase. If a completion message is issued during this time, it is injected mid-way into the ongoing transaction This leads to a mix-up where the D2D interprets the completion header as part of the data for the original RDI transaction. As a result, this misinterpretation can trigger a serious error—potentially an uncorrectable internal error—causing the link to shift into the **LINKERROR** state.

6.3.3. Simulation results

A. Code Coverage: Since we haven't mainly changed any of the SB Tx and Rx stimulus only the connections SB Tx code coverage remained the same and the SB Rx code coverage showed a slight reduction due to some missing cases from the SB Tx Stimulus causing the Rx code coverage to decrease.

```
Total Coverage By Instance (filtered view): 89.37%
```

Figure 6-7: Separate SB Tx Code Coverage

```
Total Coverage By Instance (filtered view): 88.23%
```

Figure 6-8: Separate SB Rx Code Coverage

```
Total Coverage By Instance (filtered view): 88.49%
```

Figure 6-9: Total SB Code Coverage

B. Functional coverage: These missing cases were caught by the SB Rx missed coverage bins in the figure below while the SB Tx functional coverage remained the same.

Note: Those missing cases were later added in the PHY top verification testbench to ensure exhaustive test for the whole layer.

Name	Coverage	Goal	% of Goal	Status	Included
/sb_tx_coverage/sb_tx_coverage	99.07%				
TYPE tx_cg	99.07%	100	99.07%		
CVP tx_cg::reset	100.00%	100	100.00...		
CVP tx_cg::init_pattern	100.00%	100	100.00...		
CVP tx_cg::Opcodes_rdi	100.00%	100	100.00...		
CVP tx_cg::Opcodes_phy	100.00%	100	100.00...		
CVP tx_cg::dest	100.00%	100	100.00...		
CVP tx_cg::States	100.00%	100	100.00...		
CVP tx_cg::rdi_read	100.00%	100	100.00...		
CVP tx_cg::encoder_read	100.00%	100	100.00...		
CVP tx_cg::completion	100.00%	100	100.00...		
CVP tx_cg::reg_acc_req	100.00%	100	100.00...		
CVP tx_cg::comp_msg	100.00%	100	100.00...		
CVP tx_cg::Prog_stop_cp	100.00%	100	100.00...		
CVP tx_cg::Phy_data_cp	100.00%	100	100.00...		
CVP tx_cg::Phy_send_cp	100.00%	100	100.00...		
CVP tx_cg::error	100.00%	100	100.00...		
CVP tx_cg::pattern_done	100.00%	100	100.00...		
CVP tx_cg::serial_done	100.00%	100	100.00...		
CVP tx_cg::serial_data	100.00%	100	100.00...		
CVP tx_cg::{#cov_item_Lclk.lp_cfg_vld__0#}	100.00%	100	100.00...		
CROSS tx_cg::SBINIT_PAT	100.00%	100	100.00...		
CROSS tx_cg::Parity_bits	100.00%	100	100.00...		
CROSS tx_cg::Sb_frame_flag	100.00%	100	100.00...		
CROSS tx_cg::rdi_and_encoder_reads	100.00%	100	100.00...		
CROSS tx_cg::Opcodes_RDI	100.00%	100	100.00...		
CROSS tx_cg::Opcodes_PHY	75.00%	100	75.00%		
CROSS tx_cg::Destination	100.00%	100	100.00...		
CROSS tx_cg::RAR_process	100.00%	100	100.00...		
INST \/sb_tx_coverage::sb_tx_coverage::tx_cg	99.07%	100	99.07%		

Figure 6-10: Separate SB Tx Functional Coverage

Chapter 7: End-To-End Verification Environment

7.1. Methodology:

7.1.1. Verification Reuse

Verification reuse refers to the practice of leveraging existing verification assets—such as code, environments, methodologies, and components—to achieve verification objectives more efficiently in new projects or design iterations. This concept extends the broader software engineering principle of code reuse.

In the context of hardware verification, verification reuse aims to fulfill verification goals by effectively reusing verification elements, thereby reducing development time, improving consistency, and enhancing quality.

Reuse Paradigms

Verification reuse is often categorized into two primary paradigms:

- **Horizontal Reuse:** The reuse of verification components or environments at the same hierarchical level or across similar modules.
- **Vertical Reuse:** The reuse of verification assets across different hierarchical levels, such as from block-level to system-level verification.

in this work, we employed **vertical reuse** as a key strategy for the development of the end-to-end (E2E) verification environment.

Vertical Reuse Motivation:

Top-level environment is not always enough in complex design verification where debugging become extremely hard where the needed time to debug the system from top level only is considered the biggest drain on engineering resources so we need for internal checks to guide as through debugging and to confine the bugs with in certain smaller blocks instead of debugging the whole system not only but also testing smaller blocks make it possible to reach all scenarios which might be uncovered in the top system such as having safe guard line in codes or a full flag which would never be asserted.

Verification concerns at the top level

At the top-level, verification concerns extend beyond the scope of individual blocks and focus on system-wide correctness and integration. Key aspects include:

- Ensuring the functional correctness and performance of the complete application or use-case scenario.
- Verifying the interaction between modules, subsystems, and shared resources, such as memory, interconnects, and I/O interfaces.

Confirming the connectivity and integration of all blocks and subsystems, ensuring that design intent is preserved during integration and that no connectivity mismatches or configuration issues arise.

7.1.2. Encapsulation

When many block-level components are brought together into a single top-level verification environment, the scale of integration introduces challenges that often go unnoticed during block-level development. Suddenly, issues like encapsulation, configuration, namespace conflicts, and interface mismatches become far more significant. These concerns may seem minor or even irrelevant when working in isolation, but they can quickly turn into major roadblocks when components are expected to function together as part of a larger system.

For example, a block environment that wasn't designed with reuse in mind—perhaps with hardcoded settings or loosely defined interfaces—can cause real headaches at the integration stage. What worked fine on its own may now clash with other components, break shared configurations, or behave unpredictably in a more complex context. Figure X illustrates this contrast: the difficulties that arise from poorly encapsulated block-level reuse compared to the cleaner, more predictable integration made possible by well-encapsulated environments.

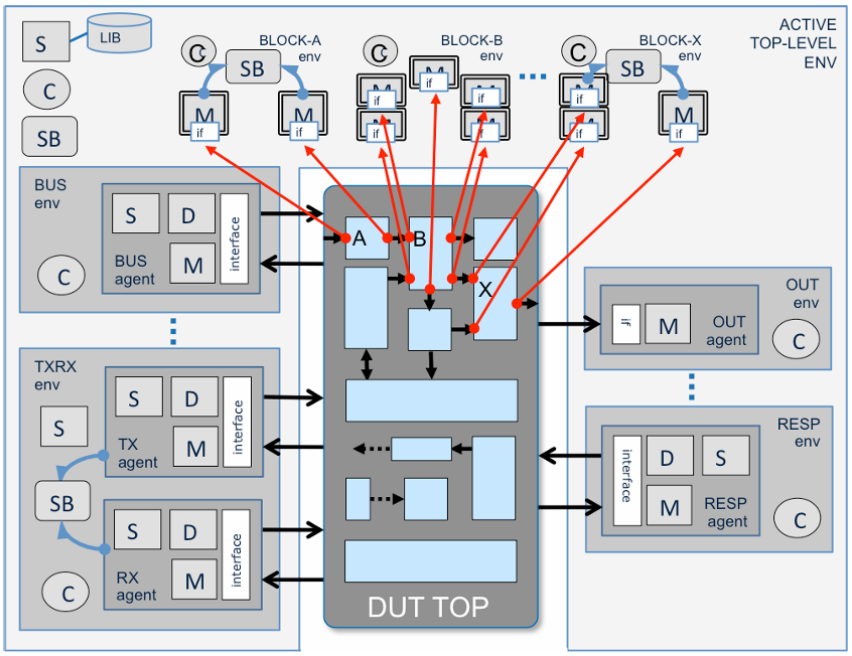


Figure 7-1:Un-Encapsulated Architecture

7.2. Hierarchical Verification Flow-steps

In this section we discuss our steps toward the hierarchical verification we used in E2E verification between two instantiated dies starting from the block level environments.

7.3.1. Active/Passive mode

All the environment should support the two modes as at block level verification each environment should be in active mode to be capable of driving it exhaustively with all the possible scenario, where at the top integration we instead use all these environments in passive modes with only one top environment which is capable of driving the DUT. Where active components like sequencers and drivers are not constructed in passive mode.

7.3.2. Solving Namespace Collision Issue

In large-scale top-level verification environments, the risk of namespace collisions becomes significantly higher due to the integration of multiple packages within a shared scope. Unlike block-level verification, where package usage is typically limited and well-contained, top-level environments often require importing various packages—sometimes with wildcard operators—which increases the likelihood of naming conflicts. These collisions frequently involve macros, constants, type definitions, or enumeration literals that are not uniquely named. To address this, it is considered best practice to prefix all package-visible identifiers with a unique, component-specific tag. This naming convention enhances modularity, prevents integration errors, and facilitates safe reuse of verification assets across different levels of the design hierarchy.

7.3.3. Hierarchal interface versus a top interface with multiple modports

While hierarchical interfaces offer structural clarity in RTL designs by encapsulating multiple signal groups into a single, organized interface hierarchy, their use within the UVM verification framework presents several challenges.

One key limitation arises from the SystemVerilog language itself, where UVM does not inherently support direct connection or visibility of signals located within nested hierarchical interfaces. In UVM-based environments, connections to DUT interfaces are typically made through virtual interfaces. When interfaces are deeply nested or structured hierarchically, accessing internal sub-interfaces or subfields becomes non-trivial and often requires additional indirection or complex binding mechanisms. So instead of using hierarchical interfaces for their complexity we used a single interface for the top layer and divided it into multiple modport each block environment has it's own modport.

7.3.4. Scoreboard Architecture Design Strategy

The initial stage in designing a functional verification scoreboard involves determining whether a single, centralized scoreboard is sufficient, or whether the design can be modularized into multiple smaller scoreboards. The latter approach—divide and conquer—offers significant advantages in terms of scalability, reuse, and maintainability. Several design scenarios can guide this decision, as outlined below.

7.3.4.1. Independent Functional Paths

In some designs, the Device Under Test (DUT) may contain independent functional paths, each performing isolated operations without requiring awareness of the others. Although such architectures are relatively uncommon, they present an ideal case for employing multiple independent scoreboards. Each scoreboard can be tailored to monitor a specific path, eliminating unnecessary complexity and interdependencies.

Even in cases where there is limited interdependence between paths, a hybrid approach may be employed: a top-level scoreboard can handle shared responsibilities and dispatch data to sub-scoreboards that operate semi-independently. This hierarchical architecture enables easier maintenance, improves readability, and supports reuse across similar designs.

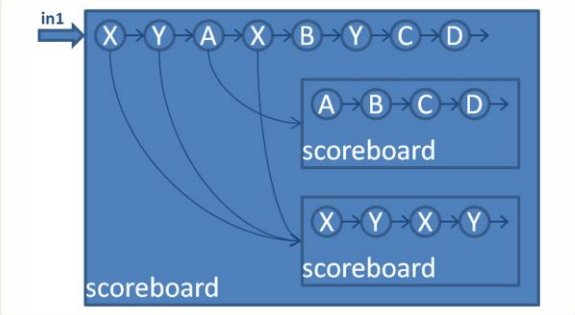


Figure 7-2: Hierarchical scoreboards for Independent Paths

7.3.4.2. Shared Midpoint Interfaces

Another common scenario involves a DUT with **internal modularity**, where data flows through a well-defined **intermediate interface**. For instance, input data (A) may generate an intermediate output (X), which is then processed further to produce the final output (Y). Such structures often indicate a pipeline or chained submodule design.

Even when the intermediate interface is not externally exposed, it may be accessible in a white-box fashion, enabling internal visibility for verification purposes. In such cases, a **chained scoreboard architecture** is appropriate, comprising:

- A scoreboard monitoring the path from A to X.

A separate scoreboard for the path from X to Y.

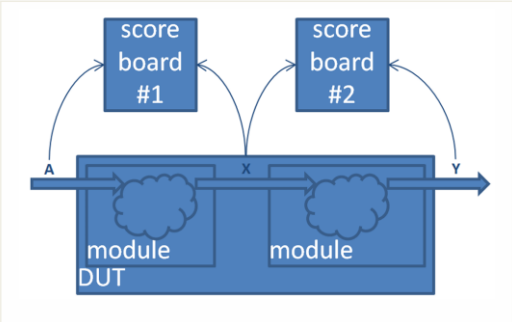


Figure 7-3: Chained Scoreboards Architecture

These scoreboards can operate independently, although they may share common configuration parameters. This separation fosters a modular and scalable design, particularly useful when verifying complex or pipelined architectures.

7.3.4.3. Design Implications

In all cases, the potential for design simplification and reuse should be thoroughly evaluated. A well-partitioned scoreboard architecture facilitates verification of large and complex systems, reduces debug effort, and promotes modular reuse across multiple projects or verification environments. Therefore, modular scoreboard design should be regarded not merely as a convenience, but as a strategic design decision critical to effective verification planning.

7.3.4.4. A Hybrid Scoreboard Approach

Our verification scoreboard takes a best-of-both-worlds approach by combining the benefits of both independent and chained scoreboard strategies. We've structured it to support multiple, separate functional paths—each with its own dedicated scoreboard—making the system more modular, easier to maintain, and reusable across different parts of the design. At the same time, within each of these paths, we use a chained scoreboard setup to track data as it moves through different internal stages, like from input to an intermediate point and then to the final output. This gives us detailed insight into each step of the process while keeping the overall scoreboard organized and scalable at the same time we check end to end data receiving and transmitting in the top scoreboard using queues which store the sent data in each die and compare the received data in the other die. The result is a flexible, well-structured verification environment that can grow and adapt with the design.

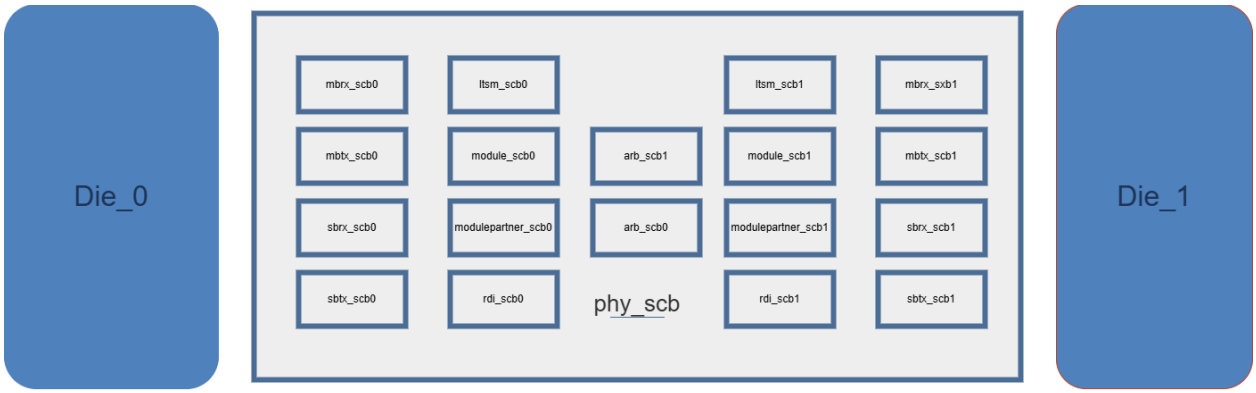


Figure 7-4:Final Scoreboard Architecture

7.3. E2E-UVM Architecture

The verification architecture is described using a top-down approach. At the highest level, the test serves as the integration point for various verification components. Within the test, we instantiate the phy_top_scoreboard, a hierarchical scoreboard that encapsulates all the block-level scoreboards, as previously discussed. These block-level environments are also instantiated within the top active environments, which function as shadow environments to facilitate targeted debugging and reuse. Where the connection between the ports of the scoreboard and their monitor is done at the test final architecture is found below.

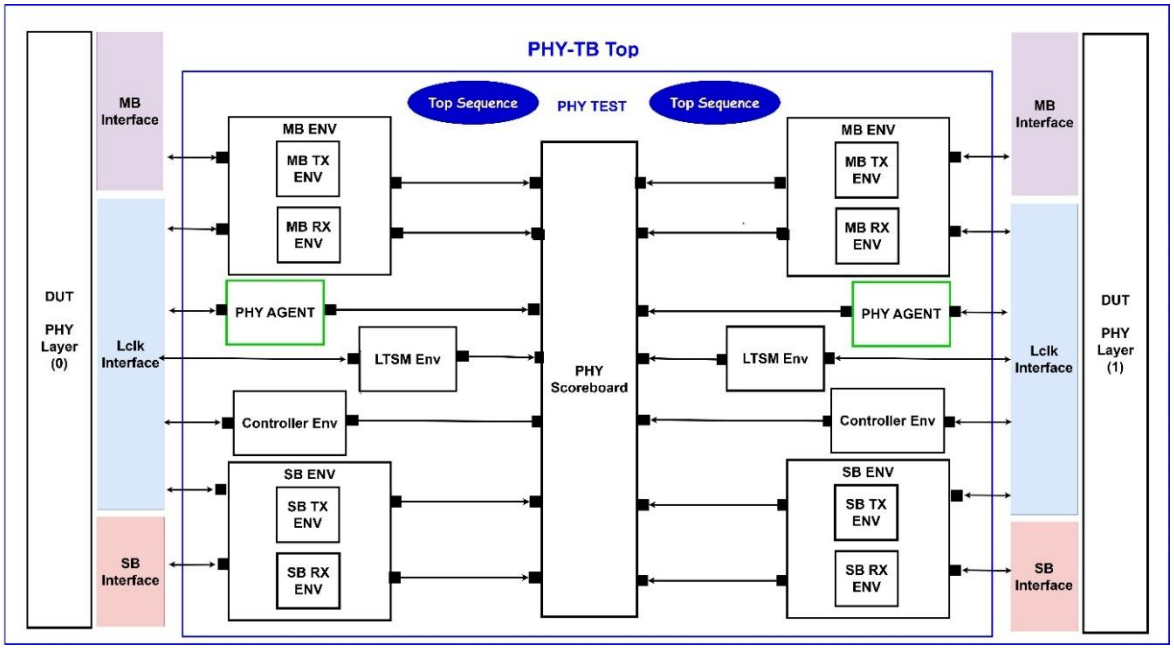


Figure 7-5 E2E UVM Architecture

At the core of the setup, the top-level DUT consists of two PHY modules instantiated to communicate with one another. We have two top level environment each includes an active agent, each dedicated to driving one of the two PHY instances. These agents are configured in the test and are responsible for simulating the behavior of independent dies, enabling realistic, die-to-die communication scenarios to be verified within a unified testbench enabling the test of lane blackouts,valid corruption and 2 dies compatibility.

Each of the two dies has its own dedicated interface, which is instantiated at the top level and passed down via a UVM configuration object. These configuration objects are retrieved within the test environment, ensuring that each die's corresponding block-level environment receives the appropriate interface handle.

Within each block-level environment, a specific modport of the shared interface is accessed, enabling precise and organized control over signal directionality and access. This structure ensures modularity, promotes reuse, and simplifies interaction between components by clearly separating responsibilities across modports, while maintaining consistency across the die-specific environments.

7.4. Sequences & Simulation Scenarios

The PHY-level UVM sequences are designed to emulate the expected protocol behaviors across different link states: **Reset**, **Bring-up**, and **Active**. Each sequence is highly configurable, allowing precise control over link conditions and fault injection, which is essential for verifying protocol robustness and recovery mechanisms under various edge cases.

7.4.1. Reset_flow Sequence

- **Purpose**:
  To mimic the layer's initial reset condition, ensuring the link starts from a clean, known state.

- **Behavior**:
  Activates the system asynchronous reset and drives `lp_state_req = Nop` for a protocol-defined number of cycles with optional suppression of `lp_linkerror`. This is done for Specification defined 4 ms residency timeout.

- **Configurable Elements**:
  - Initiator: This parameter is used to differentiate between both dies so the sequence knows which die it is driving. It only matters in this sequence to determine which `Reset_done` flag to assert after the sequence finishes as the test is waiting for it to kill the sequence.
- **Exit Condition**:
  The sequence ends once the required reset time elapses and the test-controlled `stop` signal is asserted via calling the `enough()` function.

7.4.2. Bringup_flow Sequence

- **Purpose**:
  Simulates the RESET to ACTIVE LTSM flow required to bring up the Link correctly via different tests and sideband messages up to reaching ACTIVE state.

- **Behavior**:
  After the PHY completes the required **4 ms in** RESET and meets all defined reset exit conditions, one die (initiator) will try to begin the link connection either by the adapter `lp_state_req = Active` or `sw_exit` conditions. Meanwhile, the other die will respond to this initiator die request request for bringing up the link. The sequence will then stimulate the correct adapter behavior for both dies while differentiating between them as both require different independent stimulus. For example, the idle die's adapter shouldn't ask Active entry on the RDI unless it sees the assertion of the `pl_inband_pres` signal. This Adapter emulation should run up until **both** dies enter ACTIVE.
- **Key Highlights**:
  - Highly depends on configurability & reactive stimulus methodology.
  - Uses other configurable options to introduce realistic conditions like lane corruption, blackout, and clock interruptions.
  - Supports asymmetric flow completion—dies can reach `ACTIVE` state independently.
  - Contains mechanisms for ending after finding fatal errors (either unintentional i.e. a Bug, or intentional) by the PHY layer and entering `Trainerror/Linkerror`.

- **Configurable Elements**:
  - The `Initiator` parameter defines whether the sequence is responsible to emulate the initiating die or the idle one.
  - `lane_blackout`, `lane_reverse`, `valid_corrupt`, `sb_lane_corrupt`, etc. are important configurations as each one triggers a different training path in the LTSM or even causes intentional failure during the training. These configurable error injecting elements simulate real-world issues and verify fault handling.

- **Exit Condition**:
  After completion of the RDI 4-way Active entry handshakes and both dies enter the ACTIVE state, each sequence sets their respective `Bringup_done` flag, and the test kills both sequences, or after entering `Trainerror` state if an intentional critical fault is triggered, each sequence sets their respective `entered_trainerror` flag, and the test kills both sequences.

Table 7-1: Conigurable Sequence Parameters

Parameter	Role
initiator	Lets the sequence know which die it is driving.
d2d_err	Lets the sequence know whether to allow the assertion of lp_linkerror or not (lp_linkerror means the adapter is informing the PHY layer that an adapter related fatal error has occurred and the phy should go to Trainerror/Linkerror and bring the link down).
reversal	Lets the sequence know that the current Mainband lanes connection are reversed.
lane_blackout [15:0]	Introduces blackouts (always zero) to specified mainband lanes.
lane_blackout_random [15:0]	Same as the previous one, but the blackouts start at a random point during simulation not at the start.
valid_corrupt	Introduce noise on the valid lane.
valid_corrupt_random	Same as the previous one, but the noise start at a random point during simulation not at the start.
sb_lane_blackout	Introduces a blackout (always zero) to the sideband data lane.
sb_lane_blackout_random	Same as the previous one, but the blackout start at a random point during simulation not at the start.
sb_lane_corrupt	Introduce noise on the sideband data lane.
sb_clk_blackout	Introduces a blackout (always zero) to the sideband clock lane.

7.4.3. Active_flow Sequence

- **Purpose**:
  Models link behavior during normal operation in ACTIVE state and handles potential transitions to PHYRETRAIN or TRAINERROR.

- **Behavior**:
  Stays in normal ACTIVE state behavior and performs multiple chunk transfers, then exits the ACTIVE state in one of two ways, either by a Retrain trigger or Linkerror trigger. A Retrain trigger is achieved either by inducing a framing error (PHY initiated retrain) or via an adapter request (D2D initiated retrain). A Linkerror trigger can be induced by d2d_err parameter or any lane blackout/corruption as discussed in the table above.

- **Key Features**:
  - Dual-mode exit paths:
    - `Retrain` triggered via `lp_state_req` and validated via `pl_state_sts`.
    - `Trainerror` recovery through stall/ack protocols.
  - Handles handshake completion before allowing link re-entry to `ACTIVE`.

- **Configurable Elements**:
  Same error injecting parameters as `Bringup_flow`, with additional tuning for number of active chunks, Active exit type of trigger and corruption triggering.

- **Exit Condition**:
  If a retrain exit was induced then the exit condition was that both dies perform the Active 4-way handshake and re-enter the active state, and if a trainerror exit was induce the exit condition was that both dies perform both the Trainerror and the Linkerror handshakes.

7.4.4. Simulation Scenarios

After developing the three main configurable sequences, they were utilized to flexibly and efficiently simulate various PHY Layer behaviors under controlled conditions. Enabling the injection of targeted faults or protocol variations to validate system robustness and protocol compliance. The following scenarios were defined and tested:

- Scenario 1:
  No injected errors during the Bring-up phase. Retrain is later triggered through an adapter request, verifying the correct transition behavior without any external fault influence.

Table 7-2: Scenario 1 Configurations for Both Dies.

Die 0 - Configurations	Die 1 - Configurations
# ==========	# ==========
# Bringup_flow Sequence Die (0) Configuration	# Bringup_flow Sequence Die (1) Configuration
# ==========	# ==========
# initiator : ON	# initiator : OFF
# d2d_err : OFF	# d2d_err : OFF
# stop : OFF	# stop : OFF
# reversal : OFF	# reversal : OFF
# lane_blackout : OFF	# lane_blackout : OFF
# lane_blackout_random : OFF	# lane_blackout_random : OFF
# valid_blackout : OFF	# valid_blackout : OFF
# valid_corrupt : OFF	# valid_corrupt : OFF
# valid_corrupt_random : OFF	# valid_corrupt_random : OFF
# sb_lane_corrupt : OFF	# sb_lane_corrupt : OFF
# sb_lane_blackout : OFF	# sb_lane_blackout : OFF
# sb_lane_blackout_random : OFF	# sb_lane_blackout_random : OFF
# sb_clk_blackout : OFF	# sb_clk_blackout : OFF
# Test with error : OFF	# Test with error : OFF
# ==========	# ==========
# ==========	# ==========
# ACTIVE_FLOW Sequence Die (0) Configuration	# ACTIVE_FLOW Sequence Die (1) Configuration
# ==========	# ==========
# initiator : ON	# initiator : OFF
# d2d_err : OFF	# d2d_err : OFF
# stop : OFF	# stop : OFF
# reversal : OFF	# reversal : OFF
# lane_blackout : OFF	# lane_blackout : OFF
# lane_blackout_random : OFF	# lane_blackout_random : OFF
# valid_blackout : OFF	# valid_blackout : OFF
# valid_corrupt : OFF	# valid_corrupt : OFF
# valid_corrupt_random : OFF	# valid_corrupt_random : OFF
# sb_lane_corrupt : OFF	# sb_lane_corrupt : OFF
# sb_lane_blackout : OFF	# sb_lane_blackout : OFF
# sb_lane_blackout_random : OFF	# sb_lane_blackout_random : OFF
# sb_clk_blackout : OFF	# sb_clk_blackout : OFF
# ==========	# ==========

- Scenario 2:
  Lanes 1, 3, 5, and 7 are disabled to induce a width degrade condition during Bring-up. After stabilization, a TRAINERROR is triggered via lp_linkerror to validate error handling and escalation.

Table 7-3: Scenario 2 Configurations for Both Dies.

Die 0 - Configurations	Die 1 - Configurations
# ==========	# ==========
# Bringup_flow Sequence Die (0) Configuration	# Bringup_flow Sequence Die (1) Configuration
# ==========	# ==========
# initiator : ON	# initiator : OFF
# d2d_err : OFF	# d2d_err : OFF
# stop : OFF	# stop : OFF
# reversal : ON	# reversal : ON
# lane_blackout : OFF	# lane_blackout : OFF
# lane_blackout_random : OFF	# lane_blackout_random : OFF
# valid_blackout : OFF	# valid_blackout : OFF
# valid_corrupt : OFF	# valid_corrupt : OFF
# valid_corrupt_random : OFF	# valid_corrupt_random : OFF
# sb_lane_corrupt : OFF	# sb_lane_corrupt : OFF
# sb_lane_blackout : OFF	# sb_lane_blackout : OFF
# sb_lane_blackout_random : OFF	# sb_lane_blackout_random : OFF
# sb_clk_blackout : OFF	# sb_clk_blackout : OFF
# Test with error : OFF	# Test with error : OFF
# ==========	# ==========
# ==========	# ==========
# ACTIVE_FLOW Sequence Die (0) Configuration	# ACTIVE_FLOW Sequence Die (1) Configuration
# ==========	# ==========
# initiator : ON	# initiator : OFF
# d2d_err : OFF	# d2d_err : OFF
# stop : ON	# stop : ON
# reversal : ON	# reversal : ON
# lane_blackout : OFF	# lane_blackout : OFF
# lane_blackout_random : OFF	# lane_blackout_random : OFF
# valid_blackout : OFF	# valid_blackout : OFF
# valid_corrupt : ON	# valid_corrupt : ON
# valid_corrupt_random : OFF	# valid_corrupt_random : OFF
# sb_lane_corrupt : OFF	# sb_lane_corrupt : OFF
# sb_lane_blackout : OFF	# sb_lane_blackout : OFF
# sb_lane_blackout_random : OFF	# sb lane blackout random : OFF
# sb_clk_blackout : OFF	# sb_clk_blackout : OFF
# ==========	# ==========

- Scenario 3:
  Lanes 8, 10, 12, and 14 are disabled, simulating a different width degrade configuration during Bring-up. Once the link stabilizes, a Retrain is triggered to test system recovery and handshake integrity.

Table 7-4: Scenario 3 Configurations for Both Dies.

Die 0 - Configurations	Die 1 - Configurations
# ========== Bringup_flow Sequence Die (0) Configuration ========== # initiator : ON # d2d_err : OFF # stop : OFF # reversal : OFF # lane_blackout : ON # lane_blackout_random : OFF # valid_blackout : OFF # valid_corrupt : OFF # valid_corrupt_random : OFF # sb_lane_corrupt : OFF # sb_lane_blackout : OFF # sb_lane_blackout_random : OFF # sb_clk_blackout : OFF # Test with error : OFF # ========== # ========== ACTIVE_FLOW Sequence Die (0) Configuration ========== # initiator : ON # d2d_err : OFF # stop : ON # reversal : OFF # lane_blackout : ON # lane_blackout_random : OFF # valid_blackout : OFF # valid_corrupt : OFF # valid_corrupt_random : OFF # sb_lane_corrupt : OFF # sb_lane_blackout : OFF # sb_lane_blackout_random : OFF # sb_clk_blackout : OFF # ==========	# ========== Bringup_flow Sequence Die (1) Configuration ========== # initiator : OFF # d2d_err : OFF # stop : OFF # reversal : OFF # lane_blackout : ON # lane_blackout_random : OFF # valid_blackout : OFF # valid_corrupt : OFF # valid_corrupt_random : OFF # sb_lane_corrupt : OFF # sb_lane_blackout : OFF # sb_lane_blackout_random : OFF # sb_clk_blackout : OFF # Test with error : OFF # ========== # ========== ACTIVE_FLOW Sequence Die (1) Configuration ========== # initiator : OFF # d2d_err : OFF # stop : ON # reversal : OFF # lane_blackout : ON # lane_blackout_random : OFF # valid_blackout : OFF # valid_corrupt : OFF # valid_corrupt_random : OFF # sb_lane_corrupt : OFF # sb_lane_blackout : OFF # sb_lane_blackout_random : OFF # sb_clk_blackout : OFF # ==========

- Scenario 4:
  Lane reversal is enabled and correctly detected during Bring-up, with no additional errors injected. After entering Active, a Retrain is initiated via valid frame errors occurring in the Mainband (MB).

Table 7-5: Scenario 4 Configurations for Both Dies.

Die 0 - Configurations	Die 1 - Configurations
# ==========	# ==========
# Bringup_flow Sequence Die (0) Configuration	# Bringup_flow Sequence Die (1) Configuration
# ==========	# ==========
# initiator : ON	# initiator : OFF
# d2d_err : OFF	# d2d_err : OFF
# stop : OFF	# stop : OFF
# reversal : OFF	# reversal : OFF
# lane_blackout : ON	# lane_blackout : ON
# lane_blackout_random : OFF	# lane_blackout_random : OFF
# valid_blackout : OFF	# valid_blackout : OFF
# valid_corrupt : OFF	# valid_corrupt : OFF
# valid_corrupt_random : OFF	# valid_corrupt_random : OFF
# sb_lane_corrupt : OFF	# sb_lane_corrupt : OFF
# sb_lane_blackout : OFF	# sb_lane_blackout : OFF
# sb_lane_blackout_random : OFF	# sb_lane_blackout_random : OFF
# sb_clk_blackout : OFF	# sb_clk_blackout : OFF
# Test with error : OFF	# Test with error : OFF
# ==========	# ==========
# ==========	# ==========
# ACTIVE_FLOW Sequence Die (0) Configuration	# ACTIVE_FLOW Sequence Die (1) Configuration
# ==========	# ==========
# initiator : ON	# initiator : OFF
# d2d_err : OFF	# d2d_err : OFF
# stop : ON	# stop : ON
# reversal : OFF	# reversal : OFF
# lane_blackout : ON	# lane_blackout : ON
# lane_blackout_random : OFF	# lane_blackout_random : OFF
# valid_blackout : OFF	# valid_blackout : OFF
# valid_corrupt : OFF	# valid_corrupt : OFF
# valid_corrupt_random : OFF	# valid_corrupt_random : OFF
# sb_lane_corrupt : OFF	# sb_lane_corrupt : OFF
# sb_lane_blackout : OFF	# sb_lane_blackout : OFF
# sb_lane_blackout_random : OFF	# sb lane blackout random : OFF
# sb_clk_blackout : OFF	# sb_clk_blackout : OFF
# ==========	# ==========

- Scenario 5:
  Lane reversal with Width Degradation are enabled and correctly detected during Bring-up, with no additional errors injected. After entering Active, a sb_lane_corrupt is introduced (increased the BER) to validate error handling and escalation.

Table 7-6: Scenario 5 Configurations for Both Dies.

Die 0 - Configurations	Die 1 - Configurations
# ==========	# ==========
# Bringup_flow Sequence Die (0) Configuration	# Bringup_flow Sequence Die (1) Configuration
# ==========	# ==========
# initiator : ON	# initiator : OFF
# d2d_err : OFF	# d2d_err : OFF
# stop : OFF	# stop : OFF
# reversal : ON	# reversal : ON
# lane_blackout : ON	# lane_blackout : ON
# lane_blackout_random : OFF	# lane_blackout_random : OFF
# valid_blackout : OFF	# valid_blackout : OFF
# valid_corrupt : OFF	# valid_corrupt : OFF
# valid_corrupt_random : OFF	# valid_corrupt_random : OFF
# sb_lane_corrupt : OFF	# sb_lane_corrupt : OFF
# sb_lane_blackout : OFF	# sb_lane_blackout : OFF
# sb_lane_blackout_random : OFF	# sb_lane_blackout_random : OFF
# sb_clk_blackout : OFF	# sb_clk_blackout : OFF
# Test with error : OFF	# Test with error : OFF
# ==========	# ==========
# ==========	# ==========
# ACTIVE_FLOW Sequence Die (0) Configuration	# ACTIVE_FLOW Sequence Die (1) Configuration
# ==========	# ==========
# initiator : ON	# initiator : OFF
# d2d_err : OFF	# d2d_err : OFF
# stop : OFF	# stop : OFF
# reversal : ON	# reversal : ON
# lane_blackout : ON	# lane_blackout : ON
# lane_blackout_random : OFF	# lane_blackout_random : OFF
# valid_blackout : OFF	# valid_blackout : OFF
# valid_corrupt : OFF	# valid_corrupt : OFF
# valid_corrupt_random : OFF	# valid_corrupt_random : OFF
# sb_lane_corrupt : ON	# sb_lane_corrupt : ON
# sb_lane_blackout : OFF	# sb_lane_blackout : OFF
# sb_lane_blackout_random : OFF	# sb_lane_blackout_random : OFF
# sb_clk_blackout : OFF	# sb_clk_blackout : OFF
# ==========	# ==========

7.5. Simulation Results

The simulation transcript below showcases the execution of multiple PHY-layer flows, each starting from a reset sequence and progressing through bring-up, active operation, and various termination conditions. Across these runs, different protocol paths are exercised—including clean bring-up followed by retrain, bring-up with lane reversal and valid frame corruption leading to retrain, bring-up with induced width degradation and triggering a trainerror condition. These flows collectively demonstrate the PHY's robustness across diverse operational and error-handling scenarios.

```
# [0] CONFIGURATION START: Zero Errors During Bringup & Adapter Initiated Retrain Exit After 1000 cycles in ACTIVE
# [22623000] DIE 1 - IDLE DIE -  has entered active state
# [22630000] DIE 0 - INITIATOR DIE -  has entered active state
# [23792000] DIE 1 - IDLE DIE -  has entered Retrain state
# [24070000] DIE 0 - INITIATOR DIE -  has entered Retrain state
# [37007000] DIE 1 - IDLE DIE -  has entered active state
# [37015000] DIE 0 - INITIATOR DIE -  has entered active state
# [37015000] CONFIGURATION START: Lane Reversal Setup & Framing Error InitiatedRetrain Exit After 1000 cycles in ACTIVE
# [60282000] DIE 0 - INITIATOR DIE -  has entered active state
# [60290000] DIE 1 - IDLE DIE -  has entered active state
# [62135000] DIE 1 - IDLE DIE -  has entered Retrain state
# [62322000] DIE 0 - INITIATOR DIE -  has entered Retrain state
# [75259000] DIE 1 - IDLE DIE -  has entered active state
# [75268000] DIE 0 - INITIATOR DIE -  has entered active state
# [75268000] CONFIGURATION START: LSB Lanes Blackout During Bringup & Trainerror Exit After 1000 cycles in ACTIVE
# [97888000] DIE 1 - IDLE DIE -  has entered active state
# [97895000] DIE 0 - INITIATOR DIE -  has entered active state
# [98910000] a die entered trainerror state
# [98912000] both dies entered trainerror state
# [98912000] CONFIGURATION START: MSB Lanes Blackout During Bringup & Retrain Exit After 1000 cycles in ACTIVE
# [121533000] DIE 1 - IDLE DIE -  has entered active state
# [121540000] DIE 0 - INITIATOR DIE -  has entered active state
# [122739000] DIE 1 - IDLE DIE -  has entered Retrain state
# [122834000] DIE 0 - INITIATOR DIE -  has entered Retrain state
# [135772000] DIE 1 - IDLE DIE -  has entered active state
# [135780000] DIE 0 - INITIATOR DIE -  has entered active state
```

Figure 7-6: Final Transcript for 4 scenarios at once.

7.5.1. Bugs Found & Fixed

1 sbinit_out_of_reset Handling

Description:
The receiver is designed to detect the `sbinit_out_of_reset` message through the pattern detector. Upon detection, the **pattern generator is disabled** and the **main receiver is enabled.** However, the receiver logic does not internally recognize the message code associated with this specific message. The transmitter continues to send the `sbinit_out_of_reset` message until its corresponding receiver detects it.

In the observed scenario, **die-0** successfully detects the message, disables its pattern detector, and enables the main receiver. Meanwhile, **die-1** continues to transmit the same message. Since die-0's receiver does not recognize the message code, it erroneously interprets these repeated messages as invalid, resulting in the assertion of a **false error flag.**

Fix:
The receiver logic was updated to **explicitly ignore** the `sbinit_out_of_reset` message code after initial detection. This prevents the error flag from being incorrectly asserted during ongoing reception of this message from the remote die.

2 Transmitter Generator Path Selection Handling

Description:
In the PHY module, after receiving the sbinit_out_of_reset message, the path_sel signal is expected to guide the transition out of the sbinit state. However, when path_sel is deasserted, the module does not immediately exit the sbinit state. Instead, it waits for the partner module to signal sb_exit.
This led to unintended behavior where, upon deassertion of path_sel and before the actual state transition occurs, the transmitter incorrectly restarts the initialization flow and reasserts path_sel, rather than waiting for a valid sbinit state transition.
Fix:
The PHY module was modified to register the path_sel value internally and prevent re-entry into the initialization flow. The flow will now only proceed after detecting a valid state transition out of sbinit, ensuring proper synchronization with the partner module.

3 Test_Resp- assertion for tx

Description
The test-resp signal is not asserted in the VALTRAIN_CENTER test so the main band hang in this stage waiting for the signal that initate the test and the module partner send that the test has failed without making the test
Fix
The signal is asserted in the valtrain center test

4 lane reversal-detection

Description
When lane reversal test is repeated after lane reversal is detected the signal is de-asserted while it must stay asserted so that the main band operates correctly
Fix:
Lane reversal is registered until the die start training again
Wave:

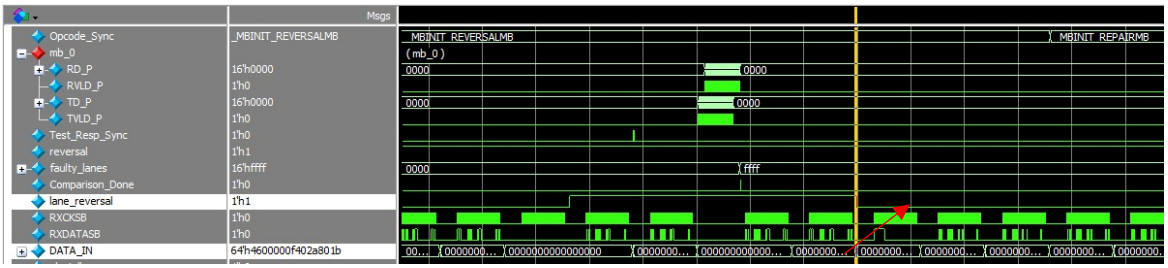


Figure 7-7 lane reversal bug

5 Arbitration Issue

Description
The arbiter is capable of bypassing a header with its data only 2 consecutive headers is not allowed while module and module partner send two consecutive message without data
Fix:
Arbiter now by pass 2 consecutive message without data

6 Valid Bit Contradiction

Description
The module partner sends the result of valid bit in message in bit 45 while the module checks the message bit number 44 so the module detects valid test result always as a failed test
Fix:
Correct the bit number in module

7.5.2. *Coverage Achieved*

7.6.2.1. Code Coverage

The conditional coverage at the top-level environment is relatively low, mainly due to the lower coverage reported for the module and module partner components. This is largely because the stimulus used at the top level is designed to reflect real-world use cases, as all the corner cases were tested by block level stimulus.
Even though we introduce faults—like corrupting RXDATA-SB or forcing a blackout on lanes we can't push the module logic as hard as we do at the block level. In block-level tests, we have more control and can directly drive specific conditions, which allows us to exercise more of the design's internal decision-making paths. But at the top level, those kinds of directed scenarios are harder to isolate and apply without breaking the overall flow.
This is a natural trade-off: the top-level test ensures everything works together in a realistic way, while the block-level tests help fill in the gaps by stressing specific logic. Together, they provide a balanced and thorough verification strategy.

```
================================================================================
=== Design Unit: work.PHY_TOP
================================================================================
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                      2604      1961       643    75.30%
    Conditions                    1267       411       856    32.43%
    Expressions                    391       279       112    71.35%
    FSM States                      61        58         3    95.08%
    FSM Transitions                151        77        74    50.99%
    Statements                    5867      4285      1582    73.03%
    Toggles                      43699     36852      6847    84.33%


Total Coverage By Design Unit (filtered view): 68.93%
```

Figure 7-8 E2E code coverage

7.6.2.2. Functional Coverage

The functional coverage analysis reflects a generally solid verification effort, with several modules achieving coverage levels above 90% as shown in figure. However,
coverage gaps observed at the top level do not indicate design issues, but rather reflect the limitations of system-level stimulus generation. In contrast:
These same modules, when verified independently at the block level, achieved near-complete coverage, including full stimulation of error paths and rare transitions.
This was possible because the block-level tests used directed and constrained stimulus specifically designed to target edge cases—though such stimulus may not be protocol-compliant or realistic in a full system context.
The top-level verification, by contrast, prioritizes realistic, end-to-end behavior and therefore does not aggressively drive these edge conditions. As a result, while the top-level environment verifies the correctness of interactions and integration, it may leave some internal control paths unexercised.

Name	Class Type	Coverage	Goal	% of Goal	Status	Included
/rx_cov_coll_pkg/rx_cov_coll		94.44%				
TYPE rx_cvg		94.44%	100	94.44%		✓
/coverage_collector_pkg/ltsm_coverage		93.84%				
TYPE cvrgp		93.84%	100	93.84%		✓
/sb_tx_coverage/sb_tx_coverage		89.30%				
TYPE tx_cg		89.30%	100	89.30%		✓
/c_module_coverage_collector_pkg/c_module_coverage		86.82%				
TYPE cg_data_out		88.46%	100	88.46%		✓
TYPE cg_data_in		85.18%	100	85.18%		✓
/coverage_collector2_pkg/SB_RX_coverage		86.47%				
TYPE cvrgp_2		86.47%	100	86.47%		✓
/coll_pkg/collector		78.86%				
TYPE cvrgp		78.86%	100	78.86%		✓
/tx_coverage/tx_coverage		74.39%				
TYPE tx_cg_mbclk		74.39%	100	74.39%		✓
/c_module_p_coverage_collector_pkg/c_module_p_coverage		72.10%				
TYPE cvrgp		72.10%	100	72.10%		✓

Figure 7-9 E2E functional coverage

Chapter 8: FPGA Flow

8.1. FPGA Overview

The FPGA implementation for this project was performed using the **Vivado Design Suite**, a comprehensive toolchain for RTL-based digital design. Below are the important aspects of the design environment and configuration:

Design Tool: Xilinx Vivado Design Suite
Version Used: Vivado **2018.2**
Target FPGA Device: **XCKU15P-FFVE1517-2-E**, a member of the **Kintex UltraScale+** family, optimized for high-speed and data-intensive applications
Design Language: **Verilog HDL**
Development Focus: RTL design, synthesis, implementation, timing simulation, and partial verification

This setup provided the necessary capabilities to handle high-throughput interfaces and complex control logic throughout the project.

The typical Vivado FPGA design flow includes the following steps:

1. **Design Entry**
   all RTL design sources written in **System Verilog** were added to the Vivado project. These included structural and behavioral modules describing the core functionality of the system. In addition, a set of **constraints** was provided through an **XDC (Xilinx Design Constraints)** file, which precisely defined the system-level requirements such as clock frequencies and reset signals.

2. **Simulation**
   Pre-synthesis behavioral simulation was performed using the **Vivado Simulator**. This step verified the logical correctness of the design and ensured that the RTL code behaved as expected under functional test scenarios before any transformation into gate-level representation.

3. **Synthesis**
   In this step, the RTL design is translated into a **gate-level netlist**, performing logic optimization, technology mapping, and resource estimation. The synthesis process produced detailed reports on **timing**, **area utilization**, **power**, and **logical hierarchy**, which are essential for evaluating design feasibility and preparing for implementation.

4. **Post synthesis functional simulation**
   This simulation validated the synthesized netlist against the original design behavior. It ensured that no functional errors were introduced during synthesis. The netlist was simulated using the same testbench as in the pre-synthesis stage to verify output correctness, which is achieved as after performing this step, it gave out the same expected results as that of Questa simulation.

5. **Post synthesis timing simulation**
   This step analyzed the **propagation delays** and **timing behavior** of the design by incorporating gate-level delays. It was used to verify that the design met the specified **setup and hold times**, particularly under worst-case conditions, and to ensure the design would function correctly when implemented on hardware.

6. **Implementation**
   Implementation consisted of two main steps:
   - **Placement**, which assigned logic elements (LUTs, registers, etc.) to specific physical locations on the FPGA fabric.
   - **Routing**, which connected these placed elements using the available interconnect resources.

   During this stage, Vivado conducted a **Static Timing Analysis (STA)** to evaluate whether all defined timing constraints were met. If any violations were found, adjustments were made to improve timing closure.
7. **Bitstream Generation**
   After successful implementation and timing closure, a **configuration bitstream (.bit)** file is generated. This file encoded the complete logic configuration of the design, which could then be loaded into the FPGA device.

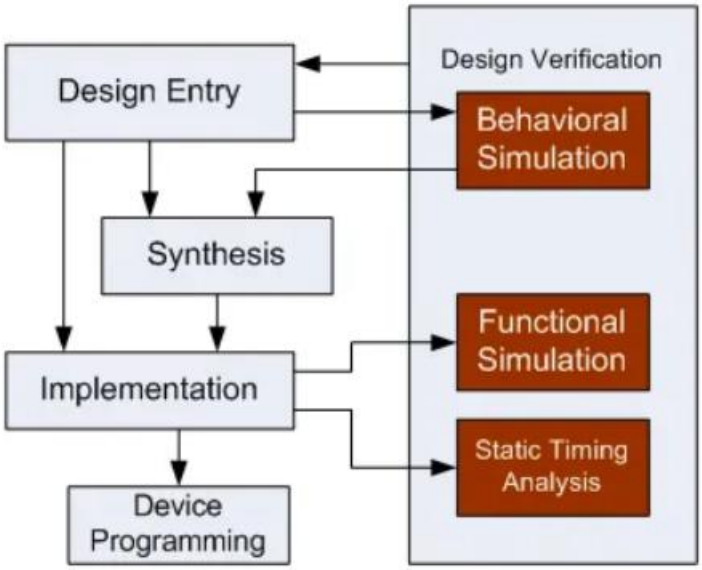


Figure 8-1: FPGA Flow

8.2. FPGA Flow

We proceeded through the design flow and successfully addressed the issues that emerged during the early stages, including synthesis and implementation. However, we encountered problems during the post-synthesis timing simulation phase, which prevented us from advancing further. Due to time constraints, we were unable to complete the debugging process or proceed to bitstream generation. Resolving these timing issues remains part of our planned future work to ensure the system's full functionality and timing closure.

In the upcoming subsections, we will walk through each step of the design flow in detail and highlight the key violations that arose during the process. This analysis will focus on the most critical issues encountered, their root causes, and how they impacted progression through the flow.

8.2.1. Linting

Linting is a **static analysis process** that checks your HDL (Hardware Description Language) code—typically Verilog or VHDL—for common **coding errors, style issues, or potential logic problems**

before simulation or synthesis. It does not execute the code but analyzes it line by line to ensure it adheres to syntax, design rules, and **good coding practices**.

8.2.1.1 Why Linting Is Important in the FPGA Flow:

Early Error Detection: Linting catches syntax errors, undefined signals, mismatched ports, unconnected modules, and unused logic before deeper stages like simulation or synthesis—saving time and debugging effort.
Improves Code Quality and Readability: It enforces naming conventions, modularity, and style rules that make the code easier to maintain, review, and reuse.
Prevents Functional Bugs: Many subtle bugs, such as unintended latches, incorrect sensitivity lists, or asynchronous resets, can be detected by linting tools.
Reduces Risk of Late-Stage Failures: Catching problems early in the flow avoids spending hours debugging during timing simulation or implementation.
Tool Compatibility Check: Linting ensures the code is well-formed and compatible with synthesis and implementation tools, avoiding downstream toolchain errors.

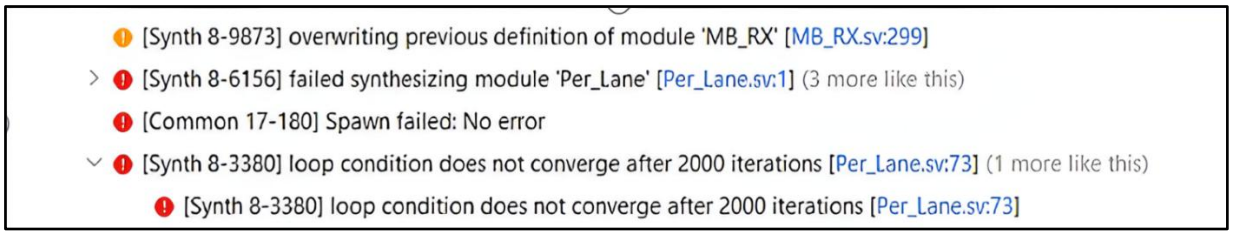


Figure 8-2 Linting Violations 1

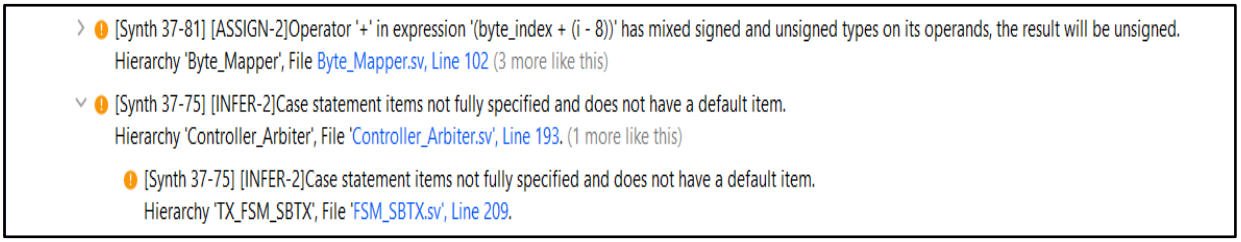


Figure 8-3 Linting Violations 2

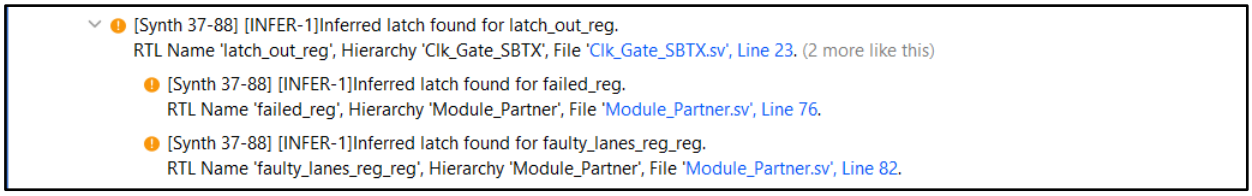


Figure 8-4 Linting Violations 3

Fig. 8-2 This error occurred due to a for loop whose iteration count was based on a parameter representing active lane width (for handling lane degradation) where This issue arises because **loop bounds in synthesizable Verilog must be statically determinable at compile time**. The Vivado Linting tool, which performs static code analysis, requires fixed loop limits to ensure safe and bounded logic generation. Since the loop condition was dependent on a s**imulation-time parameter or runtime signal (which may vary dynamically)**, the analysis could not confirm whether the loop would ever terminate, hence failing to converge.

Solution: configure the Parameter to be a Local Parameter to the Module!

Fig. 8-3 This figure shows 2 Linting Warnings:

- **Mixed Signed-ness in Arithmetic**: The first issue involves adding a signed and an unsigned value together. Specifically, the loop variable in a `for` loop—being signed by default—was added to an unsigned logic variable, leading to unintended arithmetic behavior and potential synthesis mismatches.
- **Missing Default Case in FSM**: The second issue stems from omitting a `default` case in a `case` statement within a finite state machine (FSM). Without a default condition, undefined behavior may occur if the FSM enters an unhandled state, which could compromise functional correctness and synthesis safety.

Fig. 8-4 The Clock Gate Module in the Sideband Transmitter triggered a **latch inference warning** during linting, but this behavior is **intentional!**

The module is designed to generate a **strobe signal** by conditionally passing the clock when an enable signal is asserted and gating it otherwise.

Clock gating is commonly implemented using a **latch and an AND gate** to ensure glitch-free clock transitions. This structure must be routed precisely to maintain timing integrity and avoid clock skew issues. In this design, the module was initially described using an **always_latch** block to explicitly define the latch behavior. This allows the inferred latch to later be replaced by a **dedicated FPGA clock gate primitive (BUFGCE)** during synthesis and implementation using the Vivado FPGA library, ensuring proper integration into the clocking architecture.

Solution:

```
//internal signals
logic latch_out;

always_latch begin

    if (!CLK)
        latch_out = Enable | Load;

end

assign SBTX_CLK = latch_out & CLK;
```

```
//------------------------------------------------------------------
// 2. Gated clock spine (BUFGCE)  ?  internal net clk_bufg
//------------------------------------------------------------------
wire clk_bufg;   // ***wire, one driver only***

BUFGCE u_bufgce (
    .I  (CLK),
    .CE (clk_en),
    .O  (clk_bufg)
);

//------------------------------------------------------------------
// 3. DDR forwarder (ODDRE1)
//------------------------------------------------------------------
wire sbtx_clk_int;   // ***wire, one driver only***

ODDRE1 u_oddre1 (
    .Q  (sbtx_clk_int),
    .C  (clk_bufg),
    .D1 (1'b1),
    .D2 (1'b0),
    .SR (1'b0)
);
```

Figure 8-5: Clock Gating Latch Solution

BUFGCE is used to implement a glitch-free clock gate for the strobe signal inside the FPGA. It allows enabling or disabling the strobe clock cleanly by gating the input clock with a clock enable (CE) signal, ensuring low skew and avoiding glitches that could corrupt timing-critical control signals like strobes.

Once the strobe clock is gated and stabilized by the BUFGCE, the ODDRE1 DDR output register is used to **forward the strobe signal off-chip to the Die Partner**. ODDRE1 outputs the strobe on both rising and falling edges of the reference clock, effectively doubling the data rate and providing a well-controlled duty cycle and minimal clock-to-output delay. This is crucial for the high-speed,

source-synchronous interfaces typical in UCIe, where the strobe acts as a timing reference for data capture on the receiving side.

Together, BUFGCE and ODDRE1 enable generation and forwarding of a high-quality strobe signal: BUFGCE ensures clean, glitch-free gating of the internal strobe clock, and ODDRE1 provides precise, low-jitter DDR output of that strobe to the Die Partner, facilitating reliable synchronization in UCIe chiplet communication.

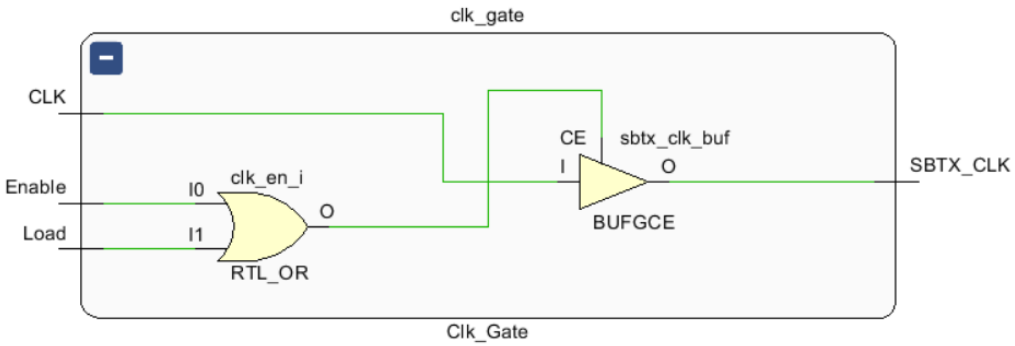


Figure 8-5 BUFGCE

8.2.2 Synthesis

CDC

Before running synthesis, it was essential to verify that all **Clock Domain Crossings (CDCs)** in the design were properly handled. This involved analyzing the interaction between modules operating under different clocks and examining the nature of the crossing signals. The design includes **three clock domains**:

MBCLK (approx. 2 GHz) — the fastest,
LCLK (1 GHz) the system clock, and
SBCLK (800 MHz) — used for control message handling and considered the slowest.

The system is composed of four main blocks:

Controller (LCLK),
MB block (MBCLK),
SB block (uses both LCLK and SBCLK),
LTSM (LCLK).

Each crossing case was reviewed, and appropriate **synchronizers** were selected based on whether the signal was a level, pulse or data bus. For FIFO-based crossings, the handling was already discussed in detail in the Design chapter. To assist the synthesis tools and ensure proper timing exception handling, the "`set_clock_groups -asynchronous`" constraint was applied in the **XDC file** to declare asynchronous relationships between these clock domains.

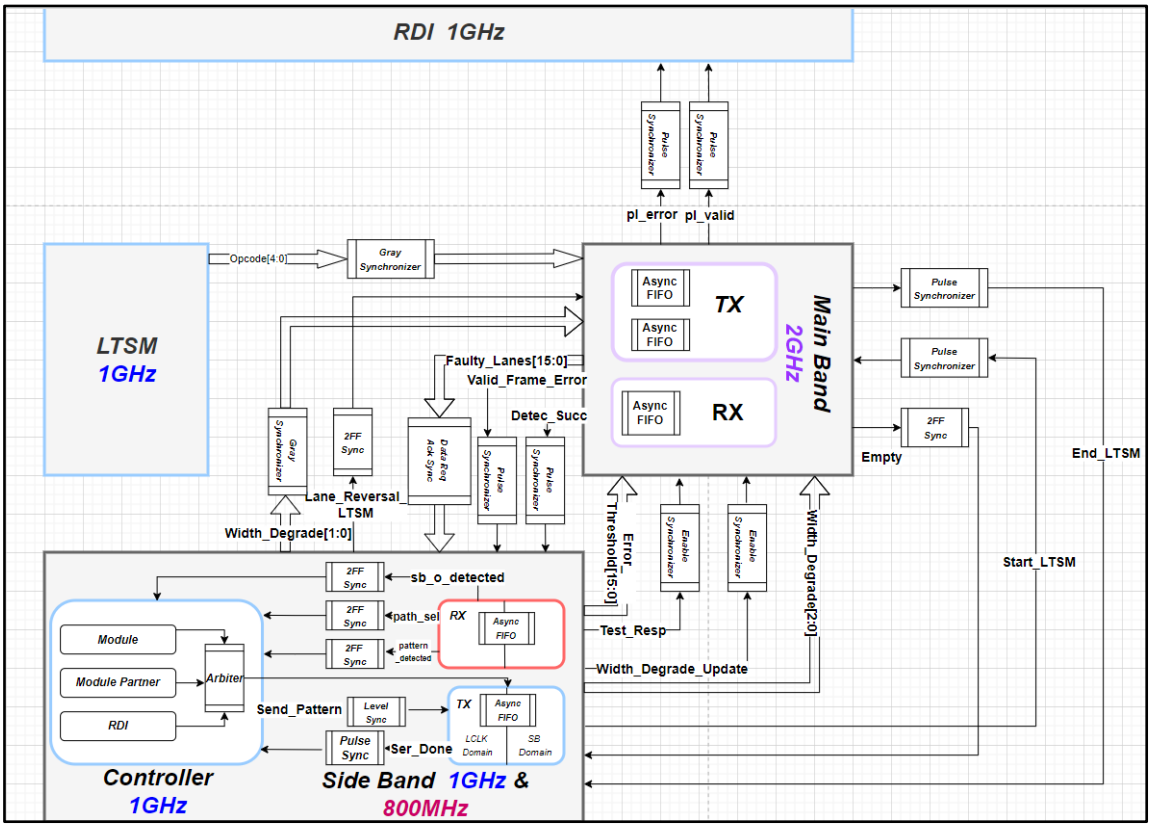
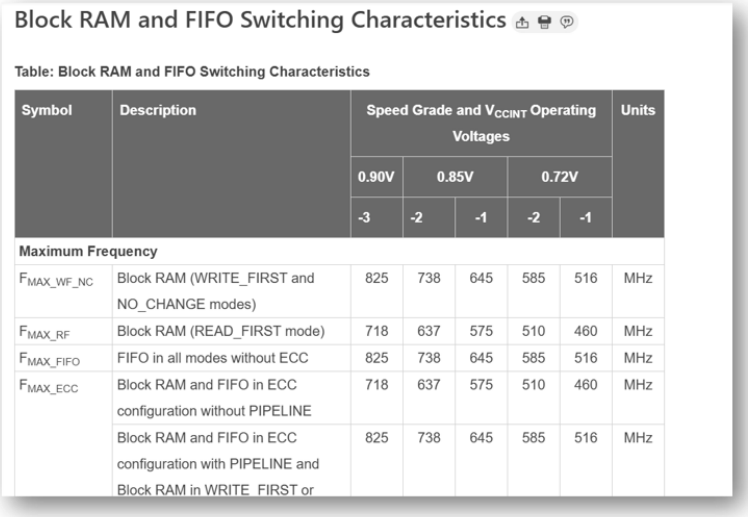


Figure 8-6 Whole System CDC Paths

Frequency Compatibility

The second critical point considered in the design was **frequency compatibility**. Initially, the system was designed to operate at high frequencies such as **1 GHz**; however, this proved unsuitable for the **Kintex UltraScale+** FPGA family. Although a **Virtex** device might support such high internal clocking, the Kintex UltraScale+ series has limitations—**internal LUTs, BRAMs, and logic cannot reliably operate above ~700 MHz**. While it is technically feasible to achieve the target **32 Gbps data rate** using **SERDES, DDR techniques**, and **dedicated FPGA IPs**, the internal fabric imposes a bottleneck. To resolve this, we applied a **frequency degradation factor of 5**, reducing the internal logic frequency to a level compatible with the target FPGA device, while still meeting system performance requirements. This resulted in a 400 MHz MB clock, a 200 MHz LCLK, and a 160 MHz SB clock—frequencies that fall within the reliable operating range of the Kintex UltraScale+ FPGA.



Block RAM and FIFO Switching Characteristics

Table: Block RAM and FIFO Switching Characteristics

Symbol	Description	Speed Grade and V_{CCINT} Operating Voltages					Units
		0.90V	0.85V		0.72V		
		-3	-2	-1	-2	-1	
Maximum Frequency							
$F_{MAX_WF_NC}$	Block RAM (WRITE_FIRST and NO_CHANGE modes)	825	738	645	585	516	MHz
F_{MAX_RF}	Block RAM (READ_FIRST mode)	718	637	575	510	460	MHz
F_{MAX_FIFO}	FIFO in all modes without ECC	825	738	645	585	516	MHz
F_{MAX_ECC}	Block RAM and FIFO in ECC configuration without PIPELINE	718	637	575	510	460	MHz
	Block RAM and FIFO in ECC configuration with PIPELINE and Block RAM in WRITE_FIRST or	825	738	645	585	516	MHz

Figure 8-7 Frequency Limitation

Problems Faced:

```
INFO: [Synth 8-6157] synthesizing module 'FIFO_SBTX' [C:/Users/mm/Desktop/10.PHY_Top/10.PHY_Top/FIFO_SBTX.sv:9]
    Parameter FIFO_WIDTH bound to: 64 - type: integer
    Parameter FIFO_DEPTH bound to: 64 - type: integer
    Parameter max_fifo_addr bound to: 6 - type: integer
WARNING: [Synth 8-5788] Register mem_reg in module FIFO_SBTX is has both Set and reset with same priority. This may cause simulation mismatches. Consider rewriting code
WARNING: [Synth 8-4767] Trying to implement RAM 'mem_reg' in registers. Block RAM or DRAM implementation is not possible; see log for reasons.
Reason is one or more of the following :
    1: RAM is sensitive to asynchronous reset signal. this RTL style is not supported.
RAM "mem_reg" dissolved into registers
```

Figure 8-8 Synthesis Problem 1

FPGA block RAMs do not support asynchronous resets on their internal memory cells. Using asynchronous resets in your RTL code can prevent synthesis tools like Vivado from correctly inferring block RAMs, leading to inefficient implementations or functional problems. This is because actual BRAM hardware only supports synchronous resets or initialization, so asynchronous reset signals cause incompatibility with the FPGA's dedicated memory resources.

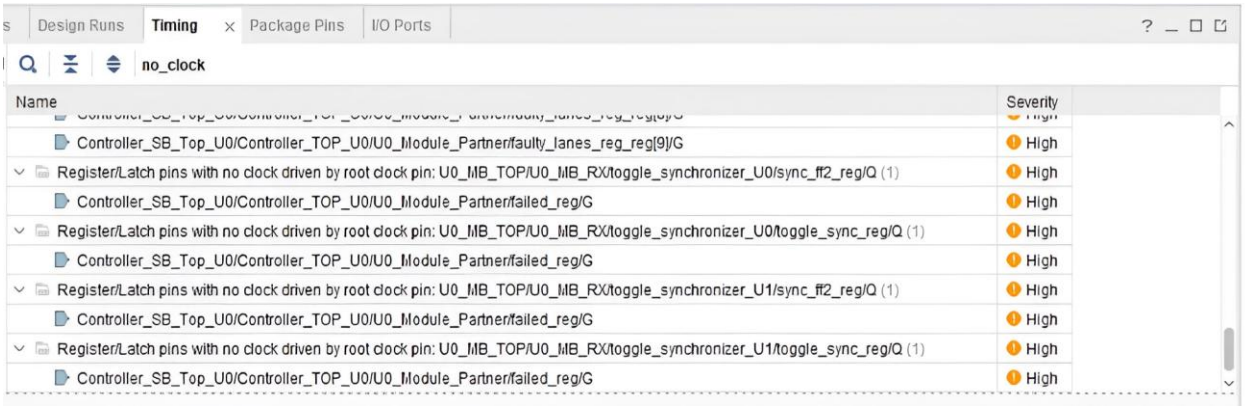


Figure 8-9 Synthesis Problem 2

During synthesis, Vivado detected that certain variables inferred unintended **latches**, and it flagged the associated timing paths as having **no clock driven by a root clock pin**. This happened because those variables were written inside **combinational `always @*` blocks** but were **not assigned in all conditional branches**, prompting the synthesis tool to infer latches to hold their values. The **outputs of synchronizers**, which are asynchronous, were used as control conditions in these blocks, causing Vivado to interpret the inferred latch **clocks as being derived from asynchronous signals** rather than from a proper global clock. As a result, these paths were marked as **having undefined or**

invalid clocking, which not only impacts timing analysis but can also lead to unreliable or unpredictable behavior in the final implementation.

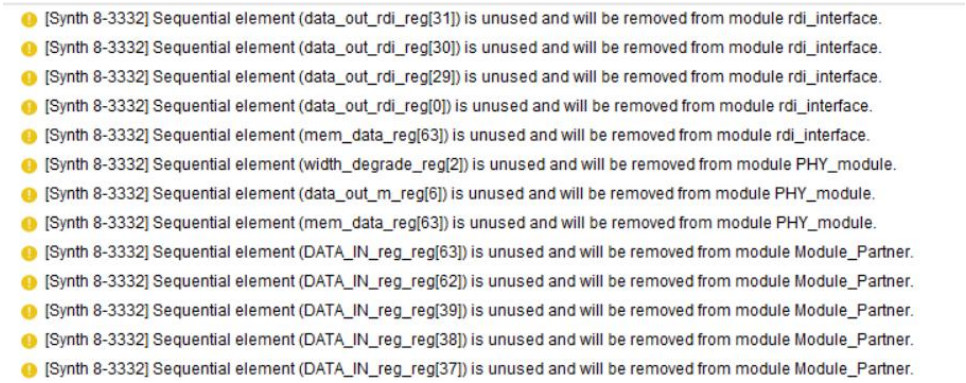


Figure 8-10 Synthesis Problem 3

During synthesis, several warnings were generated; however, **these do not affect the correctness or intended behavior** of the design. One set of warnings relates to **reserved bits** that are mandated by the protocol interface but are **not actively used or validated** in the current implementation, leading Vivado to flag them as unused inputs. Another set of warnings stems from the **bit masking strategy** used to streamline control message generation from the PHY. Specifically, **certain bits are intentionally left unassigned** by the modules generating the PHY messages, so they can later be inserted by a dedicated **Information Insertion Entity**. While Vivado interprets these unassigned bits as unused logic, this behavior is **fully intentional** and **compliant with the design specification and** thus **does not represent a design or synthesis issue**.

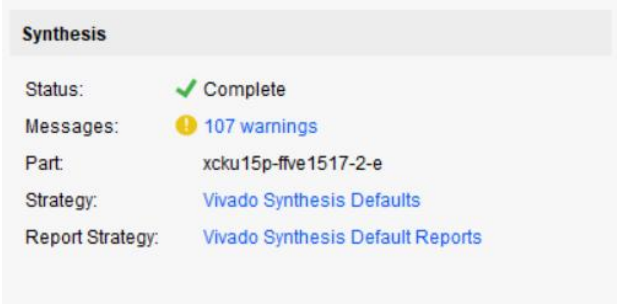


Figure 8-11 Synthesis Final Status

8.2.3 Implementation

Clock Port Assignment Constraints and GC Pin Usage in FPGA Implementation

One important consideration during the **Implementation phase** was the proper assignment of **clock ports** to dedicated **Global Clock (GC) pins** on the FPGA. Not all FPGA I/O pins are suitable for driving clock networks, and using inappropriate pins can result in failed implementation or degraded timing performance. Specifically, for **single-ended clock signals** (as opposed to differential clocks with P and N pairs), it is essential to assign the clock to a **P-type GC pin** while **leaving the corresponding N-type pin unused**. This ensures compatibility with the FPGA's internal clock routing resources and avoids any misconfiguration that could affect clock distribution or timing closure.

E18	IO_L15N_T2L_N5_AD11N_71	2L	71	HP	NA
E19	IO_L15P_T2L_N4_AD11P_71	2L	71	HP	NA
F16	IO_L14N_T2L_N3_GC_71	2L	71	HP	NA
F17	IO_L14P_T2L_N2_GC_71	2L	71	HP	NA
F18	IO_L13N_T2L_N1_GC_QBC_71	2L	71	HP	NA
F19	IO_L13P_T2L_N0_GC_QBC_71	2L	71	HP	NA
G19	IO_L12N_T1U_N11_GC_71	1U	71	HP	NA
H19	IO_L12P_T1U_N10_GC_71	1U	71	HP	NA
G16	IO_L11N_T1U_N9_GC_71	1U	71	HP	NA
G17	IO_L11P_T1U_N8_GC_71	1U	71	HP	NA
F14	IO_L10N_T1U_N7_QBC_AD4N_71	1U	71	HP	NA
G15	IO_L10P_T1U_N6_QBC_AD4P_71	1U	71	HP	NA
G14	IO_L9N_T1L_N5_AD12N_71	1L	71	HP	NA

Figure 8-12: Dedicated Clock Pins (1).

[Place 30-876] Port 'sb_clk' is assigned to PACKAGE_PIN 'F18' which can only be used as the N side of a differential clock input.
Please use the following constraint(s) to pass this DRC check:
set_property CLOCK_DEDICATED_ROUTE FALSE [get_nets {sb_clk_IBUF_inst/O}]

[Place 30-99] Placer failed with error: 'Implementation Feasibility check failed, Please see the previously displayed individual error or warning messages for more details.'
Please review all ERROR, CRITICAL WARNING, and WARNING messages during placement to understand the cause for failure.

Figure 8-14: Dedicated Clock Pins (2).

I/O Pin Limitation and Interface Width Degradation Due to Physical Layer Isolation

Another significant issue encountered during implementation was the insufficient number of I/O pins on the selected FPGA part. **Although the LUT utilization was very low**, the I/O pin utilization exceeded **200%!** This occurred because the implemented design only covers the physical layer of the UCIe protocol, and the bus interface between the physical layer and the upper D2D protocol layer is exposed as a top-level interface. As a result, the design required many I/O pins. To address this constraint, the width of the RDI interface was degraded from 64 bytes to 8 bytes, allowing the design to fit within the available I/O resources.

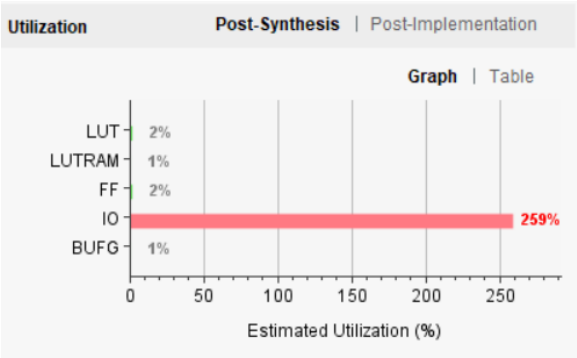


Figure 8-13 Design Utilization due to IO Pins Limitation

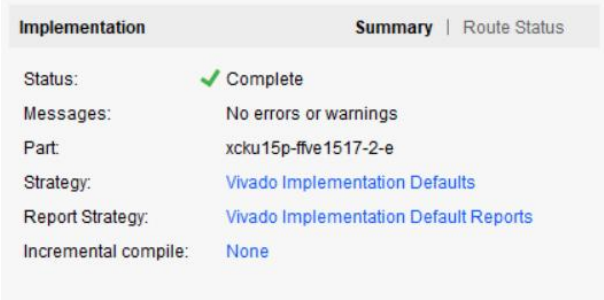


Figure 8-14 final Implementation Status

8.2.4 Simulation

Task-Based Testbench for Functional Verification

For the simulation phase, we developed a T**ask-Based back2back testbench** to verify the main functionalities of the system. The test bench was structured to simulate the complete system behavior, including:

Transitioning out of the **Reset state**
Traversing through all **Training states**
Sending **Sideband (SB) messages** in the Active state, such as:
Local Register Access Requests (Local RAR)
Remote Register Access Requests (Remote RAR)
Advertising Capabilities messages
Sending and receiving **Main-band (MB) data**

Behavioral Simulation

Limited Language Support
The FPGA design flow is primarily based on synthesizable Verilog, which lacks support for several advanced System Verilog constructs. Notably, **2-dimensional arrays** are not supported in the synthesis tools used for this device, necessitating manual restructuring of data storage and access patterns. This limitation increased the complexity of modeling UCIe structures and required additional effort to transform multidimensional data into a format compatible with the FPGA toolchain.

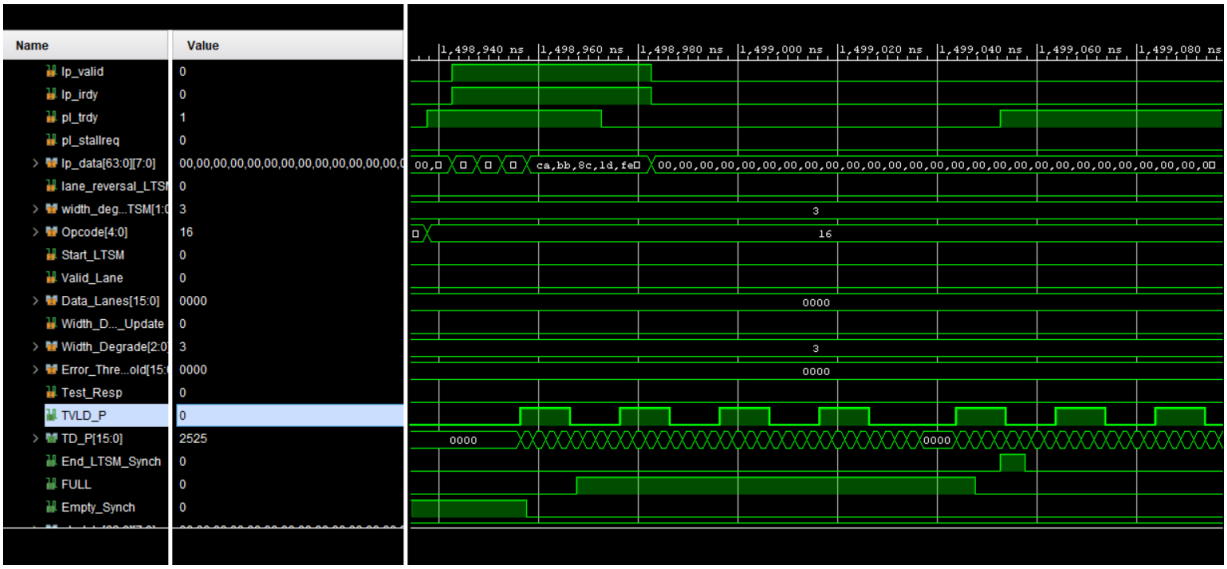


Figure 8-15 Behavioral Simulation Result (MB Data sent)

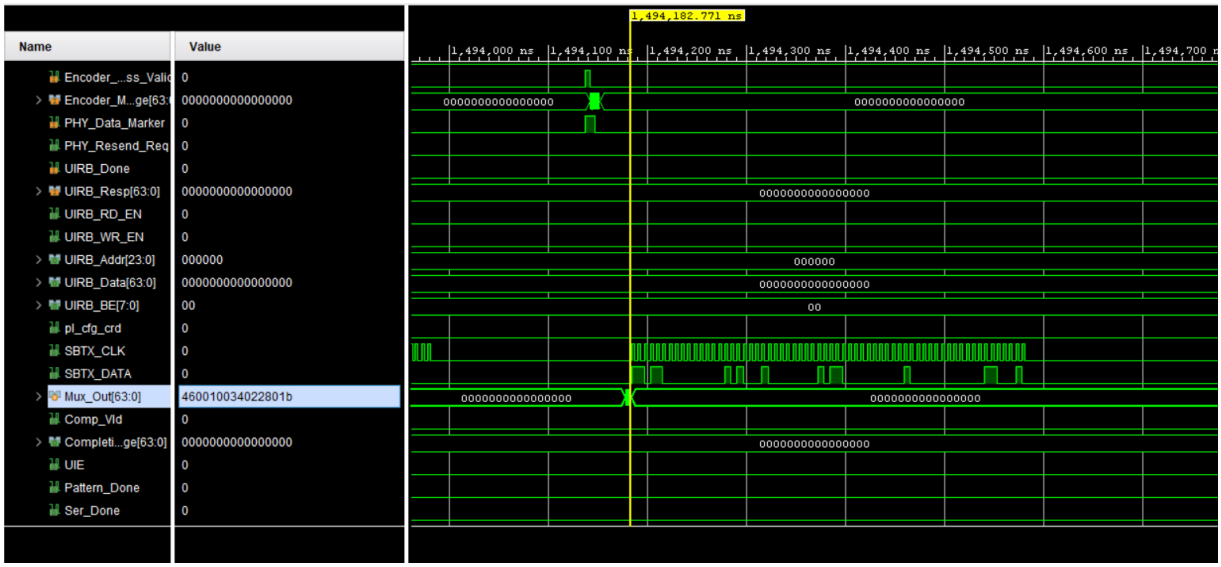


Figure 8-16 Behavioral Simulation Result (SB Message Sent)

8.3. Challenges and Modifications:

During the FPGA implementation phase, several limitations were encountered due to the mismatch between the UCIe specification requirements and the capabilities of the target FPGA device (XCKU15P-FFVE1517-2-E). The following challenges were observed:

1. **Frequency Limitations**
   UCIe requires the use of multiple high-frequency clocks to meet performance targets, including a **2 GHz MB clock**, a **1 GHz Lclk**, and an **800 MHz SB clock**. However, typical FPGA fabrics, including the Kintex UltraScale+ series, are not capable of operating at these high frequencies across large logic areas due to inherent architectural and timing limitations. As a result, achieving UCIe-compliant clock domains directly on the FPGA posed a significant challenge.

2. **I/O Bandwidth and Port Constraints**
   While the target FPGA provides a large number of I/O pins (up to 512), UCIe demands extremely high data throughput. Specifically, the interface to the Adapter Layer requires **2 × 64-byte data buses** (128 bytes total) for simultaneous send and receive operations. This is equivalent to **1024 bits** of data width, which alone exceeds the number of available FPGA I/O pins when accounting for other essential signals.

3. **Clock Ports Distribution**
   A challenge encountered was the physical separation of clock ports when special pin assignments were not specified, leading to increased routing complexity and potential timing issues.

4. **Limited Language Support**
   The FPGA design flow is primarily based on synthesizable Verilog, which lacks support for several advanced SystemVerilog constructs. Notably, 2-dimensional arrays are not supported in the synthesis tools used for this device, necessitating manual restructuring of data storage and access patterns. This limitation increased the complexity of modeling UCIe structures and required additional effort to transform multidimensional data into a format compatible with the FPGA toolchain.

Design Modifications Implemented:

1. **Frequency Scaling:**
   To address the FPGA's inability to operate at UCIe's required high frequencies, all clock domains were divided by a factor of five. This resulted in a 400 MHz MB clock, a 200 MHz Lclk, and a 160 MHz SB clock—frequencies that fall within the reliable operating range of the Kintex UltraScale+ FPGA.

2. **I/O Bandwidth Reduction:**
   To meet the I/O limitations, the Raw D2D Interface (RDI) width was reduced to 8 bytes.
   - On the transmit path, this 8-byte data was replicated internally to form a full 64-byte word, which was then delivered to the Main-Band transmitter.
   - On the receive path, the least significant 8 bytes from the 64-byte Main-Band receiver output were selected and forwarded to the D2D interface.

3. **Clock Port Assignment:**
   dedicated clock capable I/O (CCIO) pins were manually specified in the **XDC constraints file**. By assigning each clock signal to an appropriate and nearby CCIO location, clock routing was optimized, timing violations were reduced, and reliable operation across all clock domains was ensured.

4. **SystemVerilog Compatibility:**
   Due to the limited support for 2-dimensional arrays in synthesizable Verilog, all 2D arrays were initially converted into 1-dimensional representations for synthesis compatibility. These flattened arrays were then restructured back into 2-dimensional form within the logic using indexing techniques, allowing the design to preserve its intended functionality while remaining compliant with the FPGA synthesis tools.

Chapter 9: Future Work:

9.1. Migration to UCIe x64 Advanced Package

Transitioning from the current UCIe x16 Standard Package to the x64 Advanced Package enables a wider range of capabilities within the physical and logical PHY layers. This expansion facilitates greater bandwidth and scalability, laying the foundation for more complex interconnect scenarios involving multiple chiplets or modules. It also aligns with industry movement toward higher-density, high-performance die-to-die interfaces in heterogeneous integration.

9.2. Lane Repair Mechanism

With the x64 Advanced Package, the design can support the lane repair mechanism, which adds a level of redundancy and fault tolerance during link bring-up. If any lane is found to be non-functional or degraded, the system can automatically map it out and reroute traffic using spare lanes. This significantly improves yield and reliability, especially in large-scale designs where manufacturing defects are statistically more likely.

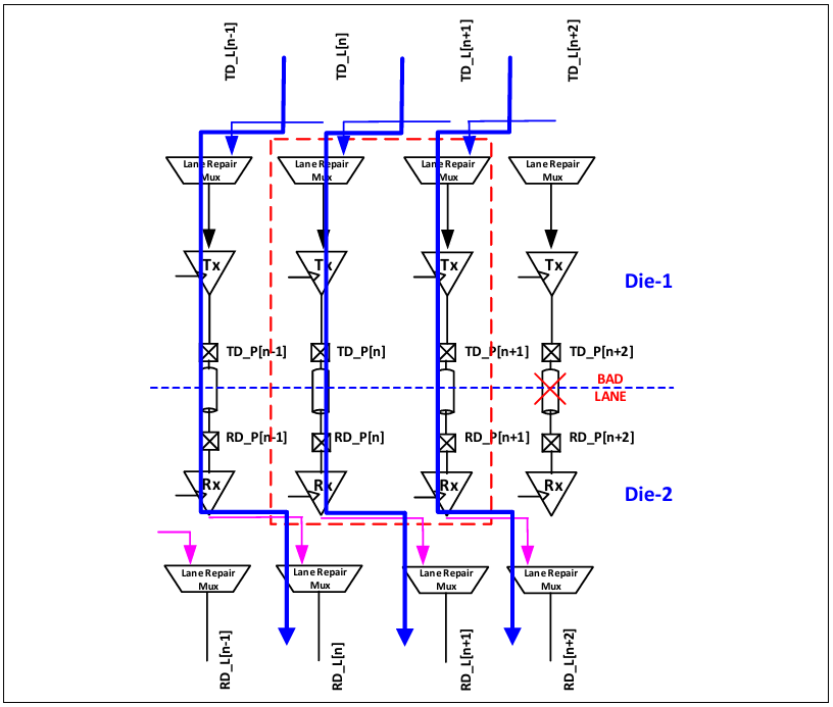


Figure 9-1: Example of Single Lane remapping implementation

9.3. MMPL (Multi-Module PHY Logic) Integration

Other simpler additions include the introduction of MMPL—a centralized control logic unit within the PHY that coordinates communication between the RDI and multiple internal modules (e.g., accelerators, memory controllers). MMPL effectively acts as an arbiter, managing access conflicts and ensuring fair and efficient data transfer across chiplet boundaries. Its inclusion would demand additional protocol logic and verification around arbitration and traffic shaping.

9.4. PHY Interoperability and Multi-Module Configuration Support

As UCIe adoption continues across various packaging types, future work must focus on ensuring PHY-level interoperability—especially between x64 and x32 modules. Supporting both Full Width and Degraded Width Modes through **Multi-Module PHY Logic (MMPL)** is key. MMPL enables two x64 modules to operate as a single x32 interface by dynamically handling lane mapping and coordination, allowing the system to match bandwidth with the link partner without hardware changes. This flexibility is essential for scalable multi-die and cross-package setups.

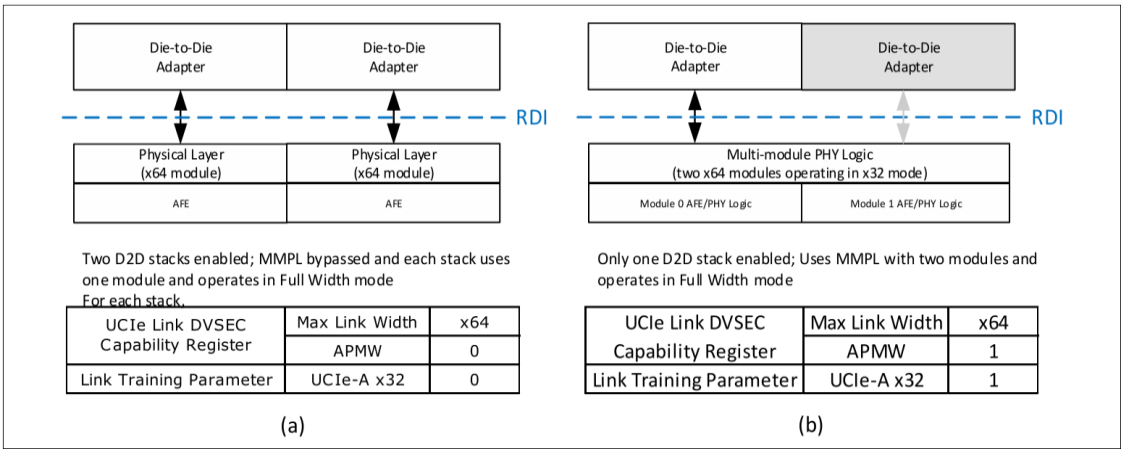


Figure 9-2:Implementation Example Showing Two Different Operating Modes

9.5. Integration of External Reference IPs as Predictors

A key direction for enhancing the verification environment is the integration of commercial or open-source UCIe-compliant IPs into the existing UVM testbench—specifically as reference models for the scoreboard or predictor component. These IPs, developed by established vendors, offer mature, specification-accurate behavior and can serve as a highly reliable golden reference against which DUT outputs can be compared.

By incorporating such IPs into the UVM environment, the verification flow benefits from cycle-accurate predictions and robust mismatch detection, especially in areas where functional correctness is subtle—like error signaling, state recovery, and timing-sensitive transitions. This approach would not only improve confidence in DUT behavior, but also reduce the overhead of manually modeling expected results, particularly as the design complexity increases with wider link widths or added features like lane repair or MMPL.

This strategy also sets the foundation for future co-simulation or mixed-language integration, should the external IP be delivered in a language other than SystemVerilog.

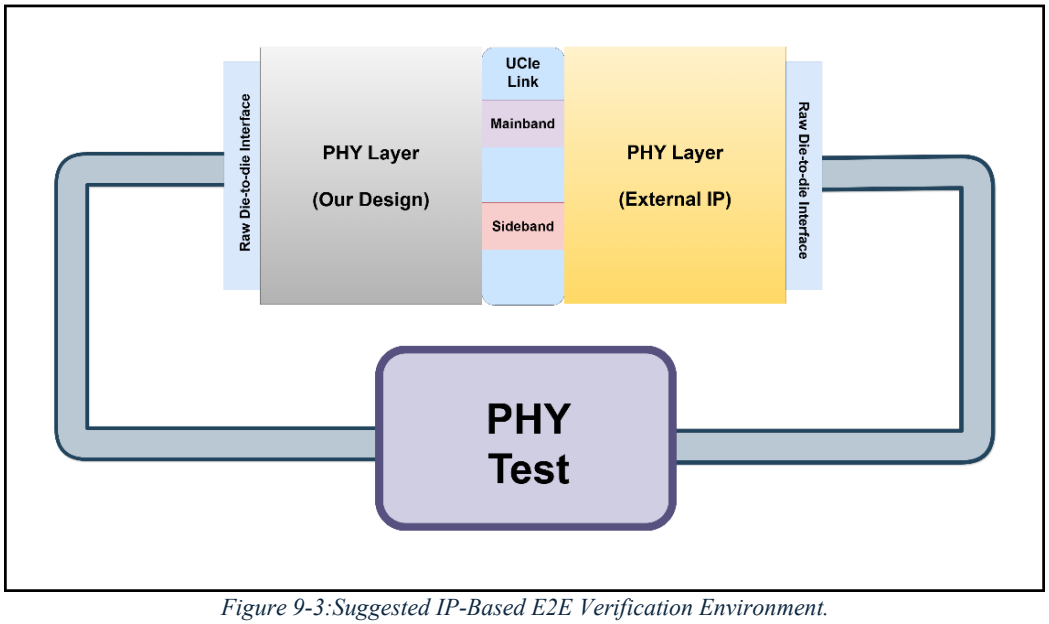


Figure 9-3:Suggested IP-Based E2E Verification Environment.

References

[1] T. Fitzpatrick, "UVM Reactive Stimulus Techniques," *DVCon U.S. 2020 Conference*, San Jose, CA, USA, 2020. [Best Paper Award – 1st Place].

[2] C. E. Cummings, "OVM/UVM Scoreboards – Fundamental Architectures," *Sunburst Design, Inc.* [Online]. Available: https://www.sunburst-design.com

[3] C. E. Cummings and J. Bergeron, "Using UVM Virtual Sequencers & Virtual Sequences," presented at *DVCon*, San Jose, CA, USA, Feb. 2012. [Online].

[4] M. Litterick, J. Sproat, and J. Bromley, "Advanced UVM Tutorial: Taking Reuse to the Next Level," presented at *DVCon U.S.*, San Jose, CA, USA.

[5] M. Litterick, "Pragmatic Verification Reuse in a Vertical World," 2013.

[6] Siemens Digital Industries Software, *UVM Cookbook*, Siemens EDA. [Online]. Available: https://eda.sw.siemens.com

[7] F. Cerisier, "Advanced UVM Scoreboarding Techniques," presented at DVClub, [Online]. Available: https://www.dvclub.org (or insert correct URL if known), 2016.

[8] Accellera Systems Initiative, "Universal Verification Methodology (UVM) 1.2 User Guide," Accellera, 2015. [Online]. Available: https://accellera.org/downloads/standards/uvm

[9] G. Allan, "Keeping Score: Part 1 of 2 - Architectural Considerations for Scoreboard Design," Mentor Graphics Corp., Fremont, CA. [Online]. Available: https://www.mentor.com/products/fv/verification-horizons/volume9/issue1/keeping-score-part1-scoreboard-design

[10] S. Churiwala and S. Garg, Principles of VLSI RTL Design: A Practical Guide. Springer, 2011.

[11] K. Ismail, T. Ismail, and H. Mostafa, "Design and Implementation of CDR and SerDes for High Speed Optical Communication Networks Using FPGA," in Proc. 18th International Conference on Transparent Optical Networks (ICTON), Trento, Italy, Jul. 2016, pp. 1-4. [Online]. Available: https://scholar.cu.edu.eg/sites/default/files/hmostafa/files/icton.pdf

[12] N. Sharif, N. Ramzan, F. K. Lodhi, O. Hasan, and S. R. Hasan, "Synchronizers: Clock Domain Crossing Perspective," School of Electrical Engineering and Computer Science, National University of Sciences and Technology (NUST), Islamabad, Pakistan, and Department of Electrical Engineering, École Polytechnique de Montréal, Montréal, Canada, 2011. [Online]. Available: https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/76984066/1a2ce918-a932-4fe4-a380-60a3a92f89f8/IEEE-Synchronizers.pdf

[13] AMD Inc., "IOB 3-state Output Switching Characteristics," Kintex UltraScale+ FPGA Data Sheet: DC and AC Switching Characteristics (DS922), 2023. [Online]. Available: https://docs.amd.com/r/en-US/ds922-kintex-ultrascale-plus/IOB-3-state-Output-Switching-Characteristics.

[14] AMD Inc., Kintex UltraScale+ FPGA Data Sheet: DC and AC Switching Characteristics (DS922), 2023. [Online]. Available: https://docs.amd.com/r/en-US/ds922-kintex-ultrascale-plus.

[15] G. Ramesh, V. Shivaraj Kumar, and K. Jeevan Reddy, "Asynchronous FIFO Design with Gray code Pointer for High Speed AMBA AHB Compliant Memory Controller," IOSR Journal of VLSI and Signal Processing (IOSR-JVSP), vol. 1, no. 3, pp. 32–37, Nov.-Dec. 2012. [Online]. Available: https://www.iosrjournals.org/iosr-jvlsi/papers/vol1-issue3/E0133237.pdf.